

中国操作系统领创者 给世界更好的选择

统信UOS 开发者应用指南

统信软件技术有限公司 著

WWW.UNIONTECH.COM

统信 UOS 开发者应用 指南

序

早年间我国还是一个“缺芯少魂”的国家，当时 Deepin 社区像一支光束宣告要补足我国的这块短板，统信 UOS 是 Deepin 的发行版。

《统信 UOS 开发者应用指南》作为一本专注于统信 UOS 操作系统开发技术的指导手册，面向开发者生态全面地介绍在以 UOS 为基础进行程序开发的各个面。该书可以说是应用 UOS 操作系统开展程序开发的指南，它不仅覆盖基于 UOS 操作系统原生程序开发的基础知识，包括多种编程语言和应用场景的支持、开发工具和应用框架的使用等，还深入介绍了迁移 Windows 生态应用、Android 生态应用到 UOS 操作系统的基础知识，展现作为优秀开发环境的 UOS 系统，开发桌面应用、Web 应用、甚至安卓应用和 AI 应用的全方位支持能力。每一章节都充满专业性和实用性，为开发者们提供了全景视角，助力开发者在应用开发时面临的选型、开始、展开等过程的指导。

为统信 UOS 做序也表达我们对国内基础软件生态领域发展的热情和关注。

Handwritten signature in black ink, reading '陆肖群' (Lu Xiaoqun).

前言

统信软件作为中国操作系统领创者，切身体会到，开发者是操作系统生态的最重要支撑之一。帮助开发者快速和全面的了解并熟悉如何使用统信 UOS 进行开发，是完善和巩固操作系统软硬件生态的重中之重。开发者是生态的成长土壤，统信 UOS 势必要向下扎根，才能茁壮成长。

对于使用统信 UOS 操作系统进行开发工作，很多开发者对系统支持的编程语言、开发框架及开发工具的安装、配置、部署等方面缺乏系统的了解渠道。为了解决上述问题，统信产品开发团队根据开发者提出的具体问题，并结合统信 UOS 操作系统的实际情况，推出了《统信 UOS 开发者应用指南》。

本指南以讲解统信 UOS 操作系统上如何完成开发与迁移为例，介绍了如何在 UOS 操作系统完成原生开发、第三方应用开发、软件迁移等，希望借此能让更多企业、组织和个人开发者了解到统信 UOS 操作系统上的开发工具、开发技巧，提升开发者体验，推进统信 UOS 生态完善。

本指南注重理论与实践的结合，通过以实际案例讲解在统信 UOS 操作系统完成开发与迁移。指南总计十个章节，讲述 UOS 操作系统面向原生代码开发、安全接口开发、Web 应用开发、迁移方案、基础服务与运维、开发者平台等多个维度，为开发者提供全面的指引。

本指南介绍了主流的编程语言、主流框架、跨平台迁移方案、第三方应用开发的安装，配置，开发介绍，常见问题解决等，帮助开发者选择适用于自身的方法。

案；提供 UOS 默认的工具版本及适用于 UOS 系统的版本，避免用户在实际开发中使用不同版本间带来的兼容性问题；收录了一些成功的用户开发案例，希望能够解决用户的开发问题，作为后续开发者的重要参考依据。帮助开发者快速上手统信 UOS 的开发与迁移工作，希望能够解决读者面临的主要挑战。

作者

2023 年 12 月 18 日

《统信 UOS 开发者应用指南》问卷调查

我们深知在一本指南里面很难解答开发者面临的所有问题，不断改进和完善统信 UOS 的平台能力是我们一贯的坚持。为了不断收集和了解开发者的需求和反馈。因此，我们特地设计了一份问卷调查，以便更全面地了解开发者在使用《统信 UOS 开发者应用指南》时遇到的问题、挑战以及改进建议。



目录

第一章 统信操作系统	1
1.1. 统信 UOS 产品简介	1
1.2. 统信 UOS 桌面版	1
1.3. 统信 UOS 服务器版	2
第二章 开发者应用指南	5
2.1. 本章目的	5
2.2. 指南价值	5
2.3. 开发方案	6
2.3.1. 原生开发方案	6
2.3.2. 迁移适配方案	7
2.4. 指导对象	10
第三章 UOS 原生应用开发	11
3.1. 本章目的	11
3.2. 编程语言	12
3.2.1. C/C++开发配置	12
3.2.2. C#开发配置	22
3.2.3. Java 开发配置	25
3.2.4. Python 开发配置	27
3.2.5. Perl 开发配置	31
3.2.6. Shell 开发配置	35
3.2.7. 其他编程语言开发配置	38
3.3. 开发套件	44
3.3.1. Qt 应用开发	45
3.3.2. DTK 应用开发	53
3.3.3. OpenGL 应用开发	64
3.3.4. Vulkan 应用开发	75
3.4. D-Bus 进程间通信开发	83
3.4.1. 通用 D-Bus 接口	85
3.4.2. UOS 扩展 D-Bus 接口	90
3.5. UOS 集成开发工具	94
3.5.1. UnionCode	95
3.5.2. Qt Creator	97
3.5.3. VS Code	101
3.5.4. Eclipse	105

3.5.5. Android Studio	108
3.6. 小结	112
第四章 UOS 安全接口开发	114
4.1. 本章目的	114
4.2. 系统安全接口	114
4.2.1. 环境安装	115
4.2.2. 环境配置	116
4.2.3. 开发案例	116
4.2.4. 常见问题	119
4.3. USKI 接口	119
4.3.1. 环境安装	120
4.3.2. 环境配置	121
4.3.3. 开发案例	130
4.3.4. 常见问题	138
4.4. pam 认证接口	139
4.4.1. 环境安装	139
4.4.2. 环境配置	139
4.4.3. 开发案例	139
4.4.4. 常见问题	142
第五章 UOS “Web” 应用开发	144
5.1. 本章目的	144
5.2. 浏览器扩展插件开发	144
5.2.1. 浏览器扩展开发	145
5.2.2. 浏览器插件开发	148
5.3. Web 应用开发	154
5.3.1. ASP.NET 应用开发	155
5.3.2. Laravel 应用开发	161
5.3.3. Ruby on Rails 应用开发	168
5.3.4. Django 应用开发	172
5.3.5. Spring Boot 应用开发	179
第六章 UOS 迁移方案	188
6.1. 本章目的	188
6.2. deepin-wine 迁移	188
6.2.1. 环境安装	189

6.2.2. 环境配置	189
6.2.3. 开发案例	194
6.2.4. 常见问题	196
6.3. 跨平台开发框架	202
6.3.1. Electron 应用开发	202
第七章 UOS 第三方应用开发	207
7.1. 本章目的	207
7.2. Android 应用开发	207
7.2.1. Flutter Android 应用开发	208
7.2.2. React Native Android 应用开发	217
7.3. AI 应用开发	225
7.3.1. CUDA 应用开发	225
7.3.2. OpenCV 应用开发	236
7.3.3. TensorFlow 应用开发	242
7.3.4. PyTorch 应用开发	249
7.4. 小结	257
第八章 UOS 搭建基础服务与运维	260
8.1. 本章目的	260
8.2. 基本服务安装与部署	260
8.2.1. HTTP 服务	260
8.2.2. DNS 服务	262
8.2.3. DHCP 服务	266
8.2.4. FTP 服务	268
8.2.5. NFS 服务	269
8.2.6. SAMBA 服务	270
8.2.7. iSCSI 存储服务	271
8.3. 虚拟化与容器	273
8.3.1. KVM 虚拟机管理	273
8.3.2. Docker 容器	275
8.4. 典型常用应用	277
8.4.1. NGINX 服务	277
8.4.2. 数据库（MariaDB 为例）	279
8.5. 服务器的管理	281
8.5.1. Ansible 管理	281

第九章 UOS 开发者平台	282
9.1. 平台价值	282
9.2. 开发者模块介绍	283
9.2.1. 文档支持	283
9.2.2. 常见问题	284
9.2.3. 开发者论坛	284
9.2.4. 商业化支持	284
第十章 开发案例介绍	286
10.1. 中国移动“移动办公”开发案例	286
10.2. 中国移动“移信云”平台开发案例	288
10.3. 北京城建“i 城建”平台开发案例	291
10.4. 无锡研究院开发案例	294
10.5. 无锡研究院开发案例	296
10.6. 云达软件资产管理系统开发案例	298
10.7. 广州掌动智能 AI 性能一体化保障平台 V3.0 开发案例	300
10.8. 交大慧谷案件管理系统开发案例	302
10.9. 交大慧谷通用融合法庭开发案例	303
10.10. 交大慧谷司法行装管理系统开发案例	305
10.11. 交大慧谷政务管理系统开发案例	306
第十一章 总结	308
附录一：统信软件介绍	309
附录二：通用开发问题	311
附录三：版本推荐	313

第一章 统信操作系统

1.1. 统信 UOS 产品简介

统信操作系统（Uniontech OS）是由统信软件基于 Linux 内核开发的操作系统。围绕国内市场需求，统信软件已经建立全场景支持的产品体系，打造了统信 UOS 桌面版、服务器版、智能终端版、云原生版四大 OS 产品，集中域管、应用商店、内网更新、畅写文档、“有”系列迁移等增值产品，终端 OS 迁移、CentOS 替换等众多解决方案。统信软件的产品和服务已经深入应用到党政、关键行业等众多领域。统信软件无论在技术实力和人才团队，还是市场拓展和生态建设，以及人才培养，都取得了优异的成绩。

1.2. 统信 UOS 桌面版

统信 UOS 桌面版是基于 Linux 开源内核的通用操作系统，同源异构支持全系列 CPU 架构，提供高效简洁的人机交互、美观易用的桌面应用与安全稳定的系统服务，是真正好用的自主操作系统。

统信 UOS 桌面版提供完善的生命周期支持，自主研发的桌面环境（DDE），已与 RedHat 领导的 GNOME、SUSE 领导的 KDE 成为全球主流的操作系统桌面环境；预装 50 余款自研应用，包含浏览器、联系人、日历、邮箱等，开箱即用；内置应用商店，可下载海量应用，一键安装，还跨平台支持 Windows 常规应用（AMD64/ARM64）百余款、安卓应用（AMD64/ARM64）数万款，全

面满足办公、生活、娱乐需求，应用均经过签名认证安全稳定；广泛适配打印机、扫描仪、高拍仪、指纹仪、扫码枪等外设，并提供外设管理应用；提供安全中心、生物识别、多因子认证等全方位的安全防护，并支持国密，全方位保障用户信息安全；提供远程协助、系统监视器、日志收集工具等多种运维工具，提升企业运维效率，降低运维成本；提供使用帮助、功能说明、设置引导等多种线上帮助功能，帮助用户轻松熟悉系统。

在最新版本中，统信 UOS 桌面版基于用户实际需求出发，在提升生产力、安全增强等方面做了众多更新，如增加“仅安全更新”，保证用户在不升级系统的前提下获得长久的安全补丁；嵌入毕昇引擎，解决老旧打印机没有 Linux 驱动问题，保证用户连接打印机即插即用并支撑用户的利旧需求；此外还增加了应用分屏、截图 OCR、长截图、跨端协同、面容识别、虹膜识别等众多功能。除了功能上的快速迭代外，统信 UOS 桌面版还持续优化交互细节、用户体验，快速修复系统缺陷，持续提升系统的性能以及稳定性。

1.3. 统信 UOS 服务器版

统信 UOS 服务器版是基于 Linux 开源内核的通用操作系统，同源异构支持全系列 CPU 架构，广泛适用于高可用集群、中间件、云计算、容器等应用场景，支撑电信、金融、政府等政企规模化应用需求。

统信 UOS 服务器版与开放原子开源基金会的 openEuler 社区（贡献率第三）、OpenAnolis 社区（贡献率第二）均有紧密合作。产品在稳定性方面，LT

P 用例测试通过率提升至 98% 以上；在安全方面，率先支持全栈国密特性；在 O/S 性能方面，Unixbench 等综合评分提升 10% 以上，Lmbench 在进程处理综合提升 15%；在兼容性方面，提供主流的数据库、中间件依赖软件栈 API 列表，并引入应用程序流机制；在生态方面，除 intel、AMD 以外，还全面支持海光、兆芯、鲲鹏、飞腾、龙芯等国内主流 CPU 平台，实现 21 万余款软硬件产品兼容适配组合。截止 2023 年 5 月，统信 UOS 服务器版小版本更新 22 次、日常补丁推送 53 次、安全更新修复 CVE 超过 4 万个，着重在稳定性、安全性、性能、OS 解决方案、兼容性和生态扩充方面开展工作。

统信软件基于服务器操作系统的应用需求，推出了多项增值组件，包括：统信服务器迁移软件-统信有易 UYi、统信高可用集群部署管理软件-统信有备 UHA、统信服务器运维监管平台-统信有幄 UMOP 等。

统信软件还推出了多个产品级联合解决方案，助力用户加速推进服务器应用领域迁移和数字化转型。

统信服务器系统迁移解决方案-统信有易，面向已部署业务应用于 CentOS 系列操作系统且具有自主替换需求的用户，帮助其快速、平滑、稳定并安全地迁移至统信 UOS 环境，支持全面的环境检测、智能的兼容性评估、简洁的流程操作，既能保障原有硬件和应用环境不变的情况下完成平滑迁移，也能针对新增扩容的场景提供兼容性评估，辅助用户进行系统迁移。

统信高可用解决方案-统信有备，致力于提升系统与应用高可用能力，提供集群与资源管理、故障注入式高可用能力演练、一键环境检测等功能。满足电信

级可用性要求，可实现数据实时同步和秒级切换，确保故障发生时用户业务系统的连续不中断。

统信运维管理解决方案-统信有幄，基于模块化、插件化设计理念，集“监、管、控、安全”为一体，适用于传统数据中心及云数据中心场景，切实减轻企业快速数字化转型进程中与日俱增的运维压力，为企业提供简单、易用、安全、高效的全方位一体化解决方案。

第二章 开发者应用指南

2.1. 本章目的

本指南将从 UOS 操作系统生态建设中原生代码开发与兼容适配开发等方面所面临的痛点出发，针对不同时期的痛点，总结开发过程相关的方法论，并给出应用开发的最佳实践和建议，希望读者在跨平台应用开发中更加专业、顺利、完整地实施开发工程。

2.2. 指南价值

本指南有助于帮助开发者全面掌握 UOS 操作系统应用开发相关的内容，让更多的应用软件厂商和开发者更快的参与到 UOS 生态建设中来。

其主要价值体现在：

- 通过快速兼容方案，可帮助用户将部分现有 Windows、安卓应用快速完成适配突破，解决短期内 UOS 操作系统无应用可用的局面。
- 通过方法论为开发人员提供不同应用在 UOS 操作系统下兼容的适配方案。
- 为开发者提供不同开发语言跨平台开发提供指导，降低开发者的入门学习难度。
- 提供服务能力与相关开发工具能力的建议。

- 迁移过程中典型问题的分析与常见误区的解析。
- 通过应用软件代码原生开发支持，并在 UOS 操作系统及多种 CPU 架构中提供代码优化、资源调整等方面的建议。
- 通过案例讲解为开发人员提供详细代码编写过程，解决开发者最关心的问题。

通过该指南，开发人员可以更有效地解决这些问题，确保应用程序顺利开发，在新环境并保持稳定运行。

2.3. 开发方案

虽然我们的 UOS 操作系统在生态建设上已经初具规模，但是相对 Windows 系统 30 多年的积累来说，我们的生态建设还是远远不足。基于国内市场的发展需求，大量的在 Windows 系统下开发的软件需要在 UOS 操作系统平台上重新开发或迁移。

值得注意的是，并不是所有的 Windows 应用产品都能在 UOS 操作系统平台上找到兼容方案，我们目前主流方案主要有两种，原生开发方案、迁移适配方案。

2.3.1. 原生开发方案

如果应用使用了在 Linux 上完全不兼容的技术进行开发，那么这些源码在 Linux 上将完全无法使用，需要进行重写。截至到 2023 年，主流语言基本不存在只支持 Windows 的情况，但是很多开发框架还是无法在 Linux 上使用，这些典

型的开发框架包括 MFC、WPF、Windows API 等。重写需要根据现有开发资源的能力，选择合适的技术框架进行开发，这些可选的技术栈包括前面所说的 java、go、Qt、Electron 等。

选择原生开发方案的优势在于，应用软件从设计到开发，完全依托于 UOS 操作系统，因此对于 UOS 操作系统的兼容性是最好的。但是缺点就是需要应用厂商投入大量的人力和时间，对现有的应用产品进行重写，相当于重新开发一款新的应用软件。

2.3.2. 迁移适配方案

2.3.2.1. 编译适配方案

Windows 下的主流开发语言和框架都支持基于 Linux 的 UOS 操作系统。跨平台开发语言包括：C#/C/C++/Python/Java/PHP。跨平台开发框架包括：Qt/Electron/Java/.Net 等，通过应用编译适配的软件主要优势：

- 大部分都可以在 UOS 操作系统上运行。
- 具备丰富的 API 接口封装，甚至还包括正则表达式的处理功能。支持 2D/3D 图形渲染，支持 OpenGL，以及大量的开发文档。
- 对源码较少改动情况下进行重新编译，例如基于 Qt 开发的应用程序，主要是调整编译开关，修改平台相关的接口。
- 实现解释型语言的虚拟机，支持基于该语言的应用程序直接运行，例如 Java/.Net 框架本身就已经实现了完整的跨平台支持，如 Electron，其开发

语言（HTML/JavaScript）是解释型语言，且官方提供了完整的运行环境（基于 Chromium 浏览器核心）。这种基本重新编译打包即可以完成。但在选择应用编译适配方案时的缺点也是很明显的。

- 必须要有多名熟悉各类跨平台开发语言的专业人员。
- 技术人员要具备除代码研发知识外，还要具备系统工程师和运维工程师等多维度的知识储备。在实施方案的同时需要准备相应的运行环境和开发环境。

2.3.2.2. 跨平台迁移方案

Deepin-Wine 对于使用跨平台框架/语言技术开发的应用，可以利用框架/语言的跨平台特性减少工作量，但还是有很多应用是基于原生的 Windows 接口进行开发的。同时，即使是使用跨平台框架/语言开发的程序，为了实现某些平台相关的特性，也会使用原生的 Windows 接口进行相关功能开发。对于这些使用原生 Windows 接口的源码，正常情况下需要完成重写才能满足应用迁移的需求。Wine（“Wine Is Not an Emulator”的首字母缩写）是一个能够在多种 POSIX-compliant 操作系统（诸如 Linux，macOS 及 BSD 等）上运行 Windows 应用的兼容层。

- 迁移方案的优势。

Wine 通过 API 级别的接口翻译，可以实现复杂应用程序不需要要编译就能在 UOS 操作系统上运行，其主要迁移工作在于 Wine 项目本身，对应应用程序的修改较少。

Deepin-wine 是统信公司基于上游 wine 的独立维护的一个分支，Deepin-wine 的开发团队会定期把解决的修复补丁提交到上游 wine 社区。目前该团队迁移了常用 top 应用和行业应用近百款 Windows 软件，基本满足用户的日常使用。

跟 Crossover 比较，Deepin-wine 是直接把迁移好的 Windows 程序打包成 Linux 的 deb 包，提交到统信 UOS 的仓库和应用商店，使得用户安装 Windows 程序更加便捷。

在应用程序兼容性方面，Crossover 主要适配的是国外的应用，而 Deepin-wine 对常用应用做了更深入的适配，解决了更多 Linux 系统的兼容性问题。

Wine 方案是一套开源的 Windows API “翻译”方案，可以在不修改原有应用程序的情况下，将 Windows 下面的应用程序迁移到 UOS 操作系统平台。同时，统信在 Wine 基础上开发了自己的商业解决方案 Deepin-wine，针对 UOS 应用程序进行单独的适配，其方案兼容更加优秀。

但同时，这套 Wine 应用的迁移方案也有不足地方。

- 迁移方案的不足。

只支持用户态程序，不支持内核态驱动。对于复杂的软件适配周期长，比如使用 IE 和 ActiveX 控件的 web 应用程序，大型 CAD 和 EDA 类软件。某些特定程序没有根本的解决方案，需要用特殊的绕过办法来解决，比如跟 Linux 桌面环境集成部分的窗口刷新、托盘图标显示等问题。

2.4. 指导对象

本指南作为 UOS 操作系统生态建设中最为重要的应用开发指导手册，其受众大致为以下人员。

- UOS 操作系统生态建设的从业技术人员。
- 应用软件厂商的产品及技术开发人员。
- 有应用软件开发意向、开发需求的用户技术人员。
- 使用 UOS 操作系统的社区开发者用户及技术爱好者。
- 高等院校师生及从事或有意愿从事 UOS 操作系统应用软件开发的相关人员。

第三章 UOS 原生应用开发

3.1. 本章目的

统信 UOS 操作系统基于的 Linux 平台也有了多年的技术发展,在编程语言、数据库、中间件、基础开发工具等方面,其支持程度和 Windows 并未有很大的差距,除了开发工具外,基本都能在 Linux 平台找到合适的替代。本章节主要介绍 Windows 下主流的通用技术在 UOS 操作系统平台上如何使用。

对于编程语言,Windows 下主流的开发语言包括 C、C++、C#、Python、Java 等,这些编程语言在 Linux 平台均有较好的支持,软件厂商需要处理好平台相关的接口和特性即可。

在数据库方面,Windows 下主流的数据库包括 SQL Server、DB2、MySQL 等,这些数据库中的数据 and 接口均可以通过工具转换到 UOS 操作系统平台的数据库。

对于中间件,其本身就是为了解决操作系统差异而存在的,国内像金蝶、东方通、普元提供的中间件本身就提供 Linux 平台支持。而上游开源的 Apache、Nginx 也支持多个操作系统平台。对于和 Windows 本生绑定的 IIS 服务,则可以通过 Nginx 来提供同样功能,其业务规则也可通过工具进行转换。

在开发工具方面,Windows 上有大量开发生态和专有的开发工具,但是随着 VSCode 这种通用开发工具的发展,统信操作系统上的差距也被逐渐补齐。W

indows 上的主流开发工具有 Visual Studio、VSCode、Qt Creator、Eclipse、Android Studio 等。在这些开发工具中，Visual Studio 是无法在 Linux 中使用的，对于使用 Visual Studio 的软件厂商，建议切换到同是微软开发的 VSCode 上。其他开发工具均支持统信操作系统，后面将详细介绍其使用方式。

3.2. 编程语言

Windows 下主流的编程语言包括 C/C++、C#、Java、Python 等,本章节将详细介绍这些主流编程语言在统信操作系统中的编译环境构建,这些编程语言在统信操作系统均有较好的支持,软件厂商需要处理好操作系统相关的接口和特性即可。

3.2.1. C/C++开发配置

C/C++应用程序是使用 C 或 C++编程语言编写的应用程序,统信 UOS V20 使用 C/C++应用程序 8.3.0 版本,安装这些应用程序可以在 Windows、Linux、Mac、UOS 等操作系统上进行独立编译、运行。但已开发的应用程序期望运行在不同操作系统上,需要解决包括编译器和开发工具链、系统调用和 API、文件路径和分隔符、GUI 界面等方面的兼容性问题。本章从实际出发,介绍如何将 Windows C/C++应用程序在 UOS 系统中如何使用。

3.2.1.1. 环境安装

在 UOS 平台上常用的编译器是 GCC 或 Clang,首先打开终端安装编译工具和构建系统,运行安装 build-essential 命令,将会安装一系列软件包,包括 gc

c、g++和 make 关于如何使用 GNU/Linux 开发的手册，以及软件调试工具 gdb 和构建工具 CMake，分别由下列安装命令实现：

```
sudo apt update
sudo apt install build-essential
sudo apt-get install manpages-dev
sudo apt install gdb
sudo apt install cmake
```

在编写代码时，需要在 UOS 系统上安装代码编辑器，例如 QtCreator。在 UOS 应用商店进行安装，或者可以前往 QtCreator 官方网站下载.deb 格式安装包。或者使用集成开发环境常见的如 QtCreator 可以在终端运行命令：

```
sudo apt install qtcreator
```

3.2.1.2. 环境配置

在完成上一节基本环境的安装后，可以使用以下命令验证其版本：

```
gcc --version
g++ --version
```

如果没有显示版本信息，则可以运行以下命令安装 gcc 和 g++：

```
sudo apt install gcc
sudo apt install g++
```

在 UOS 系统上成功配置了 C/C++开发环境后，可以使用熟悉的编辑器编写代码，并在终端使用 gcc 或 g++编译和运行程序。如果使用的 C/C++应用使用到了更新的语法特性，默认的 gcc 和 g++已经无法通过编译，可以通过安装多个 gcc 和 g++的版本解决问题：

```
sudo apt install gcc-<version> g++-<version>
```


其中<version>是想要安装的具体版本号，如 7、8、9 等。如果在后续项目中，想修改默认的版本，可以使用 update-alternatives 命令：

```
sudo update-alternatives --config gcc
sudo update-alternatives --config g++
```

如果安装了 gdb，可以使用默认配置在终端运行 gdb 命令对应用程序进行调试。在 UOS 下使用 gcc 编译一个基本的 C/C++ 程序非常简单，以 HelloWorld 为例，打开文本编辑器并创建以下文件：

```
deepin-editor helloworld.c
```

编写代码：

```
#include <stdio.h>
int main()
{
    printf ("Hello World!\n");
    return 0;
}
```

保存文件，并且将它编译成可执行文件：

```
gcc helloworld.c -o helloworld
```

这将在命令执行的同级目录下创建一个二进制文件，名称为 helloworld，给该程序添加可执行权限，并在当前目录下打开终端运行这个程序，并输出执行结果：

```
sudo chmod +x helloworld
./helloworld
Hello World!
```

3.2.1.3. 开发案例

有一些 C/C++ 应用的构建工具不是跨平台的，在 UOS 系统工作的第一步就是集成开发工具，假设有一个 Windows 上的 C/C++ 项目，使用 Visual Studio 的解决方案文件（.sln）进行构建，并且项目结构如下：

```
MyProject/  
|—— MyProject.sln  
|—— src/  
|   |—— main.cpp  
|   |—— foo.cpp  
|   |—— foo.h  
|—— include/  
|—— bar.h
```

要将该项目在 UOS 上使用 CMake 进行构建，可以按照以下步骤进行：

- 创建 CMakeLists.txt 文件：在项目根目录 MyProject/ 下创建一个名为 CMakeLists.txt 的文件，内容如下：

```
cmake_minimum_required(VERSION 3.12)  
project(MyProject)  
# 设置源文件  
set(SOURCES  
src/main.cpp  
src/foo.cpp  
)  
# 设置头文件目录  
include_directories(include)  
# 添加可执行文件生成规则  
add_executable(MyProject ${SOURCES})
```

- 编写 main.cpp、foo.cpp 代码并解决因系统调用或系统 API 引起的编译问题。

- 执行 CMake 构建：打开终端，进入项目根目录 MyProject/，执行以下命令来进行 CMake 构建：

```
# 读取当前目录的 CMakeLists.txt，并在 build 文件夹下生成 build/Makefile
cmake -B build
# 让 make 读取 build/Makefile，并开始构建，并在项目根目录(build 目录中)生成可执行文件 MyProject
cmake --build build
```

- 运行程序：进入到项目根目录，并执行以下命令来运行程序：

```
cd build
./MyProject
```

以上是一个简单的示例，展示了如何将 Windows 的.sln 项目在 UOS 上并使用 CMake 进行构建。在实际项目中，可能需要根据项目的具体需求和依赖项进行适当的调整和配置，此时的 CMake 可能会非常复杂，需要参考更多官方 CMake 的资料进行编写。

接下来需要进行代码的 API 调用调整，首先要注意的是系统调用的差异，UOS 和 Windows 使用不同的系统调用 API。在通常情况下，需要使用条件编译和宏来判断当前系统平台并使用相应的系统调用。下面是一个示例，展示了如何使用宏来判断平台并调用不同的系统调用：

```
#ifdef _WIN32
// Windows 平台上的代码
// 使用 Windows 系统调用 API
#else
// UOS 平台上的代码
// 使用 UOS 系统调用 API
```

```
#endif
```

下面是一个简单的例子，演示了一个将 Windows C/C++ 应用程序到 UOS 平台如何使用情况：

```
#include <iostream>
#ifdef _WIN32
#include <windows.h>
#else
#include <unistd.h>
#endif
int main() {
#ifdef _WIN32
// Windows 平台上的代码
MessageBox(NULL, "Hello, Windows!", "Message", MB_OK);
#else
// UOS 平台上的代码
write(1, "Hello, UOS!\n", 14);
#endif
return 0;
}
```

在此示例中，根据平台的不同，使用了不同的系统调用 API。在 Windows 上调用 MessageBox 函数显示消息框，在 UOS 上使用 write 函数将消息打印到标准输出。还需要注意文件路径分隔符，Windows 使用反斜杠 \ 作为路径分隔符，而 UOS 使用正斜杠 /。在代码中使用跨平台的路径分隔符可以增加可移植性，这里也可以使用条件编译和宏来判断：

```
#ifdef _WIN32
const std::string pathSeparator = "\\";
#else
const std::string pathSeparator = "/";
```

```
#endif  
std::string filePath = "path_to_file" + pathSeparator + "filename";
```

除了自身的代码外，还需要注意外部依赖库的差异，在 windows 系统上，一些开发项目通常会依赖于 Windows 上的特定库，这类项目在 UOS 系统使用时，需要找到相应的 Linux 版本或类似的替代库，并进行相应的调整。

为了减少应用的开发成本，建议更多的使用 C/C++标准库，或者跨平台的库（如 Qt、Boost 等）来进行业务功能的编码。

3.2.1.4. 常见问题

- UOS 上构建工具和 Windows 一样吗？

Windows 上使用 Visual Studio 类似的 IDE 工具开发 C/C++程序，通常使用 MSBuild 来进行构建。在 UOS 上，建议使用跨平台的 CMake 来构建 C/C++项目，使用 QtCreator 或 Visual Studio Code 做为 IDE 工具。

- UOS 程序编译器和 Windows C/C++是否一致？

Windows 编译器和 UOS 编译器并不完全一致。它们在编译器本身、系统调用和标准库等方面存在一些差异。

在 Windows 平台上，常用的编译器是 Microsoft Visual C++ (MSVC)，它使用不同的编译器前端和后端，与 UOS 平台上的编译器有所不同。MSVC 使用的是 Windows 特定的 PE(Portable Executable)格式来生成可执行文件。而在 UOS 平台上，常用的编译器是 GNU Compiler Collection(GCC)或 Clang。GCC 和 Clang 采用不同的编译器前端和后端，它们使用的是 ELF(Executable and

Linkable Format)格式来生成可执行文件。

- UOS 与 Windows 操作系统 API 差异性，主要表现在哪些地方？

不同的操作系统之间的系统调用 API 是有差异的。代码中使用的 Windows 特定 API 函数可能在 UOS 上不存在，或者具有不同的函数名、参数或行为。需要查找适当的 UOS 替代函数或使用跨平台库来处理操作系统特定功能。主要表现在：

- 1) 系统调用：Windows 和 UOS 使用不同的系统调用接口。例如，打开文件、创建进程、网络通信等操作在两个操作系统上使用不同的系统调用函数。
- 2) 文件系统：Windows 和 UOS 使用不同的文件系统，因此在文件路径表示、文件权限和文件操作等方面存在差异。Windows 使用驱动器字母（如 C:\）来表示文件路径，而 UOS 使用正斜杠（/）来表示路径。
- 3) 网络编程：Windows 和 UOS 在网络编程方面也有差异。两个操作系统使用不同的网络协议栈和套接字 API。因此，在进行网络通信时，需要根据操作系统选择适当的 API 和协议。
- 4) GUI 编程：Windows 和 UOS 上的图形用户界面(GUI)编程也有差异。Windows 使用 Windows API 或者一些框架如 MFC、WinForms、WPF 或一些三方定制皮肤框架进行 GUI 开发，而 UOS 上常用的图形界面库是 GTK 或 Qt。
- 5) 多线程和进程管理：Windows 和 UOS 在多线程和进程管理方面也存在差异。比如在 Windows 下，创建进程和线程可以通过调用 Windows 的 A

PI 来完成，或者通过调用 MFC 提供的并发处理类库来实现。在 UOS 下创建进程通常使用 fork 函数，使用 POSIX 的 PTHREAD 线程库来创建线程。

6) Windows 程序图形界面与 UOS 程序图形界面差异大么？

7) 如果应用程序涉及图形界面，Windows 和 UOS 图形模型差异极大，这一点是 UOS 和 Windows 程序开发最大的差别。UOS 使用 XWindow 系统 GUI，而 Windows 使用 GDI。需要根据 UOS 上的图形库和工具进行较大的调整和修改。建议图形界面统一使用 Qt 相关库进行替换。

- UOS 和 Windows 网络编程主要有哪些不同呢？

Windows 和 UOS 使用不同的 Socket API。Windows 使用 Winsock API，而 UOS 使用 BSD (Berkeley Software Distribution) Socket API。这意味着在编写网络应用程序时，需要使用不同的函数和参数：

1) 套接字地址结构：Windows 和 UOS 使用不同的套接字地址结构。Windows 使用 SOCKADDR_IN 结构，而 UOS 使用 sockaddr_in 结构。这些结构在字段名称和顺序上略有不同。

2) 端口号：Windows 和 UOS 在分配端口号时有所不同。Windows 使用动态端口范围（49152-65535），而 UOS 使用静态端口范围（0-1023）和动态端口范围（1024-65535）。

3) 字节顺序：在 Windows 和 UOS 网络编程中，字节顺序（或称为字节序）的差异是一个重要的问题。Windows 使用小端字节序（Little Endian），UOS 使用大端字节序（Big Endian）。在网络编程中，通常使用网络字节

序（Network Byte Order）来进行数据传输。网络字节序是一种统一的字节序，它与大端字节序相同。因此，为了进行字节序的转换，可以使用一些函数或宏来处理，如 `htons`、`htonl`、`ntohs`、`ntohl` 等。这些函数或宏可以将数据从主机字节序转换为网络字节序或从网络字节序转换为主机字节序，以确保数据在网络传输中的正确性和一致性。

4) I/O 模型：Windows 和 Linux 在 I/O 模型上也有不同。Windows 支持同步阻塞、同步非阻塞和异步 I/O 模型，而 Linux 支持阻塞、非阻塞和多路复用（如 `select`、`poll`、`epoll`）等模型。

- 如何实现 UOS、Windows 程序代码的最大化兼容性？

要实现 UOS 和 Windows C++/C 程序代码的最大化兼容性，可以考虑以下几个方面：

- 1) 使用标准的 C++/C 语言特性：遵循 C++/C 的标准规范，避免使用特定于某个平台或编译器的扩展功能。使用标准的语言特性可以增加代码的可移植性。
- 2) 使用跨平台的编译器：选择支持多个平台的编译器，如 GCC、Clang。这些编译器提供了对标准 C++/C 的广泛支持，并且在不同平台上具有一致的行为和特性。
- 3) 使用跨平台的库和框架：选择跨平台的库和框架，如 Boost、Qt、STL（标准模板库）等。这些库和框架提供了一致的 API 和功能，可以在 Windows 和 UOS 上进行编译和运行。

- 4) 处理文件路径和分隔符：在处理文件路径时，使用跨平台的方法来处理路径和分隔符的差异。例如，使用库函数来处理文件路径，而不是手动拼接路径字符串；
- 5) 处理系统调用和 API 的差异：了解 Windows 和 UOS 上的系统调用和 API 的差异，并使用条件编译（如：#ifdef）来区分不同平台的代码。根据平台选择适当的系统调用和 API。
- 6) 进行跨平台测试和验证：在开发过程中，进行跨平台的测试和验证，确保应用程序在不同平台上的编译和运行都能正常工作。

3.2.2. C#开发配置

统信 UOS V20 使用 dotnet 7.0 amd64 版本，为 C#开发者提供了一个完善的开发环境。通过安装 VS Code 及对应 C#插件快速搭建开发调试环境。此外，统信 UOS 还支持使用多种跨平台的图形界面库，创建图形界面应用程序。

3.2.2.1. 环境安装

dotnet 环境安装，执行以下命令：

```
sudo apt-get update # 更新源地址
sudo apt-get install dotnet-sdk-7.0
dotnet --version # 查看版本，验证 .NET 是否成功安装
```

3.2.2.2. 环境配置

IDE 环境 VS Code 软件安装，应用商店搜索 vscode，或执行以下命令：

```
sudo apt-get install com.visualstudio.code
```

新建项目(该项目为 demo 项目，程序只会打印 “Hello World!”)：

```
dotnet new console -n MyConsoleApp
```

构建项目：

```
cd MyConsoleApp  
dotnet build
```

上述命令执行结果如下：

```
适用于 .NET MSBuild 版本 17.7.3+8ec440e68  
正在确定要还原的项目...  
所有项目均是最新的，无法还原。  
MyConsoleApp -> /home/uos/MyConsoleApp/bin/Debug/net7.0/My  
ConsoleApp.dll  
已成功生成。  
0 个警告  
0 个错误  
已用时间 00:00:03.36
```

运行项目：

```
dotnet run
```

3.2.2.3. 开发案例

计算两个整数的和并输出结果。使用 `Console.WriteLine()`和 `Console.WriteLine()`来输出提示和结果。`Console.ReadLine()`用于从用户输入读取字符串，并使用 `Convert.ToInt32()`将其转换为整数类型。然后，我们将两个整数相加得到和，并使用 `Console.WriteLine()`输出结果。

在 `MyConsoleApp` 目录中修改 `Program.cs`,在文件中添加以下内容。

```
using System;  
class Program
```

```
{
static void Main()
{
// 提示用户输入两个整数
Console.WriteLine("请输入两个整数：");
Console.Write("第一个整数：");
int num1 = Convert.ToInt32(Console.ReadLine());
Console.Write("第二个整数：");
int num2 = Convert.ToInt32(Console.ReadLine());
// 计算两个整数的和
int sum = num1 + num2;
// 输出结果
Console.WriteLine("两个整数的和是：" + sum);
}
}
```

通过以下命令编译执行代码：

```
dotnet build
```

通过以下命令运行编译后示例：

```
dotnet run
```

运行结果如下：

```
请输入两个整数：
第一个整数： 1
第二个整数： 2
两个整数的和是： 3
```

请注意，确保你的 C# 程序在 Linux 上没有依赖于 Windows 特定的功能或库。某些 Windows 专用 API 或库在 Linux 上可能无法使用。

3.2.2.4. 常见问题

- 图形界面支持。

C#在 Linux 上可以使用跨平台的图形界面库，如 Gtk#或 Avalonia。这些库可以帮助你创建图形界面应用程序。确保已经安装了所需的库，并按照其文档进行设置和使用。

- 调试工具。

在 Linux 上，可以使用调试器来调试 C#代码。例如，可以使用 VS Code 的 C#扩展来进行调试。前提是确保你已经正确配置了调试器，并按照调试器的文档进行操作。

3.2.3. Java 开发配置

统信 UOS V20 支持 JAVA8 与 11 版本。Java 是一门面向对象的编程语言，不仅吸收了 C++语言的各种优点，还摒弃了 C++里难以理解的多继承、指针等概念，因此 Java 语言具有功能强大和简单易用两个特征。Java 语言作为静态面向对象编程语言的代表，极好地实现了面向对象理论，允许程序员以优雅的思维方式进行复杂的编程。

3.2.3.1. 环境安装

安装默认的 JAVA 运行环境 JRE (Java Runtime Environment)

```
apt-get install default-jre
```

安装默认的 JAVA 开发环境 JDK(Java Development Kit)

```
apt-get install default-jdk
```

3.2.3.2. 环境配置

通过默认的方式安装后即可直接使用无需配置。

当配置完成可以在终端中确认完成并查看当前的版本。

```
java -version
```

选择一个适合你的 Java IDE，在 UOS 中进行 Java 开发。一些常见的 Java IDE 包括 Eclipse、IntelliJ IDEA 和 NetBeans，你可以使用 apt 安装这些 IDE。

3.2.3.3. 开发案例

在当前目录打开终端，创建并打开一个 java 文件（可以通过右键新建文档）。

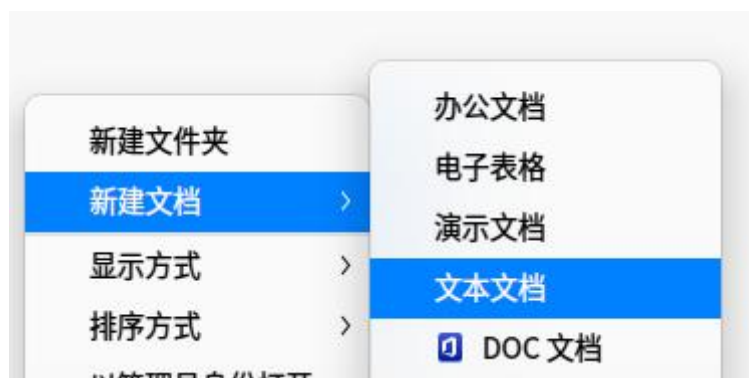


图 1 右键新建文档

修改文件名字为 Hello.java。

通过 VS Code 编辑器写入代码并保存。

```
public class Hello{  
    public static void main(String args[]){  
        System.out.println("Hello World!");  
    }  
}
```

终端中输入指令编译 java 文件。

```
javac Hello.java
```

编译后会生成一个二进制文件 Hello.class，执行它。

```
java Hello
```

3.2.3.4. 常见问题

- 在不同的 Java 版本之间遇到兼容性问题。

统信操作系统源中包含多个版本的 Java，不同的 Java 版本之前可能会遇到兼容性问题，这些问题通常涉及到新版本引入的特性、修改的 API 或对现有行为的修改，因此研发人员需要熟悉目标 Java 版本的特性和变化，根据业务需要选择合适的 Java 版本，确保使用的库和框架与目标 Java 版本兼容。如果统信操作系统源中 Java 版本无法满足业务需求，开发者可以手动切换 Java 版本，切换方法如下：

切换 Java 版本需要修改环境变量，修改环境变量配置文件路径为:/etc/profile，在文件最后添加以下内容。

```
export JAVA_HOME=<jdk 安装目录>
export PATH=.:$JAVA_HOME/bin:$PATH
```

配置文件修改后，需要手动更新配置信息才能生效，命令如下：

```
source /etc/profile
```

3.2.4. Python 开发配置

统信 UOS 支持 Python 2 和 Python 3 系列，Python 开发者能够根据实际需要灵活的选择使用哪个版本。作为一种易学易用的编程语言，Python 在系统管理、数据分析、机器学习、Web 应用等方面使用广泛。除此之外，利用 Python 也能开发图形化界面的应用程序（像 Tkinter 库），并用来创建游戏（像 Py

game 库)。

3.2.4.1. 环境安装

可以通过仓库安装，其中 x.x 表示版本号，如 2.7 或者 3.7：

```
sudo apt install pythonx.x
```

3.2.4.2. 环境配置

- 安装 pip(2.9.7 自带 pip)。

```
sudo apt install python-pip  
sudo apt install python3-pip #Python3 使用该命令
```

- 配置 pip。

在 HOME 目录下创建 .pip 文件夹，进入 .pip 文件夹创建 pip.conf 的配置文件，写入国内源地址。

```
mkdir -p $HOME/.pip  
vim $HOME/.pip/pip.conf
```

内容如下：

```
[global]  
index-url = https://pypi.tuna.tsinghua.edu.cn/simple  
[install]  
trusted-host = https://pypi.tuna.tsinghua.edu.cn
```

- 独立虚拟环境配置。

安装虚拟环境工具：

```
pip install virtualenv
```

创建虚拟环境，参数 venv 创建名字为 venv 的虚拟环境：

```
virtualenv venv
```

如系统中存在多个 Python 版本，创建虚拟环境时，可指定 Python 版本：

```
virtualenv -p /usr/bin/python2.7 venv  
virtualenv -p /usr/bin/python3.7 venv
```

或者：

```
python2 -m venv venv  
python3 -m venv venv
```

3.2.4.3. 开发案例

本案例用于说明在 UOS 上如何使用 Django 开发 web 服务。Django 本身基于 MVC 模型，MVC 模式使后续对程序的修改和扩展简化，并且使程序某一部分的重复利用成为可能。

- 使用 pip3 安装 Django。

```
pip3 install django
```

- 创建项目。

```
django-admin startproject mysite
```

- 创建虚拟环境(不使用系统的 python 虚拟环境，是为了防止多个项目使用同一个包但版本不同而造成影响)

```
python3 -m venv venv
```

- 准备环境，自动生成默认模板代码。

```
pip install daphne Django django-filter \  
django-cors-headers djangorestframework  
django-admin startproject mysite
```

- 启动服务，并访问 8000 端口。

```
cd mysite && python manage.py runserver
```


另外，可以导出当前项目中引用的各个三方包以及版本：

```
pip freeze > requirements.txt (未成名规定取名为 requirements.txt)
```

下次其他地方使用代码，调用下面命令直接使用：

```
pip install -r requirements.txt
```

3.2.4.4. 常见问题

- 临时使用指定仓库安装依赖包。

```
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple/ requests
```

未升级 pip 导致安装依赖包失败

```
pip install --upgrade pip  
pip install --upgrade setuptools
```

- 双击 python 脚本无法运行。

检查脚本首行是否为“#!/usr/bin/env python”，并增加脚本权限。

```
chmod+x some.py
```

- 导入模块失败。

```
pip install yourmod #安装缺少的模块
```

如果找不到对应模块，或者模块安装失败，确认模块是否是 Windows 平台独有的库，如 win32 的 ole/com 提供的包。

- 引用路径失败。

检测引用路径是否为/，而非 windows 的\。

- 编码问题。

在 Windows 上，用 UltraEdit 或 EditPlus 等工具先将脚本编码转换，再放

到 Linux 中执行。转换方式如下（UltraEdit）：File > Conversions > DOS >

UNIX 即可。如果文本中存在中文编码导致解码问题，则在文件第一行加入“#coding=utf-8”。

3.2.5. Perl 开发配置

统信 UOS 对 Perl 编程语言提供了默认支持。Perl 以胶水语言著称，尤其擅长正则表达式和文本处理。Perl 早年常用来开发 CGI 脚本，如：cPanel、Bugzilla 等，当下在系统维护、文本处理、网络编程等方面仍发挥着重要作用。

3.2.5.1. 环境安装

UOS 已经预装 Perl，可以通过在终端输入 `perl -v` 来检查 Perl 安装的具体版本。

如果需要对 Perl 进行升级，可以使用 apt 包管理器：

```
sudo apt update
sudo apt install perl
```

3.2.5.2. 环境配置

在安装 Perl 后，可能需要对 Perl 的环境进行一些配置。

- 环境变量配置。

在 Linux 上，Perl 的可执行文件通常位于 `/usr/bin/perl`。确保 `/usr/bin` 在系统的 PATH 环境变量中，这样就可以直接在终端中运行 Perl 脚本。

- 模块管理器。

Perl 提供了 CPAN（Comprehensive Perl Archive Network）作为

官方的模块管理器。使用 CPAN 可以安装、更新和卸载 Perl 模块。在首次使用 CPAN 时，需要进行一些初始化配置，比如选择镜像站点等。

- 编辑器集成。

如果使用文本编辑器来编写 Perl 脚本，需要配置编辑器以支持 Perl 语法高亮、代码补全等功能。一些流行的编辑器如 Vim、Emacs、Sublime Text、Visual Studio Code 等都有 Perl 插件或扩展可以使用。

- 调试工具。

要调试 Perl 脚本，可以考虑安装调试工具，比如 Perl 自带的 `perldebug`、`Devel::ebug` 模块等。

3.2.5.3. 开发案例

在配置好 Perl 环境后，可以开始编写 Perl 脚本来解决各种问题。以下是两个简单的开发案例：

- 文件处理。

一个常见的 Perl 开发案例是文件处理。编写 Perl 脚本能够读取、写入、解析和操作各种类型的文件。比如，可以编写一个 Perl 脚本来批量处理文本文件，提取特定格式的数据，然后进行分析或者转换成其他格式。

```
#!/usr/bin/perl
# 读取文件示例
open(my $fh, '<', 'input.txt') or die "无法打开文件：$!";
while (my $line = <$fh>) {
    chomp $line;
    # 对每一行进行处理
```

```
# ...  
}  
close($fh);
```

- Web 开发。

Perl 也可以用于 Web 开发，可以使用 Perl 来编写 CGI 脚本或者使用现代的 Web 框架来构建 Web 应用程序。比如，使用 CGI 模块能够处理 Web 表单提交，或者使用 Mojolicious、Dancer 等框架来构建 Web 应用程序。

```
#!/usr/bin/perl  
# 简单的 CGI 脚本示例  
use CGI qw(:standard);  
print header, start_html("Hello, World!"), h1("Hello, World!"), end_html;
```

这些只是 Perl 开发中的两个简单案例，Perl 还可以用于系统管理、网络编程、文本处理、数据库访问等各种领域。一旦熟悉了 Perl 的语法和特性，可以根据需求进行更复杂的开发。

3.2.5.4. 常见问题

在 Perl 开发过程中，可能会遇到一些常见的问题，以下是相应的处理方案：

- 依赖管理问题。

在开发过程中，可能会用到许多 Perl 模块，管理这些模块的依赖关系会变得复杂。

处理方案：使用 CPAN 或者其他工具来管理 Perl 模块的依赖关系。使用 cpanm 或者 Carton 等工具来管理项目的依赖关系，并且可以将依赖关

系列表保存在 cpanfile 中。

- 性能问题。

Perl 脚本的性能可能会成为一个问题，特别是在处理大量数据或者需要快速响应的场景下。

处理方案：使用工具来进行性能分析，比如 Devel::NYTProf 模块可以找出 Perl 脚本中的性能瓶颈。另外，也可以考虑使用更高效的算法和数据结构来优化性能。

- 安全性问题。

编写不安全的 Perl 代码可能会导致安全漏洞，比如代码注入、跨站脚本攻击等。

处理方案：遵循 Perl 的最佳实践，比如使用 `use strict;`和 `use warnings;`来帮助捕获潜在的问题。另外，也可以使用安全的模块来处理用户输入，比如 CGI::Escape 来转义用户输入。

- 跨平台兼容性问题。

Perl 脚本需在不同操作系统上运行时，可能会遇到跨平台兼容性问题。

处理方案：编写具有良好可移植性的 Perl 代码，避免使用特定于操作系统的功能或路径。另外，可以使用 File::Spec 等模块来处理文件路径问题，以确保在不同操作系统上都能正常工作。

- 调试问题。

当 Perl 脚本出现问题时，可能需要进行调试来找出问题所在。

处理方案：使用 Perl 的调试工具，比如 perldebug、Devel::ebug 等来进行调试。另外，也可以使用 Data::Dumper 或者 Carp 模块来输出调试信息。

3.2.6. Shell 开发配置

统信 UOS 的默认 Shell 是 Bash。Bash 既能用于交互的执行系统命令，也可读取并执行 Shell 脚本。利用 Shell 脚本，开发者可以将开发过程中需要经常处理的工作流程自动化，从而大幅提高开发效率。

3.2.6.1. 环境安装

Shell 作为一种命令行解释器，用于解释用户输入的命令并执行相应的操作，它通常是操作系统的一部分，不需要额外安装。

3.2.6.2. 环境配置

- Bash 环境配置。

用户级别的 Bash 配置文件通常是 ~/.bashrc，用于配置用户的 Bash 环境变量、别名、函数等。系统级别的 Bash 配置文件通常是 /etc/bashrc，用于配置系统范围的 Bash 环境变量和函数。

- Bash 行为配置。

用户级别的 Bash 行为配置文件通常是 ~/.bash_profile，用于配置用户登录时的 Bash 行为，比如设置 PATH 变量、执行指定的命令等。另一个用户级别的 Bash 行为配置文件是 ~/.bash_login，在某些系统中用于配置用户登录时的 Bas

h 行为。如果 ~/.bash_profile 和 ~/.bash_login 都不存在，Bash 会尝试读取 ~/.profile 作为登录时的配置文件。

- 其他配置文件。

在系统中，还可能存在其他 Bash 配置文件，比如 /etc/profile 用于系统范围的 Bash 环境配置，以及 /etc/profile.d/ 目录下的脚本文件用于配置全局的环境变量。在这些配置文件中，可以设置环境变量、别名、函数、命令提示符等，以及执行一些初始化操作。这些配置文件的具体作用和加载顺序可能会因操作系统和 Bash 版本的不同而有所不同。

3.2.6.3. 开发案例

- 批量处理文件。

这个简单的 Bash 脚本用于批量处理指定目录中的文件，将文件名中的空格替换为下划线。

```
#!/bin/bash
# 指定要处理的目录
directory="/path/to/your/directory"
# 进入指定目录
cd "$directory" || { echo "无法进入目录 $directory"; exit 1; }
# 处理文件名中的空格
for file in *; do
    if [ -f "$file" ]; then
        newname=$(echo "$file" | tr ' ' '_')
        mv "$file" "$newname"
        echo "已将文件 $file 重命名为 $newname"
    fi
done
```

```
echo "批量处理完成"
```

在这个脚本中，首先指定要处理的目录，然后使用 `cd` 命令进入该目录。接着使用一个 `for` 循环遍历目录中的每个文件，对于每个文件，使用 `tr` 命令将文件名中的空格替换为下划线，并使用 `mv` 命令将文件重命名。最后，显示处理完成的消息。

- 自动备份文件。

这个 Bash 自动备份脚本，用于定期备份指定目录中的文件到另一个目录。

```
#!/bin/bash

# 源目录和目标目录
source_dir="/path/to/source/directory"
target_dir="/path/to/backup/directory"

# 创建备份目录（如果不存在）
mkdir -p "$target_dir" || { echo "无法创建备份目录"; exit 1; }

# 备份文件
timestamp=$(date +%Y%m%d_%H%M%S)
backup_file="backup_${timestamp}.tar.gz"
tar -czf "$target_dir/$backup_file" -C "$source_dir" .

# 检查备份是否成功
if [ $? -eq 0 ]; then
    echo "备份成功: $backup_file"
else
    echo "备份失败"
fi
```

在这个脚本中，首先指定了源目录和目标目录，然后使用 `mkdir -p` 命令创建备份目录（如果它不存在）。接着，用 `tar` 命令将源目录中的文件打包成一个

压缩文件，并将其保存到目标目录中。最后，检查备份操作是否成功，并显示相应的消息。

3.2.6.4. 常见问题

- 文件处理。

在处理文件时，应该考虑文件权限、文件路径中可能存在的空格或特殊字符等情况。可以使用 `-f`、`-d` 等参数来判断文件或目录是否存在，以避免对不存在的文件进行操作。

- 错误处理。

可以使用 `trap` 命令捕获脚本中的错误信号，同时可以使用 `echo` 或 `logger` 命令记录错误信息到日志文件中。

- 跨平台兼容性。

在编写 Bash 脚本时，应该尽量避免使用特定于某个操作系统的命令或语法，或者根据不同的操作系统进行条件判断，以确保脚本在不同平台上的兼容性。

3.2.7. 其他编程语言开发配置

本章节涵盖了多种编程语言在统信操作系统中的开发配置，旨在为读者提供在 JavaScript、Node.js、PHP 和 GoLang 等语言中进行开发的指导。

3.2.7.1. Javascript

- 开发配置。

Javascript 开发不需做额外进行环境配置，推荐使用 VSCode 编辑器开发。

- 开发案例。

- 1) 本案例用于指导开发初学者在统信操作系统中使用 Javascript 开发。
- 2) 创建 index.html 文件。
- 3) 使用 VSCode 编辑器打开 index.html 文件，在 index.html 文件中输入

以下代码：

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hello World Example</title>
</head>
<body>
  <h1>Hello World!</h1>
  <script>
    // 在这里编写你的 JavaScript 代码
    alert('Hello World from JavaScript!');
  </script>
</body>
</html>
```

- 4) 使用浏览器打开 index.html 文件，显示效果如下：

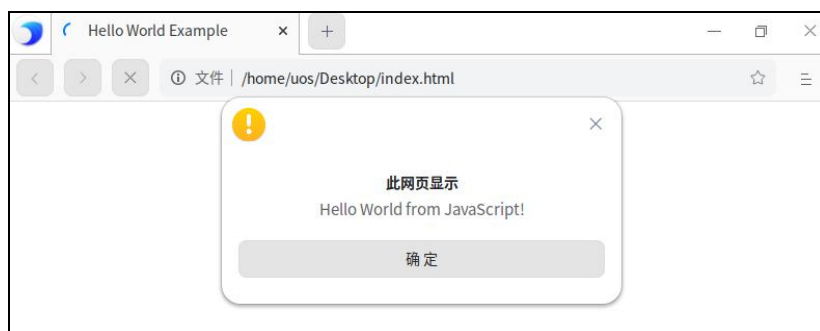


图 2 浏览器打开 index.html 文件显示效果

3.2.7.2. nodejs

- 开发配置。

- 1) 安装 nodejs。

```
sudo apt install nodejs
```

- 2) 验证 nodejs 是否安装成功。

```
nodejs -v
```

- 3) 显示内容如下：

```
v10.24.0
```

- 开发案例。

- 1) 创建目录 HellWorkNode，并在目录中创建文件 app.js 文件。

```
mkdir HellWorkNode  
cd HellWorkNode  
touch app.js
```

- 2) 使用 VSCode 编辑器打开 app.js 文件并输入以下内容。

```
console.log('Hello World from Node.js!');
```

- 3) 运行。

```
node app.js
```

- 4) 运行结果如下：

```
Hello World from Node.js!
```

3.2.7.3. php

- 开发配置。

php 为服务器开发语言，因此除了需要安装 php 本身以外还需要服务器，p

hp 开发环境安装步骤如下：

1) 安装 php。

```
sudo apt install php php-fpm
```

2) 安装 nginx。

```
sudo apt install nginx
```

3) 修改 nginx 配置文件/etc/nginx/sites-enabled/default, 找到文件中以下内容并修改（注意使用 root 权限修改文件）。

```
index index.html index.htm index.nginx-debian.html;
.....
#location ~ \.php$ {
#    include snippets/fastcgi-php.conf;
#    # With php-fpm (or other unix sockets):
#    fastcgi_pass unix:/run/php/php7.3-fpm.sock;
#    # With php-cgi (or other tcp sockets):
#    fastcgi_pass 127.0.0.1:9000;
#}
```

4) 修改为：

```
index index.php
.....
location ~ \.php$ {
    include snippets/fastcgi-php.conf;
#    # With php-fpm (or other unix sockets):
    fastcgi_pass unix:/run/php/php7.3-fpm.sock;
#    # With php-cgi (or other tcp sockets):
    #    fastcgi_pass 127.0.0.1:9000;
}
```

5) 启动 nginx、php-fpm。

```
sudo service php7.3-fpm start
sudo service nginx start
```

6) 设置 nginx 开机自启动。

```
sudo systemctl enable nginx
```

7) 查看 nginx 启动状态。

```
sudo service nginx status
```

8) 显示内容如下：

```
nginx.service - A high performance web server and a reverse pr
oxy server
Loaded: loaded (/lib/systemd/system/nginx.service; enabled; vendo
r preset: enabled)
Active: active (running) since Wed 2023-11-29 09:59:30 CST; 15mi
n ago
Docs: man:nginx(8)
Main PID: 1450 (nginx)
Tasks: 5 (limit: 4619)
Memory: 15.2M
CGroup: /system.slice/nginx.service
├─1450 nginx: master process /usr/sbin/nginx -g dae
mon on; master_process on;
├─1452 nginx: worker process
├─1453 nginx: worker process
├─1454 nginx: worker process
└─1455 nginx: worker process

11 月 29 09:59:23 uos-PC systemd[1]: Starting A high performanc
e web server and a reverse proxy server...
11 月 29 09:59:30 uos-PC systemd[1]: Started A high performance
web server and a reverse proxy server.
```

● 开发案例。

1) 删除默认文件。

```
sudo rm -rf var/www/html/*
```

2) 编写文件。

在/var/www/html/目录中创建 index.php，文件内容如下，注意需要使用 root 权限编辑和创建 index.php 文件。

```
<?php
phpinfo();
?>
```

● 案例演示。

打开浏览器，在浏览器地址栏中输入 <http://localhost>，显示内容如下图：

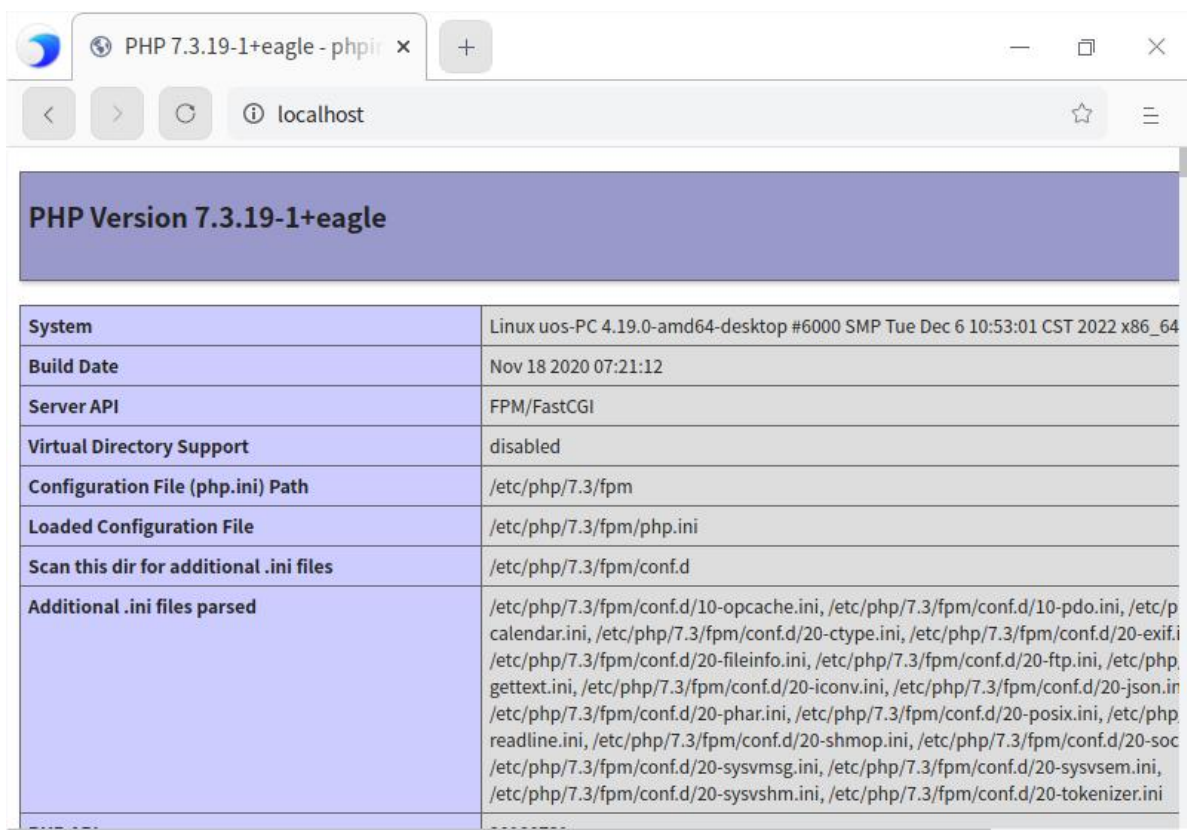


图 3 案例演示

3.2.7.4. golang

- 开发配置。

golang 开发环境安装步骤如下：

1) 安装 golang 环境。

```
sudo apt install golang
```

2) 查看 go 版本号。

```
go version
```

3) 显示内容如下：

```
go version go1.15.9.3 linux/amd64
```

- 开发案例。

1) 编写 main.go 文件，文件内容如下：

```
package main
import "fmt"
func main() {
    fmt.Println("Hello, World!")
}
```

2) 运行 go 文件。

```
go run main.go
```

3) 显示输出如下：

```
Hello, World!
```

3.3. 开发套件

在 UnionTech OS (UOS) 操作系统上，提升开发效率的关键在于充分利用其强大的开发套件。QT 作为跨平台的 GUI 开发框架，为开发者提供了丰富的工

具和组件，助力构建出色的用户界面。DTK（Desktop Tool Kit）则为桌面应用提供了全面的支持，简化了应用程序的开发流程。同时，UOS 系统积极整合了图形渲染领域的两大利器，OpenGL 和 Vulkan，为图形开发提供了高性能和跨平台的解决方案。本章节将深入介绍这些开发套件，解析它们如何共同作用，为 UOS 系统开发者提供了高效、灵活的开发环境，从而推动应用程序的创新与发展。

3.3.1. Qt 应用开发

Qt 是一个跨平台的应用程序开发框架，具有丰富的核心功能，包括 GUI 开发、多媒体支持、网络编程、数据库操作、多线程编程、XML 和 JSON 解析等。UOS 已经完全适配了 Qt，Qt 应用可以在 UOS 直接编译使用。

Qt 官方网站：<https://www.qt.io/zh-cn/>

3.3.1.1. 环境安装

在 UOS 系统上,我们可以通过从 UOS 系统的软件仓库中安装 Qt 开发环境。

Qt 的集成开发工具包、开发包安装。

```
sudo apt install qtcreator
sudo apt install qt5-default qtbase5-dev qt5-qmake qttools5-dev-tools
sudo apt install cmake g++ gcc
```

如果要开发 Qt Quick 应用需要安装以下包。

```
sudo apt install qtdeclarative5-dev qml-module-qtquick2 qml-module-qtquick-controls2 qml-module-qt-labs-settings
```


3.3.1.2. 环境配置

Qt Creator 配置参考 2.2.1 章节，其它不需要进行特别的环境配置。



图 4 Qt 应用开发环境配置

3.3.1.3. 开发案例

本节中，用基于 Qt4 的计算器应用程序作为案例工程，介绍在 UOS 下如何使用。下列是需要开发的项目工程目录结构。

```
-calculator
|-include
|-calculationunit.h
|-libs
|-libcalculationunit.dll
|-calculator.pro
|-calculatorcontrol.h
|-calculatorcontrol.cpp
|-main.cpp
|-mainwindow.cpp
|-mainwindow.h
-libcalculationunit
|-calculationunit.c
```

```
| -calculationunit.h  
| -CMakeLists.txt
```

- 源码文件编码格式转换。

从 Windows 获取过来的项目源代码文件编码格式多样，为彻底解决获取后的文件编码问题，我们可按照 Qt5 所推荐的方式，使用 UTF-8 格式的源代码文件，对于 Qt4 应用项目，通过第三方工具将源代码文件转换为 UTF-8 格式，或使用 UOS 系统的文本编辑器将其以覆盖方式另存为 UTF-8 格式。

- 依赖库获取。

在 UOS 系统开发时，如果调用了第三方库，我们需要获取该第三方库的 Linux 版本（动态库后缀为.so，静态库后缀为.a）；如果库属于自行开发，则需在 UOS 平台上重新编译。

在本案例中，需要重新编译 libcalculationunit 库。

该库的源代码及 CMake 项目文件：calculationunit.h

```
#ifndef CALCATIONUNIT_H  
#define CALCATIONUNIT_H  
double sum(double a, double b);  
double sub(double a, double b);  
double mul(double a, double b);  
double divi(double a, double b);  
#endif  
calculationunit.c  
#include "calculationunit.h"  
double sum(double a, double b)  
{
```

```
return a+b;
}
double sub(double a, double b)
{
return a-b;
}
double mul(double a, double b)
{
return a*b;
}
double divi(double a, double b)
{
return a/b;
}
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.12)
project(calculationunit VERSION 1.0.0)
SET(CMAKE_INCLUDE_CURRENT_DIR ON)
set(CMAKE_CXX_STANDARD 11)
set(CMAKE_CURRENT_SOURCE_DIR .)
set(BUILD_DIR ${CMAKE_CURRENT_SOURCE_DIR}/../build_calculation
unit)
file(GLOB_RECURSE SOURCES
"${CMAKE_CURRENT_SOURCE_DIR}/*.h"
"${CMAKE_CURRENT_SOURCE_DIR}/*.c"
)
add_library(calculationunit SHARED ${SOURCES})
```

Qt Creator 打开 CMakeLists.txt 文件，在 UOS 上重新编译即可生成 libcalculationunit.so 动态库文件。

- 语言国际化。

在应用程序源代码中使用英文作为源语言，通过 Qt 语言国际化机制将

应用程序界面显示为中文是比较推荐的做法,也避免了在源码中使用中文所带来的编码方式兼容问题。我们先将源代码中的中文字符改为英文,并用 `tr()` 包裹,为语言国际化做准备。再按照下列步骤生成对应的 .ts 和 .qm 文件来实现语言国际化。

首先要生成 .ts 文件。对于 CMake 项目,则需要在 CMakeLists.txt 文件中添加下面命令,然后执行 `cmake`,再构建即可生成 `calculator_zh_CN.ts` 文件:

```
find_package (Qt5LinguistTools)
qt5_create_translation(QM_FILES ${CMAKE_CURRENT_SOURCE_DIR}
calculator_zh_CN.ts)
add_executable(calculator ${SOURCES} ${QM_FILES})
```

上述生成 .ts 文件的方式也可以在终端中进行,进入源代码目录执行下列命令:

```
lupdate -no-obsolete . -ts calculator_zh_CN.ts
```

其中 `-no-obsolete` 参数表示删除所有无用的字符串,比如:已经废弃的翻译内容。利用 Linguist 工具完成语言国际化翻译,从 UOS 系统开始菜单打开 Qt5 Linguist 工具并从中打开生成的 `calculator_zh_CN.ts` 文件。第一次操作时,会弹出如下对话框,上面的源语言部分可以保持不变,下面的选项中根据实际情况选择,这里我们选择“Chinese(简体中文)”和“China(中文)”,这也和我们的 `calculator_zh_CN.ts` 文件名保持一致,“zh”对应中国,“CN”对应简体中文。



图 5 Qt Linguist 工具设置语言类型

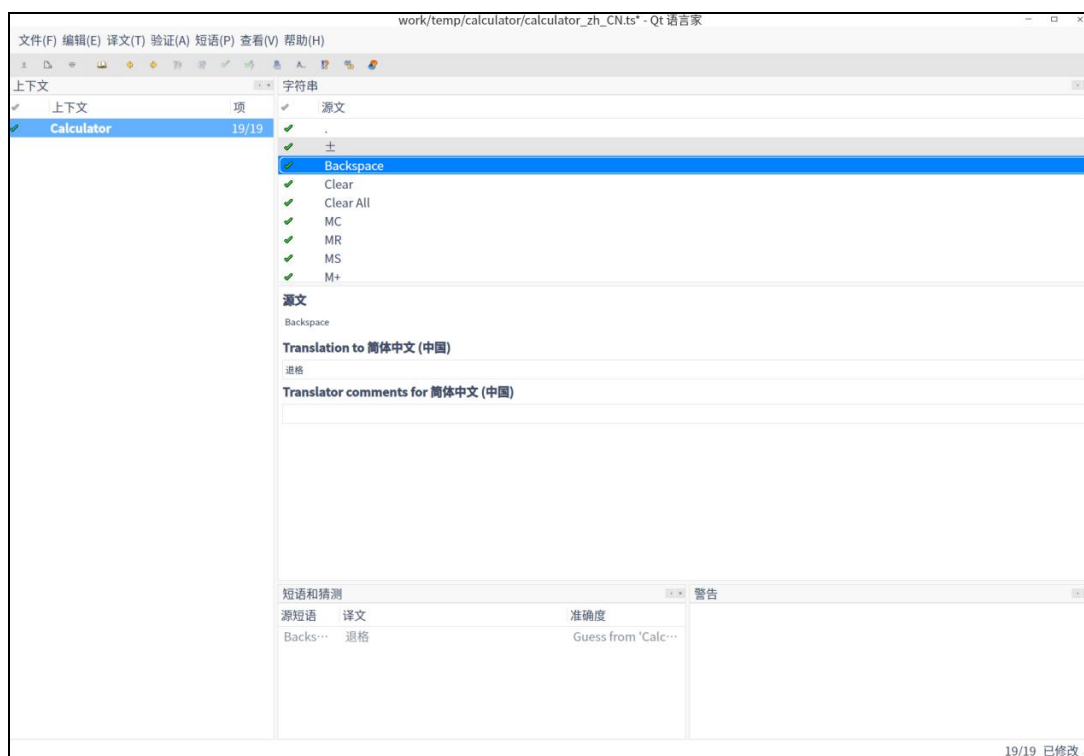


图 6 Qt Linguist 工具语言国际化翻译过程

对于翻译完成的源语言，用绿色小勾标识。对于无需翻译的源语言，应保持问号不变，这样当应用程序加载 qm 文件之后，还是会使用未翻译的字符；或者让翻译前后内容保持一致，也标上绿色小勾。

翻译完成后，从 Linguist 工具菜单栏中选择文件 > 保存，再选择文件 >

发布,之后就会在 calculator_zh_CN.ts 文件同级目录下生成 calculator_zh_CN.qm 文件了。在 UOS 系统中,应用程序的翻译文件放置在路径/usr/share/<应用名称>/translations/下面,这里我们也需要把生成的 calculator_zh_CN.qm 拷贝到路径/usr/share/calculator/translations 下面。

上述生成.qm 文件的方式也可以在终端中进行,执行下面的 lrelease 命令也可以生成 calculator_zh_CN.qm 文件。

```
lrelease calculator_zh_CN.ts
```

qm 文件生成好后需要在源码中加载翻译文件,在 main.cpp 中添加 QTranslator 类信息,调用 load 函数加载翻译文件 calculator_zh_CN.qm 即可实现文字翻译。

完成上述步骤之后,使用 Qt Creator 编译、运行计算器案例程序,就会看到中文界面了。

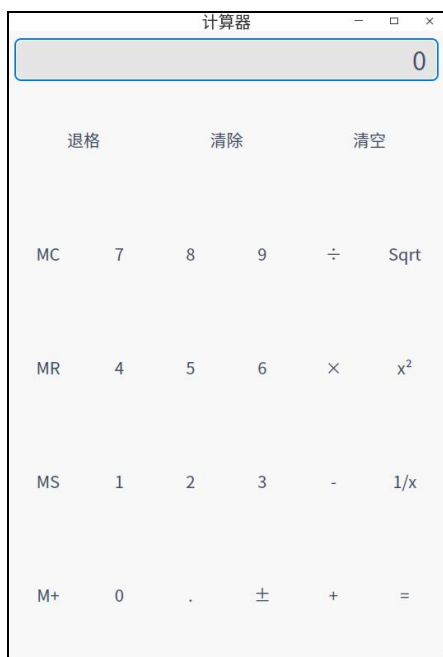


图 7 完成开发及实现语言国际化后程序界面

3.3.1.4. 常见问题

- 目录路径分隔符问题。

可以使用静态函数 `QString QDir::toNativeSeparators(const QString & pathName)` 把目录路径字符串中的斜杠分隔符转换成本地操作系统风格，在 UOS 系统中使用正斜杠 “/”，尤其是在界面显示时，斜杠的方向应与当前操作系统的风格匹配。

- 系统标准路径问题。

Qt 中的 `QStandardPaths` 类中定义了系统中的标准路径，比如：桌面路径、用户目录路径、音乐目录路径、图片目录路径等，当 Qt 应用在 UOS 平台开发时，应检查源代码中是否把 Windows 平台相关的路径用在了 UOS 平台上进行访问的情况。

- 使用了 Windows API。

某些 Windows 应用程序中由于用到了 Windows 平台相关的 API，因而包含了相应的 Windows 平台开发库的头文件，例如：`windows.h`，同时也使用了 MSVC 编译器。对于这些应用的迁移，应该首先设法去掉所涉及的 Windows 平台相关 API 及相关头文件、动态库的调用，同时在 UOS 平台上使用对应的实现方式来进行替代。

- 使用了平台特有控件。

如果被迁移的应用项目使用了 Windows 平台下的特有控件，比如：`QWinTaskbarButton` 等，这类代码在 UOS 环境下是编译不通过的，需要考虑移除。

3.3.2. DTK 应用开发

DTK(Deepin ToolKit)是基于 Qt5 开发的通用开发框架,可方便开发者统一地开发统信 UOS 桌面和统信 UOS 系列应用,主要的功能有:

- 提供单实例的接口,方便开发者直接使用,不用“造轮子”。
- 提供 XCB 窗口移动、缩放等一系列函数。
- 提供一系列美观的自绘控件,用户不用自己写 Qt 控件。

DTK 作为基础的开发包,可为整个操作系统提供整体的视觉效果。DTK 中包含 3 个核心模块、180 多个类、2000 多个函数接口,同时可无缝融合 Qt 的 14 个辅助功能模块。并且它提供了 10 多个应用和桌面扩展接口模块,可满足开发者日常图形应用、业务应用、系统定制应用的开发需求,下表中给出了 DTK 中常用的项目(或库)及其功能描述。

表 1 DTK 中常用的项目(或库)及其功能描述

项目(或库)	功能描述
dtkcore	提供应用开发中的工具类,如程序日志、文件系统监控、格式转换等工具类。
dtkgui	包含开发 GUI 应用所需的功能。主要是控制窗口主题的外观、调色板等参数。
dtkwidget	提供各种统信 UOS 风格的 DTK 基础控件。
qt5integration	提供各种 Qt 插件,方便统一 Qt 应用的风格,包括图标引擎、主题样式插件等。
qt5platform-plugins	主要是平台插件,如 XCB 插件,方便修改窗口属性。

项目的主要模块关系可参考图 DTK 主要模块关系。

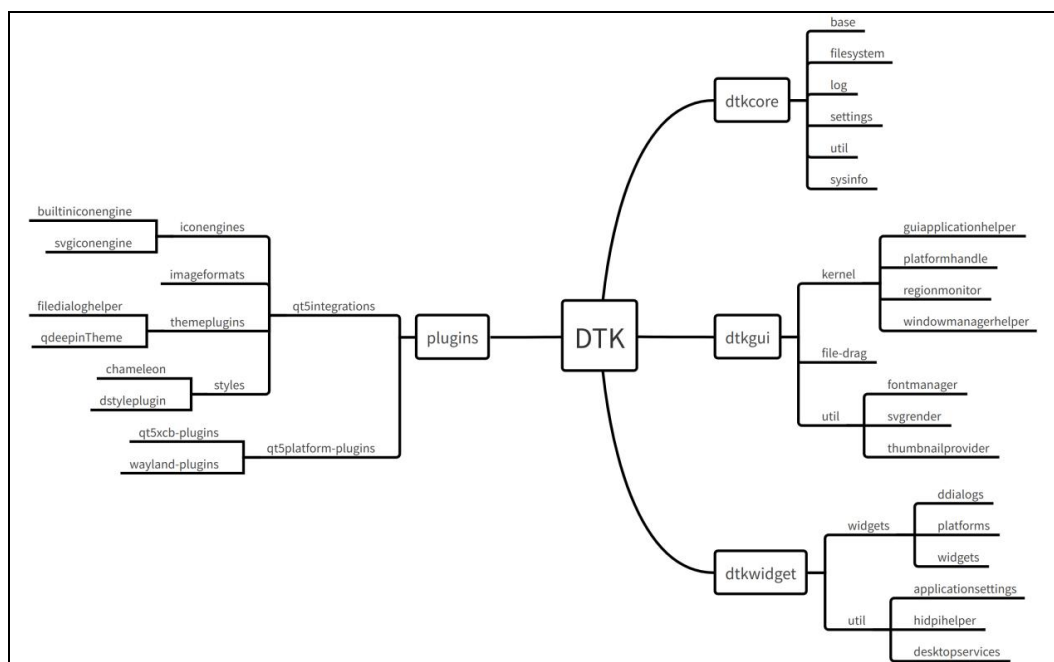


图 8 DTK 主要模块关系

DTK 对 Qt 的许多控件进行了重写，还有部分是单独的实现，这些类大都以 D 开头，源码在 dtkwidget 模块，使用这些控件，可以使程序保持一致的风格，方便通过 DTK 统一调控所有使用该控件的应用，适合进行系统主题的切换，如下图所示展示了 DTK 重绘的不同类型的按钮。

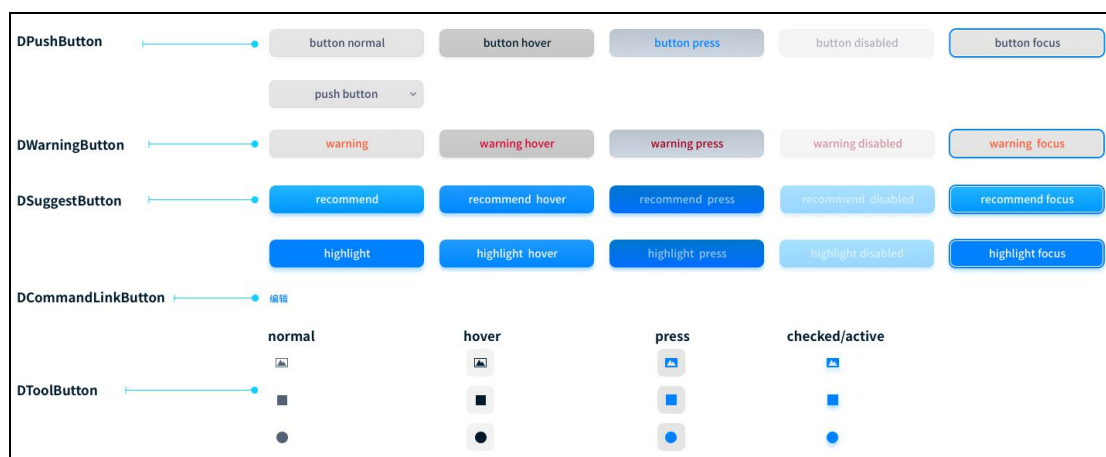


图 9 DTK 自绘按钮

在 dtkwidget 中还自带一个整合了大部分控件的示例程序，使用 Qtcreator 打开 dtkwidget 源码可直接运行，便于 DTK 开发者查看源码，并且该示例是

可交互的，可查看效果，如图 dtkwidget 示例。

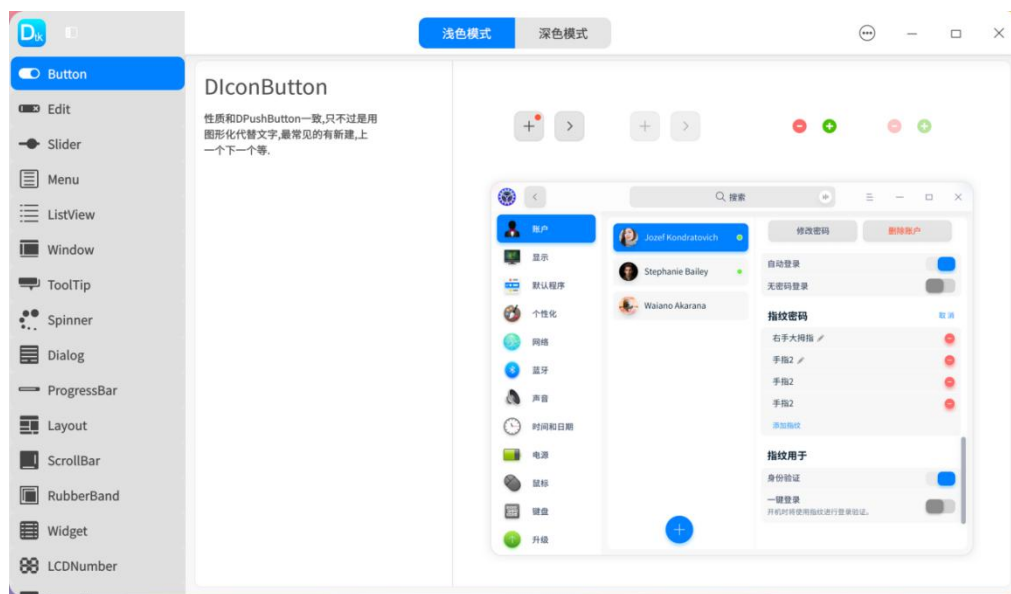


图 10 DTK 示例按钮图片



图 11 dtkwidget 示例

3.3.2.1. 环境安装

安装 DTK 开发包可以手动通过更新源地址的方式。需要安装的 DTK 开发包如下：

- libdtkwidget5-dev

- Libdtkcore5-dev
- Libdtkgui5-dev

执行如下命令一键安装：

```
sudo apt install qtbase5-dev qt5-default libdtkwidget5-dev libdtkcore5-dev libdtkgui5-dev
```

也可通过拉取远程仓库项目代码进行打包安装,DTK 源码已在 github 开源,地址为：

- dtkwidget(<https://github.com/linuxdeepin/dtkwidget>)
- dtkcore(<https://github.com/linuxdeepin/dtkcore>)
- dtkgui(<https://github.com/linuxdeepin/dtkgui>)

拉取代码之后,进入项目根目录,执行 `sudo dpkg-buildpackage -us -uc -b -j8` 进行打包,打包完会在上一级目录生成 deb 安装包,点击或使用命令对其进行安装。

安装开发环境,一般使用 Qtcreator 进行开发,gcc 或 clang 编译,qmake 或 cmake 进行构建,安装 Qtcreator 会自动安装 qmake,执行 `sudo apt install qtcreator cmake gcc`,安装 Qtcreator,gcc 和 cmake。还可安装 DTK 开发框架插件来更方便地使用 Qtcreator,安装命令如下：

```
sudo apt install qtcreator-template-dtk
```

安装完成后该插件可直接嵌入 Qtcreator,在创建新项目的页面提供 DTK 应用的选项,我们可以像创建一个 Qt 应用一样选择创建一个 DTK 应用,省去了手动配置 qmake 或者 cmake 文件的繁杂。

3.3.2.2. 环境配置

- 环境依赖。

如果需要对现有项目源码进行调试，需要安装对应项目的依赖，以 deepin 控制中心源码为例(<https://github.com/linuxdeepin/dde-control-center>), 安装依赖最简单的方式是进入该项目的根目录，执行以下命令来安装依赖。

```
sudo apt build-dep.
```

安装该项目依赖的所有的包，该命令的本质是读取 debian 目录下的 control 文件，该文件记录了该项目所依赖的所有包及其版本。如果是自己创建的项目，也可在 debian 目录下记录本项目的依赖，方便编译。

安装完依赖后可使用 Qtcreator 打开项目，选择构建套件后点击左下角的构建，构建完成后选择可执行文件(dde-control-center)，有些项目的可执行文件需要添加启动参数才能显示，如 dde-control-center 要在项目的运行设置中添加-s 参数。

开发工具 Qtcreator 配置：可在“工具-Kits-构建套件”中配置构建参数，其中 Environment 字段可添加构建参数:MAKEFLAGS=-j8 启用多核编译，提高编译速度，实际开发中比较常用。

可在“工具-构建和运行”中配置构建和运行的一些行为，如构建时文件保存，运行前部署，构建目录等等。

可在“工具-文本编辑器”中选择喜欢的字体、字号以及高亮颜色等；在“工具-环境”中选择 Qtcreator 用户界面的主题。

3.3.2.3. 开发案例

写一个 DTK 应用“control-center”，使用 DTK 风格的窗口，写一个简易的控制中心展示菜单，使用 DTK 中的 DListView 控件，代码如下：

main.cpp 文件内容如下：

```
#include <DApplication>
#include <DListView>
#include <DMainWindow>
#include <DWidget>
#include <DStyle>
#include <DHeaderView>
#include <QStyle>
#include <QLabel>
DWIDGET_USE_NAMESPACE
int main(int argc, char *argv[]){
    DApplication app(argc, argv);
    DMainWindow win;
    //新建一个标准模型
    QStandardItemModel *model = new QStandardItemModel();
    //写一个函数，函数的功能是新建显示在窗口中的条目，条目内容，设置不可编辑以及将条目添加到 model 中
    auto AddItems = [&win, &model](QIcon icon, QString name){
        DStandardItem *item = new DStandardItem(icon, name); //条目包含图标和名称
        item->setEditable(false); //设置条目不可被编辑
        model->appendRow(item); //将条目添加到 model 中
        return item;
    };
    //调用函数新建四个条目
    auto item0 = AddItems(win.style()->standardIcon(DStyle::SP_ComputerIcon), "电脑");
```

```
    auto item3 = AddItems(win.style()->standardIcon(DStyle::SP_TrashIcon), "回收站");
    auto item2 = AddItems(win.style()->standardIcon(DStyle::SP_DriveCDIcon), "媒体");
    auto item3 = AddItems(win.style()->standardIcon(DStyle::SP_TrashIcon), "回收站");
    //增加一个条目中的 action， 可用作标识、与其他条目区别开，设置位置、大小、是否可点击
    DViewItemAction *action = new DViewItemAction(Qt::AlignCenter, QSize(), QSize(), true);
    //将 QStyle 转为 DStyle
    DStyle *style = qobject_cast<DStyle *>(win.style());
    //给 action 设置图标
    action->setIcon(style->standardIcon(DStyle::SP_IndicatorChecked));
    //设置显示到哪个条目上，这里设置到 item1
    item1->setActionList(Qt::Edge::RightEdge, {action});
    //新建一个 dlistview
    DListView *listview = new DListView();
    //将上面设置好的模型加载到 listview 中
    listview->setModel(model);
    //设置 listview 背景颜色的样式
    listview->setBackgroundType(DStyledItemDelegate::BackgroundType::RoundedBackground);
    //新建一个 header 模型
    QStandardItemModel *hmodel = new QStandardItemModel();
    //设置 header 内容
    hmodel->setHorizontalHeaderLabels({"功能", "最后打开时间"});
    //新建一个 headerview
    DHeaderView *headerview = new DHeaderView(Qt::Horizontal);
    //将模型加载到 headerview
    headerview->setModel(hmodel);
    //设置 headerview 高度最大值
    headerview->setMaximumHeight(36);
    //设置 headerview 调整尺寸方式，此处设为随窗口尺寸变化
```

```
headerview->setSectionResizeMode(DHeaderView::Stretch);
//将 headerview 添加到 listview
listview->addHeaderWidget(headerview);
//为 footer 设置一个标签
QLabel *label = new QLabel("共四项内容");
//设置标签居中显示
label->setAlignment(Qt::AlignCenter);
//将标签添加到 footer
listview->addFooterWidget(label);
//将 listview 放入主窗口
win.setCentralWidget(listview);
win.resize(500, 350);
win.show();
return app.exec();
}
```

如使用 qmake 进行构建,使用如下 control-center.pro 文件,将源代码文件与如下文件置于同一目录下,使用 qtcreator 打开,可直接运行;也可将两个文件置于同目录下后在该目录打开终端,分别执行以下命令编译和打开程序。

```
qmake  #使用 qmake 生成 Makefile 文件
make   #通过 Makefile 编译生成可执行文件
./control-center  #运行可执行文件
```

control-center.pro 文件内容如下:

```
QT      += core gui
greaterThan(QT_MAJOR_VERSION, 4): QT += widgets dtkwidget #
添加 dtkwidget 模块
TARGET = control-center
TEMPLATE = app
DEFINES += QT_DEPRECATED_WARNINGS
CONFIG += c++11
SOURCES += \
```

```
main.cpp \  
HEADERS += \  
# Default rules for depl
```

CMakeLists.txt 文件内容如下：

```
cmake_minimum_required(VERSION 3.1.0) # 指定 cmake 最低版本  
project(example1 VERSION 1.0.0 LANGUAGES CXX) # 指定项目名称,  
版本, 语言 cxx 就是 c++  
set(CMAKE_CXX_STANDARD 11) # 指定 c++标准  
set(CMAKE_CXX_STANDARD_REQUIRED ON) # 指定 c++标准要求,至少  
为 11 以上  
set(target example1) # 指定目标名称  
set(CMAKE_AUTOMOC ON) # support qt moc # 支持 qt moc  
set(CMAKE_AUTORCC ON) # support qt resource file # 支持 qt 资  
源文件  
set(CMAKE_AUTOUIC ON) # support qt ui file # 支持 qt ui 文件(非  
必须)  
set(CMAKE_EXPORT_COMPILE_COMMANDS ON) # support clangd  
支持 clangd  
  
if (CMAKE_VERSION VERSION_LESS "3.7.0") # 如果 cmake 版本小于 3.  
7.0  
set(CMAKE_INCLUDE_CURRENT_DIR ON) # 设置包含当前目录  
endif()  
find_package(Qt5 COMPONENTS Widgets REQUIRED) # 寻找 Qt5 组  
件 Widgets  
find_package(Qt5 COMPONENTS Gui REQUIRED) # 寻找 Qt5 组件 Gu  
i  
  
cmake_minimum_required(VERSION 3.1.0) # 指定 cmake 最低版本  
project(example1 VERSION 1.0.0 LANGUAGES CXX) # 指定项目名称,  
版本, 语言 cxx 就是 c++  
set(CMAKE_CXX_STANDARD 11) # 指定 c++标准  
set(CMAKE_CXX_STANDARD_REQUIRED ON) # 指定 c++标准要求,至少  
为 11 以上
```



```
set(target example1) # 指定目标名称
set(CMAKE_AUTOMOC ON) # support qt moc # 支持 qt moc
set(CMAKE_AUTORCC ON) # support qt resource file # 支持 qt 资源文件
set(CMAKE_AUTOUIC ON) # support qt ui file # 支持 qt ui 文件(非必须)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON) # support clangd 支持 clangd

if (CMAKE_VERSION VERSION_LESS "3.7.0") # 如果 cmake 版本小于 3.7.0
set(CMAKE_INCLUDE_CURRENT_DIR ON) # 设置包含当前目录
endif()
find_package(Qt5 COMPONENTS Widgets REQUIRED) # 寻找 Qt5 组件 Widgets
```

如使用 cmake 进行构建，则使用如上 CMakeLists.txt，同样可以选择 Qtcreator 打开运行和命令行构建运行。使用命令行步骤如下：

```
cmake -B build #新建构建目录 build
cmake --build build #在 build 目录下生成编译文件
./build/control-center #运行可执行文件 control-center
```

运行效果如下：



图 12 control-center 应用运行效果

3.3.2.4. 常见问题

- 标题栏菜单没有「帮助」菜单项。

1) 检查 DBus, 当前应用有没有帮助服务 `com.deepin.Manual.Search ManualExists`。

2) `/usr/bin/dman` 是否存在, 以及 `/usr/share/deepin-manual` 是否存在 `appName` 的子目录。

如果 1) 返回 `false` 则不显示, 如果没有 DBus 服务, 则检查 2) 是否存在, 否则不显示。

- 滚动条显示的问题。

滚动条不占用 item 的显示区域, 是悬浮在 item 之上的。

滚动条会自动隐藏, 只有在滚动中才会显示, 停止滚动即开始隐藏动画。

可以通过设置属性来控制 `scrollArea->setProperty("_d_dtk_slider_always_show",true)`来控制 scrollbar 常显示。

- DBackgroundGroup 移除一项后, 下边的圆角没有刷新。

当使用函数 `QLayout::removeWidget` 把控件从布局中移除时:

没有把控件从 `DBackgroundGroup` 的子列表中移除, 也就是该控件的父类仍是 `DBackgroundGroup`。

也没有发出 `QEvent::ChildRemoved` 事件, 导致 `DBackgroundGroup` 不会刷新重新绘制。

解决办法:

控件 addWidget 添加布局,正常显示圆角。控件 removeWidget 移除布局,同时调用 deleteLater 脱离父子关系,正常显示圆角。

```
appendItem(m_customMtu);
removeItem(m_customMtu);
m_customMtu->deleteLater();
```

3.3.3. OpenGL 应用开发

OpenGL 是用于渲染 2D、3D 矢量图形的跨语言、跨平台的图形渲染 API 接口。这个接口由近 350 个不同的函数调用组成,用来绘制从简单的图形到比较复杂的三维景象。OpenGL 常用于 CAD、虚拟实境、科学可视化程序和电子游戏开发。

3.3.3.1. 环境安装

安装开发库、加载库、辅助库、工具、例子。

```
sudo apt update
sudo apt install build-essential           // 开发工具
sudo apt install libgl1-mesa-dev libglvnd-dev mesa-utils // GL 头文件、库文件
sudo apt install libglfw3-dev              // 窗口框架库
sudo apt install libglm-dev                // GL 图形数学库(向量、矩阵)
sudo apt install libassimp-dev             // 流行、开源的模型加载库
```

安装 glad (GL 加载库)

在 <https://glad.dav1d.de> 网站进行选择,然后生成头文件和代码文件。

图 13 生成头文件和代码文件 (一)

点击右下角 GENERATE，弹出如下页面：

图 14 生成头文件和代码文件 (二)

点击 glad.zip 下载到本地，解压至工程目录下。

3.3.3.2. 开发案例

下面例子摘自 https://www.glfw.org/docs/latest/quick_guide.html，加上了详细注释。

创建代码文件 demo2.c，代码如下：

```
#include <glad/glad.h> //使用 glad 加载库
的 GL 头文件
#define GLFW_INCLUDE_NONE //不使用 GLFW 自
带的 GL 头文件
#include <GLFW/glfw3.h> //包含 glfw 窗口库
的头文件
#include "linmath.h" //https://github.com/glfw/glfw/blob/master/d
eps/linmath.h，向量矩阵库
#include <stdlib.h>
#include <stdio.h>
static const struct { //定义顶点结构：x,y 坐标 和 rg
b 三个颜色分量
float x, y;
float r, g, b;
} vertices[3] = // 定义一个三角形的三个
顶点
{
{ -0.6f, -0.4f, 1.f, 0.f, 0.f },
{ 0.6f, -0.4f, 0.f, 1.f, 0.f },
{ 0.f, 0.6f, 0.f, 0.f, 1.f }
};
static const char* vertex_shader_text = //顶点着色器
代码
"#version 110\n" //着色器版本
"uniform mat4 MVP;\n"
```

```

"attribute vec3 vCol;\n"
"attribute vec2 vPos;\n"
"varying vec3 color;\n"
"void main()\n"
"{\n"
" gl_Position = MVP * vec4(vPos, 0.0, 1.0);\n"           //坐标变换
" color = vCol;\n"                                     //传递颜色
"}\n";
static const char* fragment_shader_text =              //片段着色器代码
"#version 110\n"                                       //着色器版本
"varying vec3 color;\n"
"void main()\n"
"{\n"
" gl_FragColor = vec4(color, 1.0);\n"                 //设置片原（像素）颜色
"}\n";
static void error_callback(int error, const char* description) //glfw
w 的错误处理回调函数
{
    fprintf(stderr, "Error: %s\n", description);
}
static void key_callback(GLFWwindow* window, int key, int scancode,
de,
int action, int mods) //glfw 的键盘事件处理回调函数
{
    if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS) //按下
Esc, 退出程序
    glfwSetWindowShouldClose(window, GLFW_TRUE);
}
int main(void) //程序主函数
{
    GLFWwindow* window; //GLFW 窗口句柄
    GLuint vertex_buffer, vertex_shader, fragment_shader, program;
//着色器相关对象的 ID 标识

```

```

    GLint mvp_location, vpos_location, vcol_location;           //存储着色
器中 uniform 变量的位置
    glfwSetErrorCallback(error_callback);                       //设置错误处理回调函数
    if (!glfwInit())                                           //GLFW 库初始化
        exit(EXIT_FAILURE);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 2); // 设置使用的 OpenGL 版本
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 0);
    //创建 GLFW 窗口提供 OpenGL 上下文
    window = glfwCreateWindow(640, 480, "Simple example", NULL,
NULL);
    if (!window)
    {
        glfwTerminate();
        exit(EXIT_FAILURE);
    }
    glfwSetKeyCallback(window, key_callback); //设置键盘事件处理回调
函数
    glfwMakeContextCurrent(window); //设置当前的 OpenGL 渲染上下文
    gladLoadGL();           //加载 opengl 函数
    glfwSwapInterval(1);
    // NOTE: OpenGL error checks have been omitted for brevity
    glGenBuffers(1, &vertex_buffer); //创建显存缓存, 传递顶点数据
    glBindBuffer(GL_ARRAY_BUFFER, vertex_buffer);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
    vertex_shader = glCreateShader(GL_VERTEX_SHADER);           //创
建、加载并编译着色器代码
    glShaderSource(vertex_shader, 1, &vertex_shader_text, NULL);
    glCompileShader(vertex_shader);
    fragment_shader = glCreateShader(GL_FRAGMENT_SHADER);
    glShaderSource(fragment_shader, 1, &fragment_shader_text, NULL);
    glCompileShader(fragment_shader);

```

```

program = glCreateProgram();
glAttachShader(program, vertex_shader);
glAttachShader(program, fragment_shader);
glLinkProgram(program);
mvp_location = glGetUniformLocation(program, "MVP");    //获取
uniform 变量位置
vpos_location = glGetAttribLocation(program, "vPos");
vcol_location = glGetAttribLocation(program, "vCol");
glEnableVertexAttribArray(vpos_location);
glVertexAttribPointer(vpos_location, 2, GL_FLOAT, GL_FALSE,
sizeof(vertices[0]), (void*) 0);
glEnableVertexAttribArray(vcol_location);
glVertexAttribPointer(vcol_location, 3, GL_FLOAT, GL_FALSE,
sizeof(vertices[0]), (void*) (sizeof(float) * 2));
while (!glfwWindowShouldClose(window))    //程序主循环
{
float ratio;
int width, height;
mat4x4 m, p, mvp;
glfwGetFramebufferSize(window, &width, &height); //得到 framebuf
fer 大小
ratio = width / (float) height;
glViewport(0, 0, width, height);
glClear(GL_COLOR_BUFFER_BIT);
mat4x4_identity(m);    //计算坐标标换矩阵
mat4x4_rotate_Z(m, m, (float) glfwGetTime());
mat4x4_ortho(p, -ratio, ratio, -1.f, 1.f, 1.f, -1.f);
mat4x4_mul(mvp, p, m);
glUseProgram(program);
//将坐标标换矩阵设置给着色器
glUniformMatrix4fv(mvp_location, 1, GL_FALSE, (const GLfloat*) m
vp);
glDrawArrays(GL_TRIANGLES, 0, 3);    //绘制三角形
glfwSwapBuffers(window);    //交换前后缓存

```



```
glfwPollEvents();           //轮询事件
}
glfwDestroyWindow(window);  //销毁窗口
glfwTerminate();            //退出 glfw 库
exit(EXIT_SUCCESS);
}
```

通过 `wget https://raw.githubusercontent.com/glfw/glfw/master/deps/linmath.h` 下载向量矩阵库头文件，拷贝到 `demo2` 目录中。再将 `glad` 加载库的头文件 `glad.h` 和源码文件 `glad.c` 拷贝到 `demo2` 目录下，形成如下文件层次结构：

```
demo2/
|—— demo2.c
|—— glad.c
|—— include/
|    |—— glad/
|    |    |—— glad.h
|—— linmath.h
|—— tree.txt
```

编译、链接：

```
gcc -o demo2 demo2.c glad.c -I./include -I./ -lglfw -lOpenGL -lm -ldl
```

打开终端，运行 `./demo2`

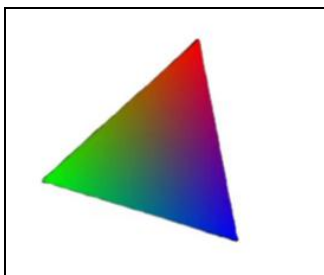


图 15 demo2 运行效果图

3.3.3.3. 环境配置

通过包管理器 apt 安装的开发库，其头文件、库文件已安装至标准目录下，不需做额外的环境配置。但是可以检查 OpenGL 硬件、驱动、性能信息，可以编写空的例子程序来验证开发环境和运行环境是否正常。

- 检查 OpenGL 环境。

终端窗口中运行 `glxinfo -B`，查看 GL、GLSL 版本、显存、驱动、是否硬件直接渲染和加速等信息。

```
glxinfo -B
```

命令执行结果如下：

```
name of display: :0
display: :0 screen: 0
direct rendering: Yes
Extended renderer info (GLX_MESA_query_renderer):
Vendor: VMware, Inc. (0xffffffff)
Device: llvmpipe (LLVM 7.0, 128 bits) (0xffffffff)
Version: 19.2.6
Accelerated: no
Video memory: 7755MB
Unified memory: no
Preferred profile: core (0x1)
Max core profile version: 3.3
Max compat profile version: 3.1
Max GLES1 profile version: 1.1
Max GLES[23] profile version: 3.0
OpenGL vendor string: VMware, Inc.
OpenGL renderer string: llvmpipe (LLVM 7.0, 128 bits)
OpenGL core profile version string: 3.3 (Core Profile) Mesa 19.2.6
OpenGL core profile shading language version string: 3.30
```

```
OpenGL core profile context flags: (none)
OpenGL core profile profile mask: core profile
OpenGL version string: 3.1 Mesa 19.2.6
OpenGL shading language version string: 1.40
OpenGL context flags: (none)
OpenGL ES profile version string: OpenGL ES 3.0 Mesa 19.2.6
OpenGL ES profile shading language version string: OpenGL ES
GLSL ES 3.00
```

- 运行例子——检查运行环境。

打开终端，运行 glxgears。

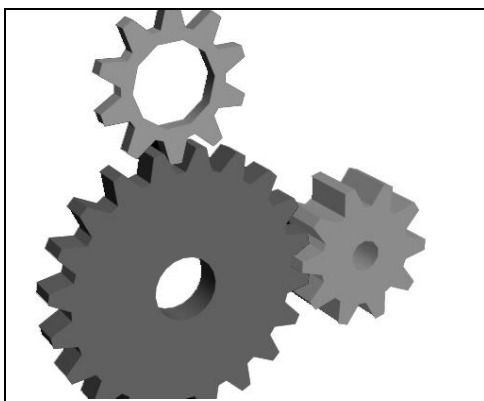


图 16 glxgears 运行效果图

可以看到 glxgears 运行正常，说明 OpenGL 环境已经部署成功。

- 编写例子——验证开发环境。

下面是一个基本例子，来源于 [https://www.glfw.org/documentation.](https://www.glfw.org/documentation.html)

html。

创建一个文件 demo1.c，代码如下：

```
#include <GLFW/glfw3.h>
int main(void)
{
    GLFWwindow* window;
```

```
/* Initialize the library */
if (!glfwInit())
return -1;

/* Create a windowed mode window and its OpenGL context */
window = glfwCreateWindow(640, 480, "Hello World", NULL, NULL);
if (!window)
{
glfwTerminate();
return -1;
}

/* Make the window's context current */
glfwMakeContextCurrent(window);

/* Loop until the user closes the window */
while (!glfwWindowShouldClose(window))
{
/* Render here */
glClear(GL_COLOR_BUFFER_BIT);

/* Swap front and back buffers */
glfwSwapBuffers(window);

/* Poll for and process events */
glfwPollEvents();
}

glfwTerminate();
return 0;
}
```

编译、链接：

```
gcc -o demo1 demo1.c -lglfw -lOpenGL
```

运行：

```
./demo1
```

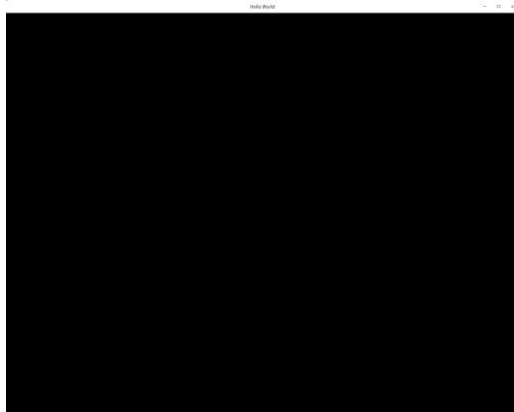


图 17 demo1 运行效果图

3.3.3.4. 常见问题

- 兼容性。

由于不同的硬件和驱动程序对 OpenGL 的支持程度不同，可能会导致应用在不同平台上表现不一致甚至出现兼容性问题。开发者需要考虑到不同硬件、操作系统和驱动程序的差异，编写具有良好兼容性的 OpenGL 应用。

- 性能优化。

OpenGL 开发中需要考虑到图形性能的优化，包括减少渲染调用、使用批处理、合并渲染状态等技术，以确保应用在各种硬件上都能够达到良好的性能表现。

- 3D 数学知识。

OpenGL 是一个底层的图形 API，需要开发者具备一定的线性代数和几何知识，以便进行矩阵变换、投影变换和其他与 3D 图形相关的计算。

- 跨平台开发。

如果你的应用需要在不同平台上运行，比如 Windows、Linux 和 macOS，你需要考虑到跨平台开发的问题，确保应用能够良好地适配不同的操作系统和硬

件平台。

- 调试和错误处理。

在 OpenGL 开发中，由于底层的特性，可能遇到各种各样的图形问题，如图形闪烁、纹理显示异常、光照计算错误等。因此，良好的调试技能和错误处理机制对于解决这些问题至关重要。

3.3.4. Vulkan 应用开发

Vulkan 是一种跨平台、低级别的图形和计算 API（应用程序接口），旨在提供高性能的图形渲染和计算功能。相对于传统的图形 API，如 OpenGL、Vulkan 提供了更多的控制权和更低的驱动开销，使开发者能够更好地优化应用程序并获得更高的性能。Vulkan 常用于图形渲染引擎、科学计算、移动设备、虚拟现实和游戏开发。

3.3.4.1. 环境安装

Vulkan SDK 是使用 Vulkan 开发应用程序必不可少的组件。它包含了 Vulkan API 的头文件，一个校验层实现，调试工具和 Vulkan 函数加载器。Vulkan 函数加载器类似 OpenGL 的 GLEW 可以在运行时查询驱动程序支持的 Vulkan API 函数。

- 安装 vulkan 驱动。

```
sudo apt update
sudo apt install libvulkan1 mesa-vulkan-drivers
```

- 安装 Vulkan SDK。

使用包管理工具安装（方便快捷）。

```
sudo apt update
sudo apt install libvulkan-dev vulkan-tools
sudo apt install vulkan-validationlayers-dev spirv-tools
```

- 从 LunarG 的网站上免费下载最新版和历史版本的 SDK。

<https://vulkan.lunarg.com/sdk/home#linux>

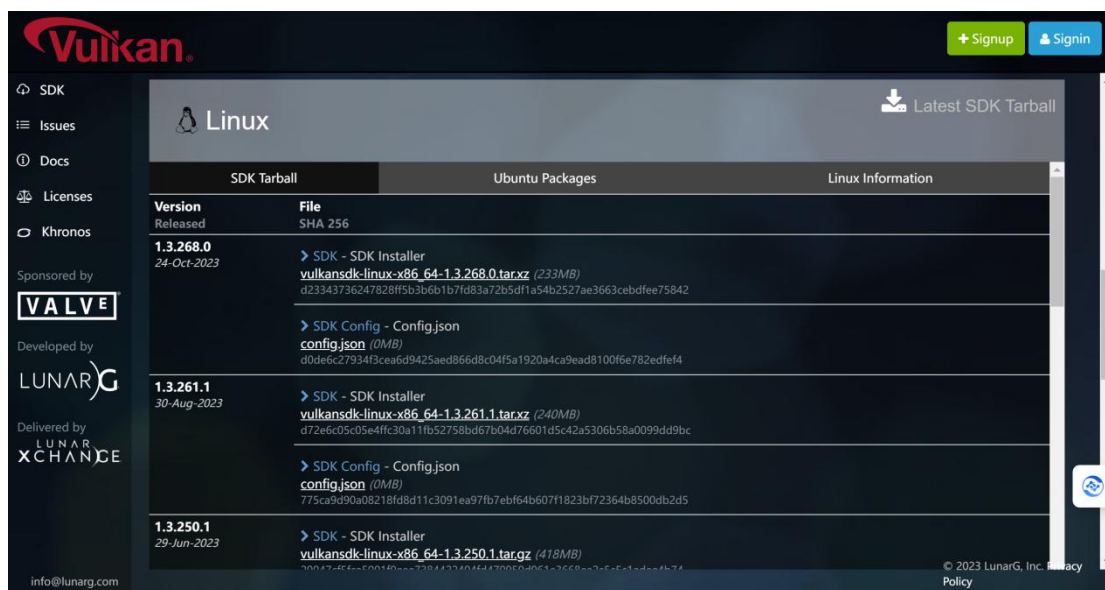


图 18 LunarG 下载界面

上图是 vulkan 各个版本的 SDK 下载的站点，点击下载 vulkansdk-linux-x86_64-1.3.250.0.tar.gz，解压缩：

```
mkdir ~/vulkan; cd ~/vulkan
mv ~/Downloads/vulkansdk-linux-x86_64-1.3.250.0.tar.gz ~/vulkan
tar xf vulkansdk-linux-x86_64-1.3.250.0.tar.gz
```

安装辅助开发包和工具：

```
sudo apt install libglfw3-dev //窗口环境库
sudo apt install libglm-dev //图形数学库，主要提供向量、矩阵等线性代数运算
```

还需要安装 shader 编译器，常用的 shader 编译器包括：glslangValidator

r、glslc。

glslangValidator 可以通过如下命令安装:

```
sudo apt install glslang-tools
```

glslc 可以通过下面 github 地址下载安装:

<https://github.com/google/shaderc/blob/main/downloads.md>

3.3.4.2. 环境配置

通过包管理器 apt 安装的开发库，其头文件、库文件已安装至标准目录下，不需做额外的环境配置。

通过官网下载的 SDK，需要配置环境：进入 sdk 目录，运行 setup-env.sh。

另外可以通过运行 sdk 自带例子检查运行环境是否正常，通过 vulkaninfo 工具检查 vulkan 硬件、驱动、性能等信息。



图 19 vkcube 示例程序

如果程序运行出现错误，检查显卡是否支持 vulkan，或者运行 vulkaninfo 查看具体的错误信息。

vulkan 信息查询工具：运行 vulkaninfo，查看显卡、驱动、性能等 vulkan 相关信息。

如果出现错误提示，请检查显卡是否支持 vulkan、vulkan 驱动是否正确安装，或通过搜索工具网上查询。

3.3.4.3. 开发案例

- 简单例子。

本示例参见 https://vulkan-tutorial.com/Development_environment#page_Setting-up-a-makefile-project，具体步骤如下：

创建一个叫 demo1.cpp 的源码文件，内容如下：

```
#define GLFW_INCLUDE_VULKAN
#include <GLFW/glfw3.h>
#define GLM_FORCE_RADIANS
#define GLM_FORCE_DEPTH_ZERO_TO_ONE
#include <glm/vec4.hpp>
#include <glm/mat4x4.hpp>
#include <iostream>
int main() {
    glfwInit();
    glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);
    GLFWwindow* window = glfwCreateWindow(800, 600, "Vulkan window", nullptr, nullptr);
    uint32_t extensionCount = 0;
    vkEnumerateInstanceExtensionProperties(nullptr, &extensionCount, nullptr);
    std::cout << extensionCount << " extensions supported\n";
    glm::mat4 matrix;
    glm::vec4 vec;
```

```
auto test = matrix * vec;
while(!glfwWindowShouldClose(window)) {
    glfwPollEvents();
}
glfwDestroyWindow(window);
glfwTerminate();
return 0;
}
```

安装依赖库：

```
sudo apt install libxxf86vm-dev libxi-dev
```

编译链接：

```
g++ -o demo1 -std=c++17 -O2 demo1.cpp -lglfw -lvulkan -ldl -pthread
-lX11 -lXxf86vm -lXrandr -lXi
```

运行：

```
./demo1
```

- 典型例子。

vulkan sdk 的 C 语言 API 使用起来非常繁琐，因此官方提供了 C++ 语言的封装（Vulkan-Hpp），方便开发人员提高效率，专注于业务逻辑的实现。

下面这个例子是 Vulkan-Hpp 项目中的一个例子，是使用 vulkan 开发三维图形应用的典型参考。

下载 vulkan-hpp 源码：

因为 vulkan-hpp 源码中包含了 5 个子项目模块 Vulkan-Headers、glfw、glm、glslang、tinyxml2，所以要用 git 命令的--recurse-submodules 参数来递归下载完整的项目代码：

```
git clone --recurse-submodules https://github.com/KhronosGroup/Vulkan-Hpp.git
```

如果网络有问题，无法递归下载到上述 5 个子项目，可以先不带--recurse-submodules 参数，仅下载 vulkan-hpp 源码：

```
git clone https://github.com/KhronosGroup/Vulkan-Hpp
```

然后再到浏览器中访问并下载 <https://github.com/KhronosGroup/Vulkan-Hpp/tree/main> 中列出来的上述 5 个子项目代码，再分别拷贝到 vulkan-hpp 目录下的与这 5 个子项目同名的子目录中，拼成一个完整的 vulkan-hpp 源码包。

确认 2 个关键的编译依赖包的版本是否符合要求：

cmake 版本不低于 3.26

gcc >= 4.8.2（早期版本可能有效，但未经测试）

进入 Vulkan-Hpp 源码目录，先安装两个需要的库包 libxinerama-dev 和 libxcursor-dev，然后开始 cmake 编译，会在 Vulkan-Hpp 目录下生成一个 build 子目录，里面是最终编译用到的 Makefile 文件：

```
cd Vulkan-Hpp
sudo apt install libxinerama-dev libxcursor-dev
cmake -DSAMPLES_BUILD_WITH_LOCAL_VULKAN_HPP=ON -DSAMPLES_BUILD=ON -DTESTS_BUILD_WITH_LOCAL_VULKAN_HPP=ON -DTESTS_BUILD=ON -DVULKAN_HPP_SAMPLES_BUILD=ON -B build -G "Unix Makefiles"
```

我们单独进入示例 15 的子目录进行编译，并测试执行示例 15：

```
cd build/samples/15_DrawCube
make
./15_DrawCube
```

3.3.4.4. 常见问题

vulkan 比较复杂，各种状态维护容易出错，建议开发测试阶段启用验证层，帮助快速定位问题。

常见错误码以及描述如下：

- VK_ERROR_OUT_OF_HOST_MEMORY 主机内存分配失败。
- VK_ERROR_OUT_OF_DEVICE_MEMORY 设备内存分配具有失败。
- VK_ERROR_INITIALIZATION_FAILED 对象的初始化可以由于特定于实现的原因而未完成。
- VK_ERROR_DEVICE_LOST 逻辑或物理设备已丢失。查看丢失的设备。
- VK_ERROR_MEMORY_MAP_FAILED 内存对象的映射失败。
- VK_ERROR_LAYER_NOT_PRESENT 请求的图层不存在或无法加载。
- VK_ERROR_EXTENSION_NOT_PRESENT 请求的扩展不支持。
- VK_ERROR_FEATURE_NOT_PRESENT 不支持请求的功能。
- VK_ERROR_INCOMPATIBLE_DRIVER 请求的 Vulkan 版本是驱动程序不支持或与特定于实现的原因。
- VK_ERROR_TOO_MANY_OBJECTS 该类型的对象过多已创建。
- VK_ERROR_FORMAT_NOT_SUPPORTED 不支持请求的格式在此设备上。
- VK_ERROR_FRAGMENTED_POOL 池分配由于以下原因而失败池内存碎片。仅当未尝试分配主机或设备时，才必须返回此信息已创建内存以适应

新的分配。这应该优先返回，但前提是实现确定池分配失败是由于碎片造成的。

- VKERROR_OUT_OF_POOL_MEMORY。
- VK_ERROR_SURFACE_LOST_KHR 曲面不再可用。
- VK_ERROR_NATIVE_WINDOW_IN_USE_KHR 请求的窗口已存在被 Vulkan 或其他 API 以阻止其再次使用。
- VK_ERROR_OUT_OF_DATE_KHR 表面发生了变化，使得它不再与交换链兼容，并进一步呈现使用交换链的请求将失败。应用程序必须查询新的图面属性并重新创建其交换链，如果他们希望继续向表面展示。



图 20 验证流程

复制 vk_layer_settings.txt 文件复制到用于调试程序的目录中，修改 VK_INSTANCE_LAYERS 环境变量，在终端运行 vulkan 程序，即可看到验证层输出的警告、错误信息。

例如：

```
cp ~/vulkan/1.3.250.0/config/vk_layer_settings.txt ~/vulkan/mydemo/bin/
```

创建名为 VK_INSTANCE_LAYERS 的环境变量：

```
export VK_INSTANCE_LAYERS=VK_LAYER_LUNARG_standard_validation
```

在终端窗口中运行 vulkan 程序，潜在的警告或错误信息会输出到终端窗口中。

- 使用 vkconfig 工具。

官方 SDK 提供了 vkconfig 工具方便让应用开发人员在程序运行器覆盖、调整 vulkan 部分层配置，得到详细的运行信息或错误提示，具体用法可参考：<https://github.com/LunarG/VulkanTools/blob/main/vkconfig/README.md>。

3.4. D-Bus 进程间通信开发

Bus 是一种高级的进程间通信系统。支持进程间一对一和多对多的对等通信，在多对多的通讯时，需要后台进程的角色去分转消息，当一个进程发消息给另外一个进程时，先发消息到后台进程，再通过后台进程将信息转发到目的进程。DBUS 后台进程充当着一个路由器的角色。

D-Bus 主要概念为总线，连接到总线的进程可通过总线接收或传递消息，总线收到消息时，根据不同的消息类型进行不同的处理。

- D-Bus 中的一些基本概念。

会话总线（Session Bus）普通进程创建，可同时存在多条。会话总线属于某个进程私有，它用于进程间传递消息。

系统总线（System Bus）在引导时就会启动，它由操作系统和后台进程使用。

Bus Name 连接名称，主要是用来标识一个应用和消息总线的连接。例如"org.kde.StatusNotifierWatcher"。

Object Paths 连接路径，标识连接中的不同服务，每个 Object Paths 表示一个服务。这个路径在连接中是唯一的。

Interfaces 接口 Object Paths 下都包含有多个接口 Interfaces，举例如下接口：

- 1) org.freedesktop.DBus.Introspectable
- 2) org.freedesktop.DBus.Properties
- 3) org.kde.StatusNotifierWatcher
- 4) Methods 和 Signals

Methods 表示可以被具体调用的方法

Signals 则表示的是信号，此信号可以被广播，而连接了这个信号的对象在接收到信号时就可以进行相应的处理。和 Qt 中的信号应该是一个意思。

- D-Bus 优势。

- 1) 易用性：具有完备的事件和通知机制，支持的库、框架众多。
- 2) 通用性：广泛应用于 Linux 桌面系统，例如许多 systemd 服务采用 D-Bus 提供接口。

- 3) 松散解耦：进程与服务间无多余依赖。
- 4) 安全：可以获取调用者 pid 和 uid，易于访问控制。

3.4.1. 通用 D-Bus 接口

基于 freedesktop 规范，系统提供了通用的 D-Bus 接口，包括系统账户管理、网络管理、蓝牙、电源管理、窗口管理、授权服务、设备管理、登录、时间、服务管理等。

```
$ busctl list --system | grep org.freedesktop
```

NAME	PROCESS	UNIT
org.freedesktop.Accounts-daemon.service	accounts-daemon	accounts-daemon.service
org.freedesktop.Avahi	-	-
org.freedesktop.DBus	systemd	init.scope
org.freedesktop.DisplayManager	lightdm	lightdm.service
org.freedesktop.GeoClue2	-	-
org.freedesktop.ModemManager1	ModemManager	ModemManager.service
org.freedesktop.NetworkManager	NetworkManager	NetworkManager.service
org.freedesktop.PolicyKit1	polkitd	polkit.service
org.freedesktop.UDisks2	udisksd	udisks2.service
org.freedesktop.UPower	upowerd	upower.service
org.freedesktop.hostname1	-	-
org.freedesktop.locale1	-	-
org.freedesktop.login1	systemd-logind	systemd-logind.service
org.freedesktop.network1	-	-

org.freedesktop.nm_dispatcher	-	-
org.freedesktop.resolve1	-	-
org.freedesktop.systemd1	systemd	init.scope
org.freedesktop.timedate1	-	-
org.freedesktop.timesync1	systemd-timesyn	systemd-tim
esyncd.service		

开发者除了通过 C 库接口，也可以通过以上服务 D-Bus 接口，方便的获取各类信息，订阅服务动态信号，管理各个模块。

3.4.1.1. 环境安装

开发 D-Bus 的库有 libsystemd sd-bus，编译依赖安装命令。

```
$sudo apt install libsystemd-dev
```

使用 QTD-Bus，安装 QT 应用开发环境，引用 qdbus 模块。

其他开发库有 golang d-bus 等。

3.4.1.2. 环境配置

D-Bus 的工具具有 busctl, dbus-send, dbus-monitor, d-feet。

busctl 是一个 D-Bus 查看和监控的基础工具，其常用命令有：

- busctl list 打印出总线中已注册的服务。
- busctl tree 查看服务 D-Bus 树结构。

```
$ busctl tree org.freedesktop.hostname1
└─/org
  └─/org/freedesktop
    └─/org/freedesktop/hostname1
```

- busctl introspect 查看服务的接口定义。

```
$ busctl introspect org.freedesktop.hostname1 /org/freedesktop/h
hostname1
```

NAME	TYPE	SIGNATURE	RESULT/VALUE	FLAGS
org.freedesktop.DBus.Introspectable	interface	-	-	-
.Introspect	method	-	s	-
org.freedesktop.DBus.Peer	interface	-	-	-
...				

- busctl monitor 监听 D-Bus 调用记录。

```
$ sudo busctl monitor org.freedesktop.hostname1
```

- busctl get-property 获取属性。

```
$ busctl get-property org.freedesktop.hostname1 /org/freedesktop
/hostname1 org.freedesktop.hostname1 Hostname
s "uos-PC"
```

- busctl send 调用接口。

```
$ sudo busctl call org.freedesktop.hostname1 /org/freedesktop/h
o
stname1 org.freedesktop.hostname1 SetHostname sb "test01" false
```

- dbus-send 发送 D-Bus 消息。

```
$ dbus-send --system --print-reply --type=method_call --dest=org.fr
e
edesktop.hostname1 /org/freedesktop/hostname1 org.freedesktop.DB
u
s.Properties.Get string:"org.freedesktop.hostname1" string:Hostname
method return time=1701875491.796218 sender=:1.211 -> destina
t
ion=:1.210 serial=3 reply_serial=2
variant string "uos-PC"
$ sudo dbus-send --system --print-reply --dest=org.freedesktop.ho
s
tname1 /org/freedesktop/hostname1 org.freedesktop.hostname1.SetH
o
stname string:Test01 boolean:true
method return time=1701875814.080582 sender=:1.226 -> destina
t
ion=:1.232 serial=12 reply_serial=2
```

- dbus-monitor 监听 D-Bus 消息。

```
$ sudo dbus-monitor --system interface=org.freedesktop.hostname
```

1

- d-feet，强大的图形化的 D-Bus 查看和调试工具。

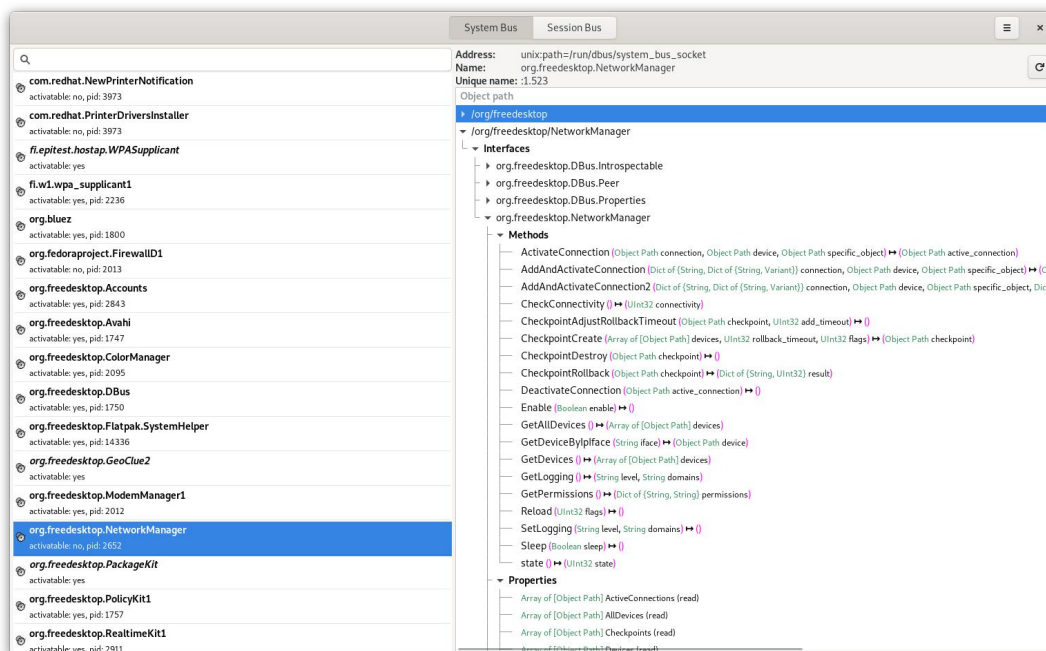


图 21 d-feet 界面

通过以上工具，观察和调试 D-Bus，简单高效。

3.4.1.3. 开发案例

下面代码演示通过 libsystemd sd-bus 调用 D-Bus，设置 hostname。

```
#include <iostream>
#include <systemd/sd-bus.h>
#define cleanup_(x) __attribute__((__cleanup__(x)))
using namespace std;
int main(int argc, char *argv[])
{
    cleanup_(sd_bus_message_unrefp) sd_bus_message *m = NULL;
    cleanup_(sd_bus_error_free) sd_bus_error error = SD_BUS_ERROR_NU
```

```
LL;

cleanup_(sd_bus_close_unrefp) sd_bus *bus = NULL;
const char *path;
int r;
/* Connect to the system bus */
r = sd_bus_open_system(&bus);
if (r < 0){
    cout << "Failed to connect to system bus: %m" << endl;
    return r;
}
r = sd_bus_call_method(bus,
    "org.freedesktop.hostname1",
    "/org/freedesktop/hostname1",
    "org.freedesktop.hostname1",
    "SetHostname",
    &error,
    &m,
    "sb",
    "test01",
    0);
if (r < 0){
    cout << "Failed to sd_bus_call_method: " << error.message << endl;
    return r;
}
cout << "SetHostname success." << endl;
r = sd_bus_call_method(bus,
    "org.freedesktop.hostname1",
    "/org/freedesktop/hostname1",
    "org.freedesktop.DBus.Properties",
    "Get",
    &error,
    &m,
    "ss",
    "org.freedesktop.hostname1",
```

```
"Hostname");
if (r < 0){
    cout << "Failed to sd_bus_call_method " << error.message << endl;
    return r;
}
char* hostname = NULL;
r = sd_bus_message_read(m, "v", "s", &hostname);
if( r < 0) {
    cout << "Failed to sd_bus_message_read: %m" << endl;
    return r;
}
cout << "hostname : " << hostname << endl;
return 0;
}
```

编译，运行：

```
# g++ test.cpp -o test `pkg-config --cflags --libs libsystemd`
$ sudo ./test
SetHostname success.
hostname : test01
```

3.4.1.4. 常见问题

- dbus 监听不到信号。

system 级 D-Bus 信号需要 root 用户才可以监听到。

3.4.2. UOS 扩展 D-Bus 接口

UOS 系统中许多系统组件、桌面、自研应用使用 D-Bus 进行进程间交互，并通过 D-Bus 接口对外提供功能。

系统级服务包括文件管理器、应用管理、锁屏、系统信息、安全管控等。

```
$ busctl list --system | grep deepin | grep -vE '^[0-9].*$'
NAME                                PROCESS    UNIT
com.deepin.anything                dd-file-manage dde-filemanager-daemon.s...
com.deepin.daemon.Accounts          dde-system-daem deepin-accounts-daemon.s...
com.deepin.daemon.AirplaneMode      dde-system-daem deepin-accounts-daemon.s...
com.deepin.daemon.Apps              dde-system-daem deepin-accounts-daemon.s...
com.deepin.filemanager.daemon       dd-file-manage dde-filemanager-daemon.s...
com.deepin.serviceandsupport        -          -
com.deepin.sync.Helper              -          -
com.deepin.system.Display           dde-system-daem deepin-accounts-daemon.s...
com.deepin.system.Power             dde-system-daem deepin-accounts-daemon.s...
com.deepin.system.SystemInfo        dde-system-daem deepin-accounts-daemon.s...
org.deepin.DeviceInfo              deepin-service- deepin-service-group@app...
org.deepin.EventLog1               deepin-event-lo dbus.service
org.deepin.PermissionManager        LocalPropertyRe permission-manager.servi...
...
```

用户级服务有商店、日历、截图、剪切板、控制中心、通知、窗口管理、设备管理器等。

```
$ busctl list --user | grep deepin | grep -vE '^[0-9].*$'
NAME                                PROCESS    UNIT
com.deepin.Calendar                -          -
com.deepin.Draw                    -          -
com.deepin.Editor                  deepin-editor session-3.scope
```

```

com.deepin.daemon          -          -
com.deepin.daemon.Appearance dde-session-dae session-3.scope
com.deepin.dde.daemon.Dock   dde-session-dae session-3.scope
com.deepin.dde.desktop      dde-desktop    session-3.scope
com.deepin.dde.lockFront     dde-lock      session-3.scope
com.deepin.dde.osd           dde-osd        session-3.scope
com.deepin.dde.shutdownFront dde-lock      session-3.scope
com.deepin.deepinid          deepin-deepinid user@1000.service
org.deepin.filemanager.server dde-file-manage ssuser@1000.se
rvice
...

```

基于 UOS 拓展 D-Bus 接口，开发者可以获取桌面、文管及其他应用信息，调用其对外接口，提升应用体验。

3.4.2.1. 环境安装

同 3.4.1.1 环境安装。

3.4.2.2. 环境配置

同 3.4.2.2 环境安装。

3.4.2.3. 开发案例

下面代码演示，应用发现需要更新，通过接口 `com.home.appstore.client.openBussinessUri`，打开应用商店并跳转到软件更新界面。

```

#include <iostream>
#include <systemd/sd-bus.h>
#define cleanup_(x) __attribute__((__cleanup__(x)))
using namespace std;
int main(int argc, char *argv[])
{

```

```

cleanup(sd_bus_message_unrefp) sd_bus_message *m = NULL;
cleanup(sd_bus_error_free) sd_bus_error error = SD_BUS_ERROR_
_NULL;

cleanup(sd_bus_close_unrefp) sd_bus *session_bus = NULL;
const char *path;
int r;
/* Connect to the system session_bus */
r = sd_bus_open_user(&session_bus);
if (r < 0){
    cout << "Failed to connect to system session_bus: %m" << e
endl;

    return r;
}
r = sd_bus_call_method(session_bus,
    "com.home.appstore.client",
    "/com/home/appstore/client",
    "com.home.appstore.client",
    "openBusinessUri",
    &error,
    &m,
    "s",
    "app_detail_info/com.qq.weixin.deepin");
if (r < 0){
    cout << "Failed to sd_bus_call_method: " << error.message <<
endl;

    return r;
}
cout << "Open appstore success." << endl;
return 0;
}

```

编译，运行：

```

# g++ test.cpp -o test `pkg-config --cflags --libs libsystemd`
# ./test

```



```
Open appstore success.
```

3.4.2.4. 常见问题

- root 用户无法调用 session 接口。

session 会话总线，用于登录的用户桌面，只能相同桌面环境的用户调用。

3.5. UOS 集成开发工具

集成开发工具（IDE）用于程序的开发、调试和发布。不同的 IDE 在应用场景和支持语言上有所偏重。大部分的 IDE 都提供了丰富的功能和扩展插件，可以帮助开发者大幅提高开发效率。本章节将针对目前主流的 IDE 如何在 UOS 上进行安装部署和运行使用做大致的介绍，以帮助研发人员快速上手。

为明确基于不同开发工具工程的开发策略，以下罗列了常用开发工具的对比信息，包含开发难度、语言支持、架构支持等信息：

表 2 常用开发工具的对比信息

IDE 工具	开发难度	支持语言	支持架构	主要问题
Qt Creator	低	C++	全架构	Qt 专属 IDE，基本不能用于其他框架开发。
VS Code	低	全部，通用型 IDE	全架构	插件配置精通困难，部分专业功能不如专业 IDE。
UnionCode	高，新 IDE 需要熟悉	C++、Python、Java 等	全架构	目前统信独立开发，需要进一步成熟。
Eclipse	低	Java、C++、Python、PHP 等	全架构	界面设计落后，不如其他新 IDE 资源占用相对较高。
Android Studio	低	Java、Kotlin	X86、ARM	由商业公司主导，对其他架构支持不佳。
Visual	建议使用	C++、C#、Java、	X86	只支持 Windows 平台。

IDE 工具	开发难度	支持语言	支持架构	主要问题
Studio	VSCode 替换	Python 等		

可以看到，和 Windows 平台强相关的 Visual Studio 不能平滑迁移，还需要做更多适配工作，故本章节不涉及该工具。UnionCode 因为推出时间较短，还未得到大规模推广，资料和功能还不完善，开发难度稍大。不过因其兼容通用的工程文件，可以平滑的替换为其它 IDE。其余 4 个 IDE 因为本身具备跨平台特性，开发难度都相对较低。

3.5.1. UnionCode

UnionCode 是深度开发的一款轻量级多语言跨平台兼容的集成开发环境，支持多种语言工程的构建、运行和调试。

3.5.1.1. 环境安装

建议在应用商店下载安装，也可以使用以下命令安装 UnionCode：

```
sudo apt install deepin-unioncode
```

3.5.1.2. 环境配置

UnionCode 部分非核心功能依赖第三方工具，为避免在使用过程中逐个安装，可以使用以下命令统一安装：

```
sudo apt install cmake gdb g++ gcc git npm
```

安装完第三方依赖后，如果需要进行套件配置，选择工具>选项>套件名。套件的参数大部分都是从样机环境扫描得到，如果需要更换某个选项可以在该界面修改。需要注意的是，部分套件的配置界面可能为空，说明本机没有安装所需

的工具，请根据缺失部分安装完整的语言工具链。

3.5.1.3. 开发案例

- 导入工程。

UnionCode 打开工程的方式和其它 IDE 有较大区别，不同于 VS Code 通过配置文件设置开发语言、工作目录等参数，UnionCode 是在打开工程时就进行选择，具体步骤为：

在菜单栏中，点击文件>打开工程>选择语言>选择套件，弹出文件选择对话框，选择想要打开的工程，完成工程导入。

- 构建与运行。

在工程视图中选中工程节点，右键选择“工程属性”，在弹出的对话框中进行编译配置和环境变量配置。完成后，在菜单“构建”中可执行编译（Ctrl+B）、重新编译、清除等操作。编译成功后在工具栏单击运行图标，工程代码将跳过断点，直接运行。

- 调试代码。

在需要中断运行的地方打上断点。

在工具栏中点击开始调试，程序会运行到第一个断点处，并打开变量监视器。变量监视器视图会记录变量的名称和在当前步骤时的值。

3.5.1.4. 常见问题

- 为什么 Qt 代码会有诊断错误？

UnionCode 依赖 clangd 实现 C/C++的语法服务，如果 clangd 版本过低会

出现上述诊断错误，建议升级到 clangd-13。

```
sudo apt install clangd-13
```

- 为什么打开工程时会报符号解析错误？

符号解析通过 Python 代码实现，当 Python 版本过低会导致符号解析功能异常，建议升级到 Python3.7。

```
sudo apt install python3.7
```

- JAVA 程序调试时为什么报环境错误？

JAVA 的运行需要 JRE 的支持，但不同平台因为兼容性问题需要的 JRE 版本存在差异。如果运行或调试时发生错误，请打开“工程属性”界面将其切换到与平台兼容的版本。

3.5.2. Qt Creator

Qt Creator 是一款跨平台集成开发环境，主要用于开发桌面应用、移动应用和嵌入式系统。它支持的语言主要为 C++、QML 和 JavaScript 等，特点是支持 Qt 框架，可以快速开发高质量的跨平台应用程序。同时，它还提供了强大的图形化界面设计工具，可以帮助开发者快速构建用户界面。

3.5.2.1. 环境安装

打开终端，使用以下命令安装 Qt Creator：

```
sudo apt install qtcreator
```

安装 Qt，可以从 Qt 官网下载安装包，也可以使用以下命令安装：

```
sudo apt install qt5-default
```

这将安装 Qt 5 的默认版本。安装其他 Qt 工具和库，可以使用以下命令：

```
sudo apt install qt5-qmake qtbases5-dev-tools qtools5-dev-tools
```

3.5.2.2. 环境配置

Qt Creator 通常会尝试自动检测已安装的编译器和 Qt 版本。如果检测成功，相关的工具链、开发包将自动在 Qt Creator 中可用。如果要添加或修改这些默认配置项，可以参照下述方法操作。

在 Qt Creator 中，套件是一组工具和设置，用于构建和运行 Qt 应用程序。要添加套件，请依次选择工具>选项>套件>添加，然后按照向导中的说明进行操作。可以选择不同的构建和运行工具，例如编译器、Qt 版本和调试器。



图 22 QtCreator 配置界面

3.5.2.3. 开发案例

本节将介绍如何在 UOS 上利用 Qt Creator 开发一个小应用。

- 生成工程。

选择文件>新建项目，进入到引导界面，按照需求配置参数后点击完成，进入代码编辑界面。

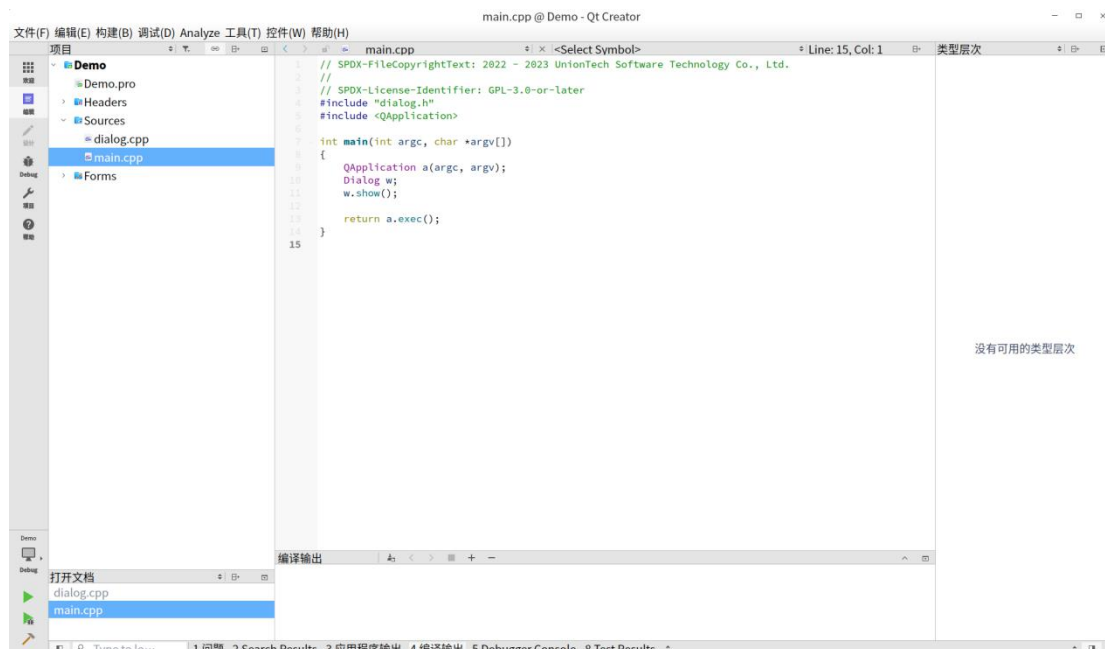


图 23 QtCreator 项目建立

- 构建与运行。

Qt Creator 可以为多个目标平台构建应用程序，也可以使用不同的编译器、调试器或 Qt 版本构建应用程序。这些配置都集中于套件中，不同的套件对应不同的构建、运行配置。所以在构建前应该选择合适的套件，可通过项目>Build&Run 进行选择。

点击构建按钮或使用快捷键 Ctrl+B 完成工程编译。编译成功后，点击运行按钮或使用快捷键 Ctrl+R 直接启动程序。默认情况下，运行应用程序会将其部署到可在桌面、设备仿真程序、模拟器或连接到开发者电脑的计算机上运行它的位置。

- 调试代码。

调试工程直接点击左下角调试图标或快捷键 F5 启动调试：

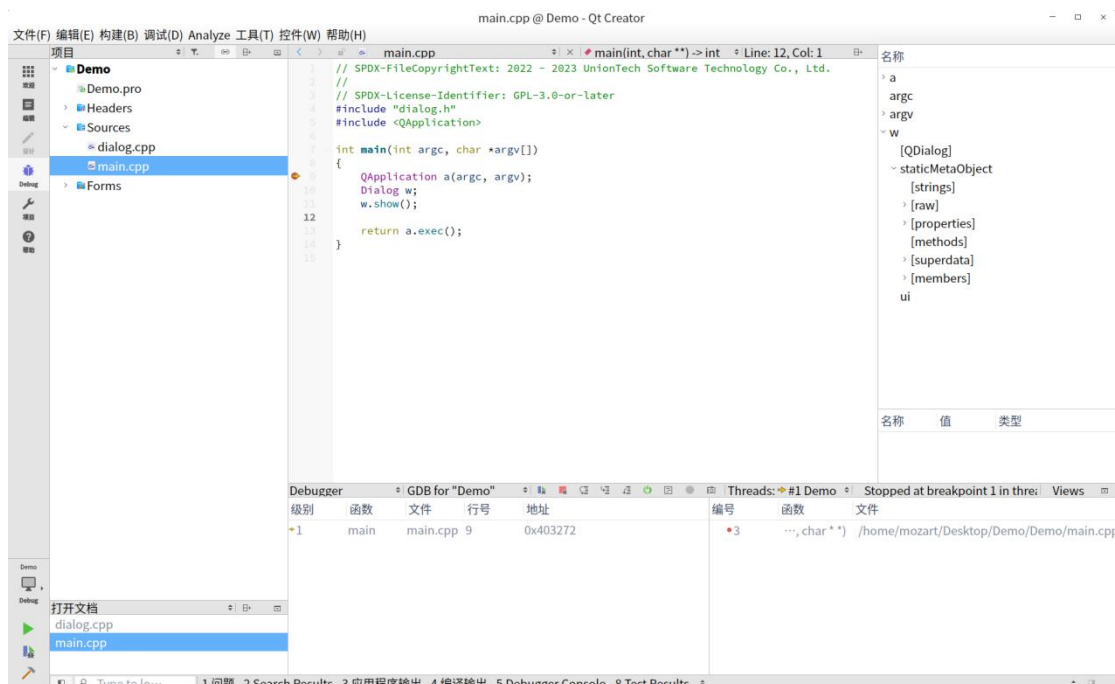


图 24 QtCreator 调试界面

3.5.2.4. 常见问题

- 为什么无法打印 Debug 级别的日志？

系统默认只打印 Warning 以上警告，如果要开启 Debug 级别日志，注意屏蔽 QT_LOGGING_RULES 环境变量。

- 编译时提示缺少编译器怎么办？

安装 Qt Creator 时默认是不会安装编译器等三方工具，建议在使用前将常用的开发工具进行安装。

```
sudo apt install clang gdb make gcc
```

- 未找到已安装的 Qt 版本怎么办？

如果已经安装了 Qt 某个版本但没有在 Qt Creator 中列出，可以通过工具>

选项>工具包>Qt 版本添加，在弹出的对话框中，选择 Qt 版本的路径和名称，并单击“确定”。

- 如何开启 Qt 源码调试？

调试 Qt 源码，打开工具>选项>调试器，然后选添加，在“源路径”填写，(也就是当前目录)，在目标路径写下载的 qt 源码路径，例如/home/user/qtbase-opensource-src-5.11.3.9，源代码可用以下命令下载：

```
apt source qtbase-opensource-src
```

本机再安装以下几个库文件，就可以开启 Qt 调试。

```
sudo apt install libqt5core5a-dbgsym libqt5gui5-dbgsym libqt5widgets5-dbgsym
```

3.5.3. VS Code

VS Code 是由微软开发的轻量级跨平台代码编辑器，支持多种编程语言，包括 TypeScript、Java 和 Python 等。它主要用于开发 Web 应用、后端应用，特点是轻量级、快速、易于扩展，支持丰富的插件和扩展。VS Code 采用的技术栈和平台无关，所以也能快速的在 UOS 完成开发。

3.5.3.1. 环境安装

打开终端，使用以下命令安装 VS Code：

```
sudo apt install com.visualstudio.code
```

或者在应用商店搜索 VS Code，点击结果可直接安装，安装完成后在启动栏或桌面点击图标运行。

3.5.3.2. 环境配置

VS Code 支持的语言是通过插件方式接入，这里以 C++ 语言作为示例语言，说明如何进行开发。

- 安装扩展。

打开 VS Code，选择“活动”栏上的“扩展”视图图标或使用键盘快捷键（Ctrl+Shift+X），搜索 C++ 语言插件，选择“安装”。打开或创建 *.cpp 文件时，在编辑器中将具有语法突出高亮显示、智能补全、智能提示以及错误检查。

- 安装编译器。

C/C++ 扩展不包括 C++ 编译器或调试器，需要手动安装这些工具或使用计算机上已安装的工具。安装命令如下：

```
sudo apt install gcc clang
```

3.5.3.3. 开发案例

为了确保语言插件得到正确的安装和配置，下面将用最简单的 Hello World C++ 程序作为示例演示。

- 创建工作区。

创建一个名为 HelloWorld 的文件夹，并在该文件夹中打开 VS Code

（code.在当前文件夹中打开 VS Code）：

```
mkdir HelloWorld  
cd HelloWorld  
code .
```

code.命令在当前工作文件夹中打开 VS Code，该文件夹即是“工作区”。随后在“工作区信任”对话框中选择是，导入工程。

- 添加源代码。

创建一个名为 helloworld.cpp 的新文件，并粘贴此源代码：

```
#include <iostream>

int main()
{
    std::cout << "Hello World" << std::endl;
}
```

- 构建与运行。

从主菜单中选择终端，运行构建任务命令（Ctrl+Shift+B）。这将显示一个包含各种编译器任务选项的下拉列表，选择 g++ 编译器完成编译。在输出目录会生成一个名为 helloworld 的可执行文件，该文件将显示在文件资源管理器中。在命令提示符或新的 VS Code 集成终端中，通过键入“./helloworld”来运行程序。

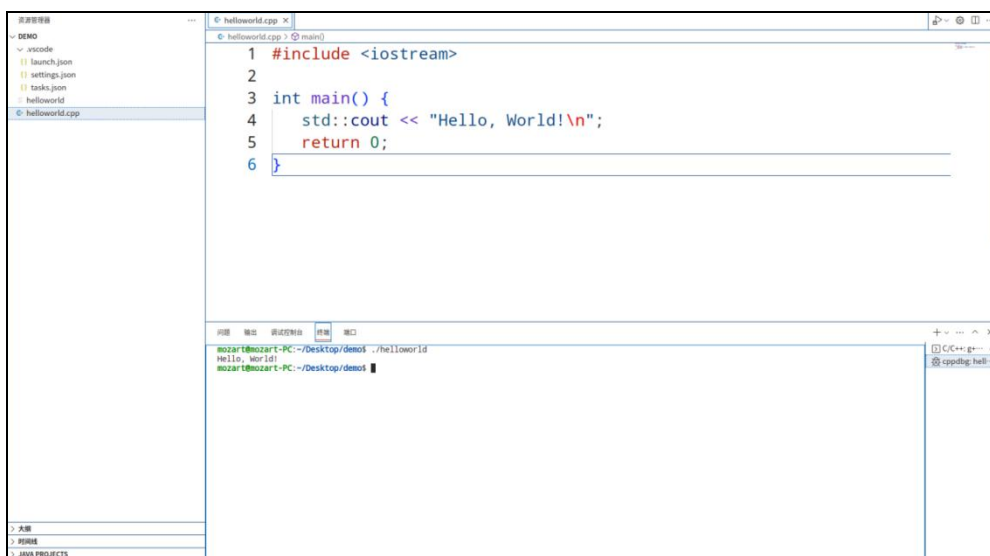


图 25 运行编译的程序

如果一切设置正确，会看到输出“Hello World”。

- 调试代码。

点击侧边栏的调试按钮，接着点击运行和调试，启动调试。如果需要定制化调试参数，点击“创建 launch.json 文件”生成调试配置。随后点击右下角的“添加配置”按钮，选择（gdb）启动，生成模板配置：

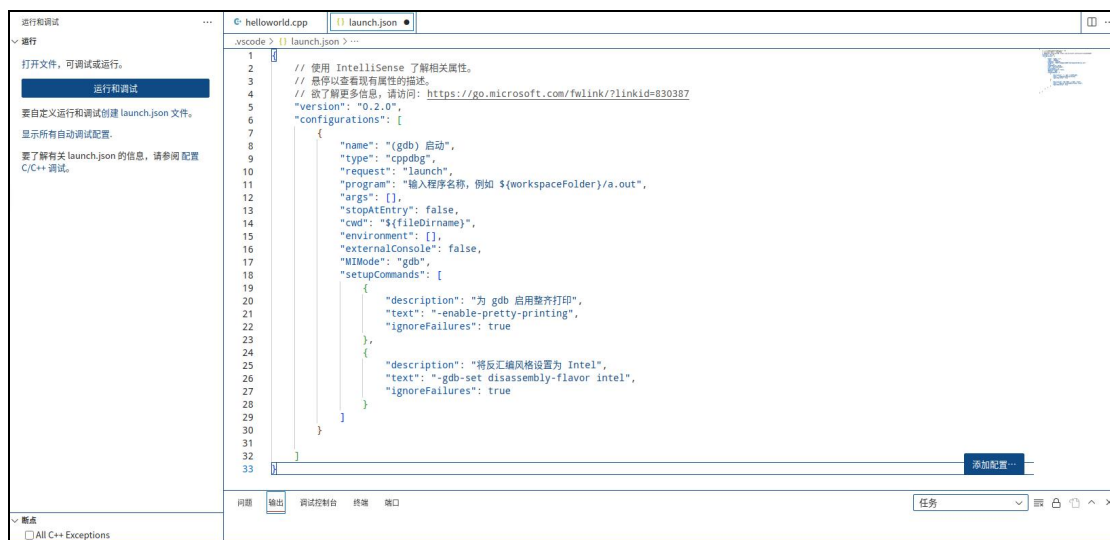


图 26 生成调试模板文件

根据需要配置每个参数，完成配置后 F5 启动调试。

3.5.3.4. 常见问题

- 为什么 VS Code 显示异常？

VS Code 使用的 Electron shell 在某些 GPU（图形处理单元）硬件加速方面存在问题。如果 VS Code 显示空白（空）主窗口，可以尝试在启动 VS Code 时通过添加 Electron `--disable-gpu` 命令行开关来禁用 GPU 加速。

```
code --disable-gpu
```

如果在更新后发生这种情况，删除 GPU Cache 目录可以解决问题。

```
rm -r ~/.config/Code/GPUCache
```

3.5.4. Eclipse

Eclipse 是一款跨平台集成开发环境，由 Eclipse 基金会开发，支持多种编程语言，包括 Java、C++ 和 Python 等。它主要用于开发桌面应用和 Web 应用等，特点是强大的插件系统，用于满足不同开发者的需求。同时，它还提供了强大的调试器和版本控制工具，可以帮助开发者更加高效地进行开发。

3.5.4.1. 环境安装

打开终端，使用以下命令安装 Eclipse：

```
sudo apt install org.eclipse.eclipse-java
```

也可以直接从商店安装：打开应用商店>输入“Eclipse”>根据需要选择合适的版本。

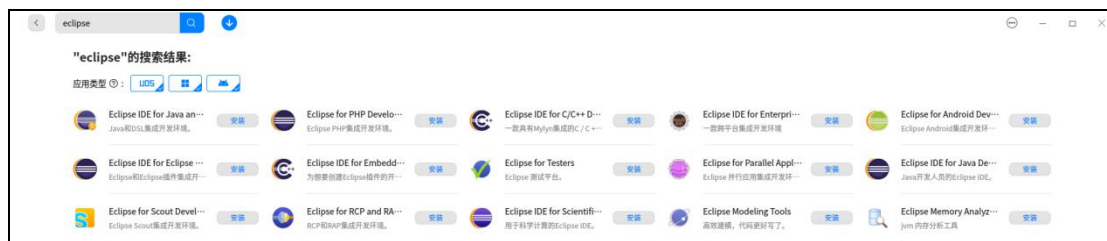


图 27 Eclipse 商店下载界面

也可以从官网下载安装：访问官网 <https://www.Eclipse.org/downloads/> 下载 Eclipse 安装程序。

3.5.4.2. 环境配置

- 工作空间配置。

Eclipse 工作空间包含以下资源：项目、文件、文件夹。项目启动时一般可

以设置工作空间，建议将其设置为默认工作空间，下次启动后无需再配置：

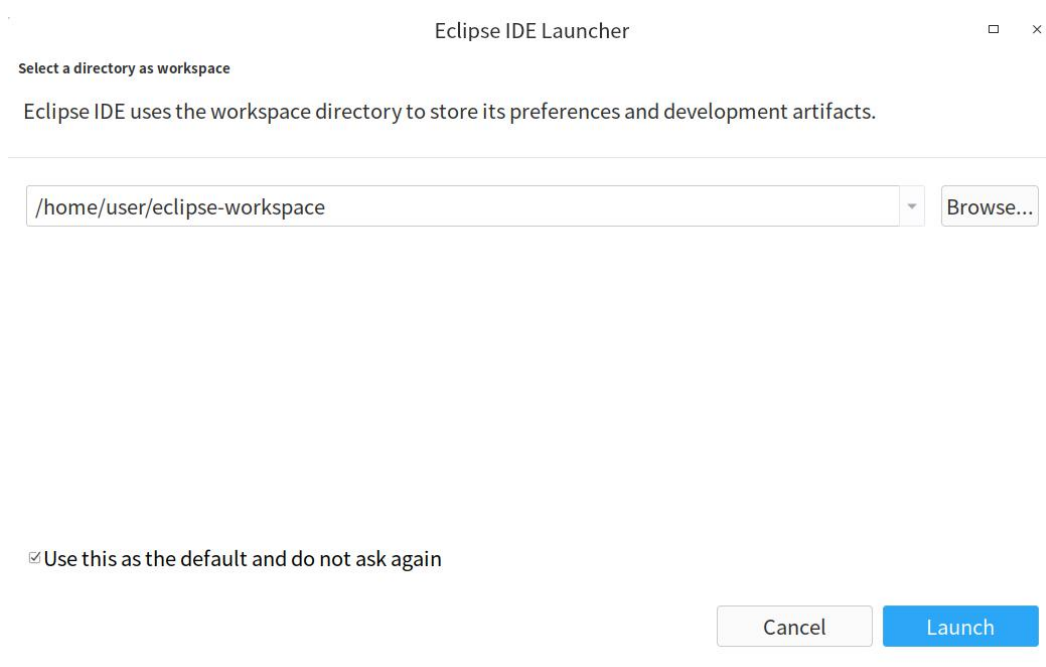


图 28 Eclipse 工作空间配置

工作空间(Workspace)有明显的层次结构。项目在最顶级，里面有文件和文件夹。可以通过资源插件提供的 API 来管理工作空间的资源。

开发语言插件配置：打开 Help>Eclipse Marketplace，进入 Eclipse Marketplace 搜索页面，根据需要安装辅助开发工具或者语言插件。以下是一些常用语言插件，建议按照括号内的名字进行搜索：

- Java Development Tools (JDT) - 用于 Java 开发的插件。
- C/C++ Development Tools (CDT) - 用于 C 和 C++开发的插件。
- PHP Development Tools (PDT) - 用于 PHP 开发的插件。
- Python Development Tools (PyDev) - 用于 Python 开发的插件。
- Ruby Development Tools (RDT) - 用于 Ruby 开发的插件。
- JavaScript Development Tools (JSDT) - 用于 JavaScript 开发的插件。

- Android Development Tools (ADT) - 用于 Android 应用程序开发的插件。

3.5.4.3. 开发案例

下面以一个 C++工程为例进行说明。首先需要安装对应的语言插件：点击菜单 Help>Eclipse Marketplace，打开 Eclipse Marketplace 搜索界面，输入 CDT 查找到 The Complete Eclipse C/C++ IDE，点击 Install 安装，成功后按照提示重启 Eclipse。

- 编译。

- 1) 通过 File>Open Projects from File System 打开 CMake 工程。
- 2) 在 Project Explorer 中选中当前工程后，点击菜单 File>Properties，根据具体需求设置工程属性。
- 3) 选中工程列表中当前工程的根节点，点击菜单栏中 Project>Build Project 菜单，编译代码。

- 调试与运行。

- 1) 选中当前工程，打开菜单 File>Properties 设置 Run/Debug Setting，点击 New 并选择 C/C++ Application，选择 Project 为当前工程，完成后点击 Apply。
- 2) 在工具栏中选择 Debug 和 “当前工程名” -Configuration，点击 Debug 图标进行调试，可进行设置断点，查看变量，单步运行等；点击 Run 图标则直接运行当前程序。

3.5.4.4. 常见问题

- Eclipse 启动时卡在 “Loading Workbench” 界面怎么办？

可以尝试删除 workspace 目录下的.metadata 文件夹，重新启动 Eclipse。

- Eclipse 中出现 “Out of Memory” 错误怎么办？

可以尝试在 Eclipse 的 Eclipse.ini 文件中增加-Xmx 参数，例如-Xmx1024m，来增加 Eclipse 的内存限制。

3.5.5. Android Studio

Android Studio 是用于开发 Android 应用的官方集成开发环境(IDE)。Android Studio 基于 IntelliJ IDEA 强大的代码编辑器和开发者工具，还提供更多可提高 Android 应用构建效率的功能，例如基于 Gradle 的灵活构建系统、快速且功能丰富的模拟器等。

3.5.5.1. 环境安装

在 UOS 上安装 Android Studio，可以按照以下步骤进行：

- 安装 Java Development Kit (JDK) 。

```
sudo apt install default-jdk
```

- 下载 Android Studio。

可以在官网下载最新版本的 Android Studio：<https://developer.android.com/studio>

- 解压缩下载的文件。

```
tar -xvzf android-studio-2020.3.1.22-linux.tar.gz
```

- 进入解压缩后的目录。

```
cd android-studio/bin
```

- 运行 Android Studio。

```
./studio.sh
```

随后按照安装向导的指示进行安装即可。安装完成后，可以在应用程序菜单中找到 Android Studio，并启动它。

3.5.5.2. 环境配置

- 项目结构设置。

更改 Android Studio 项目的各种设置，依次点击 File>Project Structure>Project Structure，弹出参数设置对话框，按需求配置各项参数。

- 设置 JDK 版本。

Android Studio 2.2 及更高版本捆绑提供了最新版本的 OpenJDK，这是推荐用于 Android 项目的 JDK 版本。如需使用捆绑的 JDK，请执行以下操作：

- 1) 在 Android Studio 中打开项目，然后依次选择 File>Settings...>Build, Execution, Deployment>Build Tools>Gradle。
- 2) 在 Gradle JDK 下，选择 Embedded JDK 选项。

默认情况下，用于编译项目的 Java 语言版本取决于项目的 compileSdkVersion。如有必要，可以通过将以下 CompileOptions 代码块添加到 build.gradle 文件替换默认 Java 版本：

```
android {  
    compileOptions {
```



```
        sourceCompatibility JavaVersion.VERSION_1_6
        targetCompatibility JavaVersion.VERSION_1_6
    }
}
```

3.5.5.3. 开发案例

Android Studio 主要偏重于对各种设备（例如手机、电视、平板）开发的 Android 应用，如果未打开任何工程，可在欢迎界面点击 Start a new Android Studio project 创建新项目。

- 创建项目类型。

New Project>Templates，根据设备类型选择对应的项目模板，Android Studio 会在项目中填充示例代码和资源，以帮助快速开发项目。

- 配置项目。

项目配置参数较多，其中有些参数需要合理设置。首先是软件包名称，在默认情况下，软件包名称会成为项目的命名空间（用于访问项目资源）和项目的应用 ID（用作发布 ID）。另外，需要配置应用支持的最低 API 级别，选择较低的 API 级别时，应用可以使用的新版 Android API 会减少，但能够运行应用的 Android 设备的比例会增大。当选择较高的 API 级别时，情况正好相反。最后是支持库的选择，默认情况下，项目配置为使用 AndroidX 库，此库取代 Android 支持库，如需改用旧版支持库，请选择 Use legacy android.support libraries。

- 构建与运行。

如需构建和运行应用，请按以下步骤操作：

- 1) 在工具栏中，从运行配置菜单中选择应用。

- 2) 从目标设备菜单中，选择要用来运行应用的设备。例如 Android TV、Google Pixel 4XL、Nexus 10 API 33 等。如未配置任何设备，则需要创建 Android 虚拟设备以便使用 Android 模拟器，或连接实体设备。

- 3) 点击 Run 按钮。

如果尝试将项目启动到存在相关错误或警告的设备，Android Studio 会发出警告。图标和样式的变更分成了错误（导致配置失败的设备选择）和警告（可能导致意外行为但仍可运行的设备选择）。

● 调试代码。

在执行调试之前，需要确认执行以下操作：启用设备的调试功能，如使用的是模拟器，则默认情况下启用调试功能，不过对于已连接的设备，需要在设备开发者选项中启用调试功能；运行可调试的 build 变体，使用 build 配置中包含 debuggable true（Kotlin 脚本中 isDebuggable=true）的 build 变体。

接下来按如下方式启动调试会话：

- 1) 在应用代码中设置一些断点。

- 2) 在工具栏中，从目标设备菜单中选择用于调试应用的设备。

- 3) 在工具栏中，点击 Debug 图标。

如果应用已在设备上运行，系统会显示一个对话框询问是否从 “Run”

切换为“Debug”，需要重启设备才能开始调试。如需让同一应用实例保持运行状态，请点击 Cancel Debug，改为将调试程序连接到正在运行的应用。否则，Android Studio 会构建一个 APK，使用调试密钥为其签名，将其安装在选择的设备上，然后运行它。如果“Debug”窗口未打开，依次选择 View>Tool Windows>Debug。

3.5.5.4. 常见问题

- 项目同步失败怎么办？

同步项目时可能收到错误消息：“Connection to the Internet denied.('Permission denied:connect')”，为了消除该错误，请执行以下操作步骤：

- 1) 在 Android Studio 中打开 gradle.properties 文件。
- 2) 将以下代码行添加到该文件：

```
org.gradle.jvmargs=-Djava.net.preferIPv4Stack=true
```

请注意，如已将其他 Gradle JVM 参数添加到 gradle.properties 文件，那么可以将此属性添加到同一代码行，如以下示例中所示：

```
org.gradle.jvmargs=-Xmx2048m -XX:MaxPermSize=512m -Djava.net.preferIPv4Stack=true
```

重启 Android Studio，以使所做的更改生效。

3.6. 小结

在本章节，主要介绍了 Windows 下的常用编程语言在统信操作系统上的使用方式，大部分编程语言在统信操作系统上均有相关支持，其对应情况如下：

表 3 编程语言在统信操作系统上的相关支持

语言	开发难度	工具链	支持架构	主要问题
C、C++	中	GCC、Clang	全架构	支持 Windows 的第三方库较多，迁移较其他编程语言困难，原生开发需重新核对 API。
C#	中	dotnet	X86、ARM、LoogArch	部分架构不支持或支持有限，和 Windows 生态绑定较深，UI 框架依赖大量 Windows 接口。
Python	低	python	全架构	跨平台支持较好，但性能一般。
Java	低	openjdk	全架构	跨平台支持较好，性能依赖平台优化。

第四章 UOS 安全接口开发

4.1. 本章目的

本章的目的是介绍如何在 UOS 系统上开发安全应用程序，重点是使用安全接口进行开发。通过学习本章，读者将了解到在 UOS 操作系统上开发安全应用程序的基本原则和方法，并掌握使用安全接口进行应用程序开发的技巧。同时，本章还将介绍一些常用的安全接口和相关工具，帮助读者理解如何利用这些接口来提高应用程序的安全性。无论是初学者还是有一定经验的开发人员，都可以通过本章的学习，获得在 UOS 系统上开发安全应用程序所需的知识和技能。

4.2. 系统安全接口

为加强对系统资源的访问控制，提高数据安全，增强防恶意攻击的能力，系统提供了安全接口，包含以下功能。

- 进程防杀。
- 设备管控，支持禁用存储、网卡、蓝牙、打印机等设备。
- 文件保护、包保护（防卸载）。
- 虚拟机管控。
- 应用联网管控，管控应用访问网络的权限。
- 内核模块防卸载，防止 ko 模块被卸载。

- 壁纸锁定。
- 网络流量审计。
- 审计，对打印，刻录、软件包安装卸载更新升级、共享、光驱文件拷贝等行为日志审计。
- 其他安全管控功能。

4.2.1. 环境安装

安装和升级安全服务相关包。

```
$ sudo apt install permission-manager permission-manager-helper uos-permissionmanager-auditd deepin-netmonitor-dkms dpkg libdpkg-perl dbus libdbus-1-3 dbus-user-session dbus-x11 libselinux1 selinux-utils python3-selinux selinux-policy-dev selinux-policy-default libburn4 auditd libaudit1 libauparse0 python3-audit libaudit-common udisks2 libudisks2-0 cups coreutils dde-dconfig-daemon dde-daemon dde-display-rules dde-desktop dde-desktop-plugins dde-file-manager dde-file-manager-server-plugins dde-file-manager-common-plugins libdfm-extension libdde-file-manager dde-file-manager-plugins dde-file-manager-daemon-plugins dde-file-manager-preview-plugins dde-disk-mount-plugin dde-kwin libdde-network-core dcc-network-plugin dock-network-plugin dss-network-plugin dde-control-center libdtckcore5 startdde deepin-home-appstore-daemon bamfdaemon libblockdev2 net-tools udev libnss-myhostname systemd-sysv libsystemd0 libpam-systemd libudev1 systemd policycoreutils libdde-network-core dcc-network-plugin dock-network-plugin dss-network-plugin org.deepin.browser net-tools deepin-cert plat-signverify deepin-elf-sign-tool deepin-elf-verify
```

启用安全功能，重启：

```
# dbus-send --print-reply --system --dest=org.deepin.PermissionManager /org/deepin/PermissionManager/PolicyManager org.deepin.PermissionManager.PolicyManager.EnableSecurity
```

4.2.2. 环境配置

系统安全服提供 D-Bus 接口。

可通过 libsystemd sd-bus 库、QT D-Bus、golang dbus 等调用。

4.2.3. 开发案例

开发一个 deb 包，禁用网卡。使用 libsystemd sd-bus 库。

安装编译依赖。

```
$ sudo apt install libsystemd-dev
```

文件结构如下：

```
$ tree ./
./
├── debian
│   ├── changelog
│   ├── compat
│   ├── control
│   └── rules
├── main.cpp
└── Makefile
1 directory, 6 files
```

main.c

```
#include <iostream>
#include <systemd/sd-bus.h>
#define _cleanup_(x) __attribute__((__cleanup__(x)))
#define DBUS_DEVICE_DESTINATION "org.deepin.PermissionManager"
```

```
#define DBUS_DEVICE_PATH "/org/deepin/PermissionManager/DeviceProtect"

#define DBUS_DEVICE_INTERFACE "org.deepin.PermissionManager.DeviceProtect"

using namespace std;
int main(int argc, char *argv[])
{
    cleanup(sd_bus_message_unrefp) sd_bus_message *m = NULL;
    cleanup(sd_bus_error_free) sd_bus_error error = SD_BUS_ERROR_NULL;
    cleanup(sd_bus_close_unrefp) sd_bus *bus = NULL;
    const char *path;
    int r;
    /* Connect to the system bus */
    r = sd_bus_open_system(&bus);
    if (r < 0)
    {
        cout << "Failed to connect to system bus: %m" << endl;
        return r;
    }
    // 调用禁用网卡接口
    r = sd_bus_call_method(bus,
        DBUS_DEVICE_DESTINATION,
        DBUS_DEVICE_PATH,
        DBUS_DEVICE_INTERFACE,
        "DisableNetwork",
        &error,
        &m,
        "s",
        "default");
    if (r < 0)
    {
        cout << " Failed to sd_bus_call_method DisableNetwork: " << error.message << endl;
    }
}
```



```
return r;
}
cout << "Network device disabled" << endl;
return 0;
}
```

Makefile:

```
#!/usr/bin/make -f
all:
    g++ -g main.cpp -o disable_network_demo `pkg-config --cflags --li
bs libsystemd`
clean:
    rm *.o disable_network_demo || true
install:
    mkdir -pv ${DESTDIR}/usr/bin/
    install -v disable_network_demo ${DESTDIR}/usr/bin/
```

debian/changelog:

```
test (0.1.0) unstable; urgency=medium
* init
-- test <test@uninotech.com> Fri, 25 Nov 2023 10:56:37 +08
00
```

debian/control:

```
Source: test
Maintainer: test <test@uninotech.com>
Section: admin
Priority: important
Build-Depends: debhelper (>=9), libsystemd-dev
Standards-Version: 4.3.0
Homepage: https://github.com/hedzr/go-faker
Package: test
Architecture: all
Description: test deb package
```

```
this package disable network device.
```

debian/rules:

```
#!/usr/bin/make -f
%:
dh $@
```

构建 deb 包:

```
# sudo apt build-dep ./
# dpkg-buildpackage -us -uc -rfakeroot -b
```

对 deb 包进行安全签名, 安装, 执行, 查看网卡状态:

```
$ sudo /usr/bin/disable_network_demo
```

4.2.4. 常见问题

- 访问安全 D-Bus 接口报错。

org.freedesktop.DBus.Error.AccessDenied 只有安全签名的应用才允许

访问本安全服务 D-Bus 接口。

4.3. USKI 接口

USKI 接口是针对统信桌面操作系统和服务端操作系统的内核安全接口给出的底层设计与使用文档。文档将给出内核安全接口的系统设计原则基础结构以及具体使用方式, 方便开发者基于此接口快速构造安全产品。USKI 接口通过沿用 LSM 提供的安全接口以确保接口多样性, 通过封装 LSM 调用机制确保 LSM 变更对开发者无感。USKI 接口会适配 LSM 接口变更从而向开发者提供稳定的使用接口, 开发者无需关注于 LSM 底层机制。

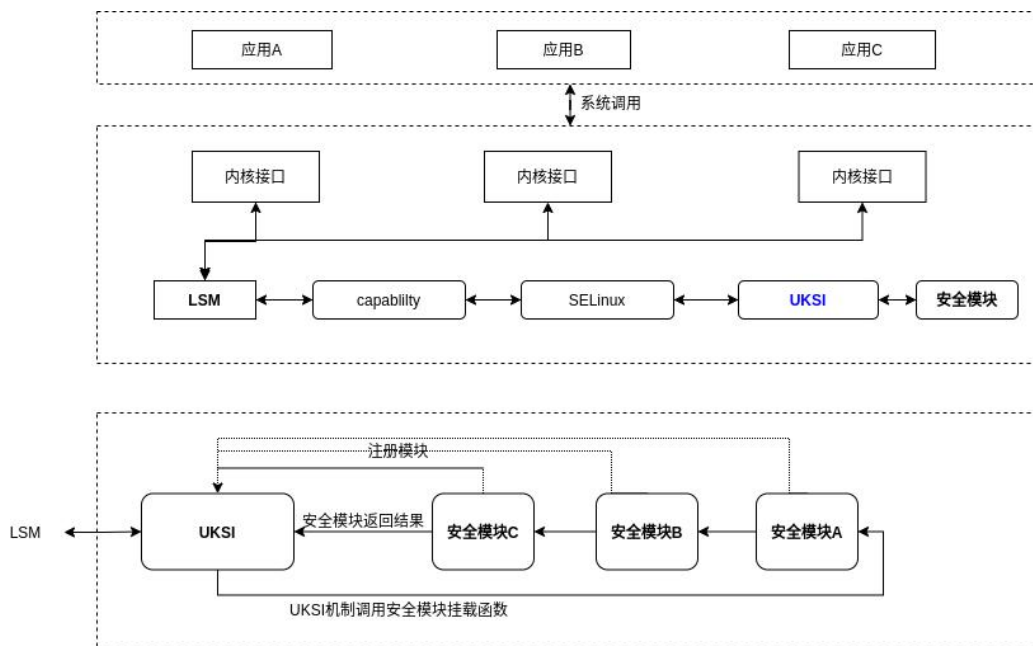


图 29 USKI 架构

USKI 整体设计如图所示，USKI 自身作为特殊的安全模块在内核启动阶段注册到 LSM 子系统，是对外提供管控第三方安全模块的接口，USKI 接口对三方安全模块进行安全检查，成功注册的三方安全模块与内核编译阶段选定的安全模块功能无异，共同作为内核安全机制的一部分。整体而言，USKI 接口具有如下优点：

- 支持同时运行多个安全模块，易于扩充。
- 接口简洁，易于理解和使用。
- 便于以模块方式开发安全模块，易于开发和调试。
- 性能开销低，基于 LSM 实现，运行高效。

4.3.1. 环境安装

无需环境安装。

4.3.2. 环境配置

USKI 提供了以下接口供开发者使用：

- 注册 hook 接口。

```
/*
 * @UOS_HOOK_LIST hook id 显式指定需要注册的 hook id
 * @struct uos_hook_cb_entry *entry 用户描述 hook 函数基地址、参
数个数以及返回值类型等信息
 */
int uos_hook_register(enum UOS_HOOK_LIST hook_id, struct uos_
hook_cb_entry *entry);
```

- 注销 hook 接口。

```
/*
 * @UOS_HOOK_LIST hook id 显式指定需要注销的 hook id
 * @char *owner 当前 hook id 对应 hook 所属的模块名称
 */
int uos_hook_cancel(enum UOS_HOOK_LIST hook_id, char *owne
r);
```

- 相关结构定义如下：

```
struct uos_hook_cb_entry {
    char *owner; // hook 函数对应的内核模块名称，注册时强制要求与 k
o 名称保持一致
    unsigned long cb_addr; // hook 函数的回调地址
    enum UOS_HOOK_RETURN_TYPE ret_type; // hook 函数的返回值
类型
    unsigned int arg_len; // hook 函数的入参个数
};

enum UOS_HOOK_RETURN_TYPE {
    UOS_HOOK_RET_TY_NONE, //标识 hook 函数为无返回值类型
    UOS_HOOK_RET_TY_INT, //标识 hook 函数为整形返回值类型
};
```

```
};
```

USKI 对外提供简洁的 hook 注册/注销接口，开发者仅仅需要按照开发规范调用 `uos_hook_register()` 接口完成 hook 注册，与之对应开发者仅仅需要调用 `uos_hook_cancel()` 接口即可注销对应 hook。

USKI 基于 LSM 针对常见场景支持安全检查函数的动态 hook，当前支持挂载的函数列表如下：

表 4 USKI 当前支持 hook 的安全检查函数

USKI hook_id	对应 LSM 安全检查函数
UOS_PTRACE_ACCESS_CHECK	<code>security_ptrace_access_check()</code>
UOS_PTRACE_TRACEME	<code>security_ptrace_traceme()</code>
UOS_CAPGET	<code>security_capget()</code>
UOS_CAPSET	<code>security_capset()</code>
UOS_CAPABLE	<code>security_capable()</code>
UOS_QUOTACTL	<code>security_quotactl()</code>
UOS_QUOTA_ON	<code>security_quota_on()</code>
UOS_SYSLOG	<code>security_syslog()</code>
UOS_SETTIME	<code>security_settime()</code>
UOS_VM_ENOUGH_MEMORY	<code>security_vm_enough_memory()</code>
UOS_BPRM_CHECK_SECURITY	<code>security_bprm_check_security()</code>
UOS_BPRM_COMMITTING_CREDS	<code>security_bprm_committing_creds()</code>
UOS_BPRM_COMMITTED_CREDS	<code>security_bprm_committed_creds()</code>
UOS_SB_ALLOC_SECURITY	<code>security_sb_alloc_security()</code>
UOS_SB_SHOW_OPTIONS	<code>security_sb_show_options()</code>
UOS_SB_STATFS	<code>security_sb_statfs()</code>

USKI hook_id	对应 LSM 安全检查函数
UOS_SB_MOUNT	security_sb_mount()
UOS_SB_UMOUNT	security_sb_umount()
UOS_SB_PIVOTROOT	security_sb_pivotroot()
UOS_SB_CLONE_MNT_OPTS	security_sb_clone_mnt_opts()
UOS_DENTRY_CREATE_FILES_AS	security_dentry_create_files_as()
UOS_LOCKED_DOWN	security_locked_down()
UOS_PATH_UNLINK	security_path_unlink()
UOS_PATH_MKDIR	security_path_mkdir()
UOS_PATH_RMDIR	security_path_rmdir()
UOS_PATH_MKNOD	security_path_mknod()
UOS_PATH_TRUNCATE	security_path_truncate()
UOS_PATH_SYMLINK	security_path_symlink()
UOS_PATH_LINK	security_path_link()
UOS_PATH_RENAME	security_path_rename()
UOS_PATH_CHMOD	security_path_chmod()
UOS_PATH_CHOWN	security_path_chown()
UOS_PATH_CHROOT	security_path_chroot()
UOS_INODE_ALLOC_SECURITY	security_inode_alloc_security()
UOS_INODE_FREE_SECURITY	security_inode_free_security()
UOS_INODE_INIT_SECURITY	security_inode_init_security()
UOS_INODE_CREATE	security_inode_create()
UOS_INODE_LINK	security_inode_link()
UOS_INODE_UNLINK	security_inode_unlink()
UOS_INODE_SYMLINK	security_inode_symlink()

USKI hook_id	对应 LSM 安全检查函数
UOS_INODE_MKDIR	security_inode_mkdir()
UOS_INODE_RMDIR	security_inode_rmdir()
UOS_INODE_MKNOD	security_inode_mknod()
UOS_INODE_RENAME	security_inode_rename()
UOS_INODE_READLINK	security_inode_readlink()
UOS_INODE_FOLLOW_LINK	security_inode_follow_link()
UOS_INODE_PERMISSION	security_inode_permission()
UOS_INODE_SETATTR	security_inode_setattr()
UOS_INODE_GETATTR	security_inode_getattr()
UOS_INODE_SETXATTR	security_inode_setxattr()
UOS_INODE_POST_SETXATTR	security_inode_post_setxattr()
UOS_INODE_GETXATTR	security_inode_getxattr()
UOS_INODE_LISTXATTR	security_inode_listxattr()
UOS_INODE_REMOVEEXATTR	security_inode_removeexattr()
UOS_INODE_NEED_KILLPRIV	security_inode_need_killpriv()
UOS_INODE_KILLPRIV	security_inode_killpriv()
UOS_INODE_GETSECURITY	security_inode_getsecurity()
UOS_INODE_SETSECURITY	security_inode_setsecurity()
UOS_INODE_LISTSECURITY	security_inode_listsecurity()
UOS_INODE_GETSECID	security_inode_getsecid()
UOS_INODE_COPY_UP	security_inode_copy_up()
UOS_INODE_COPY_UP_XATTR	security_inode_copy_up_xattr()
UOS_FILE_PERMISSION	security_file_permission()
UOS_FILE_ALLOC_SECURITY	security_file_alloc_security()

USKI hook_id	对应 LSM 安全检查函数
UOS_FILE_FREE_SECURITY	security_file_free_security()
UOS_FILE_IOCTL	security_file_ioctl()
UOS_MMAP_ADDR	security_mmap_addr()
UOS_MMAP_FILE	security_mmap_file()
UOS_FILE_MPROTECT	security_file_mprotect()
UOS_FILE_LOCK	security_file_lock()
UOS_FILE_FCNTL	security_file_fcntl()
UOS_FILE_SET_FOWNER	security_file_set_fowner()
UOS_FILE_RECEIVE	security_file_receive()
UOS_FILE_OPEN	security_file_open()
UOS_TASK_FREE	security_task_free()
UOS_CRED_ALLOC_BLANK	security_cred_alloc_blank()
UOS_CRED_FREE	security_cred_free()
UOS_CRED_PREPARE	security_cred_prepare()
UOS_CRED_TRANSFER	security_cred_transfer()
UOS_CRED_GETSECID	security_cred_getsecid()
UOS_KERNEL_ACT_AS	security_kernel_act_as()
UOS_KERNEL_CREATE_FILES_AS	security_kernel_create_files_as()
UOS_KERNEL_MODULE_REQUEST	security_kernel_module_request()
UOS_KERNEL_POST_READ_FILE	security_kernel_post_read_file()
UOS_TASK_FIX_SETUID	security_task_fix_setuid()
UOS_TASK_SETPGID	security_task_setpgid()
UOS_TASK_GETPGID	security_task_getpgid()
UOS_TASK_GETSID	security_task_getsid()

USKI hook_id	对应 LSM 安全检查函数
UOS_TASK_SETNICE	security_task_setnice()
UOS_TASK_SETIOPRIO	security_task_setioprio()
UOS_TASK_GETIOPRIO	security_task_getioprio()
UOS_TASK_PRLIMIT	security_task_prlimit()
UOS_TASK_SETRLIMIT	security_task_setrlimit()
UOS_TASK_SETSCHEDULER	security_task_setscheduler()
UOS_TASK_GETSCHEDULER	security_task_getscheduler()
UOS_TASK_MOVEMEMORY	security_task_movememory()
UOS_TASK_PRCTL	security_task_prctl()
UOS_TASK_TO_NODE	security_task_to_node()
UOS_IPC_PERMISSION	security_ipc_permission()
UOS_IPC_GETSECID	security_ipc_getsecid()
UOS_MSG_MSG_ALLOC_SECURITY	security_msg_msg_alloc_security()
UOS_MSG_MSG_FREE_SECURITY	security_msg_msg_free_security()
UOS_MSG_QUEUE_ALLOC_SECURITY	security_msg_queue_alloc_security()
UOS_MSG_QUEUE_FREE_SECURITY	security_msg_queue_free_security()
UOS_MSG_QUEUE_ASSOCIATE	security_msg_queue_associate()
UOS_MSG_QUEUE_MSGCTL	security_msg_queue_msgctl()
UOS_MSG_QUEUE_MSGSND	security_msg_queue_msgsnd()
UOS_MSG_QUEUE_MSGRCV	security_msg_queue_msgrcv()
UOS_SHM_ALLOC_SECURITY	security_shm_alloc_security()
UOS_SHM_FREE_SECURITY	security_shm_free_security()
UOS_SHM_ASSOCIATE	security_shm_associate()
UOS_SHM_SHMCTL	security_shm_shmctl()

USKI hook_id	对应 LSM 安全检查函数
UOS_SHM_SHMAT	security_shm_shmat()
UOS_SEM_ALLOC_SECURITY	security_sem_alloc_security()
UOS_SEM_FREE_SECURITY	security_sem_free_security()
UOS_SEM_ASSOCIATE	security_sem_associate()
UOS_SEM_SEMCTL	security_sem_semctl()
UOS_SEM_SEMOP	security_sem_semop()
UOS_NETLINK_SEND	security_netlink_send()
UOS_D_INSTANTIATE	security_d_instantiate()
UOS_ISMACLABEL	security_ismaclabel()
UOS_SECID_TO_SECCTX	security_secid_to_secctx()
UOS_SECCTX_TO_SECID	security_secctx_to_secid()
UOS_RELEASE_SECCTX	security_release_secctx()
UOS_INODE_INVALIDATE_SECCTX	security_inode_invalidate_secctx()
UOS_UFILE_CLOSE	uos_file_free()
UOS_INODE_NOTIFYSECCTX	security_inode_notifysecctx()
UOS_INODE_SETSECCTX	security_inode_setsecctx()
UOS_UNIX_STREAM_CONNECT	security_unix_stream_connect()
UOS_UNIX_MAY_SEND	security_unix_may_send()
UOS_SOCKET_CREATE	security_socket_create()
UOS_SOCKET_POST_CREATE	security_socket_post_create()
UOS_SOCKET_SOCKETPAIR	security_socket_socketpair()
UOS_SOCKET_BIND	security_socket_bind()
UOS_SOCKET_CONNECT	security_socket_connect()
UOS_SOCKET_LISTEN	security_socket_listen()

USKI hook_id	对应 LSM 安全检查函数
UOS_SOCKET_ACCEPT	security_socket_accept()
UOS_SOCKET_SENDMSG	security_socket_sendmsg()
UOS_SOCKET_RECVMSG	security_socket_recvmsg()
UOS_SOCKET_GETSOCKNAME	security_socket_getsockname()
UOS_SOCKET_GETPEERNAME	security_socket_getpeername()
UOS_SOCKET_GETSOCKOPT	security_socket_getsockopt()
UOS_SOCKET_SETSOCKOPT	security_socket_setsockopt()
UOS_SOCKET_SHUTDOWN	security_socket_shutdown()
UOS_SOCKET_RCV_SKB	security_socket_rcv_skb()
UOS_SOCKET_GETPEERSEC_STREAM	security_socket_getpeersec_stream()
UOS_SK_ALLOC_SECURITY	security_sk_alloc_security()
UOS_SK_FREE_SECURITY	security_sk_free_security()
UOS_SK_CLONE_SECURITY	security_sk_clone_security()
UOS_INET_CSK_CLONE	security_inet_csk_clone()
UOS_SK_GETSECID	security_sk_getsecid()
UOS SOCK_GRAFT	security_sock_graft()
UOS_INET_CONN_REQUEST	security_inet_conn_request()
UOS_INET_CONN_ESTABLISHED	security_inet_conn_established()
UOS_SECMARK_RELABEL_PACKET	security_secmark_relabel_packet()
UOS_SECMARK_REFCOUNT_INC	security_secmark_refcount_inc()
UOS_SECMARK_REFCOUNT_DEC	security_secmark_refcount_dec()
UOS_REQ_CLASSIFY_FLOW	security_req_classify_flow()
UOS_TASK_KILL	security_task_kill()
[4.19/5.10 不兼容] UOS_TUN_DEV_ALLOC_SECURITY	security_tun_dev_alloc_security()

USKI hook_id	对应 LSM 安全检查函数
UOS_TUN_DEV_FREE_SECURITY	security_tun_dev_free_security()
UOS_TUN_DEV_CREATE	security_tun_dev_create()
UOS_TUN_DEV_ATTACH_QUEUE	security_tun_dev_attach_queue()
UOS_TUN_DEV_ATTACH	security_tun_dev_attach()
UOS_TUN_DEV_OPEN	security_tun_dev_open()
UOS_SCTP_ASSOC_REQUEST	security_sctp_assoc_request()
UOS_SCTP_BIND_CONNECT	security_sctp_bind_connect()
UOS_SCTP_SK_CLONE	security_sctp_sk_clone()
UOS_XFRM_POLICY_ALLOC_SECURITY	security_xfrm_policy_alloc_security()
UOS_XFRM_POLICY_CLONE_SECURITY	security_xfrm_policy_clone_security()
UOS_XFRM_POLICY_FREE_SECURITY	security_xfrm_policy_free_security()
UOS_XFRM_POLICY_DELETE_SECURITY	security_xfrm_policy_delete_security()
UOS_XFRM_STATE_ALLOC	security_xfrm_state_alloc()
UOS_XFRM_STATE_ALLOC_ACQUIRE	security_xfrm_state_alloc_acquire()
UOS_XFRM_STATE_FREE_SECURITY	security_xfrm_state_free_security()
UOS_XFRM_STATE_DELETE_SECURITY	security_xfrm_state_delete_security()
UOS_XFRM_POLICY_LOOKUP	security_xfrm_policy_lookup()
UOS_XFRM_DECODE_SESSION	security_xfrm_decode_session()
UOS_KEY_ALLOC	security_key_alloc()
UOS_KEY_FREE	security_key_free()
UOS_KEY_GETSECURITY	security_key_getsecurity()
UOS_AUDIT_RULE_INIT	security_audit_rule_init()
UOS_AUDIT_RULE_KNOWN	security_audit_rule_known()
UOS_AUDIT_RULE_FREE	security_audit_rule_free()

USKI hook_id	对应 LSM 安全检查函数
UOS_BPF	security_bpf()
UOS_BPF_MAP	security_bpf_map()
UOS_BPF_PROG	security_bpf_prog()
UOS_BPF_MAP_ALLOC_SECURITY	security_bpf_map_alloc_security()
UOS_BPF_MAP_FREE_SECURITY	security_bpf_map_free_security()
UOS_BPF_PROG_ALLOC_SECURITY	security_bpf_prog_alloc_security()
UOS_BPF_PROG_FREE_SECURITY	security_bpf_prog_free_security()

第一列为 USKI 接口定义的 hook_id，名称与 LSM 安全检查函数名称保持对应，调用 uos_hook_register()接口时需通过 hook_id 显式指定对应 LSM 安全检查函数。第二列为 LSM 机制预留在内核关键代码路径上的安全检查函数，该函数被触发时依次调用各 LSM 安全模块对应的 hook 函数，最终调用通过 USKI 注册的 hook 函数。

4.3.3. 开发案例

本章节介绍 USKI 接口的基本使用方法，请注意 USKI 本身只提供管理三方安全模块注册注销的能力，不完善的安全模块可能导致系统陷入不稳定或者崩溃的状态。开发者可以按照如下步骤开发安全模块：

- 确认 USKI 接口已启用。

USKI 接口默认启用，自身作为 LSM 安全模块在内核启动阶段完成初始化，开发者进入系统后可通过执行 cat/sys/kernel/security/lsm 确认 USKI 是否启用，如果输出中有 uosmanager 字样代表 USKI 接口已启用，否则

说明没有启用。

- 构造安全模块。

开发者以 LKM 形式构造安全模块，通过 USKI 提供的注册/注销接口构造注册 hook 函数。可参考如下示例代码进行开发。

// hook_demo.c

```
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/lsm_uos_hook_manager.h>
#include <linux/fs_struct.h>
#include <linux/version.h>
#define MODULE_NAME "hook_demo"
typedef struct td_hook_entry {
    int hook_id;
    struct uos_hook_cb_entry cb;
} td_hook_entry_t;
int td_inode_alloc_security(struct inode * inode)
{
    pr_info("current: %s %s: test\n", current->comm, __FUNCTION__);
    return 0;
}

static td_hook_entry_t entries[] = {
{
    .hook_id = UOS_INODE_ALLOC_SECURITY,
    .cb =
    {
        .owner = MODULE_NAME, //请确保.onwer 值与 ko 名称相同
        .cb_addr = (unsigned long)td_inode_alloc_security,
        .ret_type = UOS_HOOK_RET_TY_INT,
```

```
.arg_len = 1,
},
},
{
.hook_id = UOS_HOOK_NONE,
},
};

int td_uos_manager_register_hook(void)
{
int i = 0, j = 0;
int error = 0;
for (; entries[i].hook_id != UOS_HOOK_NONE; i++) {
error = uos_hook_register(entries[i].hook_id, &entries[i].cb);
if (error) {
pr_err("Failed to register hook %d, hook_id = %d\n", i, entries[i].hook_id);
break;
}
}
if (entries[i].hook_id == UOS_HOOK_NONE)
return 0;
for (; j < i; j++) {
error = uos_hook_cancel(entries[j].hook_id, entries[j].cb.owner);
if (error)
pr_err("Failed to cancel hook %d\n", j);
}
return -1;
}

void td_uos_manager_cancel_hook(void)
{
int i = 0;
int error = 0;
for (; entries[i].hook_id != UOS_HOOK_NONE; i++) {
error = uos_hook_cancel(entries[i].hook_id, entries[i].cb.owner);
```

```
    if (error) {
        pr_err("Failed to cancel hook %d, hook_id = %d\n", i, entries[i].hook_id);
    }
}

static int __init uski_demo_init(void)
{
    int error = 0;
    pr_info("register hooks\n");
    error = td_uos_manager_register_hook();
    if (error) {
        pr_err("Failed to register hooks\n");
        return -1;
    }
    pr_info("register hooks,Done!\n");
    return 0;
}

static void __exit uski_demo_exit(void)
{
    td_uos_manager_cancel_hook();
    pr_info("remove hooks,Done!\n");
}

module_init(uski_demo_init);
module_exit(uski_demo_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("caokewu@uniontech.com");
MODULE_DESCRIPTION("A simple demo for USKI");
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/lsm_uos_hook_manager.h>
#include <linux/fs_struct.h>
#include <linux/version.h>
```



```
#define MODULE_NAME "hook_demo"
typedef struct td_hook_entry {
    int hook_id;
    struct uos_hook_cb_entry cb;
} td_hook_entry_t;
int td_inode_alloc_security(struct inode * inode)
{
    pr_info("current: %s %s: test\n", current->comm, __FUNCTION__);
    return 0;
}
static td_hook_entry_t entries[] = {
    {
        .hook_id = UOS_INODE_ALLOC_SECURITY,
        .cb =
        {
            .owner = MODULE_NAME, //请确保.onwer 值与 ko 名称相同
            .cb_addr = (unsigned long)td_inode_alloc_security,
            .ret_type = UOS_HOOK_RET_TY_INT,
            .arg_len = 1,
        },
    },
    {
        .hook_id = UOS_HOOK_NONE,
    },
};
int td_uos_manager_register_hook(void)
{
    int i = 0, j = 0;
    int error = 0;
    for (; entries[i].hook_id != UOS_HOOK_NONE; i++) {
        error = uos_hook_register(entries[i].hook_id, &entries[i].cb);
        if (error) {
            pr_err("Failed to register hook %d, hook_id = %d\n", i, entries[i].hook_id);
        }
    }
}
```

```
break;
}
}
if (entries[i].hook_id == UOS_HOOK_NONE)
return 0;
for (; j < i; j++) {
error = uos_hook_cancel(entries[j].hook_id,entries[j].cb.owner);
if (error)
pr_err("Failed to cancel hook %d\n", j);
}
return -1;
}
void td_uos_manager_cancel_hook(void)
{
int i = 0;
int error = 0;
for (; entries[i].hook_id != UOS_HOOK_NONE; i++) {
error = uos_hook_cancel(entries[i].hook_id,entries[i].cb.owner);
if (error) {
pr_err("Failed to cancel hook %d, hook_id = %d\n", i, entries[i].hook_id);
}
}
}
static int __init uski_demo_init(void)
{
int error = 0;
pr_info("register hooks\n");
error = td_uos_manager_register_hook();
if (error) {
pr_err("Failed to register hooks\n");
return -1;
}
pr_info("register hooks,Done!\n");
return 0;
}
```

```
}
static void __exit uski_demo_exit(void)
{
    td_uos_manager_cancel_hook();
    pr_info("remove hooks,Done!\n");
}
module_init(uski_demo_init);
module_exit(uski_demo_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("caokewu@uniontech.com");
MODULE_DESCRIPTION("A simple demo for USKI");
```

Makefile:

```
ifneq($(KERNELRELEASE))
obj-m:=hook_demo.o
else
KDIR:= /lib/modules/$(shell uname -r)/build
PWD:= $(shell pwd)
all:
make -C $(KDIR) M=$(PWD) modules
clean:
make -C $(KDIR) M=$(PWD) clean
endif
```

- 注册/注销安全模块。

编写完内核模块源码和 Makefile 文件后，执行 `make && sudo insmod hook_demo.ko` 注册安全模块，在特定场景下安全模块挂载的 hook 函数会被自动调用。执行 `sudo dmesg` 可观察到如图所示输出，说明安全模块中注册的 `td_inode_alloc_security()` 函数已被触发。

```
[ 81.352411] current: systemd-journal td_inode_alloc_security: test
[ 81.352416] current: systemd-journal td_inode_alloc_security: test
[ 81.352453] current: systemd-journal td_inode_alloc_security: test
[ 81.352474] current: systemd-journal td_inode_alloc_security: test
[ 81.352498] current: systemd-journal td_inode_alloc_security: test
[ 81.352509] current: sudo td_inode_alloc_security: test
[ 81.352597] current: sudo td_inode_alloc_security: test
[ 81.352690] current: sudo td_inode_alloc_security: test
[ 81.352714] current: sudo td_inode_alloc_security: test
[ 81.353367] current: sudo td_inode_alloc_security: test
[ 81.353556] current: sudo td_inode_alloc_security: test
[ 81.353561] current: sudo td_inode_alloc_security: test
[ 81.353579] current: sudo td_inode_alloc_security: test
[ 81.353583] current: sudo td_inode_alloc_security: test
[ 81.353586] current: sudo td_inode_alloc_security: test
[ 81.353588] current: sudo td_inode_alloc_security: test
[ 81.353591] current: sudo td_inode_alloc_security: test
[ 81.353594] current: sudo td_inode_alloc_security: test
[ 81.353596] current: sudo td_inode_alloc_security: test
[ 81.353599] current: sudo td_inode_alloc_security: test
[ 81.358658] current: UMDnsService td_inode_alloc_security: test
[ 81.358681] current: UMDnsService td_inode_alloc_security: test
[ 81.358811] current: UMDnsService td_inode_alloc_security: test
[ 81.358838] current: UMDnsService td_inode_alloc_security: test
[ 81.358884] current: UMDnsService td_inode_alloc_security: test
[ 81.358925] current: UMDnsService td_inode_alloc_security: test
[ 81.358933] current: UMDnsService td_inode_alloc_security: test
[ 81.358936] current: UMDnsService td_inode_alloc_security: test
[ 81.358939] current: UMDnsService td_inode_alloc_security: test
```

图 30 注册 hook

执行 `sudo rmmod hook_demo` 移除 hook_demo 模块，本例中该模块被移除时会调 USKI 提供的 `uos_hook_cancel()` 接口注销 hook，注销成功时会输出 `remove hooks,Done`，如图所示：

```
[ 341.359092] current: sudo td_inode_alloc_security: test
[ 341.359121] current: sudo td_inode_alloc_security: test
[ 341.359130] current: sudo td_inode_alloc_security: test
[ 341.359132] current: sudo td_inode_alloc_security: test
[ 341.359289] current: systemd-journal td_inode_alloc_security: test
[ 341.359356] current: systemd-journal td_inode_alloc_security: test
[ 341.359371] current: sudo td_inode_alloc_security: test
[ 341.359386] current: systemd-journal td_inode_alloc_security: test
[ 341.359433] current: systemd-journal td_inode_alloc_security: test
[ 341.359437] current: systemd-journal td_inode_alloc_security: test
[ 341.359468] current: systemd-journal td_inode_alloc_security: test
[ 341.359478] current: sudo td_inode_alloc_security: test
[ 341.359489] current: systemd-journal td_inode_alloc_security: test
[ 341.359511] current: systemd-journal td_inode_alloc_security: test
[ 341.359574] current: sudo td_inode_alloc_security: test
[ 341.359599] current: sudo td_inode_alloc_security: test
[ 341.360192] current: sudo td_inode_alloc_security: test
[ 341.360425] current: sudo td_inode_alloc_security: test
[ 341.360427] current: sudo td_inode_alloc_security: test
[ 341.360437] current: sudo td_inode_alloc_security: test
[ 341.360443] current: sudo td_inode_alloc_security: test
[ 341.360446] current: sudo td_inode_alloc_security: test
[ 341.360448] current: sudo td_inode_alloc_security: test
[ 341.360451] current: sudo td_inode_alloc_security: test
[ 341.360454] current: sudo td_inode_alloc_security: test
[ 341.360458] current: sudo td_inode_alloc_security: test
[ 341.360461] current: sudo td_inode_alloc_security: test
[ 341.362322] current: rmmod td_inode_alloc_security: test
[ 341.362340] exit the uos hook demo
```

图 31 注销 hook

4.3.4. 常见问题

- 4.19 与 5.10 内核不兼容。

代表此 hook 接口存在兼容性风险，以 UOS_TASK_KILL & security_task_kill()为例：4.19 内核对应的函数原型为 `int security_task_kill(struct task_struct *p, struct siginfo *info, int sig, const struct cred *cred)`，而 5.10 内核对应的函数原型为 `int security_task_kill(struct task_struct *p, struct kernel_siginfo *info, int sig, const struct cred *cred)`，第二个参数类型有改变。编写基于 USKI 的安全模块时，可通过条件编译解决内核版本不同引发的兼容性问题。

- hook_demo.c 中 ko 注册失败。

需要确保模块源码中的 MODULE_NAME 与 Makefile 中指定的目标文件名相同。最新版本的 USKI 接口强制要求 `td_hook_entry_t.cb.owner` 的值与当前 ko 名称保持一致才能注册成功。

- uname -r 函数定义失败。

`/include/linux/lsm_uos_hook_manager.h` 中定义了当前环境下能允许注册的 HOOK_ID，实际编写安全模块时请以此文件为准；对应 LSM 函数原型定义可以参考 `/usr/src/linux-headers-`uname -r`/include/linux/lsm_hooks.h`。

4.4. pam 认证接口

PAM 全称为“可插拔认证模块 (Pluggable Authentication Modules)”。它通过提供一些动态库以及一套通用接口和流程实现身份认证。用户可以根据需要给不同的服务配置不同的认证方式而无需更改服务程序,同时也可以向系统中添加新的认证手段,实现差异化需求。

4.4.1. 环境安装

若使用 PAM 开发库,则需要安装 libpam0g-dev 开发库,安装命令如下:

```
sudo apt install libpam0g-dev
```

4.4.2. 环境配置

无需特殊环境配置。

4.4.3. 开发案例

自定义 PAM 身份验证,若使用自定义 PAM 身份验证,需要提供自定义 PAM 身份认证插件,示例代码如下:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <security/pam_appl.h>
#include <security/pam_modules.h>
// 身份验证函数
PAM_EXTERN int pam_sm_authenticate(pam_handle_t *pamh, int flags, int argc, const char **argv) {
    const char *username;
    const char *password;
```

```
struct pam_conv *conv;
struct pam_message msg;
const struct pam_message *msgs[1];
struct pam_response *resp;
// 获取用户名
if (pam_get_user(pamh, &username, "Username: ") != PAM_SUCCESS || username == NULL) {
    return PAM_AUTH_ERR;
}
// 获取 PAM 会话交互结构体
if (pam_get_item(pamh, PAM_CONV, (const void **)&conv) != PAM_SUCCESS || conv == NULL) {
    return PAM_AUTH_ERR;
}
// 准备要传递给 conv 函数的消息
msg.msg_style = PAM_PROMPT_ECHO_OFF;
msg.msg = "Password: ";
msgs[0] = &msg;
resp = NULL;

// 调用 conv 函数与用户交互获取密码
int retval = conv->conv(1, msgs, &resp, conv->appdata_ptr);
if (retval != PAM_SUCCESS || resp == NULL || resp->resp == NULL) {
    return PAM_AUTH_ERR;
}
password = resp->resp;
// 在实际应用中，这里应该有密码验证逻辑
// 这里只是一个简单的示例，假定密码为"password123"
if (strcmp(password, "password123") == 0) {
    return PAM_SUCCESS; // 密码匹配成功
} else {
    return PAM_AUTH_ERR; // 密码错误
}
```

```
}
```

CMAKE 如下：

```
cmake_minimum_required(VERSION 3.0)
project(my_custom_pam_module C)
# 设置源文件
set(SOURCE_FILES pam.c)
# 包含 PAM 头文件路径
find_path(PAM_INCLUDE_DIR security/pam_modules.h)
include_directories(${PAM_INCLUDE_DIR})
# 生成共享库
add_library(pam_demo SHARED ${SOURCE_FILES})
SET_TARGET_PROPERTIES(pam_demo PROPERTIES PREFIX "")
```

将生成的 pam_demo.so 放在/usr/lib/x86_64-linux-gnu/security 下（amd 在次路径，其它架构 man pam）

编写自定义 pam 配置文件：例如在/etc/pam.d/目录下生成文件 demo，示例如下：

auth	required	pam_demo.so
------	----------	-------------

编写自定义调用程序代码，示例如下：

```
#include <security/pam_appl.h>
#include <security/pam_misc.h>
#include <stdio.h>
// gcc main.c -o main -lpam -lpam_misc
static struct pam_conv conv = {
    misc_conv,
    NULL
};
int main(int argc, char *argv[])
{
```



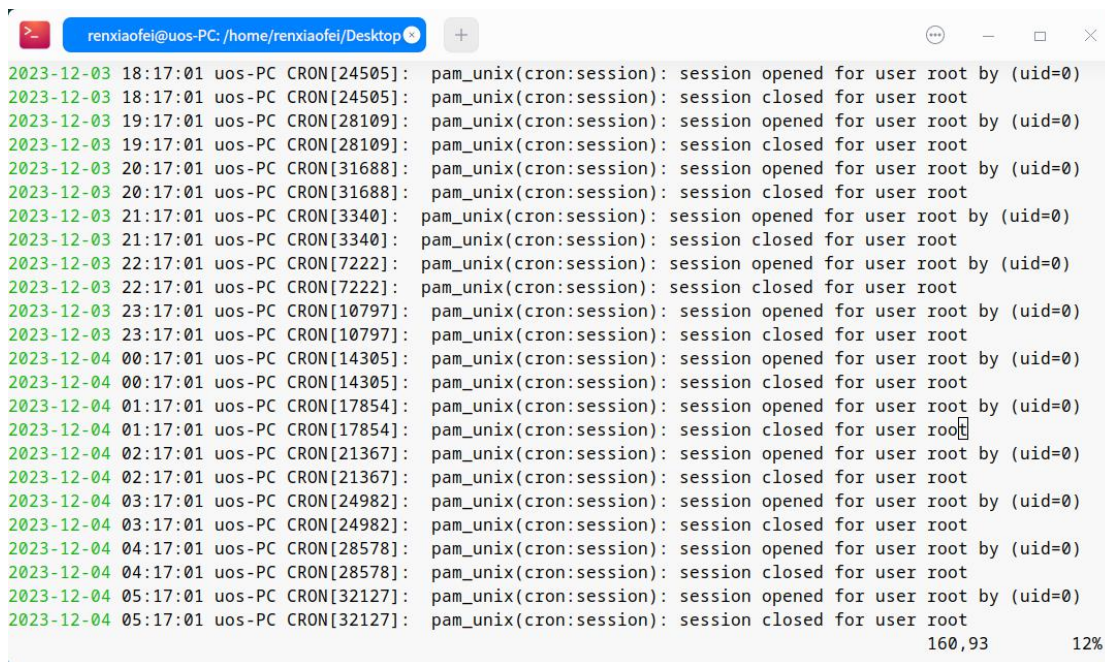
```
pam_handle_t *pamh=NULL;
int retval;
const char *user="uos";
if(argc == 2) {
    user = argv[1];
}
if(argc > 2) {
    fprintf(stderr, "Usage: check_user [username]\n");
    exit(1);
}
// pam_start 需要跟自己在/etc/pam.d/目录下生成文件名称一致
retval = pam_start("demo", user, &conv, &pamh);
if (retval == PAM_SUCCESS)
    retval = pam_authenticate(pamh, 0);    /* is user really u
ser? */

if (retval == PAM_SUCCESS){
    fprintf(stdout, "Authenticated\n");
}else{
    fprintf(stdout, "Not Authenticated\n");
}
if (pam_end(pamh,retval) != PAM_SUCCESS) {
    pamh = NULL;
    fprintf(stderr, "check_user: failed to release authenticator\n");
    exit(1);
}
return ( retval == PAM_SUCCESS ? 0:1 );
}
```

4.4.4. 常见问题

- 怎么调试？

查看/var/log/auth 对应日志：



```
renxiaofei@uos-PC: /home/renxiaofei/Desktop
2023-12-03 18:17:01 uos-PC CRON[24505]: pam_unix(cron:session): session opened for user root by (uid=0)
2023-12-03 18:17:01 uos-PC CRON[24505]: pam_unix(cron:session): session closed for user root
2023-12-03 19:17:01 uos-PC CRON[28109]: pam_unix(cron:session): session opened for user root by (uid=0)
2023-12-03 19:17:01 uos-PC CRON[28109]: pam_unix(cron:session): session closed for user root
2023-12-03 20:17:01 uos-PC CRON[31688]: pam_unix(cron:session): session opened for user root by (uid=0)
2023-12-03 20:17:01 uos-PC CRON[31688]: pam_unix(cron:session): session closed for user root
2023-12-03 21:17:01 uos-PC CRON[3340]: pam_unix(cron:session): session opened for user root by (uid=0)
2023-12-03 21:17:01 uos-PC CRON[3340]: pam_unix(cron:session): session closed for user root
2023-12-03 22:17:01 uos-PC CRON[7222]: pam_unix(cron:session): session opened for user root by (uid=0)
2023-12-03 22:17:01 uos-PC CRON[7222]: pam_unix(cron:session): session closed for user root
2023-12-03 23:17:01 uos-PC CRON[10797]: pam_unix(cron:session): session opened for user root by (uid=0)
2023-12-03 23:17:01 uos-PC CRON[10797]: pam_unix(cron:session): session closed for user root
2023-12-03 23:17:01 uos-PC CRON[14305]: pam_unix(cron:session): session opened for user root by (uid=0)
2023-12-04 00:17:01 uos-PC CRON[14305]: pam_unix(cron:session): session closed for user root
2023-12-04 01:17:01 uos-PC CRON[17854]: pam_unix(cron:session): session opened for user root by (uid=0)
2023-12-04 01:17:01 uos-PC CRON[17854]: pam_unix(cron:session): session closed for user root
2023-12-04 02:17:01 uos-PC CRON[21367]: pam_unix(cron:session): session opened for user root by (uid=0)
2023-12-04 02:17:01 uos-PC CRON[21367]: pam_unix(cron:session): session closed for user root
2023-12-04 03:17:01 uos-PC CRON[24982]: pam_unix(cron:session): session opened for user root by (uid=0)
2023-12-04 03:17:01 uos-PC CRON[24982]: pam_unix(cron:session): session closed for user root
2023-12-04 04:17:01 uos-PC CRON[28578]: pam_unix(cron:session): session opened for user root by (uid=0)
2023-12-04 04:17:01 uos-PC CRON[28578]: pam_unix(cron:session): session closed for user root
2023-12-04 05:17:01 uos-PC CRON[32127]: pam_unix(cron:session): session opened for user root by (uid=0)
2023-12-04 05:17:01 uos-PC CRON[32127]: pam_unix(cron:session): session closed for user root
160,93 12%
```

图 32 查看/var/log/auth 对应日志

- 添加自定义 pam 模块没有生效。

查看/var/log/auth，报错“cannot open shared object file: No such file or directory”，查看/etc/pam.d/目录下自己添加的 pam 文件中*.so 与安装在/usr/lib/x86_64-linux-gnu/security 下 so 名称是否一致，若一致还报错，需要确认现架构下*.so 需要存放的路径。

第五章 UOS “Web” 应用开发

5.1. 本章目的

Windows 有着极其丰富的应用生态，其桌面领域垄断性的市场占有率也导致新技术都会考虑对 Windows 平台的适配。在 Windows 平台主要运行的应用包括桌面应用、Web 应用以及目前流行的大模型 AI 应用等。

本章节将介绍 Windows 下不同应用使用的开发框架以及在 UOS 操作系统上使用这些开发框架的过程。这些常见的开发框架分类如下：

- 桌面应用开发技术：包括 WPF 应用开发、Qt 应用开发、Electron 应用开发、OpenGL 应用开发、Vulkan 应用开发。其中，WPF 框架无法在 Linux 平台使用，建议使用 Qt 框架进行替换。
- Web 应用开发技术：包括 Laravel 应用开发，Ruby on Rails 应用开发、Django 应用开发、Sprint Boot 应用开发。
- 大模型 AI 应用：包括 CUDA 应用开发、OpenCV 应用开发、TensorFlow 应用开发、PyTorch 应用开发。

5.2. 浏览器扩展插件开发

浏览器的插件和扩展是用于补充浏览器功能的程序，浏览器插件开发过程如遇某些浏览器功能无法满足用户需求，可以通过开发定制的插件或者扩展来满足

用户。本章节将详细介绍浏览器 NPAPI 插件和浏览器扩展的功能以及开发流程，用以指导开发者开发定制化的浏览器插件和扩展。

5.2.1. 浏览器扩展开发

Chrome 扩展是用于扩充 Chrome 浏览器功能的程序，主要用于对浏览器功能的增强，它更强调与浏览器相结合。如 Chrome 扩展可以在浏览器的工具栏和地址栏中显示图标，它可以更改用户当前浏览网页中的内容，也可以更改浏览器代理服务器的设置等。扩展是一系列文件的集合，这些文件包括 HTML 文件、CSS 样式文件、JavaScript 脚本文件、图片等静态文件以及 manifest.json。

5.2.1.1. 环境安装

Chrome 插件没有严格的项目结构要求，只要保证本目录有一个 manifest.json 即可，也不需要专门的 IDE，普通的 web 开发工具即可。这里推荐使用 VS Code 进行开发。

5.2.1.2. 环境配置

浏览器地址栏输入 chrome://extensions，如下图所示。



图 33 浏览器扩展管理界面

打开扩展商店（自研商店或者 chrome 扩展商城）进行安装，或者打开开发者模式进行安装 crx 的压缩包文件。

5.2.1.3. 开发案例

如下是一个最简单的扩展案例：

```
{
  "name": "My Extension",
  "version": "1.0",
  "manifest_version": 2,
  "description": "A simple extension that adds a new button to the
  browser toolbar.",
  "content_scripts": [
    {
      "matches": ["https://*/*"],
      "js": ["contentscript.js"]
    }
  ],
  "permissions": ["tabs"]
}
```

这个 manifest.json 文件定义了一个名为"My Extension"的扩展。扩展的版本为 1.0, 描述为"A simple extension that adds a new button to the browser toolbar."。扩展有一个 content script, 它将在所有网页上运行, content script 将加载"contentscript.js"文件, 扩展需要"tabs"权限, 以便可以访问浏览器的标签页。

以下是 manifest.json 文件的各个字段的说明:

- name:扩展的名称。
- version:扩展的版本。
- manifest_version:manifest 文件的版本。
- description:扩展的描述。
- content_scripts:content script 列表。
- permissions:扩展需要的权限列表。

以下是一些 manifest.json 文件的其他常用字段:

- browser_action:定义扩展的浏览器按钮。
- page_action:定义扩展的页面按钮。
- background:定义扩展的背景脚本。
- options_page:定义扩展的选项页面。

web_accessible_resources:定义扩展可以访问的网络资源。

对于更复杂的扩展, 可以使用 manifest.json 文件来配置扩展的其他功能, 例如:

- 扩展的图标。
- 扩展的语言。
- 扩展的扩展点。

有关 manifest.json 文件的更多信息，请参阅 Chrome 扩展开发文档。

5.2.1.4. 常见问题

- 如何扩展调试？

浏览器地址栏输入 `chrome://extensions`, 进入扩展管理页面，选中某个扩展，然后 F12 打开调试工具，即可进入调试。

- 扩展开发文档位置。

浏览器的扩展开发请参考官方文档：

<https://developer.chrome.com/docs/extensions/reference/>

5.2.2. 浏览器插件开发

网景插件应用程序编程接口（NPAPI, Netscape Plugin Application Programming Interface）是用于在浏览器中执行外部应用程序的通用的接口，

由一组单纯的 C Plugin API，用动态库进行注入，用规定的 API 进行通信，用字符串描述插件能力，浏览器会根据能力描述，动态加载，并负责插件调用的流程和生命周期管理，而插件本身负责用户界面和相关功能的设计和实现。如下图所示 wps 插件, 可以看成在浏览器中运行的 word 处理软件，用户可以在浏览器中直接处理 wps 文档。

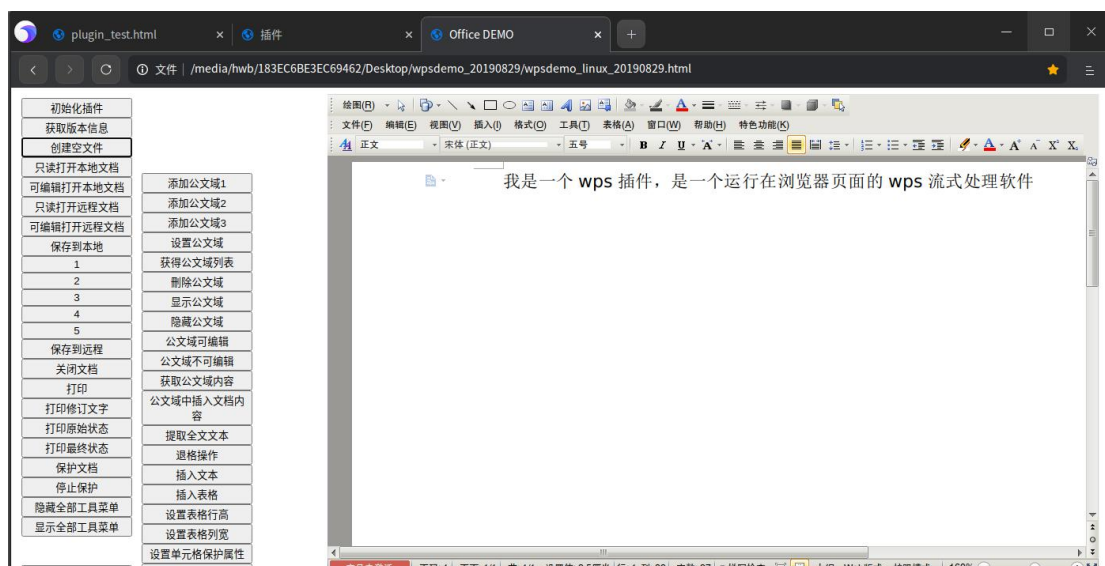


图 34 浏览器 wps 插件实例

5.2.2.1. 环境安装

浏览器插件主要由 C API 提供，其开发环境主要为 C 语言开发环境，其配置可参考“C/C++开发配置”章节。

5.2.2.2. 环境配置

开发者开发的插件在 Linux 下表现的是一个动态库即.so 文件，例如 flash 插件(libflashplayer.so)，wps 插件(libbrowsergrapher.so)。

这些动态库我们拿到后，有的是一个安装包，有的就是一个动态库，不管是自动安装，还是手动拷贝，我们需要保证这些动态库能够在默认的 plugins 路径下找到。

对于 firefox 默认路径可以是：/usr/lib/mozilla/plugins/或者\$HOME/.mozilla/plugins 或者/usr/lib64/mozilla/plugins/。

5.2.2.3. 开发案例

NPAPI 标准定义在一组包含了数据结构和接口函数的头文件中。在实际的开发中,需要将这些头文件加入到工程中,并对头文件定义的部分接口编写实现。用到的主要四个头文件有 npapi.h, npfunction.h, npruntime.h, nptype.h。

简单介绍一下三类接口函数

前缀 NP_XXX 是 npapi 的插件库提供给浏览器的最上层的接口,一般为动态链接库的导出接口。主要有 NP_GetEntryPoints、NP_Initialize、NP_GetMIMEDescription、NP_GetValue、NP_Shutdown 的等几个函数,不同平台的接口可能略有不同,但基本功能都是一样的,都是通过接口来初始化、销毁以及认知此动态库。

前缀 NPP_XXX 即 NP Plugin 是插件本身提供给浏览器调用的接口,主要被用来填充 NPPluginFuncs 的结构体,主要包括: NPP_New、NPP_Destroy、NPP_SetWindow、NPP_GetMIMEDescription、NPP_NewStream、NPP_DestroyStream、NPP_StreamAsFile、NPP_WriteReady、NPP_Write、NPP_Print、NPP_HandleEvent、NPP_URLNotify、NPP_GetValue、NPP_SetValue 等

前缀 NPN_XXX 类接口一般为浏览器引擎提供给 plugin 调用的接口主要包括 NPN_GetURL、NPN_PostURL、NPN_GetValue、NPN_SetValue、NPN_Status 等

实现前缀 NP_XXX npapi 的插件库提供给浏览器的最上层的接口,一般为动态链接库的导出接口,如下表。

表 5 接口表

接口	描述
NP_GetEntryPoints	浏览器取得插件结构体指针。
NP_Initialize	插件初始化时,只调用一次,将浏览器结构体指针传递给插件。
NP_GetMIMEDescription	返回插件的 MIME type 等描述。
NP_GetValue	供浏览器查询插件的内部信息。
NP_Shutdown	销毁插件,与 NP_Initialize 对应。

表 6 实现 npapi.h 中声明的 NPP 函数

函数	描述
NPP_Destroy	释放与插件关联的实例数据和资源。这个浏览器在删除插件实例时调用此函数,通常是因为用户已离开包含实例的页面、关闭窗口或退出浏览器。
NPP_DestroyStream	通知插件将要删除一个流数据。
NPP_GetValue	供浏览器查询插件的内部信息。
NPP_HandleEvent	注册关心的事件,当事件发生时浏览会通知插件。
NPP_New	创建插件的一个实例。
NPP_NewStream	通知插件实例出现了新的流数据。
NPP_Print	请求嵌入式打印还是全屏打印。
NPP_SetValue	设置插件变量信息。
NPP_SetWindow	当窗口创建、移动、改变大小或者销毁时通知插件。
NPP_StreamAsFile	为数据流提供一个本地文件名。
NPP_URLNotify	当对于某个 URL 的请求完成后,浏览器通知插件。
NPP_Write	插件读取流数据。
NPP_WriteReady	调用 NPP_Write 之前,确认插件可以接收多少字节的数据。

一个简单的例子如下:

```
#include <stdio.h>
#include <npapi.h>
#include <nppfunctions.h>
// 插件初始化函数
NPPError MyPlugin_Initialize(NPNetscapeFuncs* pFuncs) {
// 保存浏览器提供的函数指针
```

```
m_pFuncs = pFuncs;
// 初始化插件
printf("Plugin initialized.\n");
return NPERR_NO_ERROR;
}
// 插件渲染函数
NPErr MyPlugin_Render(NPP npp, NPWindow* pWin) {
// 绘制插件
printf("Plugin rendered.\n");
return NPERR_NO_ERROR;
}
// 插件退出函数
void MyPlugin_Shutdown(void) {
// 释放插件资源
printf("Plugin shutdown.\n");
}
// 插件事件处理函数
void MyPlugin_HandleEvent(NPP npp, NPEvent* pEvent) {
// 处理插件事件
printf("Plugin event received.\n");
}
// 插件加载函数
NPErr MyPlugin_New(NPMIMEType pluginType, NPP instance, ns
ISupports* sinfo) {
// 保存插件实例
m_pInstance = instance;
// 返回插件名称
return NPERR_NO_ERROR;
}
// 插件卸载函数
void MyPlugin_Destroy(NPP npp) {
// 销毁插件
printf("Plugin destroyed.\n");
}
```

编译插件放入插件目录后，就可以使用该插件了。

我们在网页设计中使用 HTML 代码来使用插件，通常有如下两种标签元素，供浏览器来调用插件。

- 使用 object 元素。

使用 object 元素在网页中使用插件是推荐使用的方式，其代码如下：

```
<object type="application/np-plugin" align="left" width=640 height=480 id="npID"></object>
```

- 使用 embed 元素。

使用 embed 元素调用插件代码如下：

```
<embed type="application/np-plugin" align="left" width=640 height=480 id="npID"></embed>
```

可以创建一个 html 的测试页面，有窗口类型的插件可以设置其分辨率属性。

在浏览器中打开该测试页面即可通过标签 embed 加载到 MIME type 为 "application/np-demo" 的插件。

```
<html>
<head>
</head>
<body>
  <embed width="1080" height="720" type="application/np-demo" id="pluginDemo">
</body>
</html>
```

5.2.2.4. 常见问题

- 如何查看插件加载状态。

当浏览器启动后，在默认的 plugins 路径下加载插件，读取插件的MimeType 属性，保存在浏览器内部。在地址栏中输入 about:plugins 来查看所有当前识别出的插件的MimeType 属性，如下图所示。

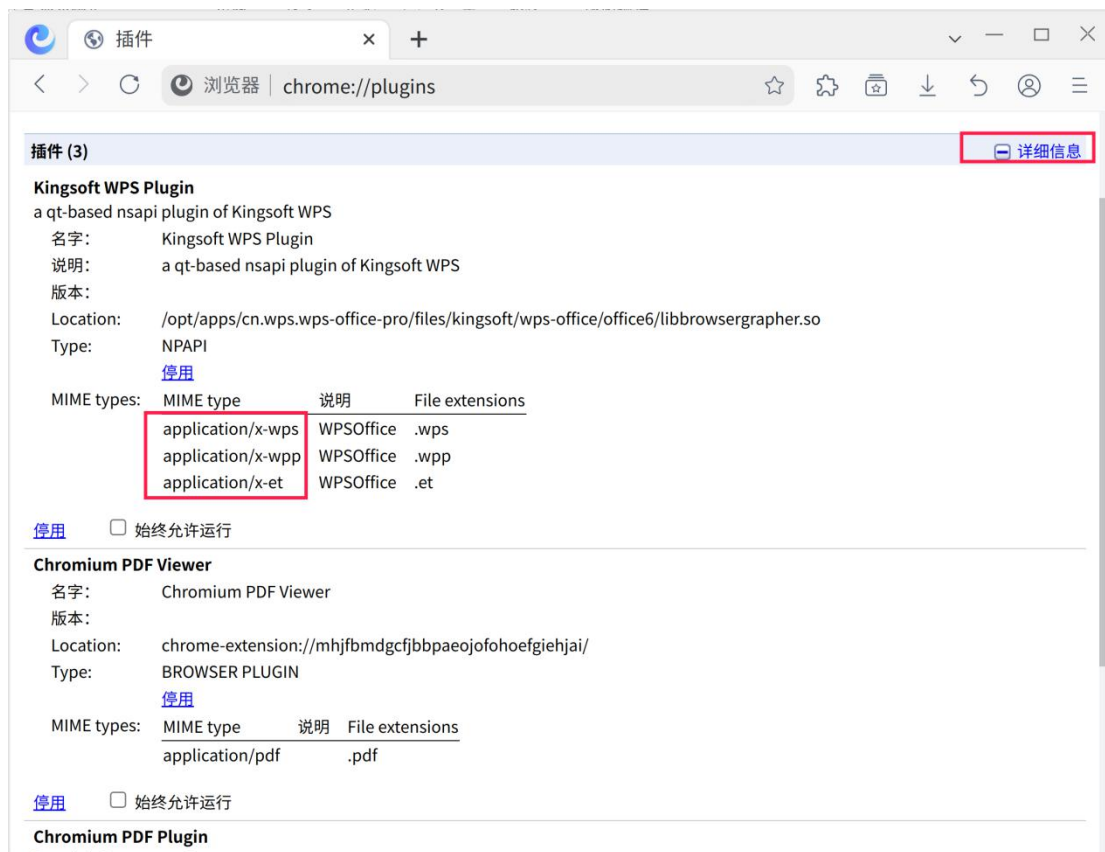


图 35 查看插件加载状态

5.3. Web 应用开发

这一章节着眼于解决用户在统信操作系统中安装、配置、以及开发 Web 应用所遇到的问题。通过详细介绍在统信操作系统上安装和配置多种主流的 Web 框架，包括 ASP.NET、Laravel、Ruby on Rails、Django 和 Spring Boot，读者将获得对这些框架的全面了解。这不仅包括框架的安装步骤，还包括如何配置环境、启动服务，并演示一个简单的开发示例。

5.3.1. ASP.NET 应用开发

ASP.NET 是由 Microsoft 开发的跨平台的 Web 应用框架，可用于构建 Web 类型的应用。当前的 .NET 中提供了 ASP.NET Core，用于替代 ASP.NET，在统信 UOS V20 上可通过安装 .NET SDK 来安装 ASP.NET Core。

5.3.1.1. 环境安装

ASP.NET Core 在 Linux 系统提供 arm32、arm64、x86 等架构的支持，可以通过下述方法安装 ASP.NET Core。

- 通过源安装。

安装方法与 .NET 的相同，参见编程语言章节中的 C# 应用开发配置。但需要额外安装 aspnetcore-runtime，命令如下：

```
sudo apt-get -y aspnetcore-runtime-<version>
```

安装时需要将其中的 <version> 换成对应的版本，如安装 .NET 7.0 则命令应为：

```
sudo apt-get -y aspnetcore-runtime-7.0
```

命令执行成功后，就完成 ASP.NET 开发环境的安装。

- 手动安装。

```
sudo apt-get update && sudo apt-get install -y libc6 libgcc1 libgsapi-krb5-2 libicu63 libssl1.1 libstdc++6 zlib1g
```

安装依赖后，进入 <https://dotnet.microsoft.com/download/dotnet> 选择需要的版本，进入版本的下载页面，下载对应架构的 SDK 的二进制文件，解压

并安装 SDK，以安装.NET 7.0 为例，安装路径为/usr/local/share/dotnet-7.0。

```
sudo mkdir -p /usr/local/share/dotnet-7.0
sudo tar -z -xf dotnet-sdk-7.0.400-linux-x64.tar.gz -C /usr/local/sha
re/dotnet-7.0
sudo chmod +x /usr/local/share/dotnet-7.0/dotnet
```

配置环境变量：配置全局变量可在/etc/profile 文件的最后添加以下内容：

```
PATH=$PATH:/usr/local/share/dotnet-7.0/
export PATH
```

临时使用可在终端中直接设置，命令如下：

```
export PATH=$PATH:/usr/local/share/dotnet-7.0/
```

5.3.1.2. 环境配置

- 环境检查。

ASP.NET 安装成功后，其开发环境就配置好了，可通过 dotnet 命令确认已安装的 ASP.NET 的信息。如下：

查看已安装的 SDK：

```
dotnet --list-sdks
```

查看已安装的 Runtime：

```
dotnet --list-runtimes
```

查看详细信息：

```
dotnet --info
```

- 配置 VS Code。

- 1) 启动 VS Code。
- 2) 打开 VS Code 扩展市场。

3) 搜索 C#扩展并安装。

如此便完成了 VS Code C#开发环境的配置。

5.3.1.3. 开发案例

- 通过终端创建 ASP.NET 应用。

ASP.NET 提供了 dotnet 命令可便捷的在终端中创建应用，如通过以下方式即可创建一个简单应用：

```
dotnet new webapp -o app1
cd app1
dotnet run
```

可在终端中看到输出：

```
$ dotnet run
正在生成...
info: Microsoft.Hosting.Lifetime[14]
Now listening on: http://localhost:5282
info: Microsoft.Hosting.Lifetime[0]
Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
Content root path: /home/uos/Projects/app1
```

打开输出中的地址，即可在浏览器中看到结果：

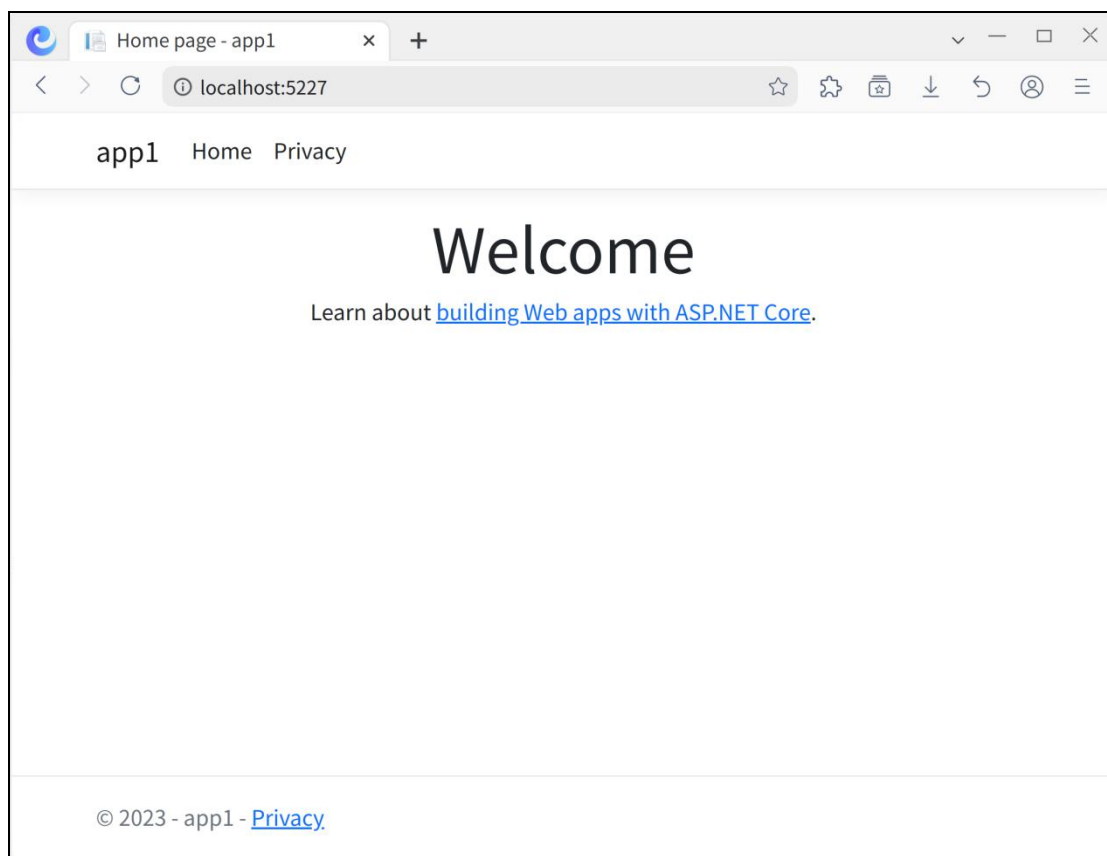


图 36 应用页面

- 通过 VS Code 创建应用。
 - 1) 启动 VS Code。
 - 2) 设置工作区，点击文件->打开文件夹菜单，选择目录并信任。
 - 3) 点击终端->新建终端菜单，打开终端，此时终端所在的路径与选择的工作区目录相同。
 - 4) 在终端中执行 `dotnet new console -o <App Name>` 创建应用。
 - 5) 此时可在 VS Code 中看到生成的源码文件，开始编辑。
 - 6) 编辑完成后点击运行->启动调试运行应用。

如在 Projects 工作区中创建应用 app1，VS Code 运行界面如下：

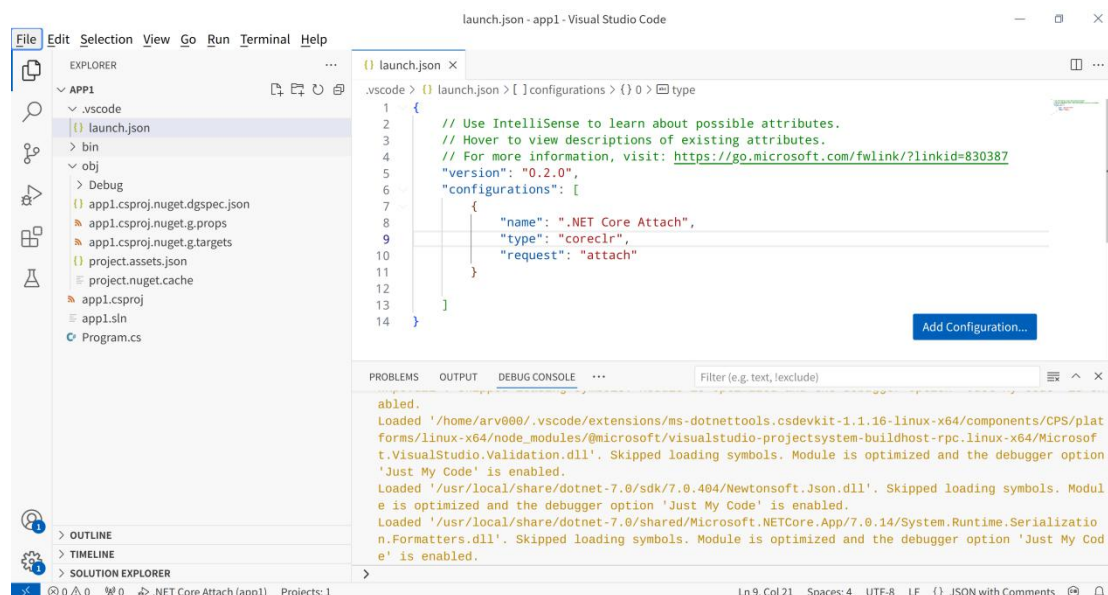


图 37 VSCode 运行 app1 应用

会自动打开浏览器，展示应用界面。

更多使用方法，如调试、发布、创建库等，可参考：

<https://learn.microsoft.com/zh-cn/dotnet/core/tutorials/with-visual-studio-code>

5.3.1.4. 常见问题

● 如何升级已安装的 ASP.NET?

若是通过 apt 源安装的，执行下述命令，若源中的 ASP.NET 有更新，则会自动更新：

```
sudo apt-get update && sudo apt-get upgrade
```

此命令会自动更新所有需要更新的软件，若想只更新 ASP.NET 可执行：

```
sudo apt-get update && sudo apt-get -y aspnetcore-runtime-<version>
```

其中的<version>为当前已安装的版本。

若是手动安装，则与安装时的操作一致，不过需要在解压前把已安装的文件删除。以.NET 7.0 为例，命令如下：

```
sudo rm -rf /usr/local/share/dotnet-7.0
sudo mkdir -p /usr/local/share/dotnet-7.0
sudo tar -z -xf dotnet-sdk-7.0.400-linux-x64.tar.gz -C /usr/local/share/dotnet-7.0
sudo chmod +x /usr/local/share/dotnet-7.0/dotnet
```

卸载已安装的 ASP.NET。

若是通过 apt 源安装的，执行下述命令，可完成.NET 的卸载：

```
sudo apt-get purge aspnetcore-runtime-<version>
```

其中<version>为当前已安装的版本。

若是手动安装的，则直接删除已安装的文件，然后取消环境变量的设置。

切换 ASP.NET 版本。

若是通过 apt 源安装的，推荐先卸载当前版本，然后安装需要切换的版本，命令如下：

```
sudo apt-get remove aspnetcore-runtime-<version>
sudo apt-get install -y aspnetcore-runtime-<new-version>
```

其中<version>为当前已安装的版本，<new-version>为需要切换的版本。

若是手动安装，则也是先删除已安装的版本，然后安装需要切换的版本。

- 使用 apt-get 安装时找不到包？

Microsoft ASP.NET Core 仅提供了 x86 平台的 deb 包，其他平台，如 arm 则需要通过手动安装的方式进行安装。

5.3.2. Laravel 应用开发

Laravel 是一个全栈 Web 应用程序框架，它提供了一套简洁、优雅的语法和丰富的功能，帮助开发人员快速构建高质量的 Web 应用程序。官方网站为：
<https://laravel.com/>

5.3.2.1. 环境安装

在 Linux 下安装 Laravel 需要同时安装 PHP、Composer 和 Laravel 本身。
以下是在 Linux 上安装 Laravel 的一般步骤：

安装 PHP：Laravel8 要求 PHP 版本至少为 7.3.0。您可以使用包管理器或手动安装 PHP。例如，在 UOS 上可以使用以下命令安装 PHP 7.3 甚至更高的版本：

```
sudo apt install php7.3 php7.3-cli php7.3-common php7.3-xml
```

安装完成后可以通过以下命令查看 php 版本：

```
php -v
```

如果您使用的是其他 Linux 发行版，请参考官方文档以了解如何安装 PHP。

安装 Composer：Composer 是 PHP 的依赖管理工具。

Laravel 使用 Composer 来管理项目的依赖关系。您可以按照 Composer 官方文档的指引进行安装。一般来说，可以使用以下命令在 Linux 上安装 Composer：

```
sudo apt install composer
```

安装完成后，您可以通过运行 `composer --version` 命令来验证 Composer

是否安装成功。

安装 Laravel：一旦您安装了 Composer，可以使用它来安装 Laravel。打开终端，进入您希望创建 Laravel 项目的目录，并执行以下命令：

```
composer global require laravel/installer
```

5.3.2.2. 环境配置

检查环境变量：

确保 Laravel 安装器的路径已经添加到系统的可执行文件路径中。可以通过运行以下命令检查：

```
echo $PATH
```

在输出的路径列表中查找 Laravel 安装器的路径（通常是 `~/.config/composer/vendor/bin`）。如果未找到，请继续下一步。

更新环境变量：

打开 `~/.bashrc` 文件,将以下内容添加到文件末尾（如果尚未添加）：

```
export PATH="$HOME/.config/composer/vendor/bin:$PATH"
```

保存文件并执行以下命令使环境变量生效：

```
source ~/.bashrc
```

查看版本信息：

```
laravel -v
```

展示内容如下：

```
Laravel Installer 4.5.1
Usage:
command [options] [arguments]
```

```
Options:
-h, --help            Display help for the given command. When
no command is given display help for the list command
-q, --quiet           Do not output any message
-V, --version         Display this application version
--ansi|--no-ansi      Force (or disable --no-ansi) ANSI output
-n, --no-interaction  Do not ask any interactive question
-v|vv|vvv, --verbose  Increase the verbosity of messages: 1 for n
ormal output, 2 for more verbose output and 3 for debug

Available commands:
completion  Dump the shell completion script
help        Display help for a command
list        List commands
new         Create a new Laravel application
```

5.3.2.3. 开发案例

Laravel 是一个全栈 Web 应用程序框架，具有富有表现力、优雅的语法。

可以通过 laravel 命令方便快捷的创建项目，如下创建示例项目 my-project，展示如何在 Laravel 中创建一个基本的路由和控制器。你可以根据需要扩展和修改这个示例，添加更多的路由、视图和功能来构建更复杂的应用程序。

```
laravel new my-project
```

查看项目目录：

```
cd my-project
tree
```

展示目录内容如下：

```
.
├── app
│   ├── Console
│   └── Kernel.php
```

```
| |—— Exceptions
| |  |—— Handler.php
| |—— Http
| |  |—— Controllers
| |  |  |—— Controller.php
| |  |—— Kernel.php
| |  |—— Middleware
| |    |—— Authenticate.php
| |    |—— EncryptCookies.php
| |    |—— PreventRequestsDuringMaintenance.php
| |    |—— RedirectIfAuthenticated.php
| |    |—— TrimStrings.php
| |    |—— TrustHosts.php
| |    |—— TrustProxies.php
| |    |—— VerifyCsrfToken.php
| |—— Models
| |  |—— User.php
| |—— Providers
| |  |—— AppServiceProvider.php
| |  |—— AuthServiceProvider.php
| |  |—— BroadcastServiceProvider.php
| |  |—— EventServiceProvider.php
| |  |—— RouteServiceProvider.php
|—— artisan
|—— bootstrap
|  |—— app.php
|  |—— cache
|—— composer.json
|—— config
|  |—— app.php
|  |—— auth.php
|  |—— broadcasting.php
|  |—— cache.php
|  |—— cors.php
```

```
| |—— database.php
| |—— filesystems.php
| |—— hashing.php
| |—— logging.php
| |—— mail.php
| |—— queue.php
| |—— sanctum.php
| |—— services.php
| |—— session.php
| |—— view.php
|—— database
| |—— factories
| |—— UserFactory.php
| |—— migrations
| |—— 2014_10_12_000000_create_users_table.php
| |—— 2014_10_12_100000_create_password_resets_table.p
hp
| |—— 2019_08_19_000000_create_failed_jobs_table.php
| |—— 2019_12_14_000001_create_personal_access_tokens_
table.php
| |—— seeders
| |—— DatabaseSeeder.php
|—— package.json
|—— phpunit.xml
|—— public
| |—— favicon.ico
| |—— index.php
| |—— robots.txt
|—— README.md
|—— resources
| |—— css
| |—— app.css
| |—— js
| |—— app.js
```



```
| | └─ bootstrap.js
| └─ lang
| | └─ en
| | └─ auth.php
| | └─ pagination.php
| | └─ passwords.php
| | └─ validation.php
| └─ views
| └─ welcome.blade.php
└─ routes
  └─ api.php
  └─ channels.php
  └─ console.php
  └─ web.php
└─ server.php
└─ storage
  └─ app
  | └─ public
  └─ framework
    └─ cache
      └─ data
      └─ sessions
      └─ testing
      └─ views
    └─ logs
  └─ tests
    └─ CreatesApplication.php
    └─ Feature
      └─ ExampleTest.php
    └─ TestCase.php
    └─ Unit
      └─ ExampleTest.php
  └─ webpack.mix.js
```

36 directories, 66 files

开启服务：

```
php artisan serve
```

浏览器地址栏输入地址：http://localhost:8000

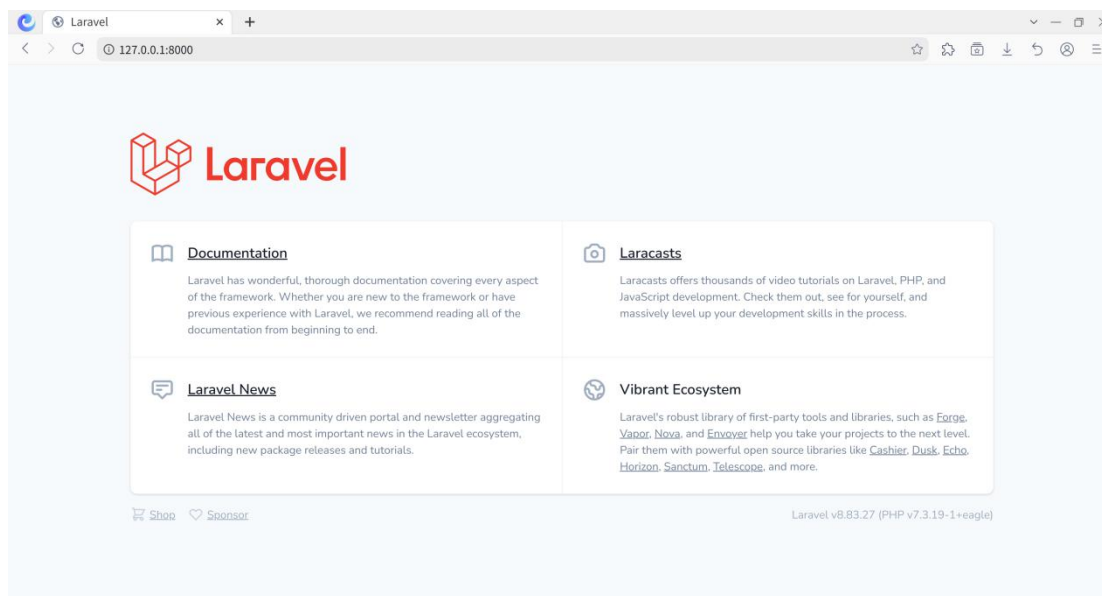


图 38 Laravel 案例网站

5.3.2.4. 常见问题

- 如何将 composer 的版本从 1.x 升级到 2。

```
composer self-update --2
```

报错信息 Command "self-update" is not defined。导致升级失败。

运行下面的脚本，sudo 运行。

```
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
```

```
php -r "if (hash_file('sha384', 'composer-setup.php') === '55ce33d7678c5a611085589f1f3ddf8b3c52d662cd01d4ba75c0ee0459970c2200a51f492d557530c71c15d8dba01eae') { echo 'Installer verified'; } else { echo 'Installer corrupt'; unlink('composer-setup.php'); } echo PHP_EOL;"
```

```
php composer-setup.php --install-dir /usr/bin --filename composer
```

```
php -r "unlink('composer-setup.php');"
```

- 创建项目 `composer create-project laravel/laravel example-app` 时报错。

```
Cannot use laravel/laravel's latest version v10.2.3 as it requires p  
hp ^8.1 which is not satisfied by your platform
```

1) 访问 php 官方网站 <https://www.php.net/downloads.php#gpg-8.2>

下载源码包。

2) 解压源码包。

3) 按照下面步骤编译和安装。

```
sudo apt install -y pkg-config build-essential autoconf bison re2c  
libxml2-dev libsqlite3-dev  
./configure  
make -j4  
sudo make install  
php -v #查看版本
```

5.3.3. Ruby on Rails 应用开发

Ruby on Rails 是一种基于 Ruby 编程语言的 Web 应用程序框架，它采用了 MVC (Model-View-Controller) 架构模式，旨在提高开发效率和代码可读性，在 UOS 系统中，通过简单命令即可安装使用。

Ruby on Rails 官方网站：<https://rubyonrails.org/>

5.3.3.1. 环境安装

在 UOS 里，在终端输入如下命令完成 ruby 的安装：

```
wget https://cache.ruby-lang.org/pub/ruby/3.1/ruby-3.1.4.tar.gz  
tar -zxvf ruby-3.1.4.tar.gz  
cd ruby-3.1.4
```

```
sudo apt-get install build-essential libssl-dev libreadline-dev zlib1g-dev libyaml-dev  
./configure  
make  
sudo make install
```

您可以通过如下命令来确认 ruby 是否成功安装：

```
ruby -v
```

安装 SQLite3

UOS 操作缺省带有 SQLite3，不需要另外单独安装。您可以通过如下命令来确认其是否正确安装：

```
sqlite3 --version
```

安装 Rails

要安装 Rails，使用 RubyGems 提供的 gem install 命令：

```
sudo gem install rails
```

我们可以通过如下命令来确认 Rails 是否已经正确安装：

```
rails --version
```

5.3.3.2. 环境配置

Ruby on Rails 不需要进行的环境配置，安装成功后即可使用。

5.3.3.3. 开发案例

Rails 内置了许多称为生成器的脚本，这些脚本旨在通过创建开始处理特定任务所需的一切来简化您的开发生活。其中之一是新的应用程序生成器，它将为提供新的 Rails 应用程序的基础，这样您就不必自己编写了额外的代码。

- 创建应用程序。

要使生成器生成应用程序，您可以打开一个终端，导航到你有权创建文件的目录，然后运行：

```
rails new blog
```

这将在 blog 目录下创建一个名为 blog 的 Rails 应用程序，并使用 bundle install 安装 Gemfile 中指定的 gem 依赖项。

创建应用程序后，切换到它的文件夹：

```
cd blog
```

这个目录包含许多生成的文件和文件夹，这些文件和文件夹构成了 Rails 应用程序的结构。这里，我们将关注点房子 app 文件夹中。

- 启动 Web 服务器。

要启动 web 服务器，你可以在 blog 目录，运行如下命令：

```
./bin/rails server
```

这将启动 Puma，一个默认使用 Rails 分发的 web 服务器。要查看应用程序的运行情况，请打开浏览器窗口并导航到 <http://localhost:3000>。这时你应该看到 Rails 的默认信息页面：

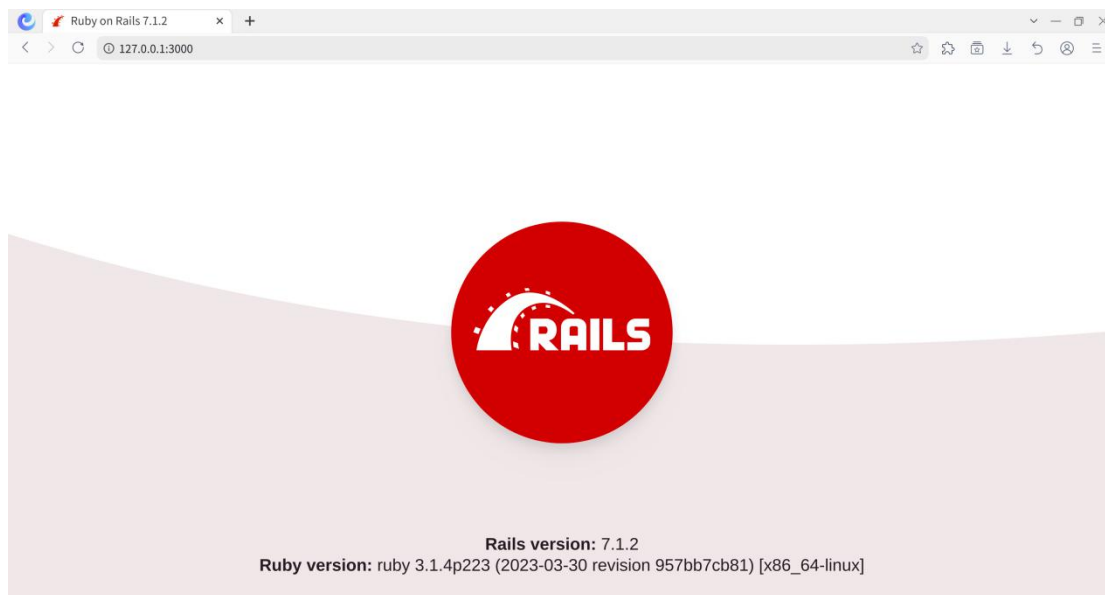


图 39 Rails 开发案例

这就代表 Ruby on Rails 一切正常。

5.3.3.4. 常见问题

- 如何安装 Ruby on Rails?

可以使用 Ruby 版本管理器 (RVM) 或 RubyGems 来安装 Ruby on Rails。具体安装步骤可以参考官方文档：https://guides.rubyonrails.org/getting_started.html

- 如何创建一个新的 Ruby on Rails 应用程序?

可以使用命令行工具创建一个新的 Ruby on Rails 应用程序。具体步骤可以参考官方文档：https://guides.rubyonrails.org/getting_started.html#create-a-new-rails-project

- 如何创建一个控制器?

可以使用命令行工具创建一个新的控制器。具体步骤可以参考官方文档：https://guides.rubyonrails.org/getting_started.html#creating-articles

- 如何创建一个模型？

可以使用命令行工具创建一个新的模型。具体步骤可以参考官方文档：https://guides.rubyonrails.org/getting_started.html#creating-the-article-model

- 如何创建一个视图？

可以使用命令行工具创建一个新的视图。具体步骤可以参考官方文档：https://guides.rubyonrails.org/getting_started.html#creating-the-article-views

- 如何使用数据库？

Ruby on Rails 支持多种数据库，包括 MySQL、PostgreSQL、SQLite 等。可以使用 ActiveRecord 进行数据库操作。具体使用方法可以参考官方文档：https://guides.rubyonrails.org/active_record_basics.html

- 如何进行用户认证和授权？

可以使用 Devise 和 CanCanCan 等 Ruby on Rails 的认证和授权库。具体使用方法可以参考官方文档：<https://guides.rubyonrails.org/security.html>

- 如何进行测试？

Ruby on Rails 内置了测试框架，可以使用测试来保证代码的质量和稳定性。具体使用方法可以参考官方文档：

<https://guides.rubyonrails.org/testing.html>

5.3.4. Django 应用开发

Django 是一个开放源代码的 Web 应用程序框架，使用 Python 编写。它遵

循 MVC（模型-视图-控制器）的设计模式，用于快速开发高质量的 Web 应用程序。一些关于 Django 的特点包括：

- 自带 ORM（对象关系映射）：可以轻松地与数据库交互，而无需编写 SQL 语句。
- 自带管理后台：可以轻松创建、读取、更新和删除数据库中的数据。
- 可扩展性：可以使用 Django 的插件系统来扩展应用程序的功能。
- 安全性：Django 提供了许多内置的安全功能，如 CSRF 保护和 XSS 防护。
- 易于测试：Django 提供了一些内置的测试工具，可以帮助您编写和运行测试。

Django 官方网站：<https://www.djangoproject.com/>

5.3.4.1. 环境安装

在使用 Django 之前，需要先安装它，下面的指南将带您完成一个简单、最小的安装，安装完毕之后，就可以正常使用 Django 了。

- 安装 Python。

安装命令如下：

```
sudo apt install python3 python3-pip
sudo apt-get install libsqlite3-dev
sudo apt-get install python3-venv
pip3 install virtualenv
python3 -m venv venv
source venv/bin/activate
```


- 设置数据库。

只有当你想使用像 PostgreSQL、MariaDB、MySQL 或 Oracle 这样的“大型”数据库引擎时，这一步才有必要的，如果我们只是使用 SQLite 的话，这一步是不必要的。

- 安装 Django。

我们可以通过如下 shell 命令来完成 Django 的安装：

```
python3 -m pip install --upgrade pip
python3 -m pip install django django-admin
```

- 验证安装结果。

要验证 Django 是否被正确安装，可以在 shell 中输入 Python。然后在 Python 提示符下，输入如下指令：

```
import django
print(django.get_version())
```

代码执行结果如下：

```
3.2.22
```

3.2.22 为 django 版本号，如果上面的 Python 代码被正确执行，就表示 Django 已经被正确安装。

5.3.4.2. 环境配置

Django 环境配置，需要激活刚才创建 python 虚拟环境，注意 `venv/bin/activate` 为使用命令 `python3 -m venv venv`，创建的虚拟环境的相对地址。

```
source venv/bin/activate
```

5.3.4.3. 开发案例

- 创建项目。

从命令行切换到你想要存储代码的目录，然后运行以下命令：

```
django-admin startproject mysite
```

这将在当前目录下创建一个 mysite 目录。

让我们来看看 startproject 创建的内容：

```
.
├── mysite
├── manage.py
├── mysite
│   ├── asgi.py
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
```

下面的内容是对这个目录结构的一个说明：

- 1) 第一层目录 mysite/

这个根目录是项目的容器。它的名字对 Django 来说并不重要;您可以将其重命名为任何您喜欢的名称。

- 2) manage.py

一个命令行实用程序，让你以各种方式与这个 Django 项目进行交互。

- 3) 第二层目录 mysite/

是项目的实际 Python 包。它的名字是 Python 包名，你需要使用它来

导入其中的任何内容(例如 `mysite.urls`)。

4) `mysite/__init__.py`

一个空文件，告诉 Python 这个目录应该被视为一个 Python 包。

5) `mysite/Settings.py`

这个 Django 项目的配置文件。

6) `mysite/urls.py`

这个 Django 项目的 URL 声明;你的 django 站点的“目录”。

7) `mysite/asgi.py`

asgi 兼容的 web 服务器的访问入口点，为您的项目提供服务。

8) `mysite/wsgi.py`

wsgi 兼容的 web 服务器的访问入口点，为您的项目提供服务。

- 启动开发者服务器。

切换到外部 `mysite` 目录，并运行以下命令：

```
cd mysite
python3 manage.py runserver
```

您将在命令行上看到以下输出：

```
Watching for file changes with StatReloader
Performing system checks...
System check identified no issues (0 silenced).
You have 18 unapplied migration(s). Your project may not work
properly until you apply the migrations for app(s): admin, auth, cont
enttypes, sessions.
Run 'python manage.py migrate' to apply them.
October 26, 2023 - 02:17:21
```

```
Django version 3.2.22, using settings 'mysite.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

这样我们就启动了 Django 开发服务器，这是一个完全用 Python 编写的轻量级 web 服务器。在 Django 中包含了这个它,但是不要在任何类似于生产环境的环境中使用此服务器,仅用于开发过程中使用它。

现在服务器已经运行，使用 web 浏览器访问 `http://127.0.0.1:8000/`,你会看到“恭喜!”页面，一枚火箭正在起飞。Django 工作起来了。

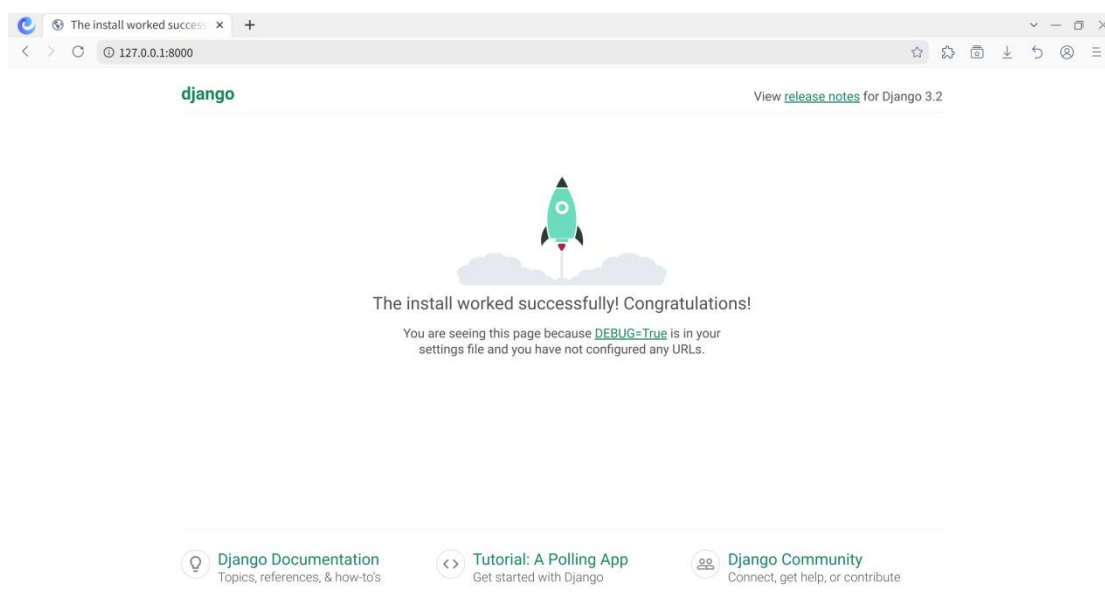


图 40 django 开发案例

5.3.4.4. 常见问题

- ModuleNotFoundError:No module named 'django'报错。

确保已正确安装 Django，您可以使用 pip3 安装 Django：`pip3 install django`。

- ImproperlyConfigured:Error loading MySQLdb module: No module named'MySQLdb'报错。

这通常是由于缺少 MySQL 数据库驱动程序引起的,您可以尝试安装 mysqlclient 库: `pip install mysqlclient`。如果您使用的是 Python 3, 请尝试 `pip3 install mysqlclient-<version>`, 其中<version>是与您的 Python 版本对应的版本号, 如 `mysqlclient-2.0.3`。

- OperationalError:no such table:<table_name>报错。

这表示 Django 无法找到指定的数据表。可能是因为您还没有进行数据库迁移。请运行 `python manage.py makemigrations` 生成迁移文件, 然后运行 `python manage.py migrate` 应用迁移并创建表。

- TemplateDoesNotExist:<template_name>报错。

这表示 Django 无法找到指定的模板文件,确保您的模板文件位于正确的目录中, 并在模板名称中使用正确的路径和扩展名,还要检查您的 TEMPLATES 设置, 确保模板的目录包含在“DIRS”列表中。

- ImproperlyConfigured:Requested setting<setting_name>not found in Django settings.

这表示您正在尝试访问不存在的 Django 设置。请确保您在 settings.py 文件中正确配置了相关设置, 并且没有拼写错误。还要确保您正在访问正确的设置名称。

5.3.5. Spring Boot 应用开发

Spring Boot 是由 Pivotal 团队提供的全新框架，其设计目的是用来简化新 Spring 应用的初始搭建以及开发过程。该框架使用了特定的方式来进行配置，从而使开发人员不再需要定义样板化的配置。

Spring Boot 官方网站：<https://spring.io/projects/spring-boot>

5.3.5.1. 环境安装

Spring Boot 需要依赖 JDK 环境配置和 Apache Maven，通过仓库命令安装：

```
sudo apt install openjdk-11-jdk maven
```

通过如下命令验证安装是否成功：

```
uos@uos:~$ java -version
openjdk version "11.0.11" 2021-04-20
OpenJDK Runtime Environment (build 11.0.11+9-post-Debian-1deb10u1)
OpenJDK 64-Bit Server VM (build 11.0.11+9-post-Debian-1deb10u1,
mixed mode, sharing)
uos@uos:~$ mvn -version
Apache Maven 3.6.3
Maven home: /usr/share/maven
Java version: 11.0.11, vendor: Debian, runtime: /usr/lib/jvm/java-11-openjdk-amd64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "4.19.0-17-amd64", arch: "amd64", family: "unix"
```

安装 Spring Boot CLI：Spring Boot CLI 是一个用于开发 Spring Boot

应用程序的命令行工具。运行以下命令进行安装：

```
curl -s https://get.sdkman.io | bash
source "$HOME/.sdkman/bin/sdkman-init.sh"
sdk install springboot
```

安装完成后，可以运行以下命令验证 Spring Boot CLI 是否正确安装：

```
uos@uos:~$ spring --version
Spring CLI v2.5.2
```

5.3.5.2. 环境配置

不需要进行配置，安装成功即可使用。

5.3.5.3. 开发案例

本案例将展示如何在 UOS 系统创建一个简单的 Spring Boot 项目。

- 通过网站创建项目。

在网站 start.spring.io 上创建一个 Web 项目。在 “Dependencies” 对话框中搜索并添加 “web” 依赖项，如图所示。点击 “生成” 按钮，下载压缩文件，并将其解压缩到计算机上的一个文件夹中。

打开网站，点击 ADD DEPENDENCIES... CTRL+B 按钮。

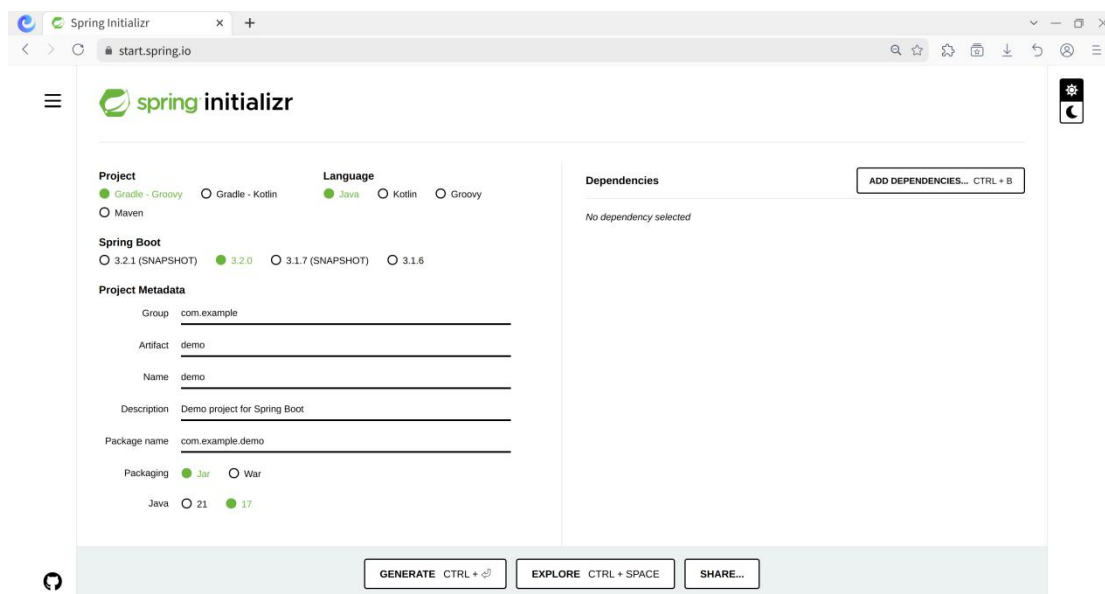


图 41 Spring Boot 项目创建步骤一

选择 Spring Web 如下图：

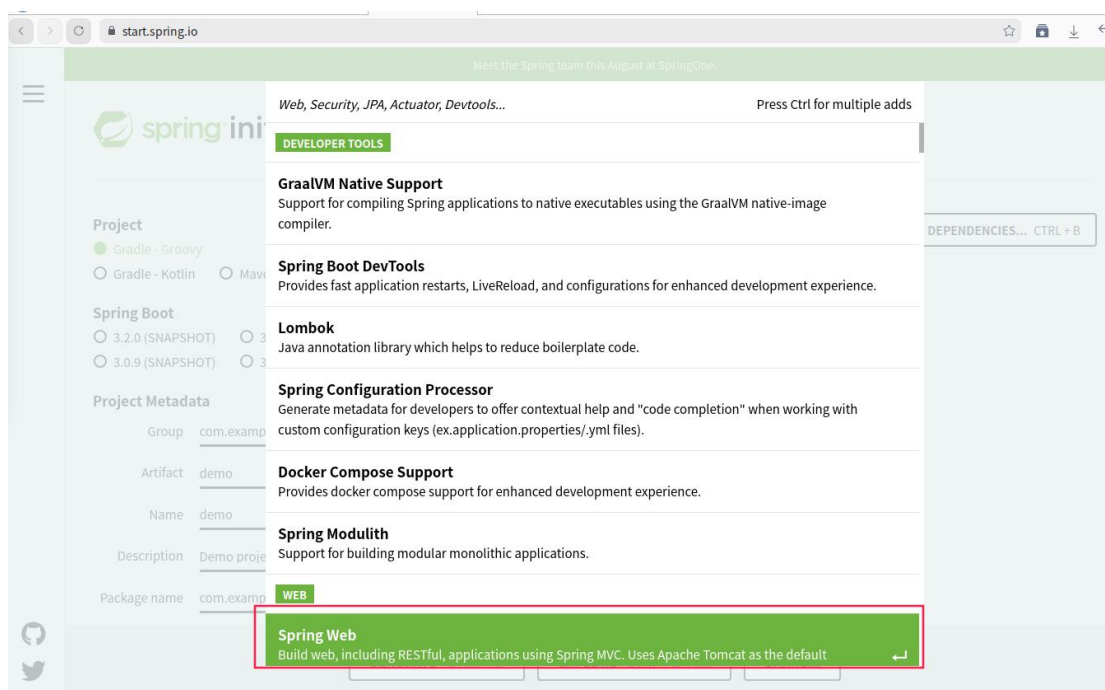


图 42 Spring Boot 项目创建步骤二

点击 GENERATE CTRL+⌘按钮下载代码。

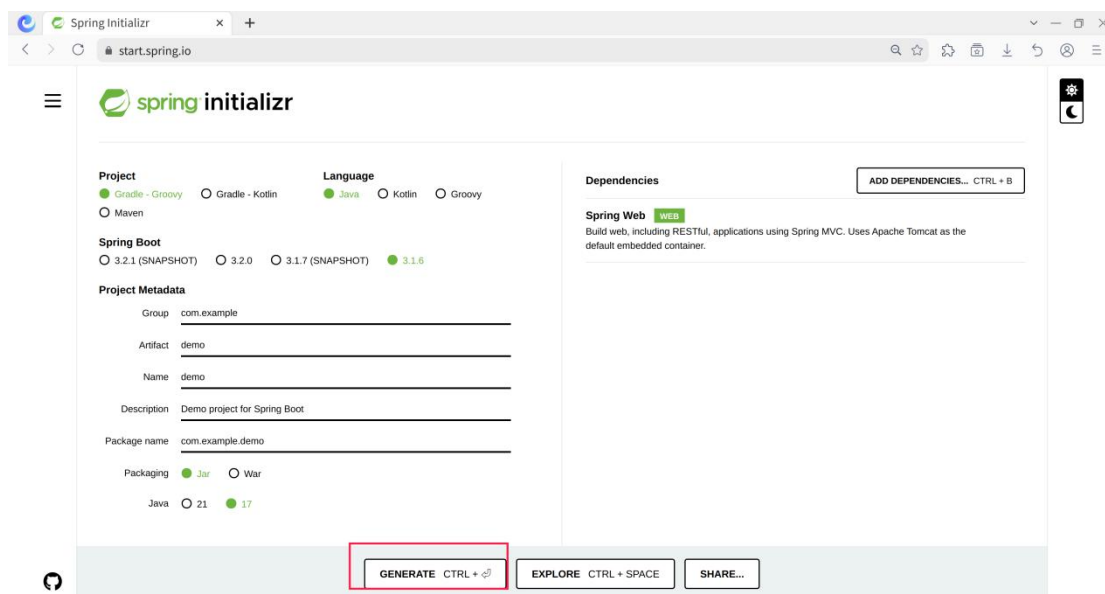


图 43 Spring Boot 项目创建步骤三

通过 start.spring.io 创建的项目包含 Spring Boot 框架，但不需要太多代码或配置。Spring Boot 是启动 Spring 项目最快、最流行的方式。

- 添加你自己的代码。

在集成开发环境里打开项目，并在 src/main/java/com/example/demo 文件夹中找到 DemoApplication.java 文件。现在，你可以参考如下内容来更改文件的内容。

```
package com.example.demo;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
@SpringBootApplication
@RestController
public class DemoApplication {
```

```
public static void main(String[] args) {
    SpringApplication.run(DemoApplication.class, args);
}
@GetMapping("/hello")
public String hello(@RequestParam(value = "name", defaultValue = "World") String name) {
    return String.format("Hello %s!", name);
}
}
```

这是在 Spring Boot 中创建一个简单的“Hello World” web 服务所需的所有代码。

我们添加的 `hello()` 方法被设计为接受一个名为 `name` 的 `String` 参数，然后将该参数与代码中的单词“hello”结合起来。这意味着如果您在请求中将自己的名字设置为“Amy”，那么响应将是“Hello Amy”。

`@RestController` 注释告诉 Spring，这段代码描述了一个应该在 web 上可用的端点。`@GetMapping("/hello")` 告诉 Spring 使用我们的 `hello()` 方法来应答发送到 `http://localhost:8080/hello` 地址的请求。最后，`@RequestParam` 告诉 Spring 期望请求中有一个名称值，但如果没有，它将默认使用单词“World”。

- 构建并运行。

现在我们可以构建并运行该程序。打开命令行(或终端)并导航到项目文件所在的文件夹,我们可以通过发出以下命令来构建和运行应用程序：

```
./gradlew bootRun
```

这时，您应该会看到一些类似于下面的输出：

```
Starting a Gradle Daemon (subsequent builds will be faster)
```



```
2023-12-09T18:09:37.059+08:00 INFO 8116 --- [           main] c
om.example.demo.DemoApplication : Started DemoApplication
in 0.78 seconds (process running for 0.945)
<=====--> 80% EXECUTING [3m 32s]
> :bootRun
```

这里的最后几行告诉我们 Spring 已经启动。Spring Boot 嵌入了 Apache Tomcat，可以充当 web 服务器，它监听本地主机端口 8080 上的请求。

打开浏览器，在顶部的地址栏中输入 `http://localhost:8080/hello`，你应该会得到如下的结果：



图 44 浏览器显示效果

5.3.5.4. 常见问题

- 如何配置数据库连接？

您可以在 Spring Boot 的配置文件（如 `application.properties` 或 `application.yml`）中配置数据库连接信息。根据您使用的数据库，您需要提供数据库 URL、用户名、密码等必要的信息。例如，对于 MySQL 数据库，您可以在配置文件中添加以下内容：

```
spring.datasource.url=jdbc:mysql://localhost:3306/mydatabase
spring.datasource.username=root
spring.datasource.password=secret
```

- 如何定义 RESTful API 端点？

在 Spring Boot 中,您可以使用@Controller 或@RestController 注解来定义 RESTful API 端点。使用@RequestMapping 注解指定端点的 URL 路径,并使用合适的 HTTP 方法注解(如@GetMapping、@PostMapping 等)定义方法。例如:

```
@RestController
@RequestMapping("/api")
public class MyController {
    @GetMapping("/hello")
    public String hello() {
        return "Hello, World!";
    }
}
```

上述代码定义了一个 GET 请求的/api/hello 端点,返回字符串"Hello, World!"。

- 如何处理请求参数和路径变量？

您可以在方法参数上使用@RequestParam 注解来接收请求参数,使用@PathVariable 注解来接收路径变量。例如:

```
@GetMapping("/user")
public String getUser(@RequestParam("id") int userId) {
    // 根据 userId 获取用户信息
    return "User: " + userId;
}

@GetMapping("/user/{id}")
public String getUserById(@PathVariable("id") int userId) {
    // 根据 userId 获取用户信息
    return "User: " + userId;
}
```

```
}
```

上述代码中的 `getUser` 方法接收名为 "id" 的请求参数，而 `getUserById` 方法接收路径中的 "id" 变量。

- 如何使用数据库访问和 ORM？

Spring Boot 提供了多种数据库访问和 ORM 的选择，包括 Spring Data JPA、Spring Data JDBC、MyBatis 等。您可以根据您的需求选择适合的技术和相关依赖，并使用它们来访问和操作数据库。您可以定义实体类、仓库接口，并利用 Spring Boot 的自动配置功能进行数据库访问的配置。

第六章 UOS 迁移方案

6.1. 本章目的

目前 Linux 系统的生态相对 Windows 来说还是比较贫乏，虽然能满足基础的办公、娱乐需求，但是很多专业属性的软件不支持 Linux 版本，例如即时通讯类软件，本章节主要讲述如何完成 Windows 应用如何迁移到 UOS 系统及跨平台开发框架的使用。

6.2. deepin-wine 迁移

deepin-wine 工具是统信 UOS 系统自带的 wine 版本，是由统信操作系统 deepin-wine 开发团队为了迁移 Windows 软件到国产操作系统而独立维护的一个 wine 分支。用户可以通过该 deepin-wine 工具在统信 UOS 系统上安装并使用 Windows 应用。

统信操作系统 deepin-wine 开发团队始终以国内用户需求为主导，在适配国内热点应用软件以及定制商业需求中，解决了大量关键性难题，先后迁移了微信、QQ、企业微信、央视影音、优酷、Foxmail、腾讯视频、格式工厂等拥有众多用户的软件到统信 UOS 系统中，使统信 UOS 系统更加满足国内用户的日常使用需求。同时，统信操作系统 deepin-wine 开发团队紧跟 wine 社区，定期将修复的补丁提交上游 wine，为 wine 项目的不断完善做出贡献。

6.2.1. 环境安装

为了在 UOS 操作系统中通过 deepin-wine 工具安装并使用 Windows 应用程序，需要安装 deepin-wine 工具及其运行时环境。需要注意的是，在本书编写期间 deepin-wine 工具的稳定版本为 deepin-wine6-stable，在后续的开发和维护中，deepin-wine 工具还会不断地进行更新和升级，其包名和命令会发生改变，可以通过下表中的命令查询更新之后的包名和版本：

```
#获取包名
apt depends com.qq.weixin.deepin | grep deepin-wine | grep -v
deepin-wine-helper | awk '{print $2}'
#获取执行命令
dpkg -L *** | grep "^/usr/bin/" | awk -F / '{print $NF}'
（注：*** 要替换为前一条命令获取包名的结果）
```

通过如下命令安装 deepin-wine 工具，以 deepin-wine 工具的包名为 deepin-wine6-stable 为例：

```
sudo apt install deepin-wine6-stable
```

安装完成之后就可以在终端中输入执行命令运行，deepin-wine6-stable 的执行命令即为 deepin-wine6-stable。

6.2.2. 环境配置

deepin-wine 工具的环境配置较为复杂，为了避免用户在配置中出现错误，deepin-wine 团队提供了多种方式自动安装配置 deepin-wine 相关环境和依赖。在安装一款以 deepin-wine 适配的软件之后，就会自动配置 deepin-wine 工具所需要的全部环境。下面介绍三种方式来配置 deepin-wine 工具的环境。

- 通过应用商店部署 Windows 应用。

统信操作系统 deepin-wine 开发团队基于 deepin-wine 迁移了大量常用软件，通过将适配成功的 Windows 应用做成单独的 deb 安装包，在经过系统测试之后上架了应用商店。用户可以通过在统信 UOS 应用商店一键进行下载和安装。在应用商店中，应用类别标识为 Windows 版本的即为 wine 应用。

下面通过在应用商店中安装部署微信为例：打开应用商店后搜索微信。



图 45 在应用商店中搜索应用

打开微信查看应用信息，选择 Windows 版本进行安装。

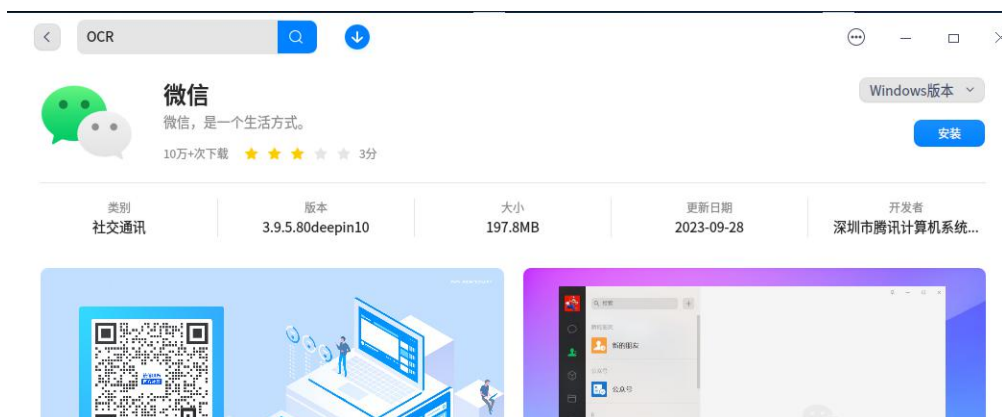


图 46 应用商店中微信的安装界面

选择 Windows 版本并安装，通过应用商店安装应用会自动安装应用所依赖的 deepin-wine 工具。安装完成之后可以通过应用商店界面或启动器中打开微信应用，首次启动 wine 应用时，会有一个容器初始化以及释放应用容器文件的过程，需要等待一段时间。之后，就可以在统信 UOS 系统上使用 Windows 版本的微信了。

- 通过终端命令行安装已适配的 Windows 应用。

在统信 UOS 系统中支持通过终端命令行安装已经经过 deepin-wine 适配的应用，例如微信的 deepin-wine 包名称为 com.qq.weixin.deepin，在终端中输入。

```
sudo apt install com.qq.weixin.deepin
```

即可自动安装 Windows 版本的微信应用以及其所需要的依赖。

```
uos@yangkun:~$ sudo apt install com.qq.weixin.deepin
请输入密码:
验证成功
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
下列软件包是自动安装的并且现在不需要了:
  deepin-wine-plugin deepin-wine-plugin-virtual gstreamer1.0-fluendo-m
  libmodplug1 libnice10 libofa0 libsoundtouch1 libsrtp2-1 libvo-aacenc
使用 'sudo apt autoremove' 来卸载它(它们)。
下列【新】软件包将被安装:
  com.qq.weixin.deepin
升级了 0 个软件包，新安装了 1 个软件包，要卸载 0 个软件包，有 821 个软
需要下载 358 MB 的归档。
解压缩后会消耗 383 MB 的额外空间。
获取:1 https://aptly.uniontech.com/pkg/nonfree-eagle/release-candidate
m.qq.weixin.deepin amd64 3.9.7deepin3 [358 MB]
已下载 358 MB，耗时 5 秒 (69.0 MB/s)
正在选中未选择的软件包 com.qq.weixin.deepin。
(正在读取数据库 ... 系统当前共安装有 270345 个文件和目录。)
准备解压 .../com.qq.weixin.deepin_3.9.7deepin3_amd64.deb ...
正在解压 com.qq.weixin.deepin (3.9.7deepin3) ...
正在设置 com.qq.weixin.deepin (3.9.7deepin3) ...
most recently configured version
do install script
正在处理用于 deepin-app-store (7.1.1.00092-1) 的触发器 ...
```

图 47 在终端命令行中安装微信

安装成功之后，就可以通过启动器或桌面快捷方式启动微信应用。

- 通过 winetricks 安装 Windows 应用。

winetricks 是一个开源辅助工具，通过使用 winetricks 可以方便地安装 windows 应用。deepin-wine 团队修改并完善了 winetricks，使其能够给统信 UOS 系统提供更多、更好的支持。

在终端中执行如下命令，下载 deepin-wine 开发团队修改的 winetricks。

```
sudo wget -O /usr/bin/winetricks
https://gitee.com/wayne-xw/winetricks_for_deepinwine/raw/master/
src/winetricks
```

在终端中执行 chmod 命令给 winetricks 添加可执行权限。

```
sudo chmod +x /usr/bin/winetricks
```

在终端中执行 winetricks 来运行 winetricks 的界面程序。

```
winetricks
```

winetricks 的主界面：



图 48 winetricks 主界面

可以方便地通过 winetricks 来安装 Windows 应用程序，选择容器即选择应

用需要安装的路径，默认的 wine 容器是用户主目录下的.wine，即~/wine。这里以安装一个游戏为例，介绍如何使用 winetricks 安装一个 Windows 游戏类别的应用软件。

在图中选择安装一个游戏，弹出的 winetricks 安装应用列表如图所示：



图 49 winetricks 安装应用程序列表

在 winetricks 安装应用列表的最上方提示当前容器的路径，可以在前一个页面进行修改。在页面中部的列表中显示了当前可以安装的应用程序信息，在左侧勾选框中选中之后，点击确定进行安装，按照安装向导逐步进行即可。

这里以安装游戏 algodoo_demo 为例进行说明，通过安装向导完成安装之后，会在容器路径中生成应用程序的目标 exe 文件，如图所示：

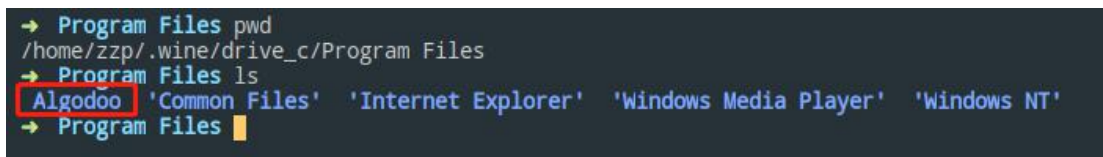


图 50 安装完成的应用路径

进入到容器目录中与 Windows 对应的 Program Files 目录，可以看到已经

生成 Algodoo 文件目录，进入其中之后，通过 `deepin-wine6-stable Algodoo.exe` 即可启动该应用。

6.2.3. 开发案例

在开发案例中，介绍如何从 Windows 应用的安装程序开始进行 `deepin-wine` 迁移。如果想安装的应用不在应用商店中或 `wine` 社区中，可以通过手动的方式进行安装，这种方式安装的应用由于没有经过 `wine` 社区测试适配可能存在缺陷，需要一定的软件调试经验。

首先需要在统信 UOS 系统中手动安装 `deepin-wine` 工具，通过以下命令安装：

```
sudo apt install deepin-wine6-stable
```

安装完成之后就可以通过 `deepin-wine6-stable` 命令执行 `wine`。

接下来需要获取到需要安装的应用对应 Windows 的安装包，这里以安装酷狗音乐为例，从酷狗音乐官网下载 Windows 版本的安装程序 `kugou_10193.exe`，从该安装程序开始执行安装。

`deepin-wine6-stable` 运行 Windows 程序可以指定若干环境变量：

表 7 名词解释

名词	解释
WINEPREFIX	指定的容器路径。
WINEDEBUG	指定的调试日志通道。
WINEARCH	指定的 cpu 架构，为 win32 或 win64。

WINEPREFIX 指定的是当前 `wine` 运行进程所在的目录，包含了当前 `wine`

运行进程的文件、配置和运行时库，默认是当前用户目录的`~/wine`，可以通过 WINEPREFIX 指定。一个 wine 容器提供了 Windows 程序的运行环境，可以类比起来成一个精简的 Windows 系统环境，一般 Windows 系统的安装目录 C 盘在容器中对应为 `drive_c` 目录。等价的，容器为当前 wine 进程提供文件支持和动态链接的运行时库，并可以模拟成不同的 Windows 版本来运行不同的应用程序。容器中的配置和文件都可以手动进行修改。

以上环境变量若不指定均为默认值，一般来说，为了避免容器之间相互影响，最好是每个应用单独指定容器目录。

接下来通过以下命令安装酷狗音乐：

```
WINEPREFIX=~/.deepinwine/com.kugou.deepin/ deepin-wine6-stable  
kugou_10193.exe
```



图 51 从安装程序开始安装酷狗音乐

通过如图所示的酷狗音乐安装界面进行安装，安装完成之后检查使用的容器路径下是否正确生成了酷狗音乐的运行 exe 程序。在容器目录中进入 `drive_c` 目录，在统信 UOS 系统中的 deep-wine 容器中的 `drive_c` 即是模拟的 Windows 操作系统中的 C 盘。如果容器的 WINEARCH 为 win32，则会生成 Program Fi

le 目录，如果容器的 WINEARCH 为 win64，则会生成 Program File 和 Program File (x86)两个目录，根据应用程序是 32 位还是 64 位不同存放。在对应的安装目录中检查，找到已经生成了酷狗音乐的运行 exe 文件。

通过 deepin-wine6-stable 命令运行酷狗音乐程序：

```
WINEPREFIX=~/.deepinwine/com.kugou.deepin deepin-wine6-stable  
~/.deepinwine/com.kugou.deepin/drive_c/Program Files/Kugou/KGMusic/  
c/KuGou.exe
```

如图所示，可以正常启动酷狗音乐。



图 52 通过 deepin-wine6-stable 命令运行酷狗音乐

6.2.4. 常见问题

一般来说，通过应用商店和 winetricks 安装的应用是经过完整 wine 社区测试适配的应用，功能比较完善。但是通过 deepin-wine 工具手动安装的应用出现缺陷的概率较大，需要通过分析日志和应用自身的提示来进行排查和修复。

deepin-wine 工具可以通过环境变量 WINEDEBUG 来打开日志通道输出，日志通道分为 trace、fixme、warn、err 几个级别。其中 trace 代表程序运行轨迹，调用哪些 API 以及关键信息输出，fixme 表示在 wine 中待完善的接口，

功能未实现或有已知缺陷，warn 和 err 代表程序运行过程中出现严重程度不同的警告和错误。打开日志的方式以日志类别和通道名称的组合，如果想使用全部通道，可以使用 all。在 wine 日志中，fixme 以上级别的日志是默认输出的。

例如，打印 file 通道的日志可以以如下方式添加：

```
WINEDEBUG=trace+file
```

日志通道的名称可以在 wine 官网下载 wine 的开源代码，在代码中如下所示的部分即为通道名称。

```
WINE_DEFAULT_DEBUG_CHANNEL(file)
```

可以根据应用表示出的问题来查找对应通道的日志，来判断以及解决问题。

部分应用对于安装的 Windows 版本和基础组件有相关要求，可以通过 winecfg 来对应用所在容器进行配置，达到与 Windows 操作系统相同的基础配置。使用方法如下所示：

```
WINEPREFIX=~/.deepinwine/com.kugou.deepin deepin-wine6-stable  
winecfg
```

上面以酷狗音乐的容器为例，在具体使用中需要替换为目标应用所在的容器。执行完 winecfg 的命令后会弹出 winecfg 的用户交互界面。



图 53 winecfg 的页面展示(1)

通过如图所示的 winecfg 应用程序页面下方可以设置 wine 容器的对应 Windows 版本，可以使 wine 容器在配置之后模拟成所选择的 Windows 版本，以达到应用软件对 Windows 系统的要求。

在函数库界面可以选择哪些动态链接库使用 Windows 原生或禁用，由于部分 wine 实现的 Windows API 存在缺陷或由于 Windows 的内部组件存在未公开的自定义行为，这部分可以通过 winecfg 配置直接使用 Windows 的原生动态链接库，通过 winecfg 配置了 quartz.dll 使用 Windows 原生，那么应用程序在运行时就会优先使用软件当前目录下或容器中系统目录 system32 中的原生 quartz.dll。



图 54 winecfg 的页面展示(2)

同样，可以在这个页面指定禁用的组件。例如在应用程序中，存在明显缺陷的子进程但不影响软件主体使用功能的，可以予以禁用，例如软件自动更新进程，通常为 xxx-update.exe。

winecfg 的其它页面，可以配置显示、磁盘、显示分辨率、字体、声音的相关内容，可以根据用户对于应用软件的需求进行选择。

部分应用程序在 windows 安装时会添加注册表项来支撑应用软件的运行和使用，在 deepin-wine 中，可以通过 regedit 来添加容器中的注册表项，方法如下所示：

```
WINEPREFIX=~/.deepinwine/com.kugou.deepin deepin-wine6-stable  
regedit
```

输入命令后会弹出如图所示的界面：

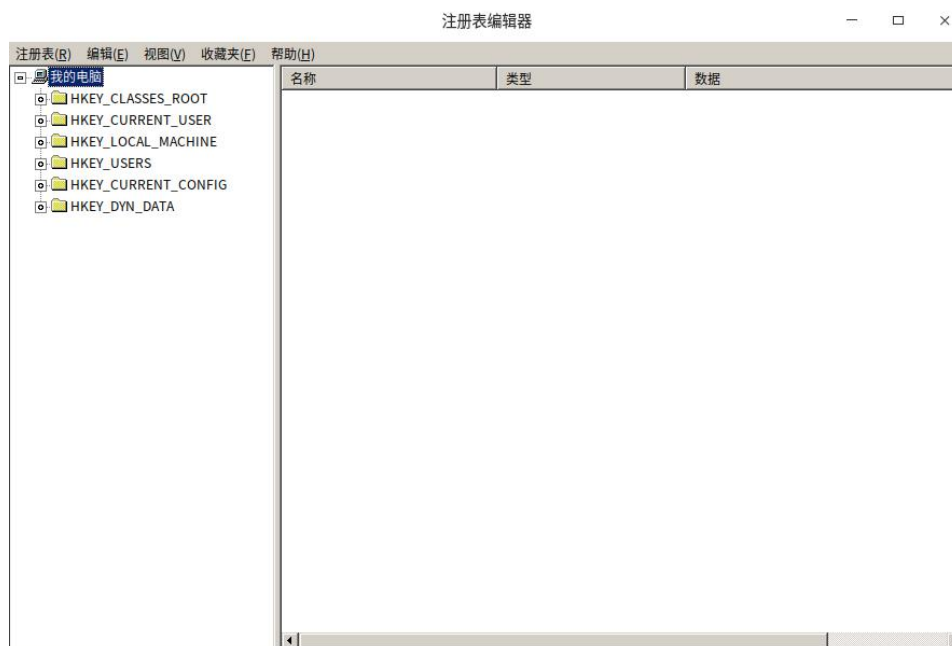


图 55 regedit 的交互界面

在 regedit 中的交互界面中，可以手动修改对应的注册表项，也可以通过上方导航菜单栏中的选项进行注册表的导入和导出。

部分应用软件需要其它 Windows 组件的支撑，如 mfc(Microsoft Foundation Classes 微软基础类库)，需要在对应的容器中安装。下面以需要 mfc 支持的 Windows graphedit 进行说明。

通过 wine 命令运行 graphedit 发现运行不起来，打开 module 的日志通道输出结果如图所示：

```
uos@yangkun:~/deepinwine/ge$ WINEPREFIX=~/deepinwine/ge WINEDEBUG=err+module deepin-wine6-stable ~/111/graphedt.exe
wine version: 6.0
0024:fixme:module:process_init load main dll 0x10000000 - 0x1003d000 override 0x10000000 - 0x50000000
0024:err:module:import_dll Library MFC42.dll (which is needed by L"Z:\\home\\uos\\111\\graphedt.exe") not found
0024:err:module:LdrInitializeThunk Importing dlls for L"Z:\\home\\uos\\111\\graphedt.exe" failed, status c0000135
```

图 56 运行 graphedit 时的错误日志

发现在运行过程中缺少运行时库 MFC42.dll，该运行时库属于 Windows MFC 类库，下面通过 winetricks 安装 MFC 相关组件。

通过前面介绍的命令启动 winetricks，选择安装 Windows 组件，在组件列

表中选择 mfc42，如图所示：



图 57 在 winetricks 中安装 mfc42 运行时库

点击确定之后按照提示进行安装，安装完成之后再通过命令运行 graphedit，发现运行成功，如图所示：

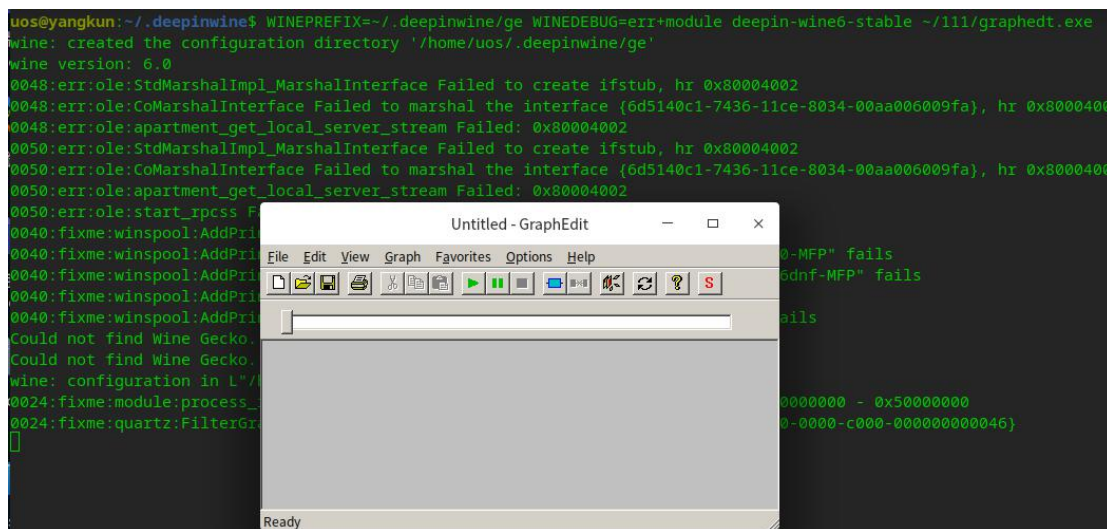


图 58 graphedit 正确运行示例

6.3. 跨平台开发框架

6.3.1. Electron 应用开发

如果您能够搭建一个网站，那么您就可以搭建一个桌面应用。Electron 是这样一个框架，它可以帮助您使用 JavaScript/Html/Css 等网站相关技术，非常快速而容易地搭建一个原生应用。它允许开发人员使用现有的网络开发技能来构建在 Windows、macOS 和 Linux 操作系统上运行的类原生应用。

Electron 结合了 Chromium（Google Chrome 的开源项目）用于渲染应用程序的用户界面，以及 Node.js 用于访问系统资源和与文件系统交互。

Electron 的一个关键优势是能够使用单一代码库创建跨平台应用程序。开发人员可以使用网络技术编写应用程序一次，然后将其打包为不同操作系统的独立可执行文件。这极大地简化了开发过程，并减少了对特定平台代码的需求。

Electron 被广大开发人员使用，并已被各种公司和组织采用来构建流行的应用程序，如 VS Code、Slack 和 Discord。其丰富的生态系统提供了各种工具、库和插件，使其成为使用网络技术开发桌面应用程序的热门选择。

6.3.1.1. 环境安装

大部分开发者使用 Node.js 来创建 Electron 项目，所以需要安装 Node.js。当前社区版的 Node.js 版本为 10.21.0,专业版的 nodejs 版本为 10.24.0，官方的最新 LTS 版本为 18.16.0。官网地址：<https://nodejs.org>

- 安装 Node.js 和 npm。

通过命令安装：

```
sudo apt install nodejs npm
```

通过 Node.js 官网安装：

在 <http://www.nodejs.com.cn/download.html> 下载安装(本章开发案例选择下载 node-v18.16.1-linux-x64.tar.xz)，下载压缩包，解压至您指定的文件夹，再通过 `export PATH=/YOURPATH/node-vxx.x.x-linux-x64/bin:$PATH` 命令，来添加 Node.js 的执行目录。如果你需要在每个终端会话中都能够访问 Electron 命令，你可以将环境变量设置添加到你的操作系统配置文件（例如.bashrc 或.bash_profile）中。这样，在每次启动终端时，都会自动加载这些环境变量。

配置完毕用 `node -v` 命令查看是否成功。当出现版本号，表示成功。

```
node -v
```

- 安装 electron。

```
npm install electron --save-dev
```

或通过网站下载 <https://registry.npmmirror.com/binary.html?path=electron/> 下载压缩包（本章开发案例选择下载 electron-v18.0.1-linux-x64.zip），解压后，同上一步骤一样，添加 electron 执行文件目录到 PATH。

6.3.1.2. 环境配置

不需要进行特别的环境配置。

6.3.1.3. 开发案例

创建一个新的文件夹，作为你的项目目录。

```
mkdir myElectron
cd myElectron
```

在项目目录下创建一个 package.json 文件，其中包含项目的相关信息和依赖。

```
npm init -y
```

命令执行结果如下：

```
{
  "name": "app1",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \\\"Error: no test specified\\\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

编写一个简单的 index.js 文件，作为主进程的入口文件

```
const { app, BrowserWindow } = require('electron');
function createWindow() {
  const win = new BrowserWindow({
    width: 800,
    height: 600,
    webPreferences: {
      nodeIntegration: true
    }
  });
  win.loadFile('index.html');
}
```

```
app.whenReady().then(createWindow);
```

在项目目录下创建一个 index.html 文件，并添加一些基本的 HTML 内容。

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Hello UOS!</title>
</head>
<body>
<h1>中国操作系统领创者!</h1>
<p>给世界更好的选择!</p>
</body>
</html>
```

run 你的程序，看看效果，使用 ls 命令看到你的工程目录下的文件目录应

该是这样：

```
index.html  index.js  node_modules  package.json  package-lock.j
son
```

在该工程目录下面执行命令：

```
electron .
```



图 59 Electron 开发案例

6.3.1.4. 常见问题

- 兼容问题。

源里面的 Node.js、npm 的版本不够新或者新旧不一致，会导致在调用接口的时候发生不兼容问题，建议采用文中提及的方式去官网下载相应版本，这样最大程度的减少安装环境过程中遇到的问题。

注意 package.json 中的 main 是 js 程序的入口，在编写程序的过程中，需要保持跟你的 js 主程序名称一致，npm 自动生成的 index.js，你需要修改保持跟你的主程序名称一致。

第七章 UOS 第三方应用开发

7.1. 本章目的

统信 UOS 操作系统作为一款优秀的 Linux 发行版，同样继承了 Linux 开发平台的所有优势，不但具备开源灵活性，提供稳定可靠的环境，还拥有广泛的工具和库支持，允许个性化定制，享有强大的命令行工具，而且依托庞大的社区支持和持续更新，为开发者提供了一个理想的创新舞台，助力他们构建安卓应用和 AI 应用等多样化、高质量的应用程序。本章节通过构建不同基础的开发环境，分别来满足 Android 应用和 AI 应用的开发，为开发者提供一个可靠的基础开发平台，来满足基于 UOS 进行第三方应用的开发。

7.2. Android 应用开发

当前的移动应用开发技术主要有原生开发和跨平台开发两种方式。

原生开发是指使用指定平台提供的编程语言和开发工具进行开发。比如，对于 iOS 平台，通常使用 Swift 或 Objective-C 编程语言，使用 Xcode 开发工具；对于 Android 平台，通常使用 Java 或 Kotlin 编程语言，使用 Android Studio 开发工具。使用原生开发方式可以获得最佳的性能和用户体验，但需要开发者熟悉多个平台的开发环境和编程语言，对开发者能力要求较高；而且相比跨平台开发方式增加了更多的开发和维护成本。

跨平台开发方式是使用一种支持跨平台的开发语言和框架，通过跨平台的编译器或解释器生成不同目标平台的应用程序。Flutter 和 React Native 是目前比较流行的跨平台开发框架。下面我们逐个介绍一下 UOS 系统如何使用这 2 个跨平台开发框架来开发 Android 移动应用。

7.2.1. Flutter Android 应用开发

Flutter 是谷歌开发的可移植的跨平台的 UI 工具包，使用 Dart 编程语言进行开发。Flutter 具有快速开发、高性能、美观的 UI 设计和丰富的组件库等优点，同时支持 iOS、Android、Web 和桌面平台（Windows、Linux、Mac OS X），开发者可以只使用一套代码库来开发不同平台的应用程序。除此之外，Flutter 还有以下的优势：

- 接近原生的性能。Flutter 使用编程语言 Dart 并编译成机器码。主机设备能理解此代码，从而确保快速有效的性能。
- 快速、一致且可自定义的渲染。Flutter 不依赖于特定平台的渲染工具，而是使用 Google 的开源 Skia 图形库来渲染 UI。这为用户提供了一致的视觉效果，无论使用什么平台来运行应用程序。
- 开发人员友好的工具。通过热重载等工具，开发人员可以在无效重启应用的情况下预览代码最新修改的效果。小部件检查器等其他工具可以轻松可视化解决 UI 布局问题。

7.2.1.1. 环境安装

参考第二章的 AndroidStudio 安装方法，安装好最新的 AndroidStudio 和 JDK。

安装 Flutter SDK 安装包。从 Flutter 中文网站 (<https://flutter.cn/docs/release/archive?tab=linux>) 以获得最新 stable release 版本的 Flutter SDK，将文件解压到合适的地方，例如：

```
mkdir ~/flutter-dev  
cd ~/flutter-dev  
tar xf ~/Downloads/flutter_linux_3.13.2-stable.tar.xz
```

将 flutter 工具添加到环境变量中：

```
export PATH="$PATH:`pwd`/flutter/bin"
```

用这个命令添加 PATH 仅在当前的命令行视窗生效。要将 Flutter 永久添加到环境变量中，请将该环境变量添加到当前用户的 bashrc 或 profile 文件。

安装 Flutter 插件，启动 AndroidStudio，打开插件偏好设置(位于 File>Settings>Plugins)，选择 Marketplace(扩展商店)，选择 Flutter plugin 然后点击 Install(安装)。

7.2.1.2. 环境配置

环境配置目的是让 Flutter SDK 的相关开发命令可以找到 AndroidStudio，并使用 AndroidStudio 提供的工具和平台 SDK。

首先，添加 AndroidStudio 安装目录到 Flutter SDK 里面，确保 flutter 命令已经在当前环境变量 PATH 里面，在终端里面执行如下命令：

```
flutter config --android-studio-dir <directory>
```

<directory>需要替换为本机 AndroidStudio 安装的具体路径。上述命令执行成功后，终端会输出以下报告：

Setting "android-studio-dir" value to "<directory>".

You may need to restart any open editors for them to read new settings.

The Flutter CLI developer tool uses Google Analytics to report usage and diagnostic data

along with package dependencies, and crash reporting to send basic crash reports.

This data is used to help improve the Dart platform, Flutter framework, and related tools.

Telemetry is not sent on the very first run.

To disable reporting of telemetry, run this terminal command:
flutter --disable-telemetry.

If you opt out of telemetry, an opt-out event will be sent, and then no further information will be sent.

This data is collected in accordance with the Google Privacy Policy (<https://policies.google.com/privacy>).

其次，同意 Android SDK 协议，在终端里面执行如下命令：

```
flutter doctor --android-licenses
```

最后一步，检查所有配置是否正确，在终端里面执行如下命令：

```
flutter doctor
```

确认输出的检查结果里面，Flutter、Android toolchain、Android Studio 这 3 项都是绿色打勾状态。

注意：上述设置方式随着 Flutter 的升级可能会发生变化，如果相关的内容在您安装 Flutter 时已经失效，请访问 Flutter 社区简体中文网站：<https://flutter.cn>

er.cn 的最新安装教程安装。

7.2.1.3. 开发案例

下面我们使用 Android Studio 的 Flutter 插件快速创建一个例子应用，来展示 Flutter 是如何进行页面布局和事件处理的。

该例子应用只有一个界面页，界面页最上方是一个标题栏，中间是一个文本框显示用户在手机屏幕上点击的次数，右下角是一个加号按钮，每按一次加号按钮，中间文本框的屏幕点击次数加一。

例子应用的详细创建步骤如下：

- 启动 Android Studio，在主界面单击 New Flutter Project 进入创建项目向导。
- 选择左侧类型列表 Flutter，验证 Flutter SDK 的路径。完成后选择 Next。
- 输入项目名称（例如 my_app）。
- 项目类型选择 Application。
- 平台类型只勾选 Android。
- 点击右下角的 Create(创建)。
- 等待 Android Studio 完成项目的创建。

关于项目名称的要求，是一个类似 com.example 的包名，包名（在 iOS 里叫 Bundle ID）一般都是公司域名的反写。如果应用打算上架商店，建议一开始的时候把这个全网唯一的包名设置好，因为应用上架之后就不能再修改了。

上述步骤会创建名为 my_app 的 Flutter 项目源码文件夹，重要的文件有这

些：

- pubspec.yaml 文件指定 Flutter 应用相关的基本信息，例如其当前版本、依赖项以及其随附的资源。
- analysis_options.yaml, 此文件设置 Flutter 在分析代码时的严格程度。
- lib 目录的 main.dart 文件，这是应用的入口文件，main 函数就在这个文件。
- android 目录的 gradle.properties，配置 Gradle 构建系统的全局属性文件，比如设置 JVM 选项、Maven 仓库地址、环境变量等。
- android 目录的 local.properties，配置 Gradle 构建系统的当前模块属性文件，比如设置 Android SDK 目录、Flutter SDK 目录、Flutter 的版本等。
- android 目录的 build.gradle，设置应用程序的构建方式、依赖库、插件和其他构建相关的设置，非常重要。设置 Maven 仓库的国内镜像也在这个文件 repositories 属性里面设置。
- android 目录的 settings.gradle，设置项目中包含的模块和库，以及它们之间的依赖关系。

在项目创建完成之后，就可以按照如下方法来打包 APK 和运行 APK：

- 1) 打包 APK 方法，单击 AndroidStudio 的 Build 菜单，选择 Flutter 子菜单，再选择 Build APK，即可将 Flutter 应用打包成 Android APK 安装包，用于发布和上架到第三方应用商店，打包好的 APK 保存在项目目录的这个

文件 build/app/outputs/flutter-apk/app-release.apk。

2) 运行 APK 的方法，定位到 Android Studio 的工具栏，在目标设备列表 (target selector) 中，选择一个用于运行应用的 Android 设备；如果列表为空，可以选择 Tools>AVD Manager 创建一个虚拟机；点击工具栏中的运行按钮，或者点击菜单栏中的 Run>Run。

Flutter 的热重载(hot reload)用法，Flutter 通过热重载技术提供快速开发调试程序能力，该功能支持应用程序在运行状态下重载代码，无需重新启动应用程序就更新程序运行状态。修改一下代码，然后告诉 IDE 或者命令行工具你需要热重载，然后看一下模拟器或者设备上应用的变化。

打开 lib/main.dart，修改字符串。

```
'You have pushed the button this many times'
```

改为：

```
'You have clicked the button this many times'
```

保存源码的修改，然后点击 Android Studio 工具栏的热重载 (Hot Reload) 按钮，你会发现修改后的字符串几乎马上出现在正在运行的应用程序界面上。

7.2.1.4. 常见问题

- 视图(View)在 Flutter 中对应什么概念？

Android 中的 View 是显示在屏幕上的一切的基础。按钮、工具栏、输入框以及一切内容都是 View。而 Flutter 中 View 的大致对应着 Widget。Widget 并非完全对应于 Android 中的 View，但是在熟悉 Flutter 的工作原理的过程中

可以把它们看做声明和构建 UI 的方式。

然而，widget 和 View 还是有一些差异。首先，widget 有着不一样的生命周期：它们是不可变的，一旦需要变化则生命周期终止。任何时候 widget 或它们的状态变化时，Flutter 框架都会创建一个新的 widget 树的实例。对比来看，一个 Android View 只会绘制一次，除非调用 invalidate 才会重绘。

Flutter 的 widget 很轻量，部分原因在于它们的不可变性。因为它们本身既非视图，也不会直接绘制任何内容，而是 UI 及其底层创建真正视图对象的语义描述。

- 如何更新 Widgets?

在 Android 中，可以直接操作更新 View。然而在 Flutter 中，Widget 是不可变的，无法被直接更新，需要操作 Widget 的状态。

这就是有状态(Stateful)和无状态(Stateless)Widget 概念的来源。Stateless Widget 如其字面意思—没有状态信息的 Widget。

StatelessWidget 用于描述的用户界面的一部分不依赖于除了对象中的配置信息以外的任何东西的场景。

例如在 Android 中，这就像显示一个展示图标的 ImageView。这个图标在运行过程中不会改变，所以在 Flutter 中就使用 StatelessWidget。

如果想要根据 HTTP 请求返回的数据或者用户的交互来动态地更新界面，那么就必须使用 StatefulWidget，并告诉 Flutter 框架 Widget 的状态(State)更新了，以便 Flutter 可以更新这个 Widget。

这里需要着重注意的是，无状态和有状态的 Widget 本质上是行为一致的。它们每一帧都会重建，不同之处在于 StatefulWidget 有一个跨帧存储和恢复状态数据的 State 对象。

如果有疑问，那么记住这条规则：如果一个 Widget 会变化（例如由于用户交互），它是有状态的。然而，如果一个 Widget 响应变化，它的父 Widget 只要本身不响应变化，就依然是无状态的。

- 如何布局 Widget，XML 布局文件在哪里？

在 Android 中，通过 XML 文件定义布局，但是在 Flutter 中，要通过一个 widget 树来定义布局的。

```
@override Widget build(BuildContext context) {  
  return Scaffold(  
    appBar: AppBar(  
      title: const Text('Sample App'),  
    ),  
    body: Center(  
      child: ElevatedButton(  
        style: ElevatedButton.styleFrom(  
          padding: const EdgeInsets.only(left: 20, right: 30),  
        ),  
        onPressed: () {},  
        child: const Text('Hello'),  
      ),  
    ),  
  );  
}
```

以下示例展示了如何显示一个带有间距的简单 widget：

- 如何在布局中添加或删除一个组件？

在 Android 中，通过调用父 View 的 `addChild()`或 `removeChild()`方法动态地添加或者删除子 View。在 Flutter 中，由于 Widget 是不可变的，所以没有 `addChild()`直接对应的方法。不过，可以给返回一个 Widget 的父 Widget 传入一个方法，并通过布尔标记值控制子 Widget 的创建。

- Widget 如何实现动画？

在 Android 中，既可以通过 XML 文件定义动画，也可以调用 View 对象的 `animate()`方法。在 Flutter 里，则使用动画库，通过将 Widget 嵌入一个动画 Widget 的方式实现 Widget 的动画效果。

Flutter 通过 `Animation<double>`的子类 `AnimationController` 来暂停、播放、停止以及逆向播放动画。它需要一个 Ticker 在垂直同步(vsync)的时候发出信号，并且在运行的时候创建一个介于 0 和 1 之间的线性插值。然后就可以创建一个或多个 `Animation`，并将它们绑定到控制器上。

例如，可以使用 `CurvedAnimation` 来实现一个曲线插值的动画。在这种情况下，控制器决定了动画进度，`CurvedAnimation` 计算用于替换控制器默认线性动画的曲线值，与 Widget 一样，Flutter 中的动画效果也可以组合使用。

在构建 Widget 树的时候，需要将 `Animation` 对象赋值给某个 Widget 的动画属性，例如 `FadeTransition` 的不透明度属性，并让控制器开始动画。

- 如何使用 Canvas 进行绘制？

在 Android 中，可以使用 `Canvas` 和 `Drawable` 将图片和形状绘制到屏幕上。Flutter 也有一个类似于 `Canvas` 的 API，因为它基于相同的底层渲染引擎 Skia。

因此，在 Flutter 中用画布(canvas)进行绘制对于 Android 开发者来说是一件非常熟悉的工作。

- 如何创建自定义 Widget?

在 Android 中，一般通过继承 View 类，或者使用已有的视图类，再重载或实现以达到特定效果的方法。

在 Flutter 中，通过组合更小的 Widget 来创建自定义 Widget（而不是继承它们）。这和 Android 中实现一个自定义的 ViewGroup 有些类似，所有的构建 UI 的模块代码都已存在，不过由开发者提供不同的行为—例如，自定义布局(layout)逻辑。

7.2.2. React Native Android 应用开发

React Native 是 Facebook 开发的跨平台应用开发框架，使用 JavaScript 编程语言进行开发。React Native 具有快速开发、跨平台、可扩展等优点，同时支持 iOS 和 Android 两个移动平台，开发者可以使用相同的代码库开发不同平台的应用程序。由于苹果公司目前只允许在 Mac 电脑上开发 iOS 应用，所以 Linux 下面只能开发 Android 应用。

7.2.2.1. 环境安装

参考第二章的 AndroidStudio 安装方法，安装好最新的 AndroidStudio 和 JDK。

参考 Node 官方文档来安装 Node 16 以上的版本。

安装完 Node 后建议设置 npm 镜像（淘宝源），从而直接在国内服务器下载 node 模块依赖包，命令如下：

```
# 使用 nrm 工具切换淘宝源
npx nrm use taobao
# 如果之后需要切换回官方源可使用
npx nrm use npm
```

Yarn 是 Facebook 提供的替代 npm 的工具，可以加速 node 模块的下载，安装命令如下：

```
npm install -g yarn
```

安装完 yarn 之后就可以用 yarn 代替 npm 了，例如用 yarn 代替 npm install 命令，用 yarn add 某第三方库名代替 npm install 某第三方库名。

7.2.2.2. 环境配置

环境配置目的是让 React Native 的相关开发命令可以找到 Android Studio，并使用 Android Studio 提供的工具和平台 SDK。具体的做法是把下面的命令加入到 shell 的配置文件中。如果 shell 是 zsh，则配置文件为 ~/.zshrc，如果是 bash 则为 ~/.bash_profile（可以使用 echo \$0 命令查看所使用的 shell）：

```
export ANDROID_HOME=$HOME/Android/sdk
export PATH=$PATH:$ANDROID_HOME/tools
export PATH=$PATH:$ANDROID_HOME/tools/bin
export PATH=$PATH:$ANDROID_HOME/platform-tools
export PATH=$PATH:$ANDROID_HOME/emulator
```

上述命令的 ANDROID_HOME 环境变量的路径要替换为本机的 Android Studio SDK 路径。使用 source \$HOME/.zshrc 命令来使环境变量设置立即生效

(否则重启后才生效)。可以使用 `echo $ANDROID_HOME` 检查此变量是否已正确设置。你可以在 Android Studio 的属性设置界面中看 Android SDK 的真实路径，如图所示：

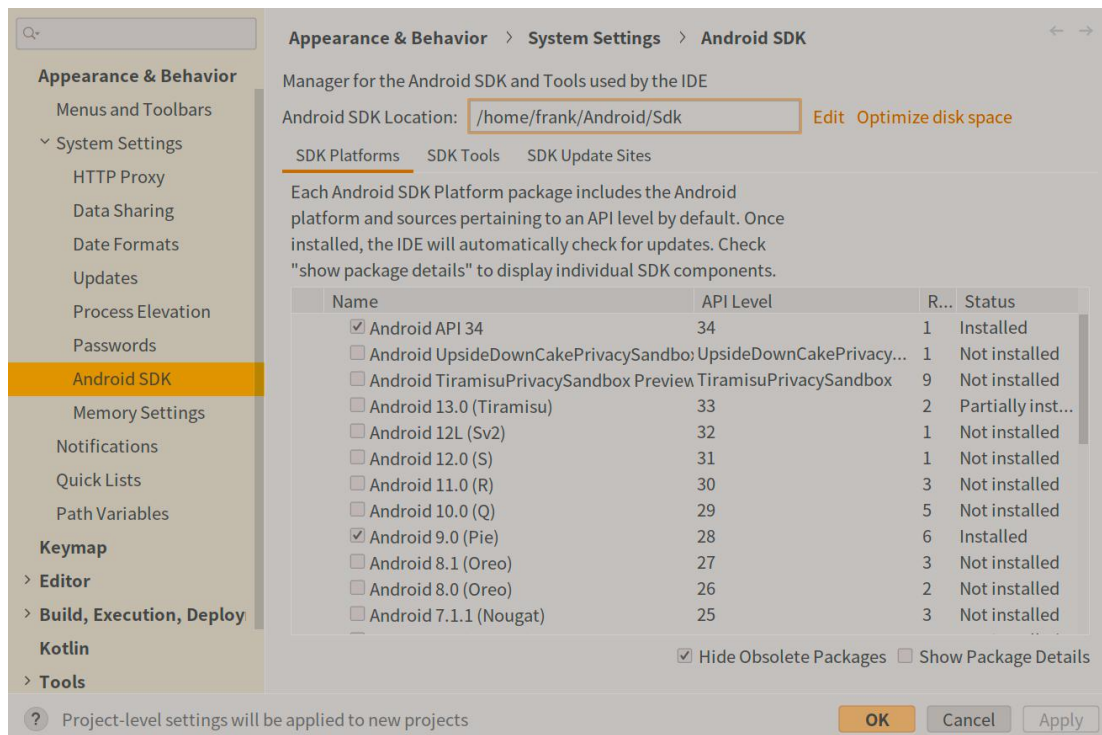


图 60 AndroidStudio SDK 设置界面

7.2.2.3. 开发案例

下面我们使用 React Native 内建的命令行工具来快速创建一个例子，用来展示 React Native 是如何进行页面布局和如何进行事件处理的。

该例子应用只有一个界面页，界面正中间是一个文本框显示用户在手机屏幕上点击的次数，右下角是一个加号按钮；每按一次加号按钮，中间文本框的屏幕点击次数加一。

例子应用的详细创建步骤如下：

如果之前全局安装过旧的 `react-native-cli` 命令行工具，请使用 `npm uninst`

`npm uninstall -g react-native-cli` 卸载掉它以避免一些冲突：

```
npm uninstall -g react-native-cli @react-native-community/cli
```

在终端里面执行如下命令，创建一个名为 `my_app` 的项目：

```
npx react-native init my_app
```

也可以额外使用 `--version` 参数（注意是两个横杠）在创建项目时候指定版本。例如：

```
npx react-native init my_app --version X.XX.X
```

上述步骤会在当前目录创建名为 `my_app` 的 React Native 项目源码目录，终端输出如下结果：

```
Welcome to React Native!
Learn once, write anywhere
✓ Downloading template
✓ Copying template
✓ Processing template
✓ Installing dependencies
Run instructions for Android:
  · Have an Android emulator running (quickest way to get started), or a device connected.
  · cd "/home/frank/project/my_app" && npx react-native run-android
```

`my_app` 源码目录重要的文件有这些：

- `App.tsx` 是应用的入口文件，在这里实现程序功能。
- `package.json`，设置项目名称、版本和依赖包。

使用任意的代码编辑器，修改 `App.tsx` 文件，将其替换为如下代码：

```
import React, { useState } from 'react';
```

```
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';

const App = () => {
  const [count, setCount] = useState(0);
  const handleButtonClick = () => {
    setCount(count + 1);
  };
  return (
    <View style={styles.container}>
      <TouchableOpacity style={styles.button} onPress={handleButtonClick}>
        <Text style={styles.buttonText}>+</Text>
      </TouchableOpacity>
      <Text style={styles.countText}>Click Count: {count}</Text>
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
  },
  button: {
    width: 60,
    height: 60,
    borderRadius: 30,
    backgroundColor: 'blue',
    justifyContent: 'center',
    alignItems: 'center',
    position: 'absolute',
    bottom: 20,
    right: 20,
  },
});
```



```
buttonText: {  
  color: 'white',  
  fontSize: 24,  
  fontWeight: 'bold',  
},  
countText: {  
  fontSize: 24,  
  marginTop: 20,  
},  
});  
export default App;
```

保存之后，就可以按照如下方法来打包 APK 和运行 APK。

- 打包 APK。

启动 Android Studio，打开 AndroidStudio 的终端窗口，在 my_app 项目目录执行命令 `./android/gradlew assembleDebug` 可打包调试版本的 APK，打包成功后 APK 文件保存在这个文件 `./android/app/build/outputs/apk/debug/app-debug.apk`；如果打包发布版本的 APK 用于发布和上架到第三方应用商店，需要额外配置签名 APK 的 keystore(密钥)文件，可详细参考这个网址：<https://reactnative.cn/docs/signed-apk-android>

- 运行 APK。

需要准备一台 Android 设备来运行 React Native Android 应用。设备既可以是真机，也可以是 Android 模拟器。确保先运行了模拟器或者连接了真机，首先打开 Android Studio 的终端窗口，然后执行命令：`npx react-native start` 或者 `yarn start` 启动 Metro 服务，终端输出如下内容：

```
Welcome to Metro v0.76.8
Fast - Scalable - Integrated
r - reload the app
d - open developer menu
i - run on iOS
a - run on Android
```

按 a 键，启动 Android 模拟器，终端继续输出如下：

```
> Task :app:installDebug
Installing APK 'app-debug.apk' on 'Pixel_2_API_34(AVD) - 14' for :a
pp:debug
Installed on 1 device.
BUILD SUCCESSFUL in 11s
38 actionable tasks: 2 executed, 36 up-to-date
info Connecting to the development server...
8081
info Starting the app on "emulator-5554"...
Starting: Intent { act=android.intent.action.MAIN cat=[android.inte
nt.category.LAUNCHER] cmp=com.my_app/.MainActivity }
BUNDLE ./index.js
LOG Running "my_app" with {"rootTag":21}
```

此时，例子应用在模拟器里面成功运行，运行效果如下图：



图 61 React Native Android 应用在模拟器里面的运行效果图

7.2.2.4. 常见问题

- 下载编译依赖包时，错误报告网络不可达？

首先通过其它途径下载缺失的依赖包，用 shasum 命令计算出这个依赖包的 sha 摘要算法值；其次，在 ~/.gradle 目录搜索 react-android 子目录，在找到的类似这个路径 ./cache/modules-2/files-2.1/com.facebook.react/react-android 的目录下的最高版本号子目录下，新建一个前一步的 sha 摘要值为目录名的子目录；最后把下载好的依赖包拷贝到前一步创建的子目录里面。

- 无法连接到 Maven 库？

打开项目的 android 子目录的 build.gradle 文件，找到 repositories 属性这一部分，新增阿里云的镜像仓库，如下：

```
allprojects {  
    repositories {  
        maven { url 'https://maven.aliyun.com/repository/google' } //新增  
        maven { url 'https://maven.aliyun.com/repository/jcenter' } //新增  
        google()  
        jcenter()  
    }  
}
```

7.3. AI 应用开发

7.3.1. CUDA 应用开发

CUDA 是英伟达（NVIDIA）推出的一种 GPGPU 编程平台和 API，它允许开发者使用 C/C++、Fortran 等编程语言在 NVIDIA 的 GPU 上进行通用计算。CUDA 最初是为了加速图形和图像处理而设计的，但随着 GPGPU 技术的发展，它已经成为了一种广泛应用于科学计算、数据分析、机器学习等领域的计算平台。

CUDA 官网：<https://developer.nvidia.com/cuda-toolkit>

7.3.1.1. 环境安装

“CUDA 工具包”包括库，调试和优化工具，编译器，文档以及用于部署应用程序的运行库。它具有支持深度学习，线性代数，信号处理和并行算法的组件。通常，CUDA 库支持所有 Nvidia GPU 系列，但在最新一代产品上表现最佳。

CUDA 工具包下载页面：<https://developer.nvidia.com/cuda-toolkit-arc>

hive。首先选取 CUDA 工具包版本，比如以安装 CUDA 11.8 为例，选取这个页面 <https://developer.nvidia.com/cuda-11-8-0-download-archive>，然后按照网页指导选择操作系统为 Linux,架构为 x86_64 或其他架构，发行版为 Debian,版本为 11，安装类型为 deb(local)，之后会弹出具体安装命令。

执行如下命令：

```
wget https://developer.download.nvidia.com/compute/cuda/11.8.0/local_installers/cuda-repo-debian11-11-8-local_11.8.0-520.61.05-1_amd64.deb
sudo dpkg -i cuda-repo-debian11-11-8-local_11.8.0-520.61.05-1_amd64.deb
sudo cp /var/cuda-repo-debian11-11-8-local/cuda-*-keyring.gpg /usr/share/keyrings/
sudo apt-get update
sudo apt-get -y install cuda
```

官方提供的命令中有 `sudo add-apt-repository contrib` 这一行，在 UOS 系统上执行会报错，可以不用执行，没有不良影响。

之后重启电脑，让系统加载新安装的 nvidia 驱动。再执行 `/usr/local/cuda-11/extras/demo_suite/deviceQuery` 命令检查一下设备状态，看到类似下面的输出说明安装成功。

```
/usr/local/cuda-11/extras/demo_suite/deviceQuery Starting...
CUDA Device Query (Runtime API) version (CUDART static linking)
Detected 1 CUDA Capable device(s)
Device 0: "NVIDIA GeForce GT 1030"
CUDA Driver Version / Runtime Version          11.8 / 11.8
CUDA Capability Major/Minor version number:    6.1
Total amount of global memory:                  1998 MBytes (2094792704 bytes)
```

```

( 3) Multiprocessors, (128) CUDA Cores/MP:      384 CUDA Cores
GPU Max Clock rate:                             1468 MHz (1.47
GHz)
Memory Clock rate:                             3004 Mhz
Memory Bus Width:                              64-bit
L2 Cache Size:                                 524288 bytes
Maximum Texture Dimension Size (x,y,z)          1D=(131072), 2D
=(131072, 65536), 3D=(16384, 16384, 16384)
Maximum Layered 1D Texture Size, (num) layers  1D=(32768), 20
48 layers
Maximum Layered 2D Texture Size, (num) layers  2D=(32768, 327
68), 2048 layers
Total amount of constant memory:                65536 bytes
Total amount of shared memory per block:        49152 bytes
Total number of registers available per block: 65536
# 省略一些内容
deviceQuery, CUDA Driver = CUDART, CUDA Driver Version = 11.
8, CUDA Runtime Version = 11.8, NumDevs = 1, Device0 = NVIDIA G
eForce GT 1030
Result = PASS

```

下一步安装cuDNN,cuDNN(CUDA Deep Neural Network library)是NVIDIA打造的针对深度神经网络的加速库,是一个用于深层神经网络的GPU加速库。如果你要用GPU训练模型,cuDNN不是必须的,但是一般会采用这个加速库。官方安装指导文档: <https://docs.nvidia.com/deeplearning/cudnn/install-guide/index.html>, 各版本下载地址: <https://developer.nvidia.com/rdp/cudnn-archive>。

使用如下命令安装 cuDNN:

```
$ wget https://developer.nvidia.com/downloads/compute/cudnn/secure/8.9.3/local_installers/11.x/cudnn-local-repo-debian11-8.9.3.28_1.0-1_amd64.deb
$ sudo dpkg -i cudnn-local-repo-debian11-8.9.3.28_1.0-1_amd64.deb
$ sudo cp /var/cudnn-local-repo-*/cudnn-local-*-keyring.gpg /usr/share/keyrings/
$ sudo apt-get update
#安装运行库
$ sudo apt install libcudnn8
# 安装开发包
$ sudo apt-get install libcudnn8-dev
# 安装代码样品
$ sudo apt-get install libcudnn8-samples
```

7.3.1.2. 环境配置

在~/.bashrc 文件中增加以下内容，配置 PATH 和 LD_LIBRARY_PATH。

```
export PATH=/usr/local/cuda-11/bin:$PATH
export LD_LIBRARY_PATH=/usr/local/cuda-11/lib64:$LD_LIBRARY_PATH
```

之后重新进入 bash，执行 `nvcc -V` 命令，输出类似如下内容：

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2022 NVIDIA Corporation
Built on Wed_Sep_21_10:33:58_PDT_2022
Cuda compilation tools, release 11.8, V11.8.89
Build cuda_11.8.r11.8/compiler.31833905_0
```

7.3.1.3. 开发案例

使用 CUDA 编译器（nvcc）编译和运行 CUDA 程序

示例 1-hello world

将以下代码保存为.cu 后缀的文件，比如 test1.cu。

```
#include <stdio.h>
#include <cuda_runtime.h>
__global__ void demo(void)
{
}
int main()
{
    demo<<<1, 1>>>();
    printf("hello, world!\n");
    return 0;
}
```

然后执行如下命令进行编译，然后再运行程序。

```
$ nvcc test1.cu -o test1
$ ./test1
hello, world!
```

看到输出了 hello, world 表示运行成功。注意这里需要使用 nvcc 编译器，不能使用其他 C 语言编译器如 gcc，并且文件名必须以 “.cu” 为后缀，否则也会在编译时报错。

这段代码里用 __global 定义了一个名为 demo 的全局函数，使用 __global__ 修饰符标识为 CUDA 的全局函数。CUDA 的全局函数由 GPU 执行，可以在 GPU 上并行地运行。这个函数没有任何参数和实际的代码体，因此它并不执行任何操作。

在 main 函数中，调用了 demo<<<1,1>>>() 语句。这是调用 CUDA 全局函数的语法，<<<1,1>>> 表示启动 1 个线程块，每个线程块中有 1 个线程。通过调用这个函数，将在 GPU 上启动一个线程来执行 demo 函数。

示例 2 向量加法

将以下代码保存为 vector_add.cu 文件。

```
/**
 * 向量加法:  $C = A + B$ .
 * 这个示例是一个非常基本的示例。
 */
#include <stdio.h>
// For the CUDA runtime routines (prefixed with "cuda_")
#include <cuda_runtime.h>
/**
 * CUDA 核设备代码，GPU 代码
 * 计算向量 A + 向量 B，将结果保存到向量 C。这三个向量都具有相同的
元素个数 numElements.
 */
__global__ void vectorAdd(const float *A, const float *B, float *C,
int numElements) {
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < numElements) {
        C[i] = A[i] + B[i] + 0.0f;
    }
}
/**
 * 宿主 main 函数
 */
int main(void) {
    // 用于检查 cuda 返回的错误代码
    cudaError_t err = cudaSuccess;
    // 打印向量长度，并计算它的内存大小。
    int numElements = 50000;
    size_t size = numElements * sizeof(float);
    printf("[Vector addition of %d elements]\n", numElements);
    // 分配宿主输入向量 A
    float *h_A = (float *)malloc(size);
```

```
// 分配宿主输入向量 B
float *h_B = (float *)malloc(size);
// 分配宿主输出向量 C
float *h_C = (float *)malloc(size);
// 验证分配是否成功
if (h_A == NULL || h_B == NULL || h_C == NULL) {
    fprintf(stderr, "Failed to allocate host vectors!\n");
    exit(EXIT_FAILURE);
}
// 使用随机数初始化宿主向量 A 和 B
for (int i = 0; i < numElements; ++i) {
    h_A[i] = rand() / (float)RAND_MAX;
    h_B[i] = rand() / (float)RAND_MAX;
}
// 分配 GPU 设备输入向量 A
float *d_A = NULL;
err = cudaMalloc((void **)&d_A, size);
if (err != cudaSuccess) {
    fprintf(stderr, "Failed to allocate device vector A (error code %s)\n",
n",
        cudaGetErrorString(err));
    exit(EXIT_FAILURE);
}
// 分配 GPU 设备输入向量 B
float *d_B = NULL;
err = cudaMalloc((void **)&d_B, size);
if (err != cudaSuccess) {
    fprintf(stderr, "Failed to allocate device vector B (error code %s)\n",
n",
        cudaGetErrorString(err));
    exit(EXIT_FAILURE);
}
// 分配 GPU 设备输入向量 C
float *d_C = NULL;
```

```
err = cudaMalloc((void **)&d_C, size);
if (err != cudaSuccess) {
    fprintf(stderr, "Failed to allocate device vector C (error code %s)!\n",
n",
    cudaGetErrorString(err));
    exit(EXIT_FAILURE);
}
// 复制宿主向量 A 和 B 内存数据到 GPU 设备相应位置中去
// 从 h_A 复制到 d_A
printf("Copy input data from the host memory to the CUDA device\n");
err = cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
if (err != cudaSuccess) {
    fprintf(stderr,
"Failed to copy vector A from host to device (error code %s)!\n",
    cudaGetErrorString(err));
    exit(EXIT_FAILURE);
}
err = cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
if (err != cudaSuccess) {
    fprintf(stderr,
"Failed to copy vector B from host to device (error code %s)!\n",
    cudaGetErrorString(err));
    exit(EXIT_FAILURE);
}
// 启动向量加法 CUDA 核
int threadsPerBlock = 256;
int blocksPerGrid = (numElements + threadsPerBlock - 1) / threadsPerBlock;
printf("CUDA kernel launch with %d blocks of %d threads\n", blocksPerGrid,
threadsPerBlock);
vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, numElements);
```

```
err = cudaGetLastError();
if (err != cudaSuccess) {
    fprintf(stderr, "Failed to launch vectorAdd kernel (error code %s)!
\n",
    cudaGetErrorString(err));
    exit(EXIT_FAILURE);
}
// 从 GPU 设备结果向量复制内存数据到宿主结果向量
printf("Copy output data from the CUDA device to the host mem
ory\n");
err = cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
if (err != cudaSuccess) {
    fprintf(stderr,
    "Failed to copy vector C from device to host (error code %s)!\\n",
    cudaGetErrorString(err));
    exit(EXIT_FAILURE);
}
// 释放 GPU 设备内存
err = cudaFree(d_A);
if (err != cudaSuccess) {
    fprintf(stderr, "Failed to free device vector A (error code %s)!\\n",
    cudaGetErrorString(err));
    exit(EXIT_FAILURE);
}
err = cudaFree(d_B);
if (err != cudaSuccess) {
    fprintf(stderr, "Failed to free device vector B (error code %s)!\\n",
    cudaGetErrorString(err));
    exit(EXIT_FAILURE);
}
err = cudaFree(d_C);
if (err != cudaSuccess) {
    fprintf(stderr, "Failed to free device vector C (error code %s)!\\n",
    cudaGetErrorString(err));
```

```
exit(EXIT_FAILURE);  
}  
// 释放宿主内存  
free(h_A);  
free(h_B);  
free(h_C);  
printf("Done\n");  
return 0;  
}
```

然后执行如下命令进行编译，然后再运行程序。

```
$ nvcc vector_add.cu -o vector_add  
$ ./vector_add  
[Vector addition of 50000 elements]  
Copy input data from the host memory to the CUDA device  
CUDA kernel launch with 196 blocks of 256 threads  
Copy output data from the CUDA device to the host memory  
Done
```

以上代码的主要流程是：

- 准备待处理数据。
- 在 GPU 设备上分配存储空间。
- 把数据从宿主拷贝到 GPU 设备。
- GPU 设备执行运算：向量加法。
- 把结果从 GPU 设备拷贝回宿主。
- 结束后释放 GPU 设备内存。
- 释放宿主内存。

GPU 设备上运行的代码称为设备代码（device code）。CPU 上运行的代

码称为宿主代码（host code）。从软件上的逻辑结构来看，GPU 设备中的 grid 里面有多个线程块（block），线程块里面有多个线程（thread）。

```
int threadsPerBlock = 256;
int blocksPerGrid = (numElements + threadsPerBlock - 1) / threadsPerBlock;
```

以上代码设置每个线程块分为 256 个线程，然后计算 grid 中有多少个线程块。

```
vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, numElements);
```

以上代码算是本例的核心代码，让 CUDA 核开始执行 vendorAdd 函数，使用<<<线程块数,线程数>>>这种记法。

```
int i = blockDim.x * blockIdx.x + threadIdx.x;
```

以上代码是 CUDA 核设备代码 vectorAdd 函数中计算当前线程应该访问向量哪个索引位置的表达式，vectorAdd 函数被各个线程执行。blockDim.x 表示每个线程块（block）中的线程数量，blockIdx.x 表示当前线程块的索引，threadIdx.x 表示当前线程在线程块中的索引。通过这个表达式可以计算出当前线程在整个向量中的索引位置 i，如此并行执行无误。

更多 CUDA 代码例子请查看官方项目 <https://github.com/NVIDIA/cuda-samples>。

7.3.1.4. 常见问题

- CUDA 支持哪些操作系统？

CUDA 支持 Windows、Linux 和 Mac 操作系统。有关完整列表，请参阅最新的 CUDA 工具包发行说明。最新版本可从 <http://docs.nvidia.com> 获取。

- 哪些 GPU 支持运行 CUDA 加速的应用程序？

CUDA 是所有 NVIDIA GeForce、Quadro 和 Tesla GPU 以及 NVIDIA GRID 解决方案的标准功能。完整列表可以在 CUDA GPU 网页 <https://developer.nvidia.com/cuda-gpus> 上找到。

- 什么是“计算能力”？

GPU 的计算能力决定了其一般规格和可用功能。有关详细信息，请参阅 CUDA C 编程指南中的计算功能部分 <http://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#compute-capabilities>。

7.3.2. OpenCV 应用开发

OpenCV 的全称是 Open Source Computer Vision Library，是一个跨平台的计算机视觉库。OpenCV 是由英特尔公司发起并参与开发，以 BSD 许可证授权发行，可以在商业和研究领域中免费使用。OpenCV 可用于开发实时的图像处理、计算机视觉以及模式识别程序。

OpenCV 官方网站：<https://opencv.org/>

7.3.2.1. 环境安装

以 python 语言为例，通过系统软件仓库安装：

```
sudo apt install python3-opencv
```

通过 pip python 包管理器安装：

```
pip3 install opencv-python
```

可以编写 python 脚本检查是否安装成功：

```
import cv2
print(cv2.__version__)
```

或者去官网 <https://opencv.org/releases/> 下载最新版本然后编译安装。

7.3.2.2. 环境配置

无需特殊环境配置。

7.3.2.3. 开发案例

- 示例 1 显示图像。

准备一张图片放置在工作目录，命名为 abc.jpg。

编写一个文件，比如命名为 read_show_img.py，输入以下代码：

```
# -*- coding: utf-8 -*-
import cv2
img = cv2.imread('./abc.jpg', cv2.IMREAD_COLOR)
cv2.imshow('image', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

执行如下命令运行代码：

```
python3 read_show_img.py
```

运行代码后将会弹出一个窗口，以彩色显示这张图片，按下任意键后程序退出。

- 示例 2 鼠标绘图。

编写一个文件，比如命名为 mouse_draw.py，输入以下代码：


```
# -*- coding: utf-8 -*-
import cv2
import numpy as np
drawing = False
# mode true 是矩形, mode false 是小圆形, 画线。
mode = False
# 绘图起点坐标
ix, iy = 0, 0
img = np.zeros((600, 600, 3), np.uint8)
# 创建回调函数
def mouse_cb(event, x, y, flags, param):
    print('mouse_cb', event, x, y, flags, param)
    global ix, iy, drawing, mode, img
    if event == cv2.EVENT_LBUTTONDOWN:
        drawing = True
        # 当按下左键时, 设置为起始位置坐标
        ix, iy = x, y
        # 当鼠标左键按下后, 再移动鼠标来绘制图形。
        elif event == cv2.EVENT_MOUSEMOVE and flags == cv2.EVENT_FLAG_LBUTTON:
            if drawing == True:
                if mode == True:
                    print('draw rectangle')
                    cv2.rectangle(img, (ix, iy), (x, y), (0, 255, 0), -1)
                else:
                    print('draw circle')
                    # 绘制圆圈, 小圆点连在一起就成了线, 3 代表了笔画的粗细
                    cv2.circle(img, (x, y), 3, (0, 0, 255), -1)
            elif event == cv2.EVENT_LBUTTONUP:
                # 当鼠标左键松开时停止绘画。
                drawing = False
                if mode == True:
                    cv2.rectangle(img, (ix, iy), (x, y), (0, 255, 0), -1)
                else:
```

```
cv2.circle(img,(x,y),5,(0,0,255),-1)
cv2.namedWindow('image')
# 设置鼠标事件处理回调函数
cv2.setMouseCallback('image', mouse_cb)
font = cv2.FONT_HERSHEY_SIMPLEX
cv2.putText(img, 'key Q quit,key M switch mode', (30, 30), font, 1,
(255,255,255), 2)
while(True):
    cv2.imshow('image', img)
    k = cv2.waitKey(1) & 0xFF
    if k == ord('m'):
        # m 键切换模式: mode true 是矩形, mode false 是小圆形, 画线。
        mode = not mode
    elif k == ord('q'):
        # q 退出
        break
```

执行如下命令运行代码：

```
python3 mouse_draw.py
```

运行代码之后，可以看到弹出一个窗口，窗口显示文字 “key Q quit,key M switch mode”，意思是 Q 键退出，M 键切换模式。一开始，按下鼠标左键后移动鼠标，可以在窗口绘制出很多红色的小点，然后按键盘上的 M 键，切换模式为绘制矩形，再按下鼠标左键后移动鼠标，可以在窗口绘制出绿色的矩形。

这个典型的例子可以帮助我们更好地理解如何使用 openCV 做一些简单的人机交互。

● 示例 3 模糊图像。

准备一张图片放置在工作目录，命名为 opencv_logo.png，最好尺寸小一些。

编写一个文件，比如命名为 blur.py，输入以下代码：

```
# -*- coding: utf-8 -*-  
import cv2  
from matplotlib import pyplot as plt  
img = cv2.imread('opencv_logo.png')  
# 原图  
plt.subplot(121), plt.imshow(img), plt.title('Original')  
plt.xticks([], plt.yticks([]))  
# 模糊图  
blur = cv2.GaussianBlur(img, (99, 99), 0)  
plt.subplot(122), plt.imshow(blur), plt.title('Blurred')  
plt.xticks([], plt.yticks([]))  
plt.show()
```

执行如下命令运行代码：

```
python3 blur.py
```

运行代码之后，可以看到一个窗口显示 2 张图片，左边是原图，右边是经过高斯模糊后的图片。

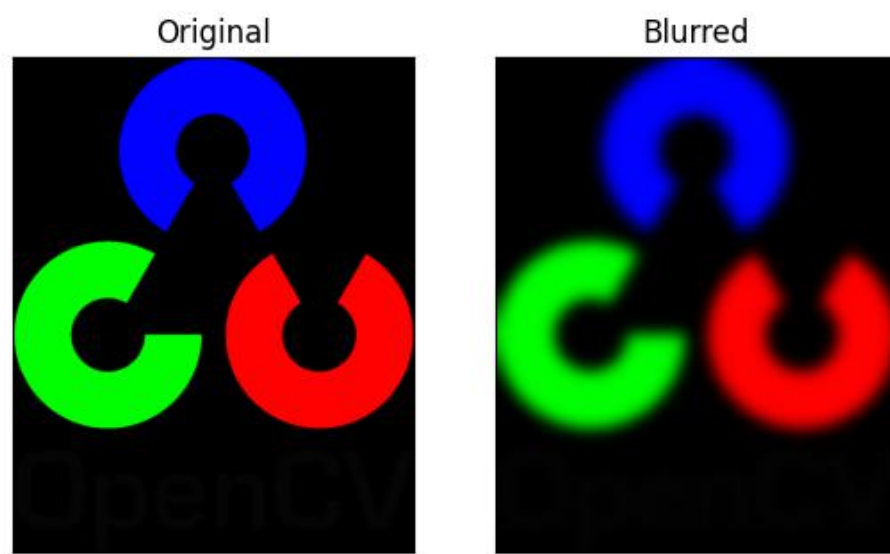


图 62 程序运行结果窗口

7.3.2.4. 常见问题

- 遇到了版本兼容性问题怎么办，提示函数未定义、参数错误？

OpenCV 有多个版本，不同版本之间的 API 和功能可能会有所不同。如果你在学习或查找示例代码时使用了与你安装的版本不兼容的代码，可能会导致错误或功能失效。确保你的代码与所使用的 OpenCV 版本兼容，并查阅相应版本的文档。仔细研究函数的参数和返回值，并注意任何变化或废弃的功能。如果需要在不同版本间切换，可以考虑使用条件语句或版本检查来适应不同的 API。另外，及时更新 OpenCV 版本并保持最新的文档和示例代码也是解决版本兼容性问题的有效方法。

- 为什么图像加载不出来，加载报错？

OpenCV 用于处理图像和视频，但在加载图像时可能会遇到问题。常见问题包括文件路径错误、文件格式不支持、图像加载失败等。请确保提供正确的文件路径，并检查所使用的图像格式是否受支持。

- 为什么图像显示不出来？

在使用 OpenCV 时，可能会遇到图像显示不正确或无法显示的问题。这可能是由于窗口未正常创建、图像尺寸不匹配、颜色通道顺序错误等引起的。确保使用适当的窗口管理函数（如 `cv2.imshow()`）和正确的图像尺寸。

- 我得到了一个图像，但它的颜色看起来很奇怪？

这和图像通道顺序有关，OpenCV 默认使用 BGR（蓝绿红）通道顺序，而不是常见的 RGB 顺序。这可能会导致图像的颜色显示不正确。需要注意在图像

处理过程中正确处理通道顺序或进行通道转换。

7.3.3. TensorFlow 应用开发

TensorFlow 是一个用于机器学习的端到端开源平台。它拥有一个由工具、库和社区资源组成的全面、灵活的生态系统，使研究人员能够推动机器学习领域的最先进技术，并使开发人员能够轻松构建和部署机器学习驱动的应用程序。

TensorFlow 最初是由谷歌大脑机器学习团队的研究人员和工程师开发的，用于进行机器学习和神经网络研究。然而，该框架具有足够的通用性，也可以用于其他领域。

TensorFlow 官方网站：<https://www.tensorflow.org>。

7.3.3.1. 环境安装

通过 pip python 包管理器安装。

对于有支持 CUDA 的显卡机器。

```
pip3 install tensorflow
```

对于只有 CPU 的机器，可以安装轻量包。

```
pip install tensorflow-cpu
```

安装完毕后，可以将以下代码保存为 test.py 文件并运行来测试能否使用 GPU 加速。

```
import tensorflow as tf
print(tf.config.list_physical_devices('GPU'))
```

在有支持 CUDA 的显卡机器上，运行 python3 test.py 之后如果可以看到

类似下面的结果，说明有 GPU 可以使用。

```
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

如果实际有 GPU,但是 tensorflow 库却不能发现 GPU,可能是没有安装 CUDA 工具包或 cuDNN 库，或者是 tensorflow 库不支持目前的 CUDA 工具包版本。

7.3.3.2. 环境配置

无需特殊配置。

7.3.3.3. 开发案例

示例：基本分类，对服装图像进行分类

这个例子将使用 tensorflow 的 keras 库（构建和训练模型的高级 API）训练一个神经网络模型，对运动鞋、衬衫等服装图像进行分类。使用了 FashionMNIST 数据集（<https://github.com/zalandoresearch/fashion-mnist>）进行训练和评估，该数据集包含 10 个类别的 70,000 个灰度图像。这些图像以低分辨率（28x28 像素）展示了单件衣物。

保存以下代码为 keras_classification.py 文件：

```
# TensorFlow and tf.keras
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
print('tf version:', tf.__version__)
# 从网络上加载 Fashion MNIST 数据集
fashion_mnist = tf.keras.datasets.fashion_mnist
```

```
(train_images, train_labels), (test_images, test_labels) = fashion_mnist.load_data()

# 服装分类
class_names = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot']

# 查看第一份训练数据
plt.figure()
plt.imshow(train_images[0])
plt.colorbar()
plt.grid(False)
plt.show()

# 预处理数据,将图像像素值从 0 到 255 之间缩小至 0 到 1 之间。
train_images = train_images / 255.0
test_images = test_images / 255.0

# 查看 5 x 5 个训练图像和标签
plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[train_labels[i]])
plt.show()

# 先配置模型的层
model = tf.keras.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10)
])

# 再编译模型
model.compile(optimizer='adam',
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
```

```
metrics=['accuracy'])
# 训练模型
# 将训练数据发给模型
model.fit(train_images, train_labels, epochs=10)
# 用测试数据评估模型
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)

print('\nTest accuracy:', test_acc)
# 进行预测
probability_model = tf.keras.Sequential([model,
tf.keras.layers.Softmax()])
predictions = probability_model.predict(test_images)
def plot_image(i, predictions_array, true_label, img):
    true_label, img = true_label[i], img[i]
    plt.grid(False)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(img, cmap=plt.cm.binary)
    predicted_label = np.argmax(predictions_array)
    if predicted_label == true_label:
        color = 'blue' # 正确蓝色表示
    else:
        color = 'red' # 错误红色表示
    plt.xlabel("{} {:.2f}% {}".format(class_names[predicted_label],
    100*np.max(predictions_array),
    class_names[true_label]),
    color=color)
def plot_value_array(i, predictions_array, true_label):
    true_label = true_label[i]
    plt.grid(False)
    plt.xticks(range(10))
    plt.yticks([])
    thisplot = plt.bar(range(10), predictions_array, color="#777777")
    plt.ylim([0, 1])
```



```
predicted_label = np.argmax(predictions_array)
thisplot[predicted_label].set_color('red')
thisplot[true_label].set_color('blue')
# 显示 5 x 3 个测试数据和预测结果
num_rows = 5
num_cols = 3
num_images = num_rows*num_cols
plt.figure(figsize=(2*2*num_cols, 2*num_rows))
for i in range(num_images):
    plt.subplot(num_rows, 2*num_cols, 2*i+1)
    plot_image(i, predictions[i], test_labels, test_images)
    plt.subplot(num_rows, 2*num_cols, 2*i+2)
    plot_value_array(i, predictions[i], test_labels)
plt.tight_layout()
plt.show()
```

以上代码的主要流程是：

- 导入 Fashion MNIST 数据集。
- 浏览查看数据：查看第一份训练数据。
- 预处理数据：数据集中图像的像素值处于 0 到 255 之间，除以 255.0

将这些值缩小至 0 到 1 之间，以相同方式对训练数据和测试集进行处理。

- 构建模型构建神经网络需要先配置模型的层，然后再编译模型。编译模型时可以配置：

optimizer 优化器：决定模型如何根据其看到的数据和自身的损失函数进行更新。

loss 损失函数：测量模型在训练期间的准确程度。你希望最小化此函数，以便将模型“引导”到正确的方向上。

metrics 指标：用于监控训练和测试步骤。这个例子使用了准确率，即被正确分类的图像的比率。

- 训练模型将训练数据馈送给模型，这里例子中训练数据在 train_images 和 train_labels 数组中，模型学习将图像和标签关联起来，要求模型对测试集（本例中的 test_images 数组）进行预测，验证预测是否与结果标签匹配（本例中的 test_labels 数组）。
- 进行预测，附加一个 Softmax 层，将结果转换为更容易理解的概率，再绘制成图表方便查看。

执行以下命令运行程序：

```
python3 keras_classification.py
```

程序运行一会儿之后，可以看到类似下面的输出，即完成了模型的训练。

```
Epoch 7/10
1875/1875 [=====] - 2s 1ms/step - loss: 0.2687 - accuracy: 0.8994
Epoch 8/10
1875/1875 [=====] - 2s 1ms/step - loss: 0.2583 - accuracy: 0.9043
Epoch 9/10
1875/1875 [=====] - 2s 1ms/step - loss: 0.2486 - accuracy: 0.9073
Epoch 10/10
1875/1875 [=====] - 2s 1ms/step - loss: 0.2398 - accuracy: 0.9109
313/313 - 0s - loss: 0.3344 - accuracy: 0.8848 - 322ms/epoch - 1ms/step
Test accuracy: 0.8848000168800354
```

经过十次训练之后，模型的准确率是 88.48%，最终会打开如下图所示的窗口。

□。

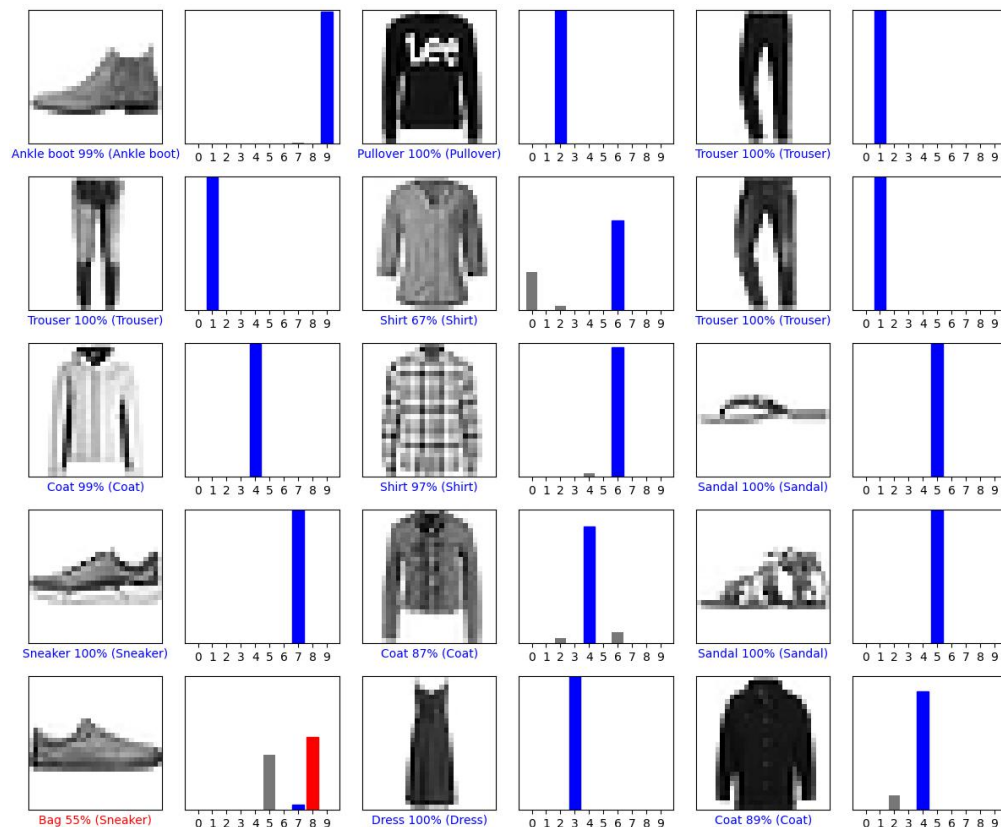


图 63 程序运行结果窗口

7.3.3.4. 常见问题

- 遇到了"ModuleNotFoundError: No module named 'tensorflow'"错误？

这个错误通常表示您的系统中没有安装 TensorFlow。请确保您已经正确安装了 TensorFlow 并且可以在 python shell 上运行 `import tensorflow` 语句。如果安装过程有问题，请尝试重新安装或查阅 TensorFlow 的安装文档。

- 如何创建一个简单的神经网络模型？

在 TensorFlow 中，您可以使用 Keras API 来创建神经网络模型。Keras 提供了一组简单易用的接口，用于定义和训练神经网络。您可以参考 TensorFI

ow 官方文档中的 Keras 教程来学习如何创建一个简单的神经网络模型。

- 如何加载和预处理数据？

TensorFlow 提供了多种数据加载和预处理工具。您可以使用 TensorFlow 的数据集 API 来加载常见的数据集，也可以使用 NumPy 或 Pandas 等库来加载自定义数据。数据预处理可以使用 TensorFlow 的数据转换函数，如图像处理函数、标准化函数等。

- 如何训练和评估模型？

在 TensorFlow 中，您可以使用模型的 `fit()` 方法来训练模型，并使用 `evaluate()` 方法来评估模型性能。您需要将输入数据和标签提供给这些方法，并指定训练的批次大小、训练轮数等参数。

7.3.4. PyTorch 应用开发

PyTorch 是一个开源的深度学习框架，该框架由 Facebook 人工智能研究院的 Torch7 团队开发，它的底层基于 Torch，但实现与运用全部是由 Python 来完成。该框架主要用于人工智能领域的科学研究与应用开发。PyTorch 最主要的功能有两个，其一是拥有 GPU 张量，该张量可以通过 GPU 加速，达到在短时间内处理大数据的要求；其二是支持动态神经网络，可逐层对神经网络进行修改，并且神经网络具备自动求导的功能。

PyTorch 官方网站：<https://pytorch.org/>

7.3.4.1. 环境安装

可以查看官方安装指导页面 <https://pytorch.org/get-started/locally/>，比如选择构建为 Stable(2.0.1)稳定版，操作系统为 Linux,包管理器为 pip,语言为 python,计算平台为 CUDA 11.8。

以通过 pip python 包管理器安装适配计算平台 CUDA 11.8 版本的 pytorch 为例，执行如下命令安装：

```
pip3 install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
```

将以下代码保存为 test.py 文件。

```
import torch
print('torch version:', torch.__version__)
print('cuda available:', torch.cuda.is_available())
```

然后执行如下命令执行这个文件。

```
python3 test.py
```

看到类似下面的输出说明，说明安装完成，并且可以使用 CUDA 加速。

```
torch version: 2.0.1+cu118
cuda available: True
```

7.3.4.2. 环境配置

无需特殊配置。

7.3.4.3. 开发案例

本例使用 CIFAR10 数据集(<https://www.cs.toronto.edu/~kriz/cifar.html>)进行图像分类器神经网络训练。它有类：“飞机”、“汽车”、“鸟”、“猫”、

“鹿”、“狗”、“青蛙”、“马”、“船”、“卡车”。CIFAR-10 中的图像尺寸为 32×32 ，即尺寸为 32×32 像素的 3 通道彩色图像。

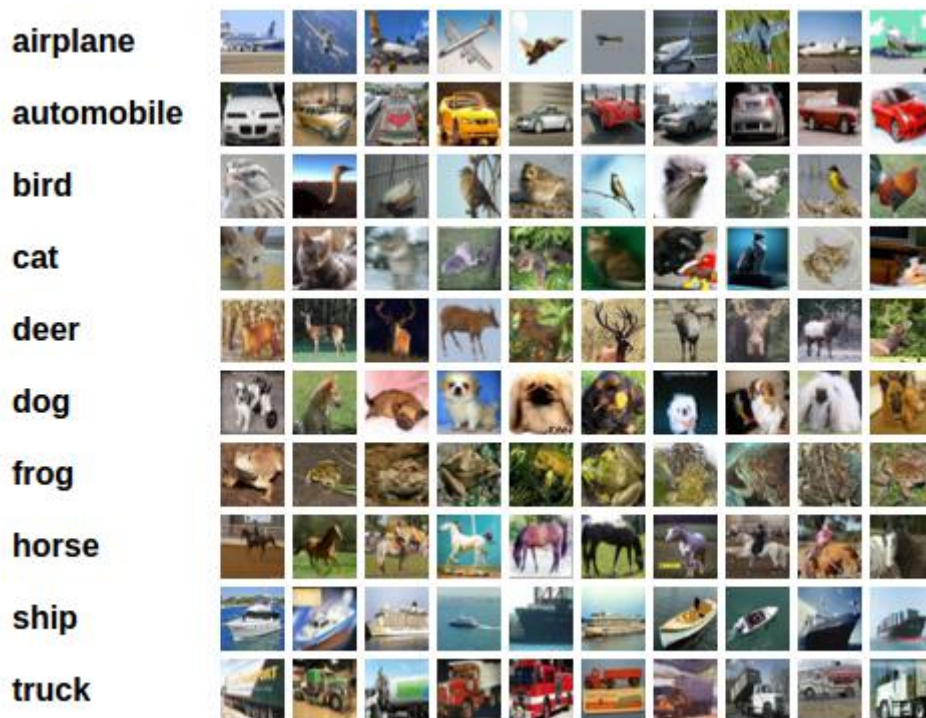


图 64 CIFAR-10 数据集示例图片

将以下代码保存为 `cifar10_tutorial.py` 文件。

```
import torch
import torchvision
import torchvision.transforms as transforms
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))])
batch_size = 4
# 加载数据集
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
    download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=batch
_size,
    shuffle=True, num_workers=2)
```

```
testset = torchvision.datasets.CIFAR10(root='./data', train=False,
download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=batch_size,
shuffle=False, num_workers=2)
# 定义分类名称
classes = ('plane', 'car', 'bird', 'cat',
'deer', 'dog', 'frog', 'horse', 'ship', 'truck')
import matplotlib.pyplot as plt
import numpy as np
# 显示图片
def imshow(img):
img = img / 2 + 0.5      # unnormalize
npimg = img.numpy()
plt.imshow(np.transpose(npimg, (1, 2, 0)))
plt.show()
# 随机获取一些训练数据
dataiter = iter(trainloader)
images, labels = next(dataiter)
# 查看图片和标签
imshow(torchvision.utils.make_grid(images))
# print labels
print(' '.join('%5s' % classes[labels[j]] for j in range(batch_size)))
import torch.nn as nn
import torch.nn.functional as F
# 定义卷积神经网络
class Net(nn.Module):
def __init__(self):
super().__init__()
self.conv1 = nn.Conv2d(3, 6, 5)
self.pool = nn.MaxPool2d(2, 2)
self.conv2 = nn.Conv2d(6, 16, 5)
self.fc1 = nn.Linear(16 * 5 * 5, 120)
self.fc2 = nn.Linear(120, 84)
```

```
self.fc3 = nn.Linear(84, 10)
def forward(self, x):
    x = self.pool(F.relu(self.conv1(x)))
    x = self.pool(F.relu(self.conv2(x)))
    x = torch.flatten(x, 1) # flatten all dimensions except batch
    x = F.relu(self.fc1(x))
    x = F.relu(self.fc2(x))
    x = self.fc3(x)
    return x
net = Net()
import torch.optim as optim
# 定义损失函数
criterion = nn.CrossEntropyLoss()
# 定义优化器
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
# 用训练数据来训练网络
for epoch in range(2): # 对数据集进行 2 次训练
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # 获取输入数据; data 类型是 [inputs, labels]
        inputs, labels = data
        # 将参数梯度归零
        optimizer.zero_grad()
        # forward + backward + optimize
        # 将数据提供给网络，再计算损失，反向传播，优化
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        # 打印统计结果
        running_loss += loss.item()
    if i % 2000 == 1999: # 每隔 2000 小批次输出一消息
        print(f'[{epoch + 1}, {i + 1:5d}] loss: {running_loss / 2000:.3f}')
    running_loss = 0.0
```



```
print('Finished Training')
# 保存模型为文件
PATH = './cifar_net.pth'
torch.save(net.state_dict(), PATH)
# 重新创建网络，并从之前保存的数据加载状态。
net = Net()
net.load_state_dict(torch.load(PATH))
# 在测试数据上测试网络
dataiter = iter(testloader)
images, labels = next(dataiter)
# 查看一些测试图片
imshow(torchvision.utils.make_grid(images))
print('GroundTruth: ', ' '.join('%5s' % classes[labels[j]] for j in range(4)))

# 用数据测试网络
outputs = net(images)
_, predicted = torch.max(outputs, 1)
# 输出预测结果
print('Predicted: ', ' '.join('%5s' % classes[predicted[j]] for j in range(4)))
correct = 0
total = 0
# 因为我们没有训练，所以我们不需要计算输出的梯度
with torch.no_grad():
    for data in testloader:
        images, labels = data
        # 将图片数据传给网络，计算输出结果
        outputs = net(images)
        # 我们选择数值最大的类别作为预测
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        # 统计正确数量
        correct += (predicted == labels).sum().item()
# 计算并打印正确率
```

```
print(f'Accuracy of the network on the 10000 test images: {100 *  
correct // total} %')
```

执行如下命令运行程序：

```
python3 cifar10_tutorial.py
```

程序会先从网络下载数据集，这里已经下载过了，没有重复下载，可以看到

类似如下输出内容：

```
Files already downloaded and verified  
Files already downloaded and verified  
bird  cat  dog  deer  
[1, 2000] loss: 2.215  
[1, 4000] loss: 1.888  
[1, 6000] loss: 1.714  
[1, 8000] loss: 1.594  
[1, 10000] loss: 1.527  
[1, 12000] loss: 1.484  
[2, 2000] loss: 1.413  
[2, 4000] loss: 1.396  
[2, 6000] loss: 1.374  
[2, 8000] loss: 1.343  
[2, 10000] loss: 1.337  
[2, 12000] loss: 1.284  
Finished Training  
GroundTruth:  cat  ship  ship  plane  
Predicted:  cat  car  truck plane  
Accuracy of the network on the 10000 test images: 52 %
```

这里模型的准确率为 52%，比概率要好得多，概率为 10%（从 10 个类别中随机选择一个类别）。

7.3.4.4. 常见问题

- 遇到"ModuleNotFoundError:No module named 'torch'"错误?

这个错误通常表示你的 Python 环境中没有安装 PyTorch。你需要确保已经正确安装了 PyTorch 并且在 Python 环境中可用。可以通过在终端或命令行中运行 `pip list` 或 `conda list` 命令来检查已安装的包列表中是否包含了 PyTorch。

- 如何创建和操作张量 (Tensors) ?

张量是 PyTorch 中的核心数据结构，类似于多维数组。你可以使用 `torch.tensor()` 函数创建张量，并使用各种张量操作进行数学运算和变换。可以参考 PyTorch 的官方文档或教程来学习如何创建和操作张量。

- 如何在 GPU 上运行 PyTorch?

PyTorch 支持在 GPU 上进行计算，以加速训练和推理过程。要在 GPU 上运行 PyTorch，你需要确保你的计算机上有可用的 GPU，并且已经安装了与你的 PyTorch 版本兼容的 GPU 驱动程序。然后，你可以使用 `torch.cuda.is_available()` 检查 GPU 是否可用，并使用 `to()` 方法将张量或模型移动到 GPU 上进行计算。

- 如何加载和处理数据集?

在深度学习中，数据集的加载和处理是非常重要的步骤。PyTorch 提供了 `torchvision` 模块，其中包含了许多常用的计算机视觉数据集（如 MNIST、CIFAR-10 等）的加载器。你可以使用这些加载器来加载数据集，并使用 PyTorch 的数据转换功能对数据进行预处理和增强。

- 如何定义和训练神经网络模型？

在 PyTorch 中，你可以使用 torch.nn 模块来定义神经网络模型的结构，包括各种层、激活函数等。训练模型时，你需要定义损失函数和优化器，并使用训练数据进行迭代优化模型参数。PyTorch 提供了灵活且强大的 API 来支持模型定义、训练和推理过程。

7.4. 小结

在本章节，主要介绍了 Windows 下的应用开发技术在 UOS 操作系统完成开发的过程，主要包含桌面应用开发技术、Web 应用开发技术、AI 应用开发技术。

其中，桌面应用开发是 Windows 平台的优势，部分开发框架无法在 UOS 操作系统平台使用，需要使用新的框架进行重新开发，其主要方案总结如下：

表 8 开发工具替代

框架	替代方案	开发难度	支持架构	主要问题
WPF	DTK、Qt	中	全架构	需要重新开发，工作量大。
Qt	原生支持	低	全架构	需要解决 Windows API 切换到 UOS 操作系统接口的问题。
OpenGL、Vulkan	原生支持	低	全架构	本身可移植性较好，但是一般会结合其他 UI 框架使用，需要考虑其依赖框架。需要考虑不同显卡硬件兼容性，目前国内显卡性能和兼容性一般。

在浏览器扩展于插件方面，extension 和 NPAPI 有完全不同的兼容情况：

表 9 浏览器插件替代

框架	替代方案	开发难度	支持架构	主要问题
extension	原生支持	低	全架构	默认就是平台无关的。
NPAPI	原生支持	高	全架构	本身可移植性较差，针对不同平台需要重新编译适配。

在 Web 后台服务方面，基于 Linux 的 UOS 操作系统平台对主流的 Web 开发技术有着极其优秀的兼容性，得益于 Linux 在服务器、容器市场的绝对优势，Web 开发技术一般会优先适配 Linux 平台。从 Windows 开发切换到 UOS 操作系统平台开发的方案总结如下：

表 10 web 开发工具替代

框架	替代方案	开发难度	支持架构	主要问题
ASP.NET	原生支持	低	X86、ARM	目前主要由微软官方维护，对其他架构的支持较弱。
Laravel	原生支持	低	全架构	本身可移植性较好，主要需要考虑是否使用了其他和平台相关的工具或库。
Ruby on Rails	原生支持	低	全架构	
Django	原生支持	低	全架构	
Spring Boot	原生支持	低	全架构	

Android 开发与 Web 开发非常类似，非常容易从 Windows 开发切换到 UOS 操作系统平台，但目前只支持 X86、ARM 架构，方案总结如下：

表 11 Android 开发替代

框架	替代方案	开发难度	支持架构	主要问题
Flutter	原生支持	低	X86、ARM	本身可移植性较好，主要需要考虑是否使用了其他和平台相关的工具或库。
React Native	原生支持	低	X86、ARM	

对于 AI 开发方面，基于 Linux 的 UOS 操作系统平台对于主流的 AI 开发技

术支持很好，也是得益于 Linux 在服务器市场的优势，方案总结如下：

表 12 AI 开发替代

框架	替代方案	开发难度	支持架构	主要问题
CUDA	原生支持	低	X86、ARM	目前由 NVIDIA 官方维护，对其他架构的支持较弱。
OpenCV	原生支持	低	全架构	本身可移植性较好，主要需要考虑是否使用了其他和平台相关的工具或库。
TensorFlow	原生支持	低	全架构	
PyTorch	原生支持	低	全架构	

由于 OpenCV 部分模块、TensorFlow 和 PyTorch 依赖 CUDA 进行显卡加速，因此可能在一些国内处理器架构上不能使用 CUDA 相关功能，只能利用 CPU 进行计算。

第八章 UOS 搭建基础服务与运维

8.1. 本章目的

Linux 以其强大的稳定性和丰富的服务扩展能力深受大家喜爱，统信 UOS 服务器操作系统作为 Linux 操作系统的发行版之一，同样支持并提供了常见的服务和应用程序，同时保持了和通用 Linux 下完全一致的使用体验和兼容可靠性。本章节从基本服务、虚拟化与容器、典型常用应用及运维管理工具几个维度分别展开，涵盖包括 HTTP、DNS、DHCP、FTP、NFS、SAMBAs、ISCSI、KVM、Docker、NGINX、MariaDB 及 Ansible 服务和应用的安装、配置、目录结构、常用参数和管理过程，为读者提供高效快捷的入门知识指导。

8.2. 基本服务安装与部署

8.2.1. HTTP 服务

HTTP 是世界使用排名第一的 Web 服务器软件。它可以运行在几乎所有广泛使用的计算机平台上。

HTTP 源于 NCSAhttpd 服务器，经过多次修改，成为世界上最流行的 Web 服务器软件之一。HTTP 取自"a patchy server"的读音，意思是充满补丁的服务器，因为它是自由软件，所以不断有人来为它开发新的功能、新的特性、修改原来的缺陷。HTTP 的特点是简单、速度快、性能稳定，并可做代理服务器来使

用。

8.2.1.1. 服务安装

- 安装 HTTP。

```
dnf install -y httpd
```

- 启动 HTTP 服务。

```
systemctl start httpd
systemctl enable httpd      #设置开机启动
```

- 启动完成后打开浏览器，验证：<http://192.168.200.201>。

8.2.1.2. 服务配置

- HTTP 各文件存放路径。

- 1) 默认 Web 目录：`/var/www/html/`
- 2) 配置目录：`/etc/httpd/`
- 3) 全局配置文件：`/etc/httpd/conf/httpd.conf`
- 4) 配置文件子目录：`/etc/httpd/conf.d`

- HTTP 配置文件详解。

```
<VirtualHost *:自定义端口>
    ServerName www.uos.com #在 ServerName 后加上你的网站名称。
    ServerAlias ftp.uos.com mail.uos.com    #如果你想多个网站名称都
取得相同的网站，可以加在 ServerAlias 后加上其他网站别名。别名间以空格隔
开。
```

```
    ServerAdmin uos@pxb.com #在 ServerAdmin 后加上网站管理员的电
邮地址，方便别人有问题是可以联络网站管理员
```

```
    DocumentRoot /var/www/html #在 DocumentRoot 后加上存放网站内
容的目录路径(用户的个人目录)
```



```
</VirtualHost>
<Directory /> #定义 “/” 目录区域的开始
Options FollowSymLinks #控制选项允许使用软链接
AllowOverride None #不允许隐含控制文件中的覆盖配置
Order allow,deny #访问控制策略的应用顺序
Allow from all #禁止任何人访问此区域
</Directory> #定义 “/” 目录区域的结束
```

8.2.2. DNS 服务

DNS 服务在网络中的作用就是维护一个地址数据库，其中记录了各种主机域名与 IP 地址的对应关系，以便为客户程序提供正向或反向的地址查询服务，即正向解析与反向解析。正向解析：根据域名查 IP 地址，是 DNS 服务最常用的基本功能；反向解析：根据 IP 查域名，不是很常用，应用于例如反垃圾邮件的验证等。

8.2.2.1. 服务安装

- 在 uos server1 上。

```
# yum install bind      #安装 DNS 软件 bind9
```

注意事项：BIND 安装以后，会自动增加一个名为 named 的系统服务。DNS 服务的端口：TCP 53 端口、UDP 53 端口

- 服务配置。

/etc/named.conf 是 Bind 的主配置文件，不过他并不包含 DNS 数据。查看/etc/named.conf 文件可以发现，主配置文件里面使用了 include 关键字来加载其它 3 个配置文件。Named.conf.options 文件解析：

```
options {
    directory "/var/cache/bind"; # zone 文件的默认路径
    // forwarders { #这里设置的是主 DNS 的 IP 地址
    //    0.0.0.0;      #当本地缓存中没有对应的解析时，将客户机的查询
转发到哪些 DNS 服务器，可以添加多个 DNS 服务器的地址。本地测试不设置
    // };
    dnssec-validation auto;      #开启 DNSSEC 功能，DNS 安全扩展
    listen-on-v6 { any; };      #定义 bind 的监听 IP 地址 (IPv6)
};
```

添加 DNS 区域配置：编辑 named.conf.local 添加

```
# cp /etc/bind/named.conf.local /etc/bind/named.conf.local.bak #
备份
# vim /etc/bind/named.conf.local
zone "uos.com" {          #定义 DNS 的 zone
    type master;          #定义此区为主服务器
    file "/etc/bind/db.uos.com"; #指定具体存放 DNS 记录的文件
};
zone "168.192.in-addr.arpa" {      #定义一个 IP 为 192.168.200.*反
向域区
    type master;
    file "/etc/bind/db.192.168.200";
};
```

type 类型有三种：

- master:表示定义的是主域名服务器。
- slave:表示定义的是辅助域名服务器。
- hint:表示是互联网中根域名服务器。

DNS 正向解析区域文件的配置：

```
#cp -rp /var/named/named.localhost /var/named/named.uos.com
# vim /var/named/named.uos.com
```

```

;
; BIND data file for local loopback interface
;
$TTL    604800                #正确解析数据的缓存生存周期
@       IN      SOA          uos.com. root.uos.com. (
域名      认证授权（主、从服务器      域名      邮箱地址
2        ; Serial      #序号
604800    ; Refresh      #更新间隔时间
86400     ; Retry        #更新失败再次尝试的间隔时间
2419200   ; Expire       #若一直失败，尝试多久后放弃
604800 )   ; Negative Cache TTL  #查询失败的 dns 缓存
;
@ IN NS uos.
@ IN A 127.0.0.1
@ IN AAAA ::1
www IN A 192.168.200.201      #主机记录
mail IN CNAME www

```

DNS 反向解析区域文件的配置：

```

#cp -rp /var/named/named.loopback /var/named/named.192.168.
200
# vim /var/named/named.192.168.200
; BIND reverse data file for local loopback interface
;
$TTL    604800
@       IN      SOA      uos. root.uos.com. (
                        1      ; Serial
                        604800 ; Refresh
                        86400  ; Retry
                        2419200 ; Expire
                        604800 ) ; Negative Cache TTL
;
@ IN NS localhost.
1.0.0 IN PTR localhost.

```

```
201      IN      PTR www.uos.com.
```

注意事项：声明域的时候已经有了，192.168.200 所以我们只需要输入 201 即代表 192.168.200.201；PTR 记录域名后的点，即 FQDN 记录。

记录类型：

- A：定义了一条 A 记录，即主机名到 IP 地址的对应记录。
- MX：定义了一邮件记录。
- CNAME：定义了对应主机的一个别名。
- PTR：表示反向记录。
- NS：域名服务器记录。

测试配置文件。

```
# named-checkzone db.uos.com db.127
# named-checkzone db.192.168.200 db.127
```

重启服务以重载配置。

```
# systemctl restart bind
```

部署主机域名缓存机制：在 uos server2 上配置。

- 确认本机的网络地址，主机映射，默认 DNS 服务器地址。

```
# yum install bind
```

- 配置。

```
# vim /etc/bind/named.conf.options
options {
    directory "/var/cache/bind";
    forwarders {
        192.168.200.201;
```

```
};  
    dnssec-validation auto;  
    listen-on-v6 { any; };  
};
```

- 重启。

```
# systemctl restart bind9
```

8.2.3. DHCP 服务

DHCP（动态主机配置协议）是一个局域网的网络协议。指的是由服务器控制一段 IP 地址范围的分配，客户机登录服务器时就可以自动获得服务器分配的 IP 地址和子网掩码、网关、DNS。

8.2.3.1. 服务安装

```
# yum install dhcp    #安装 dhcp  
# ifconfig            #查看目标主机的网卡名称
```

8.2.3.2. 服务配置

DHCP 服务器端的基本配置。

- 在 uos server1 上。

```
# vim /etc/default/isc-dhcp-server  
INTERFACESv4="ens33"           #将监听端口修改为对应网卡  
  
# vim /etc/dhcp/dhcpd.conf      #主配置文件  
    subnet 192.168.200.0 netmask 255.255.255.0 { #subnet 后跟子网网段，netmask 后跟子网掩码  
        range 192.168.200.201 192.168.200.250;      #分配的 IP 地址起止范围
```

```
option domain-name-servers 202.106.0.20; #DNS 服务器地址(多个地址用","隔开)
option routers 192.168.200.2;           #默认网关
default-lease-time 600;                 #最小租约 600 秒
max-lease-time 7200;                   #最大租约 36000 秒
option broadcast-address 192.168.200.255; #分发广播地址
}
```

- 重启。

```
# systemctl restart isc-dhcp-server
```

- 验证是否运行。

```
# netstat -uap |grep :67 # 查看 dhcp 是否正常运行 udp 67
```

- DHCP 客户端的基本配置。

- 1) 客户端网卡配置选择自动分配即可。
- 2) 默认情况下，UOS 在安装完成后就是自动分配状态，无需配置。
- 3) 通过以下命令或在图形界面点击网卡配置，可以使客户端立即发出自动配置请求：

打开 uos server2 客户端验证(先 down 掉网卡再重启网卡服务)

```
# nmcli connection down 有线连接
```

```
# nmcli connection up 有线连接
```

```
# ifconfig
```

- 根据 MAC 地址分配固定 IP。

- 1) 找到 uos2 客户端 MAC 地址。

```
# ifconfig
```

- 2) 在 uos1 上。

```
# vim /etc/dhcp/dhcpd.conf
```

```
host test {      #主机声明
    hardware ethernet 00:0c:29:a8:cc:8f;      #客户机网卡 MAC 地址
    fixed-address 192.168.200.202;      #制定分配的 IP 地址
}
# systemctl restart isc-dhcp-server
```

3) uos2 客户端验证

```
# nmcli connection down 有线连接
# nmcli connection up 有线连接
# ifconfig
```

得到的 IP 地址是 192.168.200.202。

8.2.4. FTP 服务

FTP 是 File Transfer Protocol（文件传输协议）的英文简称，用于 Internet 上的控制文件的双向传输。基于不同的操作系统有不同的 FTP 应用程序，分为服务端和客户端，而所有这些应用程序都遵守同一种协议以传输文件。在 FTP 的使用当中，用户经常遇到两个概念："下载"（Download）和"上传"（Upload）。"下载"文件就是从远程主机拷贝文件至自己的计算机上；"上传"文件就是将文件从自己的计算机中拷贝至远程主机上。

8.2.4.1. 服务安装

- 安装。

```
dnf -y install vsftpd
```

- 启动。

```
systemctl start vsftpd
```

- 设置开机时启动。

```
systemctl enable vsftpd
```

- 查验端口。

```
ss -nltp |grep 21
```

- 验证客户端登陆。

使用浏览器打开 ftp://<server1 IP>，以验证匿名用户可以登陆。

8.2.4.2. 服务配置

配置文件：/etc/vsftpd/vsftpd.conf，vsftpd.conf 的形式非常简单，每一行即为一项设定。若是空白行或是开头为#的一行，将会被忽略。内容的格式只有一种，如下所示：

```
option=value
```

但要注意：等号两边不能加空格。

8.2.5. NFS 服务

NFS 是 Network File System 的缩写，中文意思是网络文件系统。它的主要功能是通过网络（一般是局域网）让不同的主机系统之间可以共享文件或目录。

NFS 客户端通过挂载（mount）的方式将 NFS 服务器端共享的数据目录挂载到本地系统中。从客户端本地看，NFS 服务器端共享的目录就好像是客户端自己的目录一样。

8.2.5.1. 服务安装

- 安装。


```
dnf -y install rpcbind nfs-utils
```

- 新建测试用的三个目录以及文件。

```
mkdir /uos1 /uos2 /uos3
touch /uos1/file{1..9}
touch /uos2/file{10..19}
touch /uos3/file{100..109}
```

- 增加一个用户。

```
useradd -u 1111 -m nfsuser      #nfsuser 对应 uid 为 1001
chmod o+w /uos1
```

- 配置共享目录。

```
vim /etc/exports
/uos1 *(rw,anonuid=1001,anongid=1001,sync,no_wdelay)
/uos2 192.168.200.0/24(ro)
/uos3 192.168.200.2(rw,no_root_squash,sync,no_wdelay)
```

说明：/uos1：所有远程连接用户获得 nfsuser 身份，共享/uos；目录/uos

2：以只读方式挂载；/uos3：不压制 root 权限。

- 重启 nfs 相关服务。

```
systemctl restart nfs.service
systemctl restart rpcbind
```

8.2.5.2. 服务配置

/etc/exports 是 NFS 共享配置文件。

8.2.6. SAMBA 服务

Samba 是在 Linux 和 UNIX 系统上实现 SMB 协议的一个免费软件，由服务器及客户端程序构成。SMB（Server Messages Block，信息服务块）是一种

在局域网上共享文件和打印机的一种通信协议，它为局域网内的不同计算机之间提供文件及打印机等资源的共享服务。SMB 协议是客户机/服务器型协议，客户机通过该协议可以访问服务器上的共享文件系统、打印机及其他资源。通过设置“NetBIOS over TCP/IP”使得 Samba 不但能与局域网络主机分享资源，还能与全世界的电脑分享资源。

8.2.6.1. 服务安装

```
# dnf install -y samba
# systemctl start smb
# systemctl status smb
```

8.2.6.2. 服务配置

Samba 的配置文件/etc/samba/smb.conf。

8.2.7. ISCSI 存储服务

ISCSI: (Internet Small Computer System Interface) , 即 Internet 小型计算机系统接口。

ISCSI 技术是一种由 IBM 公司研究开发的,是一个供硬件设备使用的可以在 IP 协议的上层运行的 SCSI 指令集,这种指令集可以实现在 IP 网络上运行 SCSI 协议,使其能够在诸如高速千兆以太网上进行路由选择。ISCSI 技术是一种新储存技术,该技术是将现有 SCSI 接口与以太网(Ethernet)技术结合,使服务器可与使用 IP 网络的储存装置互相交换资料。

ISCSI 是一种基于 TCP/IP 的协议,用来建立和管理 IP 存储块设备、主机和

客户机等之间的相互连接，并创建存储区域网络（SAN）。SAN 使得 SCSI 协议应用于高速数据传输网络成为可能，这种传输以数据块级别（block-level）在多个数据存储网络间进行。

ISCSI 属于 SAN 的一种，它走的协议是通过 IP 网络，将 SCSI 块数据转换成网络封包。虽然 ISCSI 与 NAS 一样，都是通过 IP 网络来传输数据，但是在传输数据的方式协议上它还是属于 FC-SAN。一个是文件级别的（NAS），一个是 block 级别的（ISCSI）。

SCSI 结构基于客户/服务器模式，其通常应用环境是：设备互相靠近，并且这些设备由 SCSI 总线连接。ISCSI 的主要功能是在 TCP/IP 网络上的主机系统（启动器 initiator）和存储设备（目标器 target）之间进行大量数据的封装和可靠传输过程。此外，ISCSI 提供了在 IP 网络封装 SCSI 命令，且运行在 TCP 上。

8.2.7.1. 服务安装

- 安装 open-iscsi。

```
# dnf install open-iscsi
```

- 启动 iscsi 服务。

```
# systemctl restart iscsi #刷新 iqn 标识  
# systemctl start iscsid #启动 iscsi 服务
```

- 设置开机启动服务。

```
# systemctl enable iscsid
```

8.2.7.2. 服务配置

ISCSI 的配置文件：/etc/iscsi/iscsid.conf。

8.3. 虚拟化与容器

8.3.1. KVM 虚拟机管理

KVM 全称是基于内核的虚拟机（Kernel-based Virtual Machine）。它是一个开源软件，基于内核的虚拟化技术，实际是嵌入系统的一个虚拟化模块，通过优化内核来使用虚拟技术，该内核模块使得 Linux 变成了一个 Hypervisor，虚拟机使用 Linux 自身的调度器进行管理。

KVM 是基于虚拟化扩展（Intel VT 或者 AMD-V）的 X86 硬件的开源的 Linux 原生的全虚拟化解决方案。KVM 中，虚拟机被实现为常规的 Linux 进程，由标准 Linux 调度程序进行调度；虚机的每个虚拟 CPU 被实现为一个常规的 Linux 进程。这使得 KVM 能够使用 Linux 内核的已有功能。但是，KVM 本身不执行任何硬件模拟，需要客户空间程序通过/dev/kvm 接口设置一个客户机虚拟服务器的地址空间，向它提供模拟的 I/O，并将它的视频显示映射回宿主的显示屏。目前这个应用程序是 QEMU。

QEMU 是一套由 Fabrice Bellard 所编写的模拟处理器的自由软件。它与 Bochs, PearPC 近似，但其具有某些后两者所不具备的特性，如高速度及跨平台的特性。经由 kqemu 这个开源的加速器，QEMU 能模拟至接近真实电脑的速度。

libvirt 是管理虚拟机和其他虚拟化功能的软件，比如存储管理，网络管理的软件集合。它包括一个 API 库，一个守护程序（libvirtd）和一个命令行工具（virsh）；libvirt 本身构建于一种抽象的概念之上。它为受支持的虚拟机监控程序

实现的常用功能提供通用的 API。libvirt 的主要目标是为各种虚拟化工具提供一套方便、可靠的编程接口，用一种单一的方式管理多种不同的虚拟化提供方式。

我们通常所说和使用的 KVM 虚拟机，实际上是这三个软件的结合。

8.3.1.1. 服务安装

- 查看和打开 CPU 虚拟化支持。

```
egrep "(svm|vmx)" /proc/cpuinfo    #查看 cpu 是否支持虚拟化
```

- 注意事项。

- 1) vmx 对应 intel 的 cpu。
- 2) svm 对应 amd 的 cpu。
- 3) 需在主板 bios 打开虚拟化选项，多数计算机默认处于打开状态。

- 查看内核 kvm 支持。

```
lsmod|grep kvm
```

- 安装所需软件包。

```
yum install qemu qemu-img libvirt libguestfs-tools virt-install virt-manager virt-viewer #安装 KVM 和相关虚拟化工具
```

- 启动 libvirtd。

```
systemctl restart libvirtd.service    #启动 libvirtd  
systemctl status libvirtd.service     #查看启动状态
```

8.3.1.2. 服务配置

xml 配置文件也就是通常所说的虚拟机的描述文件，主要用来定义一个虚拟机的名称、UUID、CPU、内存、虚拟磁盘、网卡等各种参数设置。配置文件默

认路径为 “/etc/libvirt/qemu/”，uos.xml 配置文件说明：

```
<domain type='kvm'>
  <name>uos</name>                #虚拟机的名字
  <uuid>16e9cb49-8b05-4b34-9325-3bcf169dd32e</uuid>    #uuid 值
  <memory unit='KiB'>4194304</memory>                #虚拟机的最大内存
  <currentMemory unit='KiB'>4194304</currentMemory>   #虚拟机当前
的内存
  <vcpu placement='static'>4</vcpu>                  #该虚拟机的 cpu 数
  <os>
    <type arch='x86_64' machine='pc-i440fx-3.1'>hvm</type>    #hvm
表示全虚拟化
    <boot dev='hd'>                                          #hd 表示从硬盘启动
  </os>
  <emulator>/usr/bin/qemu-system-x86_64</emulator>    #二进制模拟
器设备的完整路径
  <disk type='file' device='disk'>    #disk 是用来描述磁盘的主要容器
    <driver name='qemu' type='qcow2'>
    <source file='/var/lib/libvirt/images/kvm/uos.qcow2'>    #指定磁盘
上文件的绝对路径
    <target dev='hda' bus='ide'>
    <address type='drive' controller='0' bus='0' target='0' unit='0'>
    <interface type='direct'                #桥接设备
    <mac address='52:54:00:0c:ae:41'>        #MAC 地址
    <source dev='enp2s0' mode='bridge'>
    <model type='e1000'>
    <address type='pci' domain='0x0000' bus='0x00' slot='0x03' functio
n='0x0'>
    </interface>
    <source network='br0'>
```

8.3.2. Docker 容器

Docker 使用 Google 公司推出的 Go 语言进行开发实现，基于 Linux 内核

的 cgroup, namespace, 以及 AUFS 类的 Union FS 等技术, 对进程进行封装隔离, 属于操作系统层面的虚拟化技术。由于隔离的进程独立于宿主和其它的隔离的进程, 因此也称其为容器。最初实现是基于 LXC, 从 0.7 以后开始去除 LXC, 转而使用自行开发的 libcontainer, 从 1.11 开始, 则进一步演进为使用 runC 和 containerd。

8.3.2.1. 服务安装

- 安装 docker。

```
# yum install -y docker
```

- 启动并设置开机启动。

```
# systemctl restart docker
# systemctl enable docker
```

- 查看版本信息。

```
# docker version
Client:
  Version:           18.09.1
  API version:       1.39
  Go version:        go1.11.6
  Git commit:        4c52b90
  Built:             Tue, 03 Sep 2019 19:59:35 +0200
  OS/Arch:           linux/amd64
  Experimental:      false
Server:
  Engine:
    Version:          18.09.1
    API version:      1.39 (minimum version 1.12)
    Go version:       go1.11.6
```

```
Git commit:      4c52b90
Built:           Tue Sep  3 17:59:35 2019
OS/Arch:         linux/amd64
Experimental:    false
```

8.3.2.2. 服务配置

用 dockerfile 构建镜像的一般过程：

- `mkdir uosdocker` #创建 Dockerfile 文件目录，进入目录并将需要用到的代码 CP 到此。
- `vim Dockerfile` #编辑 Dockerfile 内容。
- `docker build -t uos:20 uosdocker/` #构建镜像，指定镜像名称和版本。
- `docker images` #列出刚刚创建的 uos 镜像。

8.4. 典型常用应用

8.4.1. NGINX 服务

Nginx(engine x)是一个高性能的 HTTP 和反向代理 web 服务器,同时也提供了 IMAP/POP3/SMTP 服务反向代理。Nginx 是由伊戈尔·赛索耶夫为俄罗斯访问量第二的 Rambler.ru 站点（俄文：Рамблер）开发的，第一个公开版本 0.1.0 发布于 2004 年 10 月 4 日。其将源代码以类 BSD 许可证的形式发布，因它的稳定性、丰富的功能集、示例配置文件和低系统资源的消耗而闻名。

其特点是占用内存少，并发能力强，事实上 nginx 的并发能力在同类型的网页服务器中表现较好，中国大陆使用 nginx 网站用户有：百度、京东、新浪、

网易、腾讯、淘宝等。

8.4.1.1. 服务安装

- 安装。

```
dnf install nginx
nginx -v          #查看 nginx 版本
systemctl start nginx
systemctl enable nginx
```

启动后，在浏览器输入 IP 地址，即可看到 nginx 的欢迎页面。至此 nginx 安装成功。

- 源码编译安装。

```
wget http://nginx.org/download/nginx-1.14.2.tar.gz#下载 nginx 源码包
tar -xf nginx-1.14.2.tar.gz
cd nginx-1.14.2
./configure --help
./configure --prefix=/usr/local/nginx --without-http_rewrite_module
--without-http_gzip_module
#指定安装路径 #避免 Nginx 依赖于其他库（常用指定用户 组 开启加密）

make && make install #编译并安装
/usr/local/nginx/sbin/nginx 启动
netstat -anptu |grep nginx
curl http://127.0.0.1
```

8.4.1.2. 服务配置

- 进入/etc/nginx，编辑主配置文件 nginx.conf：

```
# vim /etc/nginx/nginx.conf
server {
```

```
listen      443;
    server_name  localhost;
    ssl          on;
    ssl_certificate      /root/server.crt;  #配置证书位置
    ssl_certificate_key  /root/server.key;  #配置秘钥位置
    #ssl_client_certificate ca.crt;#双向认证
    #ssl_verify_client on; #双向认证
    ssl_session_timeout  5m;
    ssl_protocols  SSLv2 SSLv3 TLSv1;
    ssl_ciphers  ALL:!ADH:!EXPORT56:RC4+RSA:+HIGH:+MEDIUM:+LOW:+SSLv2:+EXP;
    ssl_prefer_server_ciphers   on;
}
```

- #重启 nginx。

```
systemctl restart nginx
```

- 验证。

```
https://192.168.200.201  #nginx 的 https 功能可以使用了
```

8.4.2. 数据库（MariaDB 为例）

自甲骨文公司收购 MySQL 后，其在商业数据库与开源数据库领域市场的占有份额都跃居第一，这样的格局引起了业内很多的人士的担忧，因为商业数据库的老大有可能将 MySQL 闭源。

为了避免 Oracle 将 MySQL 闭源，而无开源的类 MySQL 数据库可用，MySQL 社区采用分支的方式来避开这个风险。MariaDB 数据库就这样诞生了，MariaDB 是一个向后兼容，可能在以后替代 MySQL 的数据库产品，其官方地址为：<https://mariadb.org/>

mariadb 和 mysql 几乎是一样的。首先，mariadb 就是由 mysql 的创始人

负责维护的。而 mariadb 就是 mysql 创始人女儿的名字。使用方法和 mysql 相同

8.4.2.1. 服务安装

```
yum install mariadb-server #安装 mariadb 服务端程序
systemctl start mariadb.service #启动程序
systemctl enable mariadb.service #设为开机自启动
ss -nltp |grep 3306 #mysql 服务默认的端口号是 tcp 3306
```

8.4.2.2. 服务配置

Mysql 命令行工具是 MySQL 官方提供的连接工具，用户可以通过 mysql 连接到 mysqld 上进行一系列的 SQL 操作。

mysql 工具有两种模式：交互模式和命令行模式。交互模式指令需要连接到 mysql 服务器下达，命令行模式通过特定（-e）参数读取 shell 命令行传递的指令到服务器。

命令格式：mysql [OPTIONS] [database]

- -h, --help 显示帮助信息。
- -e, --execute=name 执行指定命令，命令行模式。
- -u, --user=name 指定连接的用户。
- -h, --host=name 指定主机名。
- -p, --password 指定密码。
- -P, --port 指定端口。

8.5. 服务器的管理

8.5.1. Ansible 管理

- Ansible 是一种自动化运维工具。
- Ansible 是一种集成 IT 系统的配置管理、应用部署、执行特定任务的开源平台，它是基于 python 语言，由 Paramiko 和 PyYAML 两个关键模块构建。
- 集合了众多运维工具的优点，实现了批量系统配置、批量程序部署、批量运行命令等功能。
- Ansible 是基于模块工作的。
- Ansible 不需要在远程主机上安装 client/agents，因为它们是基于 ssh 来和远程主机通讯的。
- Ansible 也被定义为配置管理工具。

8.5.1.1. 服务安装

```
pip install -y paramiko PyYAML Jinja2 httplib2 six    #安装 ansible  
的依赖  
yum install -y ansible                                #安装 ansible
```

8.5.1.2. 服务配置

- 配置文件目录：/etc/ansible/
- 执行文件目录：/usr/bin/
- lib 依赖库：/usr/lib/python2.7/site-packages/ansible/
- help 文件：/usr/lib/python2.7/site-packages/ansible

第九章 UOS 开发者平台

为开发者提供统信 UOS 的开发技术文档、开放技术资源、统一调用系统的开放能力、商业化支持的一站式服务平台。通过开发者与平台的互动，不断促进操作系统生态体系繁荣，最终实现平台的商业价值孵化。

9.1. 平台价值

统信开发者平台作为一种共享型的产品服务模式，服务于个人开发者、企业开发者以及生态合作伙伴：

- 开发者通过平台开放的技术资源，获取优质的服务体验。
- 用户通过开发者的贡献，不断获取新的使用场景。
- 统信 UOS 通过开发者的技术支撑，获取更大的业务成功。
- 开发者平台的搭建，有助于三者相互依存，互惠互利。
- 学习平台：帮助开发者完成技术入门引导，并学习相关技术资源。
- 接口调用：开放部分操作系统、云端服务的 API 接口，供开发者直接调用。
- 需求定制：针对有开发能力的客户，根据特定需求对产品进行定制。
- 获取收益：借助平台的商业合作模式、资源以及 openAPI 获取收益和流量。

9.2. 开发者模块介绍

统信开发者平台是为 UOS 开发者提供的开放技术资源一站式服务平台。致力于帮助开发者统一调用系统的开放能力,降低开发者的开发成本,促进生态体系的繁荣。

9.2.1. 文档支持

开发者平台提供了丰富的技术文档支持,涵盖了 UOS 应用开发的全流程。这些文档内容系统整理,涵盖了应用设计、编码、测试、发布等每个开发阶段。

这些技术文档按照开发阶段进行了模块化分类,为开发者提供了有针对性的信息查阅。当前文档模块包括但不限于以下内容:

开发接口: 涵盖了 UOS ID、桌面环境功能扩展、DTK、安全开发等接口的详尽文档,帮助开发者快速获得来自官方的接口支持。

开发工具: 提供了集成开发环境 (deepin IDE), 支持开发者高效地进行应用程序的开发、调试和测试。

驱动开发: 为开发者提供了各类型驱动开发的技术文档,包括显示设备驱动、无线网络驱动、蓝牙设备驱动、打印驱动等。同时为开发者提供了调试优化和常见问题的分享,帮助开发者快速进行驱动的开发工作。

应用分发: 提供了开发者在软硬件开发完成后的商业化支持引导,提供了适配认证、应用分发、驱动投递的全流程指引,帮助开发者快速获得商业化支持。

这些技术文档将持续更新,跟进最新技术动态,确保为开发者提供准确、及

时的开发支持和指导。

9.2.2. 常见问题

在开发过程中，开发者常常面临各种技术挑战。为解决这些问题，开发者平台整合了大量常见问题的解答，并以问答形式呈现给开发者。目前，平台已根据已收集的问题进行了分类整理，覆盖了系统、应用、外设、商店、硬件、适配等多个开发场景。

开发者可在平台上搜索与其工作相关的关键词以获取专业团队提供的相关解答。这些解答来自于专业团队，确保了解决方案的准确性和实用性。

9.2.3. 开发者论坛

开发者论坛是 UOS 社区内开发者和用户交流的核心枢纽。论坛不仅仅是一个提问和解答的平台，更是一个汇聚各类话题的场所。在这里，用户可以提出您在 UOS 开发或使用过程中遇到的问题，分享个人的技术见解和最佳实践，以及提供对产品改进的建议和反馈。

除社区成员间的互动外，官方团队也会积极参与并通过论坛发布各种重要信息。开发者及用户可以通过论坛获得最新的产品更新、技术教程、重要公告和各类活动信息。同时，官方团队也在论坛中与用户保持紧密的互动，关注用户的需求和反馈，为产品的持续改进提供了有力支持。

9.2.4. 商业化支持

开发者平台不仅为开发者提供技术支持,还给予商业化运营的帮助。开发者

可以通过开发者平台实现如下的商业化支持：

开发者认证：不论您是个人开发者或者是企业开发者，都可以通过开发者平台进行开发者认证，认证通过后的主体即可获得来自官方的商业化支持。

适配认证：开发者平台为开发者提供了包括整机设备、外设（打印机、扫描仪等）、流版签、中间件、数据库、安全管理、云平台等等各类型的适配认证服务，通过适配认证测试的产品将会发放适配认证证书。

应用分发：通过应用分发服务，开发者可以将开发完成的应用上架到统信应用商店，获得来自官方的应用分发服务。

帐号服务：统信帐号服务是面向开发者提供的基于 OAuth2.0 标准协议的帐号授权登录服务，通过接入帐号服务，终端用户使用统信帐号即可快速便捷地登录开发者的应用。

驱动投递：通过驱动投递服务，硬件开发者可以将 UOS 操作系统相关的驱动信息或者驱动包投递到统信官方分发渠道，经审核验证可用后，开发者投递的信息将共享给所有 UOS 系统用户。

第十章 开发案例介绍

10.1. 中国移动“移动办公”开发案例

移动办公是中国移动为集团成员打造的一款办公软件，围绕企业新沟通场景，提供企业通讯录、即时消息、通话会议、协同办公等功能。同时为提升成员感知，开展“V 网短号升级”、“成员权益营销”、“数智办公应用”等领域融合创新，进一步凸显差异化优势。



图 65 移动办公软件界面

10.1.1. 案例背景

随着移动互联网和智能终端的不断发展，为了进一步提高企业信息流转和信

息处理效率，企业也对办公信息化和自动化提出了的新需求，移动办公成为企业办公和移动互联结合应用的一大主题。特别是在在国产化信创的背景下，移动办公软件需要与国产化硬件和软件相适应，以满足国家对信息安全和自主可控的要求。

10.1.2. 项目需求

移动办公软件的设计目标是帮助用户摆脱时间和空间的限制，随时随地随意处理工作，提高效率、增强协作。在国产化信创环境下，移动办公软件需要实现与国产操作系统兼容、适配国产 CPU 处理器、支持国产数据库和与国产中间件兼容。

10.1.3. 项目难点

移动办公的主要难点在于如何实现跨平台兼容性，提高开发效率，以及保证安全可靠。移动办公基于 Electron 框架开发，客户端 UI 层采用 web 前端技术栈开发，服务端采用 Java 语言开发，端侧核心模块封装在 C++原生模块中，通过 NodeJS 集成，相比原生桌面应用开发，具有开发效率高、版本迭代快等特点，同时在安全性和应用性能方面都有所优化，保障用户体验良好。

10.1.4. 解决方案和成效

移动办公的解决方案主要包括以下几个方面：

跨平台兼容性：Electron 是一个可以使用 Web 技术来开发跨平台桌面应用的开发框架，可以做到一套代码兼容 Windows、Mac、Linux 三个系统，具有

很好的跨平台兼容性。

开发效率高：移动办公前端采用 Angular、Vue 框架，利用组件化开发模式，将各个业务模块组件化，便于开发复用，同时利用 Electron 自身丰富的 API 能力，能够轻松实现业务需求，极大地提升开发效率。

安全可靠：移动办公将消息模块等核心业务采用 C++ 语言实现，再通过 NodeJS 集成到端侧，另外端侧在源码泄露和源码保护方面均采用了一些安全措施，保障前端代码不被泄露，提升了端侧的安全可靠性。

通过这些解决方案，移动办公软件在提供丰富功能的同时，也保证了软件的安全性和性能，为用户提供了良好的使用体验。

10.2. 中国移动“移信云”平台开发案例

中国移动移信云是一款紧跟国家和集团发展战略的产品，以终端域操作系统替代为切入点，进行业务支撑系统和管理信息系统的国产化替代演进研究。它充分利用了省内业务支撑系统和管理信息系统丰富的业务场景优势，总结了信息系统国产化面临的软硬件兼容性、业务稳定性、系统性能等关键共性问题，并设计了适用于大型复杂信息系统国产化替代的解决方案。



图 66 “移信云”平台

10.2.1. 案例背景

移信云平台通过结合云原生技术和国产化技术，利用 IT 云属地省两级协同优势和云技术能力，构建了移动信创云化共性技术平台。这个平台实现了国外云平台、云桌面、操作系统的替代，以及国产化终端和外设、国产操作系统环境的承载，和在国产化环境中业务应用的免改造访问等重要创新和突破。这些创新和突破为解决国产化替代的共性技术问题提供了低成本和高效的解决方案，有力地支持了公司内外部用户复杂信息系统的转型升级。

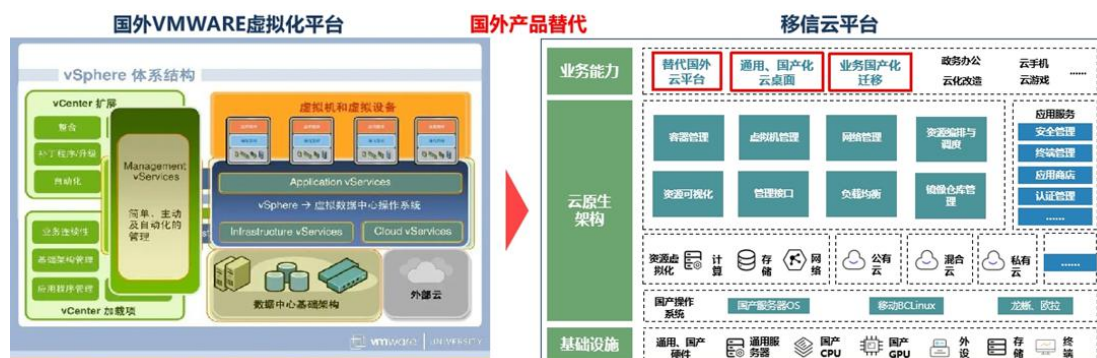


图 67 产品设计图

10.2.2. 项目需求

移信云平台的主要需求包括：信息系统端到端分层解耦及云端适配问题；基于移动自有 IT 云资源，构建支撑通用及国产化架构的云原生基础平台；基于业务虚拟化、应用虚拟化技术实现双架构业务并行；通过虚拟化迁移技术实现国外云平台、云桌面产品的替代。

10.2.3. 项目难点

移信云平台端到端支持通用及国产化双架构环境，云端集中进行业务支撑系统、管理信息系统 48 个子业务访问兼容适配。在不改造业务系统的条件下，实现了营业、办公、运维终端系统的国产化替代，大幅降低了国产化替代的难度，降低了应用系统改造的成本，有效保障了系统转型升级过程中业务的可用性和可靠性。

10.2.4. 解决方案和成效

此外，移信云平台还能发布通用及国产操作系统云桌面，解决了国外云平台的瓶颈问题。云端集中进行高拍仪、写卡器等现网数十种外设兼容适配，实现了 X86、ARM 架构国产化终端瘦终端及现网各类终端的云化使用。平台采用自有云资源实现了 4000 点国外 VMware 办公、营业云桌面的替代，支撑了现网 1 余万点营业、办公、运维终端的国产化改造上云，有效提高了系统国产化的覆盖率，提升了老旧设备的使用效率，大幅降低了系统建设运维的成本。



图 68 后台管理界面

10.3. 北京城建“i 城建”平台开发案例

i 城建是北京城建集团打造的新一代平台，通过对已有数据资源的计算分析形成集团自有的大数据决策中心，服务于企业的决策和管理在建筑行业中率先实现了工程项目成本、财务资金的统一管控。

i 城建客户端应用支持 PC 端和手机端，本次迁移的为 PC 端 i 城建应用，基于 JDK1.7 进行开发，前期是在 x86 环境+Windows 系统使用，本次终端国产化替过程中，将基础环境替换为麒麟 990 芯片+统信桌面操作系统 V20，实现 i 城建应用在全国化环境的部署和应用。

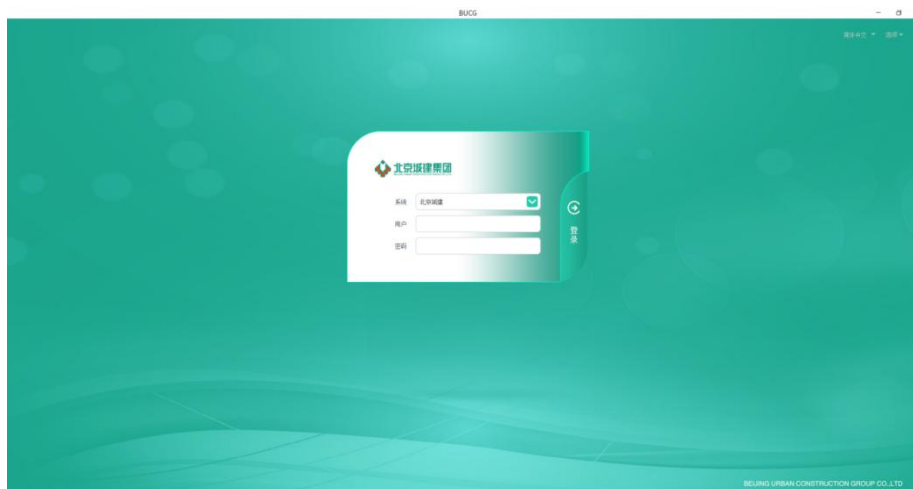


图 69 “i 城建”平台

10.3.1. 案例背景

北京城建集团有限责任公司是一家大型综合性建筑企业集团，员工 26500 余人，旗下拥有 120 余家法人企业、42 家分公司，目前国产化替换工作正在全面启动中，本案例主要涉及办公终端替换，预计替换终端约 4 万台。

10.3.2. 项目需求

i 城建为北京城建集团内部办公统一入口应用，北京城建集团客户在进行国产化替换过程中，本着真替真用的宗旨，在进行基础软硬件兼容性验证的同时，优先启动核心应用 i 城建在国产化平台上的迁移工作，顺利完成 i 城建的迁移工作，满足用户使用需求，是城建集团本次国产化终端迁移的关键因素。

10.3.3. 项目难点

在本次城建集团国产化终端替换项目中，用户硬件选型为麒麟 990 芯片，应用开发技术采用 Java。在麒麟 990 芯片上图形框架采用 Wayland，而 Wayland 并未对 jdk 进行原生适配支持，尤其用户采用的 jdk 版本为较早的 1.7 版本，基于上述基本环境，在迁移 i 城建项目过程中，遇到打印控件无法显示、应用弹出框不可点击、窗口黑屏等等棘手的问题，该类问题需要从操作系统、jdk、Wayland 技术、应用层代码多维度去分析解决。

10.3.4. 解决方案及成效

为解决上述难点问题，统信软件配合城建集团技术专家成立联合公关组，深入分析 java 环境与 Wayland 图形框架兼容问题，与 X11 环境进行细致对比，

部分问题通过修改操作系统底层代码进行规避, java 环境或 i 城建应用层的问题通过调整上层实现方式进行解决。

在采用了系统底层与基础环境, 加上应用层联合分析的解决方案后, 有效解决或规避了上述应用过程中遇到的问题, 满足了城建集团用户在麒麟 990 芯片+UOS V20 系统环境上使用 i 城建应用的需求, 统信 UOS 开发者应用指南为国产化替换过程中应用迁移、产品适配提供了一整套技术指导, 内容涉及到开发语言、开发环境以及不同类型应用的迁移方案选择, 对于初步接触国产化环境的技术人员来说, 是一套非常实用的实践操作指南, 相信该指南在各种类型的迁移场景下都能帮助开发人员快速上手, 少走弯路。

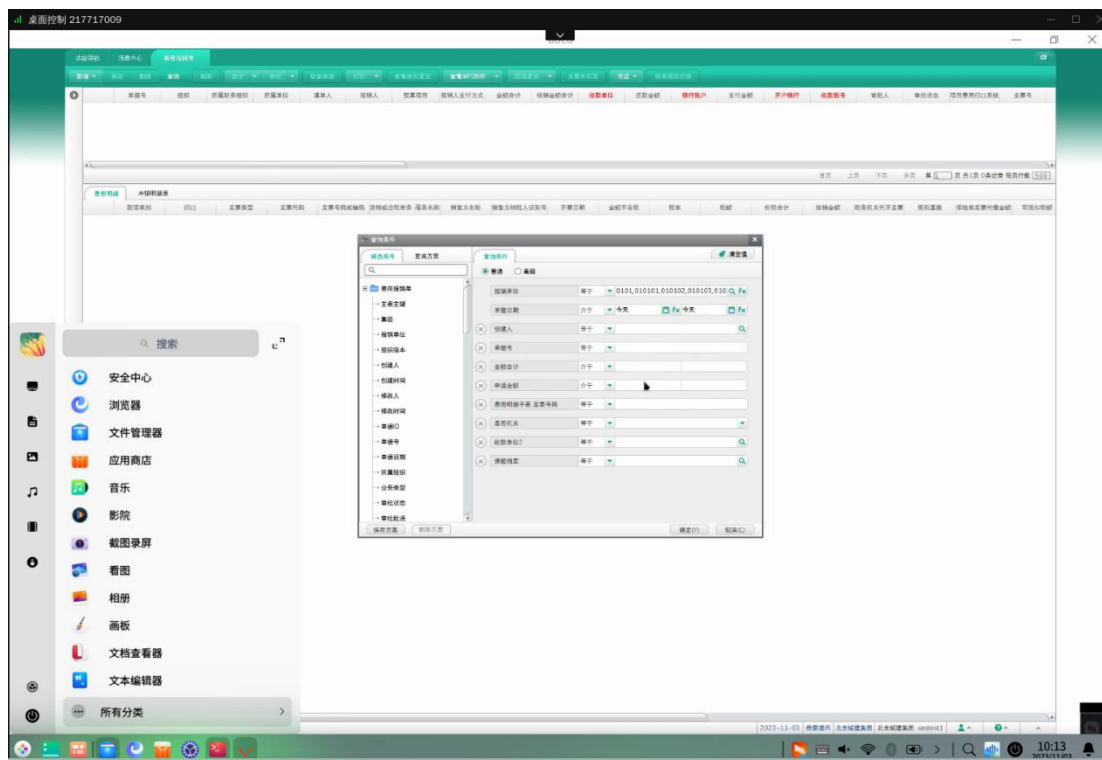


图 70 “i 城建”平台

10.4. 无锡研究院开发案例

ResNet50 是一个基于残差网络的深度学习模型，用于解决分类、回归等任务。该模型是在 ImageNet 图库上训练得到的，具有强大的图像分类能力。ResNet50 是 ResNet 模型中的一种，它具有 50 层深度，在不同的计算机视觉任务中都得到了广泛的应用。本文介绍一种基于 ResNet50 识别图像内容的应用案例，采用跨平台开发语言 Python，统信 UOS 本身带了 Python2.7 和 3.7，可以很方便的支持应用开发。

10.4.1. 案例背景

在国防领域，图像识别技术的应用越来越广泛，例如用于识别敌方军机、军舰等目标。本案例中使用 ResNet50 模型进行目标的图像识别。

10.4.2. 项目需求

项目的主要需求是开发一个能够准确识别和分类目标的图像识别系统。系统需要能够处理大量的图像数据，并能在短时间内给出准确的识别结果。

10.4.3. 项目难点

项目的主要难点在于如何提高图像识别的准确性和效率。由于目标的形状、颜色、大小等特征可能会因环境条件（如光照、天气等）和目标本身的变化（如军机的飞行姿态等）而变化，因此需要选择一个能够抽取图像深层特征的模型。此外，由于需要处理大量的图像数据，因此还需要考虑如何提高系统的处理速度。

10.4.4. 解决方案和成效

我们选择了 ResNet50 作为我们的图像识别模型。ResNet50 具有 50 层深度，能够抽取图像的深层特征，因此具有较高的识别准确性。此外，我们还使用了 Python 作为开发语言，并在统信 UOS 操作系统上进行开发，这使得我们能够方便地进行跨平台开发。

我们的系统在测试中表现出了良好的性能。我们使用了一张名为 cat.jpg 的图片进行测试，通过运行命令 `python3 resnet.py`，我们的系统能够准确地识别出图片中的目标。

```
from torchvision.io import read_image
from torchvision.models import resnet50, ResNet50_Weights
import torch

img = read_image("cat.jpg")

# Step 1: Initialize model with the best available weights
weights = ResNet50_Weights.DEFAULT
model = resnet50(weights=weights)
model.eval()

# Step 2: Initialize the inference transforms
preprocess = weights.transforms()

# Step 3: Apply inference preprocessing transforms
batch = preprocess(img).unsqueeze(0)

# Step 4: Use the model and print the predicted category
prediction = model(batch).squeeze(0).softmax(0)
class_id = prediction.argmax().item()
score = prediction[class_id].item()
category_name = weights.meta["categories"][class_id]
print(f"{category_name}: {100 * score:.1f}%")
```

图 71 执行结果

10.5. 无锡研究院开发案例

MNIST 数据集 (Mixed National Institute of Standards and Technology database) 是一个用来训练各种图像处理系统的二进制图像数据集，广泛应用于机器学习中的训练和测试。该案例基于统信 UOS 桌面版环境，原生应用开发 MNIST 数据集图像分类应用，以及 ResNet50 识别图像内容应用。采用跨平台开发语言 Python。统信 UOS 本身带了 Python2.7 和 3.7，可以很方便地支持应用开发。

10.5.1. 案例背景

整个数据集由三部分构成，训练集有 55000 条数据，验证集有 5000 条数据，测试集有 10000 条数据。虽然这个数据集听起来还是挺多的，但实际上在机器学习里，这个只是一个入门级的案例。通过这样量级的数据集，可以给我们做到一个比较好的模型训练。

10.5.2. 项目需求

本项目的需求是基于 MNIST 数据集进行图像分类应用的开发，包括训练和测试两个部分。训练部分的目标是通过学习 MNIST 数据集，训练出一个能够准确识别手写数字的模型。测试部分的目标是使用训练得到的模型，对新的手写数字图像进行识别，以验证模型的准确性。

10.5.3. 项目难点

项目的难点主要在于如何处理和识别图像数据。MNIST 数据集的图像是二进制的，需要通过特定的方式进行处理才能被模型正确识别。此外，由于手写数字的形状、大小和书写风格的差异，使得图像识别的难度增加。

10.5.4. 解决方案和成效

本项目采用 Python 语言进行开发，利用其强大的数据处理和机器学习库，对 MNIST 数据集进行处理和学习。在训练过程中，模型的 loss 在不断下降，表明模型的性能在不断提高。在测试过程中，模型的准确率达到了 98.43%，表明模型具有很高的识别准确性。

此外，本项目还在统信 UOS 桌面版环境下进行了原生应用开发，包括 MNIST 数据集图像分类应用和 ResNet50 识别图像内容应用。这些应用的开发和运行，进一步证明了统信 UOS 操作系统对国产化软件的良好支持。

```
Train Epoch: 1 [0/60000 (0%)] Loss: 2.277438
Train Epoch: 1 [640/60000 (1%)] Loss: 1.111244
Train Epoch: 1 [1280/60000 (2%)] Loss: 0.825507
Train Epoch: 1 [1920/60000 (3%)] Loss: 0.637466
Train Epoch: 1 [2560/60000 (4%)] Loss: 0.516296
Train Epoch: 1 [3200/60000 (5%)] Loss: 0.491021
Train Epoch: 1 [3840/60000 (6%)] Loss: 0.214211
Train Epoch: 1 [4480/60000 (7%)] Loss: 0.505524
Train Epoch: 1 [5120/60000 (9%)] Loss: 0.303467
Train Epoch: 1 [5760/60000 (10%)] Loss: 0.195921
Train Epoch: 1 [6400/60000 (11%)] Loss: 0.366282
Train Epoch: 1 [7040/60000 (12%)] Loss: 0.232958
Train Epoch: 1 [7680/60000 (13%)] Loss: 0.270607
Train Epoch: 1 [8320/60000 (14%)] Loss: 0.217196
Train Epoch: 1 [8960/60000 (15%)] Loss: 0.328657
Train Epoch: 1 [9600/60000 (16%)] Loss: 0.078749
Train Epoch: 1 [10240/60000 (17%)] Loss: 0.137674
Train Epoch: 1 [10880/60000 (18%)] Loss: 0.152079
Train Epoch: 1 [11520/60000 (19%)] Loss: 0.236204
Train Epoch: 1 [12160/60000 (20%)] Loss: 0.067038
Train Epoch: 1 [12800/60000 (21%)] Loss: 0.166221
Train Epoch: 1 [13440/60000 (22%)] Loss: 0.128072
Train Epoch: 1 [14080/60000 (23%)] Loss: 0.201078
Train Epoch: 1 [14720/60000 (25%)] Loss: 0.096085
Train Epoch: 1 [15360/60000 (26%)] Loss: 0.367067
Train Epoch: 1 [16000/60000 (27%)] Loss: 0.316969
Train Epoch: 1 [16640/60000 (28%)] Loss: 0.136688
Train Epoch: 1 [17280/60000 (29%)] Loss: 0.108511
Train Epoch: 1 [17920/60000 (30%)] Loss: 0.149551
```

图 72 执行结果

10.6. 云达软件资产管理系统开发案例

10.6.1. 案例背景

云达软件资产管理系统，是由云达信息技术有限公司（简称“云达信息”）自主研发的软件正版化工作综合管理信息化平台。该系统可有效辅助做好软件正版化工作，建立完善正版软件资产管理长效机制，加强软件配置、使用、处置等环节管理，用信息化产品和技术手段实现了政府机关、企事业单位软件资产全生命周期动态管理，确保用户软件使用符合法规，为用户节省软件采购、运维、集成等一系列费用，真正实现软件资产在摸清家底、盘活存量的基础上规范管理、管好增量，优化投资布局，最终实现软件资产收益最大化。



图 73 资产管理系统监控

10.6.2. 项目需求

- 软件资产管理系统以及服务，所有终端（包括国产机器）软件信息自动提交到管理后端。
- 授权与合同统一在系统上管理、软件授权与终端匹配、补齐缺失授权。

10.6.3. 项目难点

- 多平台兼容。

该系统能够支持不同芯片环境下运行 Windows、统信、麒麟等操作系统，以及国产中间件和数据库等。系统具备多平台融合的能力，使系统管理员能够统一管理不同环境下的软件资产。

- 多级网络互联。

该系统能够支持多级网络互联，在多账号系统环境下运行。这意味着需要在复杂的网络中进行软件资产管理，并确保数据的安全传输和通信。系统应具备灵活的用户权限管理和安全机制，能够根据不同用户的权限进行操作，保护敏感数据的安全性和完整性。

10.6.4. 解决方案及成就

引进安装“软件资产管理系统”及服务，所有终端（包括国产机器）软件信息自动提交到管理后端；授权与合同统一在系统上管理，软件授权与终端匹配，补齐欠缺授权，使用已有授权；实现软件资产效益最大化，避免软件厂商控告，降低企业经营风险；本服务同时包含了培训和台账梳理，大大减轻资产监管工作

的人力负担。

10.7. 广州掌动智能 AI 性能一体化保障平台 V3.0 开发案例

掌动智能 AI 性能一体化保障平台 V3.0（简称：XRunner）是一款全栈性能测试解决方案，涵盖了脚本生成，场景设置，性能监控，报告生成等性能测试全流程功能。XRunner 的主要功能包括模拟生产环境工作负载的大规模压测，测试广泛的云环境或者本地部署的应用，识别私有云/公有云，容器云，微服务/Mesh 层的性能瓶颈，诊断代码、容器、微服务级别的根源问题，优化系统部署性能，实时提供部署参数调优建议等。



图 74 功能介绍

10.7.1. 案例背景

XRunner 是掌动智能公司的核心产品，该公司是中国领先的数字化转型基础支撑软件核心技术提供商。XRunner 的开发和应用，是掌动智能响应国家对科技安全上升至国家战略层面的要求，推动基础软硬件的国产化，保障信息安全，

以及关键领域的信息化投入的重要举措。

10.7.2. 项目需求

XRunner 的开发需求主要来自于国内外的大型企业和政府机构，他们需要一款能够全面、深入地进行性能测试的工具，以保障其业务系统的稳定运行。同时，这些用户也对测试工具的易用性、灵活性和实时性有着较高的要求。此外，随着国家对科技安全的重视程度不断提高，国产化也成为了这些用户选择测试工具时的重要考虑因素。

10.7.3. 项目难点

XRunner 的开发面临的主要难点包括如何实现对各种应用环境的广泛适配，如何精准地识别和定位性能瓶颈，如何提供实时的部署参数调优建议，以及如何实现全面的国产化。这些难点需要掌动智能公司在技术研发、产品设计和市场布局等方面进行全面的考虑和策划。

10.7.4. 解决方案和成效

为了解决上述难点，XRunner 采用了一系列创新的技术和方法。在技术性能参数方面，XRunner 主要采用 BS 结构，服务器端采用 java 编程语言，核心模块均为自主研发，采用 Vue 前端框架用于构建用户界面，基于标准 HTML、CSS 和 JavaScript 构建，并提供了一套声明式的、组件化的编程模型。在国产化方面，XRunner 取得了统信操作系统的适配认证，包括各类国产 CPU 架构（龙芯、鲲鹏、飞腾、申威、兆芯、海光）的适配认证。

XRunner 的应用成效显著，它不仅提高了用户的性能测试效率和准确性，还为用户节省了大量的人力和物力资源。同时，XRunner 的成功应用也为掌动智能公司赢得了良好的市场口碑和用户信任，为公司的持续发展奠定了坚实的基础。

10.8. 交大慧谷案件管理系统开发案例

案件管理系统是一种用于管理和维护法院或其他司法机关的案件信息的系统。它通过实现案件登记、案件分发、案件办理、司法文书生成等功能，实现案件信息的快速传递和查询，提高办案的科学性和规范性。在这个案例中，我们将探讨如何将案件管理系统与信创国产化相结合。

10.8.1. 案例背景

随着信息技术的发展和国产化的推进，国产化案件管理系统的需求日益增长。国产化不仅可以满足国内市场的需求，还可以提供更佳的用户体验。信创，作为一种新的信息技术应用创新，以加强信息技术的安全、可靠、可控为核心发展目标，为案件管理系统提供了新的可能性。

10.8.2. 项目需求

案件管理系统的主要功能包括案件登记、案件分发、庭前文书生成、案件排期和盖章结案。这些功能需要通过信创国产化平台实现，以满足国内市场的需求和提供更佳的用户体验。此外，案件管理系统还需要适配主流信创软硬件方案，自主可控，全面保护信息安全。

10.8.3. 项目难点

项目的主要难点在于如何将案件管理系统与信创国产化相结合，以满足国内市场的需求和提供更佳的用户体验。此外，案件管理系统需要适配主流信创软硬件方案，自主可控，全面保护信息安全，这也是项目的一个难点。

10.8.4. 解决方案和成效

解决方案包括国产化适配，主要包括适配环境搭建、国产化服务器适配、国产化数据库适配、国产化中间件适配等。整体思路是在环境安装完成情况下，把稳定版本的应用及数据库、中间件、外设等安装到国产化环境下，并检查各项功能适配无问题。

根据系统及数据库现状和迁移需求，进行分析和理解，然后对应用系统进行现状调研，了解应用系统的功能内容、性能指标、使用情况等，根据了解到的需求情况，进行产品选型和编写系统适配方案，系统迁移方案，即可开始准备适配与迁移改造工作。

该方案可以解决客户从终端到基础硬件，再到基础软件以及应用软件替换过程中的痛点，让客户省心，放心。我们的解决方案具有高可靠性，可以系统地降低国产化过程中的风险。

10.9. 交大慧谷通用融合法庭开发案例

10.9.1. 案例背景

慧谷通用融合法庭是一种新型的法庭形态，通过将传统法庭与现代科技有机

结合，满足群众多元司法需求的科技支撑。在信创国产化的背景下，慧谷通用融合法庭采用了国产化的软硬件技术，以确保数据安全和网络安全。

10.9.2. 项目需求

慧谷通用融合法庭的主要需求包括：

- 跨域远程开庭：实现不同地域法庭之间的远程庭审，实现法庭审理过程的实时互动和全面记录。
- PC 端网上开庭：利用现代信息技术手段将实体法庭“搬到”线上，方便当事人参与庭审过程。
- 手机端开庭：案件当事人可通过手机 APP 参加庭审。
- 车载法庭开庭：采用云上庭审方式进行公开开庭，将法庭搬到商圈景区、村社院坝。

10.9.3. 项目难点

项目难点主要包括：

- 信创国产化技术的融合与应用：如何将国产化的软硬件技术应用于慧谷通用融合法庭的各个功能模块。
- 数据安全性与网络安全：确保庭审过程中的数据传输安全，防止信息泄露。

10.9.4. 解决方案和成效

慧谷通用融合法庭在信创国产化背景下，采用了国产化的软硬件技术，包括国产操作系统、数据库、中间件等。通过建立专网共享科技法庭、跨网接入远程

出庭室、云上联通在线庭审网，方便当事人“就地”“就近”参与诉讼，真正实现诉讼活动全时空泛在化。这种庭审方式可以节省当事人往返法院的时间和费用，减轻法院的工作负担，提高审判效率，同时增强司法的公正性和公信力。

10.10. 交大慧谷司法行装管理系统开发案例

10.10.1. 案例背景

司法行装管理系统是一款专门为人民法院机关设计的操作系统，主要用于管理固定资产、基本建设、环境秩序等。该系统记录了人民法院机关办公大楼的管理和维修情况，负责法院各项经费、办公用品保障和车辆管理使用、维修保养等工作。在中国的数字化转型过程中，信创产业的发展成为了必然选择，而司法行装管理系统的国产化也是这一过程的重要组成部分。

10.10.2. 项目需求

司法行装管理系统的主要功能包括行装后勤服务、财务管理、物资管理、快递收寄件、药品管理和智慧后勤等。这些功能需要在国产化的过程中得到保障和优化，以满足法院的实际需求和操作习惯。同时，系统的国产化也需要考虑到与信创环境的兼容性，以便在“进口”与国产之间实现平滑的迁移

10.10.3. 项目难点

项目的主要难点在于如何在保证功能完整的同时，实现系统的国产化。由于中国的技术发展在过去的数十年间严重依赖进口，因此在自主研发国产硬件的过

程中，可能会遇到硬件支持的复杂性问题。此外，由于硬件之间存在架构差异，软件涉及兼容和运行环境限制，因此在“进口”与国产之间的迁移也是一个具有挑战性的工作。

10.10.4. 解决方案和成效

为了解决这些难点，我们采取了一系列的措施。首先，我们进行了深入的市场调研和技术研究，了解了国内外的最新技术和产品，以便在国产化的过程中引入最适合的技术和解决方案。其次，我们与国内的硬件制造商进行了深度合作，共同研发了兼容国产硬件的软件系统。同时，我们也对现有的系统进行了优化，以便在国产硬件上运行得更加稳定和高效。

在实施了这些解决方案后，我们取得了显著的成效。司法行政管理系统的国产化不仅成功实现，而且在功能和性能上都达到了预期的目标。同时，系统的国产化也为我们节省了大量的成本，提高了我们的竞争力。

10.11. 交大慧谷政务管理系统开发案例

10.11.1. 案例背景

该案例是我们一个政务管理系统的实施过程，以及如何利用信创国产化的技术和方法来提升其效能。

随着社会信息化、数字化的深入发展，人民法院的工作也面临着新的挑战 and 机遇。如何利用先进的技术手段，提高审判效率，提升司法公正性，成为法院工作的重要课题。融合法庭，作为一种新型的法庭形态，正是应运而生，以满足社

会对司法服务的高标准、严要求。

10.11.2. 项目需求

政务管理系统的主要需求包括待办事项管理、公文管理、内部电子邮件系统以及值班管理等。这些功能可以帮助法院提高工作效率，确保在限制的时间内完成任务，并提供自动催办功能，对督办文件办结情况进行归档。

10.11.3. 项目难点

政务管理系统的实施面临的主要难点包括信息化程度的不一致，部分部门可能缺乏信息化建设的基础，导致系统的推广受阻。此外，政务管理系统的安全性问题也需要重视，政府部门涉及的信息较为敏感，系统的安全性必须得到保障，以防止信息泄露和数据被篡改的风险。

10.11.4. 解决方案和成效

政务协同 OA 管理系统作为一种新型的管理工具，正在逐渐被政府部门广泛采用。通过信息共享、协同办公和流程管理等功能，该系统提高政府部门的工作效率，提高决策的科学性和准确性，促进部门之间的协作与合作。然而，推广和应用该系统也面临一些挑战，需要政府部门加大投入和改革力度，克服各种困难，确保系统能够发挥最大的效益。

第十一章 总结

UOS 操作系统下的应用开发是构建各种应用程序的过程，涉及到多种编程语言、开发工具和框架。本书从 UOS 原生应用开发、安全接口开发、Web 应用开发、迁移方案、第三方应用开发以及基于 UOS 的基础服务运维等几个方面展开介绍，同时也介绍了 UOS 下常用的编程语言、开发套件和 IDE 工具。帮助开发人员快速完成从 Windows 到 UOS 的跨架构原生开发转换，并掌握 UOS 应用开发者所需的关键技能。

统信作为国内操作系统的领创者，一直致力于研发安全稳定、智能易用的操作系统产品，并以操作系统为核心，引领中国软硬件生态建设，同上下游软硬件厂商一同合作，为伙伴提供方便的开发环境和基础设施，共同打造操作系统创新生态，给世界更好的选择！

附录一：统信软件介绍

统信软件技术有限公司（简称：统信软件），成立于 2019 年，注册资金 6.27 亿，是中国操作系统的领创企业。在北京、武汉、上海、广州、南京、成都、西安等多地设立分支机构，现有员工 2000 人，研发技术人员占比达 70%。在全国共设有 6 大研发中心、7 大区域服务中心与 3 地生态适配认证中心，技术服务能力辐射全国。

统信软件在操作系统研发、行业定制、国际化、迁移适配、交互设计等多方面拥有深厚的技术积淀，现已形成桌面、服务器、智能终端等操作系统产品线，以及集中域管平台、企业级应用商店、“有”系列平台迁移软件等应用产品，能够满足不同用户和应用场景对操作系统产品与解决方案的广泛需求。据第三方机构统计数据，统信 UOS 操作系统在桌面端持续保持市占率第一，在服务器端增速行业第一。

统信软件还打造了中国首个面向全球的桌面操作系统根社区—deepin。目前 deepin 社区已成为中国最大、最活跃的操作系统社区，用户遍布全球 100 多个国家与地区，累计发布版本 60 余次，支持语言 34 种，累计下载量超 9000 万次，全球社区用户超 300 万，提供开源代码超过 1000 万行。近两年 Distro Watch 网站的 Linux 排行榜上稳居前列，在国际上拥有较大影响力。

统信软件积极开展以统信 UOS 操作系统为核心的软硬件生态建设，携手上

下游合作伙伴紧密合作，攻克难关，致力打造“开放、创新、共赢”的生态体系。截止 2023 年 10 月，适配合作伙伴超 6000 家，同心生态联盟成员突破 1100 家，生态适配总数超 330 万。

统信软件在产品研发实力、信息安全和质量管理上均已达到行业领先标准，专利累计申请 713 件，专利累计授权 257 件，拥有计算机软件著作权 240 项。通过了 CMMI3 国际评估认证及 GB/T 20272-2019 安全操作系统四级认证，拥有 ISO27001 信息安全管理体系认证、ISO9001 质量管理体系认证等资质。统信软件也积极参与操作系统标准制定相关工作，截止目前，已参与制定相关国标、团标、行标等近 40 项。

统信软件始终秉承以“以客户为本”的服务理念，通过科学、完善的服务流程体系，为用户提供标准化、规范化、专业化的技术服务支持，打造统信软件优质的服务口碑。截止目前，统信软件已在全国建立七大区域服务中心，31 个省级服务点，专业的服务团队能够为用户提供 7*24 小时全方位、及时性的技术服务与支持。

统信软件以“打造操作系统创新生态，给世界更好的选择”为愿景，致力于研发安全稳定、智能易用的中国操作系统产品，以操作系统为核心，引领中国软硬件生态建设，让世界见证中国科技力量！

附录二：通用开发问题

由于统信 UOS 操作系统是基于 Linux 的发行版，而 Linux 作为类 Unix 的操作系统，继承了 Unix 系统很多优秀的底层设计，因此和 Windows 有着完全不同的文件系统和目录结构实现形式。在许多 Windows 应用代码切换到 UOS 时，需要时刻注意这些区别。本章节从路径分隔符、行尾符、文件系统区分、目录结构差异、服务保留端口等几个方面，讲解两者差异情况，了解并掌握这些差异，有助于读者在应用开发过程中避免因差异代码导致开发问题的产生。

- 路径分隔符。

Windows 使用反斜杠 `\` 作为路径分隔符，而 UOS 使用正斜杠 `/`。在代码中直接使用硬编码路径分隔符可能导致在 UOS 上的路径问题。应使用跨平台的路径分隔符定义或使用操作系统提供的路径处理函数。

- 行尾符。

Windows 使用回车和换行符 `\r\n` 作为行尾符，而 UOS 使用换行符 `\n`。在将源文件从 Windows 切换到 UOS 时，需要将行尾符转换为 UOS 格式。

- 文件系统区分。

Windows 一般采用 NTFS 文件系统，UOS 采用 EXT4 文件系统，Windows 对文件名和路径不区分大小写，而 UOS 是区分大小写的，在代码中

引用文件名和路径时要注意大小写。另外不同的操作系统中都存在一些系统的限制，如打开文件句柄数的限制、Socket 等待队列的限制、进程和线程堆栈大小的限制等，因此在开发的过程中，必须考虑到这些限制因素对程序的影响。

- 目录结构差异。

Windows 平台和 UOS 平台的目录结构完全不同，因此在开发应用时，要特别注意源代码中有关应用工作目录的处理。

- Web 服务端口使用。

在 UOS 中，使用特权端口（1024 以下）需要 root 权限，如果应用没有特别要求，可以使用 1024 以上端口。如果需要在 80、443 等特殊端口提供服务，可以使用 root 权限来运行程序，或者使用如下命令给程序设置权限：

```
sudo setcap 'cap_net_bind_service=+ep' /path/to/my_program
```

附录三：版本推荐

序号	语言或工具	系统默认版本	其他支持版本
1	C/C++	8.3.0	8.3.0
2	C#	6.0	dotnet-sdk-6.0 dotnet-sdk-7.0
3	JAVA	openjdk-11、11.0.9.2	opendjdk-8 openjdk-11
4	Python	2.7.16	2.7.16、3.7.3
5	perl	5.28.1.2	5.28.1.2、6.d-2
6	shell	5.0.1.1	5.0.1.1
7	Javascript	不涉及	不涉及
8	nodejs	v10.24.0	v10.24.0
9	php	PHP 7.3.19-1+eagle	7.3.19
10	golang	1.15.9.3	1.15.9.3
11	QT	5.11.3	5.11.3
12	DTK	5.6.x	5.6.x
13	OpenGL	4.6.0	3.3.0
14	Vulkan	1.2	1.2
15	D-Bus	1.12.20	1.12.20
16	UnionCode	1.1.19	1.1.19
17	Qt Creator	4.8.2	4.8.2
18	VS code	1.83.1	1.83.1
19	Eclipse	2023.09	2023.09
20	Android Studio	2022.2.1	2022.2.1
21	PAM	1.3.2.2-1+dde	1.3.2.2-1+dde
22	USKi	不涉及	不涉及
23	系统安全接口	0.1.242-deepin1	0.1.242-deepin1
24	浏览器扩展插件	不涉及	不涉及

序号	语言或工具	使用版本	支持版本
25	浏览器插件	不涉及	不涉及
26	ASP.NET	7.0.400	7.0.400
27	Laravel	4.5.1	4.5.1
28	Ruby on Rails	3.1.4p223	3.1.4p223
29	Django	3.2.22	3.2.22
30	Spring Boot	Spring CLI v3.2.0	Spring CLI v3.2.0
31	deepin-wine	6.0.0.62-1	6.0
32	统信彩虹平台软件	2.3	2.2
33	Electron	v28.0.0	28.0.0
34	Flutter Android	3.10.0	3.0.0
35	React Native	0.71.11	0.60
36	CUDA	9.2.148	9.2.148
37	OpenCV	3.2.0	3.2.0
38	TensorFlow	1.10.1	1.10.1
39	PyTorch	1.13.1	1.13.1